CSCI-GA 2420 / MATH-GA 2010

Numerical Methods I

Course Notes

Fall 2026

Instructor: Prof. Florian Schaefer

Notes by You Li

Acknowledgements

These notes follow the Fall 2026 lectures of CSCI-GA 2420 / MATH-GA 2010 (Numerical Methods I) at the Courant Institute, taught by Prof. Florian Schaefer. Each chapter follows one lecture. Sections marked (SUPPLEMENT) go beyond the blackboard and draw on Trefethen and Bau [1], Demmel [2], and Golub and Van Loan [5]; background from linear algebra follows Axler [4].

Notation. Throughout, $\mathbb{K}$ denotes either $\mathbb{R}$ or $\mathbb{C}$. Vectors carry an arrow, $\vec{x} \in \mathbb{K}^n$; for $A \in \mathbb{K}^{m\times n}$, A^* is the conjugate transpose (equal to $A^\top$ when $\mathbb{K} = \mathbb{R}$), $\mathcal{R}(A)$ is the range (column space), and $\mathcal{N}(A)$ is the null space. The symbol $:=$ means "is defined as", with the colon on the side of the name being defined ($=:$ when that name stands on the right). Unless stated otherwise, $\|\cdot\|$ is the Euclidean norm on vectors and the induced 2-norm on matrices. Absolute values and inequalities between matrices, such as $|E| \leq |L|\,|U|$, are taken entrywise.

Contents

1 Numerical Representation of Vectors and Matrices **6**
- 1.1 Overview . . . 6
- 1.2 Vectors and Matrices . . . 6
- 1.3 Representing Real Numbers . . . 7
- 1.4 The Geometry of Floating-Point Numbers . . . 9
- 1.5 Cancellation . . . 9
- 1.6 Conditioning . . . 10
- 1.7 Stability . . . 10
- 1.8 (SUPPLEMENT) Further Topics . . . 12

2 Cache Hierarchy and Tiling/Blocking, Solving Linear Systems **14**
- 2.1 Flops Are Not the Whole Story . . . 14
- 2.2 Memory Is One-Dimensional . . . 15
- 2.3 The Memory Hierarchy . . . 15
- 2.4 Loop Reordering . . . 16
- 2.5 Tiling . . . 16
- 2.6 Linear Systems . . . 17
- 2.7 Triangular Systems . . . 17
- 2.8 Gaussian Elimination . . . 18
- 2.9 (SUPPLEMENT) Further Topics . . . 19

3 Solving Linear Systems and Stability **21**
- 3.1 Recap: From Triangular Systems to Elimination . . . 21
- 3.2 The Algebra of Lower Triangular Matrices . . . 21
- 3.3 Answering the Two Questions . . . 23
- 3.4 The In-Place Algorithm . . . 23
- 3.5 Zero Pivots . . . 24
- 3.6 Small Pivots Are Also Dangerous . . . 24
- 3.7 Error Analysis of the LU Factorization . . . 25
- 3.8 (SUPPLEMENT) Further Topics . . . 25

4 LU Factorization with Pivoting **27**
- 4.1 Backward Error of the LU Factorization . . . 27
- 4.2 Pivots and the Growth of Products . . . 29
- 4.3 Partial Pivoting . . . 29
- 4.4 The In-Place Algorithm with Partial Pivoting . . . 30

4.5 (SUPPLEMENT) Further Topics . . . 31

5 Least Squares Problems, Gram–Schmidt, and QR **33**
5.1 Is Partial Pivoting Enough? . . . 33
5.2 Non-Square Systems . . . 34
5.3 Least Squares Problems . . . 34
5.4 The Geometry of Least Squares . . . 35
5.5 Orthogonalization: The QR Factorization . . . 36
5.6 Classical Gram–Schmidt . . . 36
5.7 Modified Gram–Schmidt . . . 38
5.8 (SUPPLEMENT) Further Topics . . . 40

6 Product View on Error Bounds, Householder QR **42**
6.1 Why Write Algorithms as Matrix Products? . . . 42
6.2 The Product Form of Gram–Schmidt . . . 42
6.3 Error Amplification in Products . . . 45
6.4 Lower Triangularization: The Case of LU . . . 46
6.5 Gram–Schmidt Is Also a Lower Triangularization . . . 47
6.6 Householder Triangularization . . . 47
6.7 (SUPPLEMENT) Further Topics . . . 51

7 Modified Gram–Schmidt **53**
7.1 Picking Up from Householder QR . . . 53
7.2 Using Q . . . 53
7.3 Givens Rotations . . . 54
7.4 Full and Reduced QR . . . 56
7.5 Existence . . . 57
7.6 Uniqueness . . . 58
7.7 (SUPPLEMENT) Further Topics . . . 59

8 Overdetermined Least Squares and the Normal Equations **61**
8.1 Recap and Motivation . . . 61
8.2 Conventions . . . 62
8.3 The Optimality Condition . . . 63
8.4 What the Optimality Condition Says . . . 66
8.5 The Normal Equations . . . 66
8.6 Gram Matrices and Positive Semidefinite Matrices . . . 67
8.7 (SUPPLEMENT) Further Topics . . . 70

9 Cholesky Factorization **72**
9.1 Recap and Motivation . . . 72
9.2 Deriving Cholesky by Symmetric Elimination . . . 73
9.3 The Cholesky Algorithm . . . 75
9.4 Existence: Cholesky Never Breaks Down . . . 78
9.5 Rounding Errors . . . 79
9.6 Schur Complements . . . 81
9.7 Cholesky and QR . . . 83
9.8 (SUPPLEMENT) Further Topics . . . 84

10 Eigen-like Decompositions, Computing a Single Eigenvalue **87**
10.1 Recap and Motivation . . . 87
10.2 Eigenpairs of Hermitian Matrices . . . 88
10.3 A Negative Result: Eigenvalues Cannot Be Computed in Finitely Many Steps . . . 90
10.4 Power Iteration . . . 91
10.5 Matrix Functions: Manipulating the Spectrum . . . 96
10.6 Inverse Iteration . . . 97
10.7 Rayleigh Quotient Iteration . . . 98
10.8 (SUPPLEMENT) Further Topics . . . 102

11 Schur Factorization **105**
11.1 Recap and Motivation . . . 105
11.2 Deflation . . . 105
11.3 Subspace Iteration: Motivation . . . 107
11.4 Eigenvalue Problems as Matrix Factorizations . . . 107
11.5 Defective Matrices and Invariant Subspaces . . . 109
11.6 The Jordan Normal Form . . . 111
11.7 The Schur Factorization . . . 113
11.8 Computing Eigenvectors from the Schur Form . . . 116
11.9 (SUPPLEMENT) Further Topics . . . 117

12 Orthogonal Iteration **119**
12.1 Recap . . . 119
12.2 From Schur Factorization to Diagonalization . . . 119
12.3 Orthogonal Iteration . . . 121
12.4 Orthogonal Iteration in General . . . 123
12.5 Convergence of Eigenvector Estimates . . . 124
12.6 Convergence to Schur Factorization . . . 125
12.7 (SUPPLEMENT) Further Topics . . . 126

13 QR without Shifts **128**
13.1 Recap . . . 128
13.2 Schur and Eigendecomposition . . . 129
13.3 Convergence of Orthogonal Iteration . . . 130
13.4 Convergence to Schur Factorization . . . 131
13.5 Basic QR Iteration . . . 132
13.6 (SUPPLEMENT) Further Topics . . . 134

14 Hessenberg QR **136**
14.1 Recap: From Orthogonal Iteration to the QR Algorithm . . . 136
14.2 The Cost of the QR Iteration . . . 136
14.3 Reduction to Hessenberg Form . . . 137
14.4 The QR Iteration on Hessenberg Matrices . . . 138
14.5 Bulge Chasing . . . 139
14.6 The Implicit Q Theorem . . . 140
14.7 (SUPPLEMENT) Further Topics . . . 141

15 Shifted QR **142**

15.1 Recap: QR Iteration on Hessenberg Matrices . . . 142
15.2 Proof of the Implicit Q Theorem . . . 143
15.3 Application to Bulge Chasing . . . 144
15.4 The Shifted QR Iteration . . . 144
15.5 Choosing the Shift . . . 144
15.6 Relationship with (Inverse) Power Iteration . . . 146
15.7 (SUPPLEMENT) Further Topics . . . 147

A Linear Algebra Basics **149**
A.1 Matrix Multiplication . . . 149
A.2 Transpose and Conjugate Transpose . . . 150
A.3 Inverses, Elementary and Triangular Matrices . . . 151
A.4 Determinants . . . 152
A.5 Subspaces and Rank . . . 152
A.6 Eigenvalues and Diagonalization . . . 153
A.7 Orthogonality and Projections . . . 153
A.8 Hermitian Matrices, Quadratic Forms, and the SVD . . . 154

B Norms and Unitary Matrices **155**
B.1 Vector and Matrix Norms . . . 155
B.2 Unitary Transformations . . . 160
B.3 The 2-Norm, Singular Values, and Condition Numbers . . . 162
B.4 The Frobenius Norm . . . 163

C The Floating-Point Model **166**
C.1 IEEE 754 Formats . . . 166
C.2 Machine Epsilon and the Model of Arithmetic . . . 167
C.3 Backward Stability . . . 168

Lecture 1

Numerical Representation of Vectors and Matrices

Supplementary reading:

- [1] Lectures 12, 13, and 14.
- [2] Sections 1.3–1.5.
- [5] Section 2.4.

1.1 Overview

The topics of the semester are: numerical representation of vectors and matrices; solving linear systems by Gaussian elimination; solving least squares problems by orthogonalization; computing eigendecompositions; low-rank approximation; iterative methods; and structured matrices.

This lecture covers the first topic. A computer cannot store real numbers exactly: every number is rounded when it is stored, and every arithmetic operation rounds again, so even $(x + a) - a = x$ can fail. We proceed in two parts.

1. *Representation* (Sections 1.3 to 1.5): how real numbers are stored (floating point) and what errors this causes.
2. *Error analysis* (Sections 1.6 and 1.7): how to tell whether an error comes from the problem (*conditioning*) or from the algorithm (*stability*).

1.2 Vectors and Matrices

We write $\mathbb{K}$ for either $\mathbb{R}$ or $\mathbb{C}$, so that every result only needs to be stated once. A *vector* $\vec{v} \in \mathbb{K}^m$ is an m-tuple of elements of $\mathbb{K}$, representing a position or direction in an m-dimensional coordinate system. A *matrix* $A \in \mathbb{K}^{m \times n}$ is an $m \times n$ array of elements of $\mathbb{K}$, which may be thought of as n positions in an m-dimensional

coordinate system. We identify $\mathbb{K}^{m\times 1} \cong \mathbb{K}^m$, so a column vector is a matrix with one column:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \\ A_{31} & A_{32} \end{bmatrix} = \begin{bmatrix} \vec{a}_1 & \vec{a}_2 \end{bmatrix}. \tag{1.1}$$

Scalar multiplication and addition act entrywise. For $A \in \mathbb{K}^{m\times n}$, $\vec{v} \in \mathbb{K}^n$, and $B \in \mathbb{K}^{n\times \ell}$,

$$(A\vec{v})_i = \sum_k A_{ik} v_k, \qquad (AB)_{ij} = \sum_k A_{ik} B_{kj}, \tag{1.2}$$

so A is a linear map $\mathbb{K}^n \to \mathbb{K}^m$ and $\mathbb{K}^{n\times\ell} \to \mathbb{K}^{m\times\ell}$: it carries a region, together with the vectors $\vec{x}$ in it, to the image vectors $A\vec{x}$.

The entrywise formula is rarely the most useful way to think. Numerical linear algebra relies much more on the *column view*:

$$A\vec{v} = \sum_{j=1}^{n} v_j \vec{a}_j, \tag{1.3}$$

that is, $A\vec{v}$ is the linear combination of the columns of A with coefficients given by the entries of $\vec{v}$. This view appears throughout the course. The column space $\mathcal{R}(A)$ decides whether $A\vec{x} = \vec{b}$ is solvable, the QR factorization orthogonalizes the columns, and low-rank approximation asks how few directions suffice to approximately span them. Likewise, the j-th column of AB is A applied to the j-th column of B, so a matrix–matrix product is ℓ matrix–vector products done at once.

1.3 Representing Real Numbers

To store $\vec{v} \in \mathbb{K}^m$ or $A \in \mathbb{K}^{m\times n}$ in memory, we must represent real numbers. A computer's native objects are bits, $\mathbb{B} \coloneqq \{0, 1\}$, so every number must be built from finitely many of them. We get there in steps, each fixing a shortcoming of the previous one: unsigned integers, signed integers, fixed point, and floating point.

1.3.1 Unsigned and Signed Integers

A string $d \in \mathbb{B}^b$, with bits $d_i \in \{0, 1\}$, represents the integers 0 through $2^b - 1$ via

$$\mathrm{UInt}(d) \coloneqq \sum_{i=1}^{b} d_i 2^{i-1}. \tag{1.4}$$

To represent negative numbers, we use an offset and give the highest bit a negative weight:

$$\mathrm{Int}(d) \coloneqq \sum_{i=1}^{b-1} d_i 2^{i-1} - d_b 2^{b-1}, \tag{1.5}$$

which covers -2^{b-1} through $2^{b-1} - 1$. This is *two's complement*. Nonnegative numbers keep their unsigned encoding and the adder circuit is shared with unsigned integers. The price is an asymmetric range: the most negative number, such as `typemin(Int8)` $= -128$, has no positive counterpart, so $-x$ can overflow back to x. (A different kind of offset, the *biased* encoding that stores $x + K$, is used for floating-point exponents; see Section C.1.)

1.3.2 Fixed-Point Numbers

Fixing the position of the binary point and using $f = \lfloor b/c \rfloor$ of the bits as fractional digits gives

$$\mathrm{Fixed}(d) \coloneqq \mathrm{Int}(d) \cdot 2^{-f}. \tag{1.6}$$

That is, about $1/c$ of the bits represent the fractional part (e.g., $c = 2$: half of them).

The problem. Fixed-point numbers are equally spaced, with spacing 2^{-f} everywhere. To represent both a large number s and a small number s^{-1}, we need about $\log_2(s)$ bits before the binary point and about as many after it. So the number of bits grows *linearly* with the number of orders of magnitude we want to cover. Physical simulations routinely span a dozen orders of magnitude, so fixed point quickly becomes impractical.

1.3.3 Floating-Point Numbers

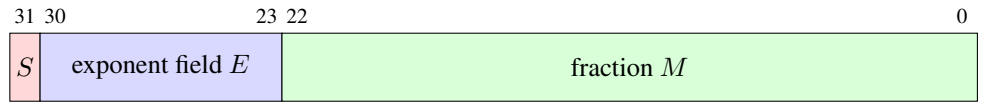
The idea of floating point is to store the order of magnitude separately as an exponent. Given $(p, e) \in \mathbb{B}^{b_p} \times \mathbb{B}^{b_e}$, define

$$\mathrm{Float}(p, e) \coloneqq (-1)^{p_1} \left(1 + \sum_{i=2}^{b_p} p_i 2^{-(i-1)}\right) 2^{\mathrm{Int}(e)}. \tag{1.7}$$

The three parts are the sign bit p_1, the *significand* (or *mantissa*) in $[1, 2)$, and the power of two. The leading 1 of the significand is *implicit* and takes no storage. The decimal analogue is $\pm 1.123 \cdot 10^{-4}$.

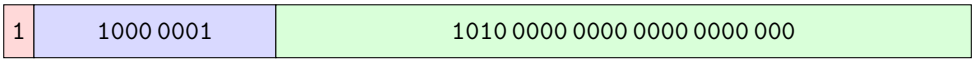
This solves the problem of fixed point: to cover numbers from s^{-1} to s, the exponent only has to reach $\pm \log_2(s)$, which takes about $\log_2(\log_2(s))$ bits. Fixed point needs about $\log_2(s)$ bits for the same range. So floating point covers huge ranges cheaply, which is why scientific computing uses it. A complex number is stored as two floating-point numbers. For details, see [6].

In memory. The picture below shows how the bits are laid out, for single precision (Float32, $b_p = 24$, $b_e = 8$).



$$\underbrace{(-1)^{p_1}}_{\text{sign}} \times \underbrace{\left(1 + \sum_{i=2}^{b_p} p_i 2^{-(i-1)}\right)}_{\text{significand } 1.M \in [1,2)} \times \underbrace{2^{\mathrm{Int}(e)}}_{\text{IEEE stores } E = \text{exponent}+127}$$

For example, $-6.5 = -110.1_2 = -1.101_2 \times 2^2$ is stored as

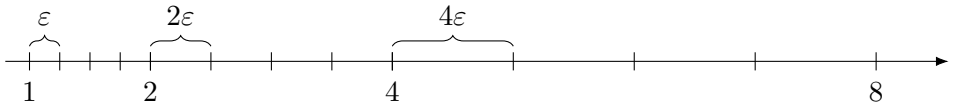


that is, `0xC0D00000`: the sign is $(-1)^1$, the exponent is $129 - 127 = 2$, and the significand is $1.101_2 = 1 + \frac{1}{2} + \frac{1}{8} = 1.625$, so the value is $-1.625 \times 2^2 = -6.5$.

The only difference from the actual IEEE format is the exponent. The formula writes it as $\mathrm{Int}(e)$ for simplicity, while IEEE stores it with a bias, as $E - 127$ (or $E - 1023$ for Float64), and reserves the all-zeros and all-ones exponent fields for 0, subnormals, ∞, and `NaN` (Section C.1).

1.4 The Geometry of Floating-Point Numbers

Plotting all floating-point numbers on the real line shows that they are *not equally spaced*: within each *binade* $[2^E, 2^{E+1})$ they are equally spaced, but crossing a power of two doubles the spacing: if the spacing near 1 is ε, it is 2ε near 2 and 4ε near 4.



Why? Within one binade the exponent E is fixed, and only the significand $1.M$ changes.

- In $[1, 2)$: $1.M$ has $b_p - 1$ fraction bits, so it takes 2^{b_p-1} equally spaced values, with spacing $\varepsilon := 2^{-(b_p-1)}$.
- In $[2^E, 2^{E+1})$: the same values times 2^E, so the spacing is $2^E\varepsilon$.

Every binade holds the same number of floating-point numbers, but each is twice as long as the previous one. Hence the spacing doubles.

So the absolute spacing grows with the numbers, but the relative spacing (spacing divided by the number) always stays between $\varepsilon/2$ and ε. In other words, floating point gives every number about the same *relative* accuracy. This suits real data, whose uncertainty is usually proportional to its size.

Because every result is rounded to this grid, the usual rules of arithmetic can fail.

Example 1 (Floating-Point Addition Is Not Associative)**.** Suppose ties are rounded up, and evaluate $(1 + \varepsilon) + 1 + 2$ in two ways.

- Left to right: $(1 + \varepsilon) + 1 = 2 + \varepsilon$ lies exactly halfway between 2 and $2 + 2\varepsilon$ and rounds up to $2 + 2\varepsilon$. Adding 2 gives $4 + 2\varepsilon$, halfway between 4 and $4 + 4\varepsilon$, which rounds up to $4 + 4\varepsilon$.
- Last two terms first: $1 + 2 = 3$ exactly, and $(1 + \varepsilon) + 3 = 4 + \varepsilon$ rounds down to 4.

The same expression yields 4 or $4 + 4\varepsilon$. Multiplication is not associative either, and the distributive law fails as well; commutativity of addition and of multiplication does hold.

1.5 Cancellation

Relative accuracy has one weak point: subtracting two nearly equal numbers can destroy it.

Example 2 (Cancellation Exposes Existing Error)**.** Suppose the stored value of $10^6 + 1$ is already off by a relative error ε, so it is $(10^6 + 1)(1 + \varepsilon)$. Subtract 10^6:

$$(10^6 + 1)(1 + \varepsilon) - 10^6 \approx 1 + 10^6\varepsilon. \tag{1.8}$$

The exact answer is 1, but we get $1 + 10^6\varepsilon$.

- Before the subtraction: the error is about $10^6\varepsilon$, on a number of size 10^6. Relative error: ε.
- After the subtraction: the error is still $10^6\varepsilon$, but now on a number of size 1. Relative error: $10^6\varepsilon$.

The error itself did not grow. The result got smaller, so the same error is now a million times larger relative to it.

1.6 Conditioning

How do errors *propagate*? Consider computing $y = f(x)$ for $f : \mathbb{R} \to \mathbb{R}$. Since the input is rounded before anything else happens, the best we can hope for is $\hat{y} = f(\hat{x})$ with $|\hat{x} - x| / |x| < \varepsilon$. Comparing relative errors,

$$\frac{|\hat{y} - y| / |y|}{|\hat{x} - x| / |x|} = \frac{|f(\hat{x}) - f(x)|}{|\hat{x} - x|} \cdot \frac{|x|}{|f(x)|} \xrightarrow[\varepsilon \to 0]{} \frac{|x|\,|f'(x)|}{|f(x)|}. \tag{1.9}$$

Definition 3 (Condition Number)

The (relative) *condition number* of f at x is

$$\kappa_f(x) := \frac{|x|\,|f'(x)|}{|f(x)|}. \tag{1.10}$$

The problem is *well-conditioned* at x if $\kappa_f(x) \approx 1$ and *ill-conditioned* if $\kappa_f(x) \gg 1$.

The condition number depends only on the problem and the input, not on the algorithm. This is the most important point here: no algorithm can rescue an ill-conditioned problem, because the difficulty is intrinsic.

Example 4 (Conditioning of Subtraction)**.** Let $f(x) = x - 10^6$ at $x = 10^6 + 1$. Then

$$\kappa_f(10^6 + 1) = \frac{|10^6 + 1| \cdot 1}{1} \approx 10^6 \gg 1. \tag{1.11}$$

This is Example 2 seen as a problem: cancellation is a conditioning phenomenon, not a defect of floating point. Even an algorithm with infinite precision turns a relative uncertainty of 10^{-16} in x into a relative uncertainty of 10^{-10} in the output.

1.7 Stability

Conditioning describes the problem; stability describes the algorithm. We write $\hat{f}$ for the algorithm as it actually runs on the computer: it starts from the stored input $\hat{x} = \mathrm{fl}(x)$ and rounds after every operation.

Definition 5 (Stability)

An algorithm $\hat{f}$ for f is *stable* if

$$\frac{\left|\hat{f}(\hat{x}) - y\right| / |y|}{|\hat{x} - x| / |x|} \lesssim \kappa_f(x). \tag{1.12}$$

The left-hand side is the relative error of the output divided by the relative error of the input. In words: the algorithm amplifies input errors no more than the problem itself does. Otherwise $\hat{f}$ is *unstable*.

The form used in practice. The input error comes from rounding, so it is at most the unit roundoff $u = \varepsilon/2$ (Definition 117), and it is 0 if x is already a floating-point number, which would make the ratio undefined. Replacing the denominator by u gives

$$\frac{\left|\hat{f}(\hat{x}) - y\right|}{|y|} \lesssim \kappa_f(x)\, u. \tag{1.13}$$

The right-hand side is the error that *any* algorithm must make: rounding the input costs u, and the problem amplifies it by $\kappa_f(x)$. A stable algorithm adds nothing beyond that. (Here $\lesssim$ means "at most a modest constant times".)

Example 6 (An Unstable Algorithm for the Identity)**.** Let $f(x) = x$, the identity, with $\kappa_f = 1$, and compute it as $\hat{f}(x) = (x + 10^6) - 10^6$. Mathematically $\hat{f} = f$. Take x to be a floating-point number, so $\hat{x} = x$ and $y = x$.

- Step 1: $x + 10^6$ is rounded, $\mathrm{fl}(x + 10^6) = (x + 10^6)(1 + \delta)$ with $|\delta| \le u$. The low-order bits of x are lost here.
- Step 2: subtracting 10^6 is exact (Lemma 7).

Hence

$$\hat{f}(x) = x + (x + 10^6)\delta, \qquad \frac{\left|\hat{f}(x) - x\right|}{|x|} = \frac{|x + 10^6|}{|x|}\,|\delta| \approx \frac{10^6}{|x|}\,|\delta|\,. \tag{1.14}$$

For $x \approx 1$ the error is about $10^6 u$, but a stable algorithm must reach $\kappa_f u = u$. So $\hat{f}$ is unstable, by a factor of about 10^6; for instance, $\hat{f}(0.1) = 0.09999999997671694$. ($x = 1$ is an exception: $10^6 + 1$ is a floating-point number, so $\delta = 0$.)

Why it fails. Look at the two steps separately: $\hat{f} = g_2 \circ g_1$ with $g_1(x) = x + 10^6$ and $g_2(z) = z - 10^6$. For $x \approx 1$ and $z = x + 10^6$, their condition numbers (Definition 3) are

$$\kappa_{g_1}(x) = \frac{|x|}{|x + 10^6|} \approx 10^{-6}, \qquad \kappa_{g_2}(z) = \frac{|z|}{|z - 10^6|} \approx 10^6. \tag{1.15}$$

Step 1 shrinks relative errors by a factor 10^6; step 2 is a cancellation and magnifies them by 10^6. What happens to an error depends on where it enters:

- An error already present in x passes through both steps: it is shrunk, then magnified, so overall it is unchanged ($\kappa_f = 1$).
- The rounding error δ of step 1 enters *after* the shrinking step. It passes only through step 2, so it is magnified by 10^6.

Lesson. A well-conditioned problem can still be solved by an unstable algorithm if one of its intermediate steps is ill-conditioned. We will see this again with Gram–Schmidt and the normal equations.

Unstable vs. ill-conditioned. These are different diagnoses with different fixes.

- *Unstable*: the algorithm is at fault. Use a different algorithm.
- *Ill-conditioned*: the problem itself is sensitive, and no algorithm can help. Change the problem (reformulate or regularize it), or use higher precision.

The IEEE formats, the two conventions for machine epsilon, the standard model of floating-point arithmetic, and the notion of backward stability are collected in Appendix C.

1.8 (SUPPLEMENT) Further Topics

1.8.1 Rewriting Formulas to Avoid Cancellation

The recipe: find the step that subtracts two nearly equal numbers, and use an algebraic identity to rewrite the formula without it. Mathematically nothing changes; numerically the error drops to $O(u)$.

Roots of a quadratic. The roots of $ax^2 + bx + c$ are $x_{1,2} = (-b \pm \sqrt{b^2 - 4ac})/(2a)$. Suppose $b > 0$ and $b^2 \gg 4ac$. Then $\sqrt{b^2 - 4ac} \approx b$, so the "+" root computes $-b + (\text{almost } b)$: a cancellation. The "−" root adds two numbers of the same sign and is accurate. So compute that one first and get the other from Vieta's relation $x_1 x_2 = c/a$:

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}, \qquad x_1 = \frac{c}{a\, x_2} = \frac{2c}{-b - \sqrt{b^2 - 4ac}}. \tag{1.16}$$

(If $b < 0$, swap the roles of the two roots.)

Variance. The formula $\mathrm{Var} = \overline{x^2} - \bar{x}^2$ (mean of squares minus square of mean) subtracts two large, nearly equal numbers when the data have a large mean and a small spread. For the data $10^8 + 1$, $10^8 + 2$, $10^8 + 3$ it returns 0 in Float64, while the true variance is $2/3$; it can even return a negative number. The fixes avoid squaring the large mean:

- *Two passes*: first compute $\bar{x}$, then $\mathrm{Var} = \frac{1}{n}\sum_i (x_i - \bar{x})^2$. The differences $x_i - \bar{x}$ are small, and all terms are nonnegative.
- *Welford's recurrence*: one pass over the data, updating the mean and the sum of squared deviations as each x_k arrives,

$$\bar{x}_k = \bar{x}_{k-1} + \frac{x_k - \bar{x}_{k-1}}{k}, \qquad M_k = M_{k-1} + (x_k - \bar{x}_{k-1})(x_k - \bar{x}_k), \tag{1.17}$$

  and finally $\mathrm{Var} = M_n/n$.

Three standard cases.

- $1 - \cos x$ for small x: for $x = 10^{-8}$, $\cos x$ rounds to exactly 1, giving 0 instead of $5 \cdot 10^{-17}$. Use $1 - \cos x = 2\sin^2(x/2)$.
- $\log(1 + x)$ for small x: already $1 + x$ loses the low bits of x (as in Example 6); for $x = 10^{-17}$ the result is 0. `log1p(x)` computes $\log(1 + x)$ directly from x (likewise `expm1(x)` for $e^x - 1$).
- $\sqrt{x+1} - \sqrt{x}$ for large x: for $x = 10^{16}$ the result is 0 instead of $5 \cdot 10^{-9}$. Rationalize: $\sqrt{x+1} - \sqrt{x} = 1/(\sqrt{x+1} + \sqrt{x})$.

1.8.2 Sterbenz's Lemma: Close Numbers Subtract Exactly

Lemma 7 (Sterbenz)

If x, y are floating-point numbers with $y/2 \le x \le 2y$, then $x - y$ is a floating-point number, so $\mathrm{fl}(x - y) = x - y$ exactly.

Intuition. Think of decimal numbers with three significant digits. $1.23 - 1.21 = 0.02$: both inputs lie on the grid of spacing 0.01, so their difference does too, and it is *smaller* than the inputs, so it needs fewer digits, not more. Nothing has to be rounded.

Proof. Say $y \le x \le 2y$, and let 2^e be the spacing of floating-point numbers near y, so y is an integer multiple of 2^e with at most b_p bits. The spacing near $x \ge y$ is 2^e or a larger power of 2, so x is an integer multiple of 2^e as well. Hence $x - y = k\,2^e$ with an integer $k \ge 0$. From $x \le 2y$ we get $x - y \le y < 2^{b_p} 2^e$, so $k < 2^{b_p}$ and $x - y$ fits in b_p bits. □

Consequence: the subtraction is not to blame. In Example 2 the subtraction itself is exact by the lemma: all of the error was already in the input, and the subtraction only made it visible. Rule: if the inputs are inexact, do not compute a small result by subtracting two large numbers.

1.8.3 Useful Techniques

Kahan summation. Adding n numbers one by one can make an error of order $nu \sum_i |x_i|$: adding a small term to a large running sum rounds away the term's low bits. Kahan's compensated summation keeps those lost bits in a variable c and feeds them back:

```
s = 0.0; c = 0.0
for x in xs
    y = x - c        # add back what was lost last time
    t = s + y        # rounded sum: low bits of y are lost
    c = (t - s) - y  # = (what was really added) - y = the lost part
    s = t
end
```

This reduces the error to about $2u \sum_i |x_i|$, essentially independent of n [7], at about four times the cost. Example: adding 0.1 ten million times gives an error of $1.6 \cdot 10^{-4}$ naively, 0 (to the printed digits) with Kahan, and $1.4 \cdot 10^{-9}$ with Julia's `sum`, which adds *pairwise* in a binary tree so that its error grows only like $u \log_2 n$.

Fused multiply-add (FMA). $\mathrm{fma}(a, b, c)$ computes $ab + c$ with a single rounding instead of two: faster and more accurate. Since a compiler may or may not use it, the same code can give slightly different results.

`-ffast-math`. This compiler flag (Julia: `@fastmath`) lets the compiler treat floating point as associative, so it may reorder sums (e.g., to vectorize them). This changes results and can break Kahan summation, whose $c = (t - s) - y$ is algebraically 0 and may be deleted. Keep it off.

1.8.4 Low-Precision Formats

Two 16-bit formats split their bits differently. Float16 has 5 exponent bits and $b_p = 11$: $\varepsilon \approx 10^{-3}$, but the largest number is only 65504. BFloat16 has 8 exponent bits, hence the same range as Float32 (up to $3.4 \cdot 10^{38}$), but only $b_p = 8$: $\varepsilon \approx 8 \cdot 10^{-3}$, about two decimal digits. Deep learning prefers BFloat16: gradients span many orders of magnitude, so what fails first is *range* (overflow, underflow), not precision; Float16 training needs *loss scaling* (multiplying the loss by a constant), BFloat16 does not. Sums are still accumulated in Float32: by Proposition 119, a dot product of length n has error up to about $nu\ |\vec{x}|^\top |\vec{y}|$. For $n = 4096$, BFloat16 ($u = 2^{-8}$) gives $nu = 16 \ge 1$, so the bound guarantees nothing, while Float32 ($u = 2^{-24}$) gives $nu \approx 2.4 \cdot 10^{-4}$. Hence tensor cores take BFloat16 (or FP8) inputs but accumulate in Float32.

Lecture 2

Cache Hierarchy and Tiling/Blocking, Solving Linear Systems

Supplementary reading:

- [2] Sections 2.3 and 2.6.
- [1] Lectures 17 and 20.

2.1 Flops Are Not the Whole Story

In Lecture 1 we asked whether we compute the *right* answer (conditioning: the problem's fundamental limit; stability: does the algorithm do something stupid?). Now we ask how *fast* we compute it.

Algorithms and computational cost. Consider the algorithm for matrix multiplication

$$A \mathrel{+}= BC, \qquad A \in \mathbb{R}^{\ell \times n},\ B \in \mathbb{R}^{\ell \times m},\ C \in \mathbb{R}^{m \times n}: \tag{2.1}$$

```
function matmuladd!(A, B, C)          # A += B*C
    for i = 1:l
        for j = 1:m
            for k = 1:n
                A[i,k] += B[i,j] * C[j,k]
            end
        end
    end
end
```

The innermost line runs $\ell \times m \times n$ times, so the computational cost is $\ell \times m \times n$. We set $\ell = 2 \cdot 10^4$, $m = 3 \cdot 10^4$ and time it with `@btime`:

n	Cost ℓmn	Time
1	$6 \cdot 10^8$	$6 \cdot 10^{-2}$ s
10^3	$6 \cdot 10^{11}$	4 s

The cost grows by a factor of 1000, so we would expect 1000×0.06 s $= 60$ s. It takes only 4 s: *more than* 10 *times faster than expected*. (The timings are for Julia's built-in multiplication, which calls BLAS.)

Flops and memory access. Does Julia use fewer operations? No: for $\ell = m = n$, the time grows roughly like the cost,

$\ell = m = n$	Cost ℓmn	Time
10^3	10^9	$7 \cdot 10^{-3}$ s
10^4	10^{12}	6 s

that is, about 1000 times the cost in about 1000 times the time. The explanation is that four operations appear in the algorithm: additions, multiplications, and reading and writing entries of the arrays (both written $[\,]$). Their costs are very different:

$$\text{cost}(+) < \text{cost}(*) \ll \text{cost}([\,]). \tag{2.2}$$

(On modern CPUs, $+$ and $*$ in fact cost about the same.) *Memory access is much slower than flops on modern hardware.*

- For $n = 1$, every entry of B is read and then used only once, so almost all of the time goes into memory access.
- For $n = 10^3$, every entry of B is used 10^3 times, so the cost of reading it is shared by many operations.

2.2 Memory Is One-Dimensional

Memory is a long string of bits, and a vector is a contiguous run of floating-point numbers $u_1, u_2, u_3, \ldots$. There is no such thing as a two-dimensional matrix in memory; it must be flattened in some order. Julia, Fortran, BLAS, and LAPACK use *column-major* order, storing entire columns contiguously:

$$A_{11},\ A_{21},\ A_{31},\ A_{41},\ A_{12},\ A_{22},\ A_{32},\ A_{42},\ \ldots \tag{2.3}$$

so that A_{ij} sits at linear index $i + (j-1)m$. C and NumPy default to row-major order. Besides the data, one also stores metadata such as sizes, strides, and preallocated buffers.

Since the storage is just a contiguous block, the same memory can be *reinterpreted* as an array of shape $m \times n \times k \times l$, $mn \times kl$, or $mnk \times l$ without any copying. Reshaping is free; transposing is not.

2.3 The Memory Hierarchy

Main memory is much slower than arithmetic, so hardware adds *caches*: small, fast memory close to the processor. Two bottlenecks remain:

- *Latency*: how long one access waits (a few cycles for L1, a few hundred for main memory).
- *Bandwidth*: how many bytes can be moved per second.

Latency is hidden by *prefetching*: the hardware fetches nearby memory, or memory in a strided pattern, before it is needed. This only works for regular access patterns, so the *regularity* of memory access often matters more than its volume. GPUs instead hide latency by running many threads. When threads access contiguous elements, the hardware merges their accesses into one, so on a GPU non-contiguous access is very slow.

2.4 Loop Reordering

Example 8 (Two Loop Orders for a Matrix–Vector Product)**.** Both loops compute $\vec{u} = A\vec{v}$ with the same number of operations.

```
# i outer: inner loop along a row of A
for i = 1:m
    for j = 1:n
        u[i] += A[i,j] * v[j]
    end
end
```

```
# j outer: inner loop down a column
for j = 1:n
    for i = 1:m
        u[i] += A[i,j] * v[j]
    end
end
```

In column-major storage, the left loop reads A along a row, in steps of m; in the right loop, every access is contiguous.

The left loop computes inner products of the rows of A with $\vec{v}$ (`dot`); the right loop adds up the columns of A times v_j (`axpy`). Julia stores matrices by columns, so the right loop is faster. The gap grows with the size of A: once A no longer fits in the cache, each strided access uses only one number from every cache line it loads. (The board says the order matters most for small matrices; we measured the opposite: the left loop is $3\times$ slower at $m = 64$ and $20\times$ at $m = 4096$.)

2.5 Tiling

Even without cache misses, the limited bandwidth slows us down. The key observation: multiplying two $m \times m$ matrices takes about m^3 operations but involves only about m^2 numbers, so each number is used about m times.

- If all m^2 numbers fit into the cache, we load each of them from main memory only once, and then do all m^3 operations in the fast cache.
- If the matrices are too large, we *tile*: split A, B, C into $b \times b$ blocks, $N = m/b$ along each side, and multiply block by block. (For simplicity, assume b divides m; otherwise the blocks at the edges are smaller.) We choose b so that the three blocks we are working on fit into the cache; Section 2.9.2 quantifies the gain.

Written out entry by entry, the tiled algorithm has *six* nested loops: the outer three choose the blocks (I, J, K count blocks, not entries), and the inner three multiply them.

```
for I = 1:N, J = 1:N, K = 1:N              # which blocks: C_IJ += A_IK * B_KJ
    for i = (I-1)*b+1 : I*b                # multiply the two b x b blocks
        for j = (J-1)*b+1 : J*b
            for k = (K-1)*b+1 : K*b
                C[i,j] += A[i,k] * B[k,j]
            end
        end
    end
end
```

We do the same m^3 operations as before, just in a different order. The inner three loops only use three blocks, $3b^2$ numbers in total. They stay in the cache and are each used about b times.

2.6 Linear Systems

Problem. Given $A \in \mathbb{K}^{m\times n}$ and $\vec{b} \in \mathcal{R}(A) \subseteq \mathbb{K}^m$, find $\vec{x} \in \mathbb{K}^n$ with

$$A\vec{x} = \vec{b}. \tag{2.4}$$

The condition $\vec{b} \in \mathcal{R}(A)$ just says that a solution exists.

- Linear systems appear in virtually all of mathematics.
- They *can* be solved by computing A^{-1}, but they *need not* be. Computing A^{-1} is in fact the wrong way: it costs more ($2m^3$ flops for a square matrix, versus $\frac{2}{3}m^3$ for the LU factorization of Lecture 3), and it usually gives a less accurate answer.

Square matrices. We begin with $A \in \mathbb{K}^{m\times m}$. Then the following three conditions are equivalent:

$$\underbrace{\mathcal{N}(A) = \{\vec{0}\}}_{\vec{x}\mapsto A\vec{x}\text{ injective}} \iff \text{the columns of } A \text{ are linearly independent} \iff \underbrace{\mathcal{R}(A) = \mathbb{K}^m}_{\vec{x}\mapsto A\vec{x}\text{ surjective}} \tag{2.5}$$

In words:

- *Left*: only $\vec{x} = \vec{0}$ is mapped to $\vec{0}$. Then different inputs give different outputs: if $A\vec{x} = A\vec{y}$, then $A(\vec{x} - \vec{y}) = \vec{0}$, so $\vec{x} = \vec{y}$. Hence a solution, if it exists, is unique.
- *Right*: every $\vec{b} \in \mathbb{K}^m$ is of the form $A\vec{x}$. Hence a solution always exists.

If these conditions hold, we say that A *has full rank*, *is invertible*, or *is nonsingular*; then $A\vec{x} = \vec{b}$ has exactly one solution for every $\vec{b}$. For rectangular matrices, injective and surjective are no longer the same; they are treated in Lecture 5.

2.7 Triangular Systems

We begin with the easy case. The small example

$$\begin{aligned} x_1 + 5 &= 15 \\ x_1 + 5x_2 &= 20 \end{aligned} \qquad \Longrightarrow \qquad x_1 = 10,\ x_2 = 2 \tag{2.6}$$

already shows the whole idea: x_1 does not depend on x_2, so the triangular structure *decouples* the unknowns and they can be solved for one at a time. For lower triangular A, *forward substitution* computes

$$x_1 = \frac{b_1}{A_{11}}, \qquad x_i = \frac{1}{A_{ii}}\left(b_i - \sum_{j=1}^{i-1} A_{ij}x_j\right), \quad i = 2, \dots, m. \tag{2.7}$$

Upper triangular systems are solved in reverse order by *back substitution*. The same formula admits two implementations, both costing $\sim m^2$ flops but with different access patterns.

Algorithm 1 Forward substitution, row-oriented (`dot` form).

1: **for** $i = 1, \dots, m$ **do**
2: $\quad x_i \leftarrow b_i$
3: $\quad$ **for** $j = 1, \dots, i-1$ **do**
4: $\quad\quad x_i \leftarrow x_i - A_{ij}x_j$
5: $\quad x_i \leftarrow x_i / A_{ii}$

Algorithm 2 Forward substitution, column-oriented (`axpy` form), the efficient variant.

$\vec{x} \leftarrow \vec{b}$
for $j = 1, \dots, m$ **do**
    $x_j \leftarrow x_j / A_{jj}$
    for $i = j+1, \dots, m$ **do**
        $x_i \leftarrow x_i - A_{ij} x_j$ ▷ remove the contribution of x_j at once

For upper triangular A, x_m is the only unknown in the last equation, so back substitution starts from the bottom:

$$x_m = \frac{b_m}{A_{mm}}, \qquad x_i = \frac{1}{A_{ii}} \left(b_i - \sum_{j=i+1}^{m} A_{ij} x_j \right), \quad i = m-1, \dots, 1. \tag{2.8}$$

The two implementations mirror those of forward substitution, with every loop over the unknowns reversed.

Algorithm 3 Back substitution, row-oriented (`dot` form).

for $i = m, \dots, 1$ **do**
    $x_i \leftarrow b_i$
    for $j = i+1, \dots, m$ **do**
        $x_i \leftarrow x_i - A_{ij} x_j$
    $x_i \leftarrow x_i / A_{ii}$

Algorithm 4 Back substitution, column-oriented (`axpy` form).

$\vec{x} \leftarrow \vec{b}$
for $j = m, \dots, 1$ **do**
    $x_j \leftarrow x_j / A_{jj}$
    for $i = 1, \dots, j-1$ **do**
        $x_i \leftarrow x_i - A_{ij} x_j$ ▷ remove the contribution of x_j at once

In column-major storage the row forms walk along rows (strided) and the column forms walk down columns (contiguous); this is the matrix–vector discussion again.

2.8 Gaussian Elimination

A general matrix is not triangular. Gaussian elimination *transforms A into upper triangular form by a sequence of row operations*, reducing the general problem to a triangular one. A 2×2 example:

$$\begin{aligned} x_1 + 2x_2 &= 10 \\ 2x_1 + x_2 &= 8 \end{aligned} \xrightarrow{\text{row } 2 - 2\times\text{row } 1} \begin{aligned} x_1 + 2x_2 &= 10 \\ -3x_2 &= -12 \end{aligned} \quad \Longrightarrow \quad x_2 = 4,\ x_1 = 2. \tag{2.9}$$

In matrix language, this elimination step is a *left multiplication*:

$$\begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} 10 \\ 8 \end{bmatrix}. \tag{2.10}$$

Theorem 9 (Elementary Lower Triangular Matrices)

Let $A \in \mathbb{K}^{m\times m}$ and suppose the first $k-1$ columns of A are already zero below the diagonal, and $A_{kk} \neq 0$. Define the multipliers $\ell_{ik} := A_{ik}/A_{kk}$ for $i = k+1, \dots, m$, the vector

$$\vec{\ell}_k := \begin{bmatrix} 0 & \cdots & 0 & \ell_{k+1,k} & \cdots & \ell_{mk} \end{bmatrix}^\top, \tag{2.11}$$

whose first k entries vanish, and the *elementary lower triangular matrix*

$$L_k := I - \vec{\ell}_k \vec{e}_k^* = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & -\ell_{k+1,k} & 1 & & \\ & & \vdots & & \ddots & \\ & & -\ell_{mk} & & & 1 \end{bmatrix} \leftarrow \text{row } k \tag{2.12}$$

Then $L_k A$ is zero below the diagonal in its first k columns. Its first k rows equal those of A; besides zeroing column k below the diagonal, only the trailing $(m-k)\times(m-k)$ block is updated.

Proof. Left multiplication by L_k performs, for each $i > k$, the row operation "row i minus ℓ_{ik} times row k". The (i,k) entry becomes $A_{ik} - \ell_{ik}A_{kk} = 0$. Row k is zero in its first $k-1$ columns, so the columns that have already been eliminated are not disturbed. □

Applying Theorem 9 for $k = 1, \dots, m-1$ (assuming that each A_{kk} is nonzero when we reach it) to both sides of $A\vec{x} = \vec{b}$ gives

$$\underbrace{L_{m-1}\cdots L_1 A}_{\text{upper triangular}}\, \vec{x} = L_{m-1}\cdots L_1 \vec{b}. \tag{2.13}$$

With $U := L_{m-1}\cdots L_1 A$ and $L := (L_{m-1}\cdots L_1)^{-1}$, we obtain $A = LU$ with U upper triangular. We stop here with two questions: *is L lower triangular, and can we compute it?* Both are answered in Lecture 3.

2.9 (SUPPLEMENT) Further Topics

2.9.1 Leading Dimension

In column-major order, the entry A_{ij} lives at

$$\mathrm{addr}(A_{ij}) = \text{base} + (j-1)\cdot \mathrm{lda} + (i-1), \tag{2.14}$$

where the *leading dimension* $\mathrm{lda} \geq m$ is the distance between consecutive columns. Allowing $\mathrm{lda} \neq m$ lets us describe a *submatrix* of a larger matrix by the quadruple (start address, m, n, lda) without copying any data. This is what Julia's `view` and `@views` do, whereas `A[1:100, 1:100]` makes a copy.

2.9.2 Computational Intensity

We can quantify this [2, §2.6]. Consider two levels of memory, a fast memory of size M and a slow memory. Let one flop take time t_f and one slow-memory access take time $t_m \gg t_f$.

Definition 10 (Computational Intensity)

If an algorithm performs f flops and m slow-memory accesses, its *computational intensity* is $q \coloneqq f/m$.

The total time is roughly

$$T = ft_f + mt_m = ft_f \left(1 + \frac{t_m}{t_f} \cdot \frac{1}{q}\right). \tag{2.15}$$

Running near the peak rate ft_f requires $q \gg t_m/t_f$, and on modern machines t_m/t_f is in the tens to hundreds. The three levels of the BLAS are distinguished precisely by q (here n is the vector length or matrix order):

Level	Typical operation	Flops f	Accesses m	q
Level 1	`axpy` $\vec{y} \leftarrow \alpha\vec{x} + \vec{y}$	$2n$	$3n$	$2/3$
Level 2	`gemv` $\vec{y} \leftarrow \alpha A\vec{x} + \beta\vec{y}$	$2n^2$	n^2	2
Level 3	`gemm` $C \leftarrow \alpha AB + \beta C$	$2n^3$	$4n^2$	$n/2$

Only Level 3 has an intensity that grows with the problem size, matching the observation that matrix multiplication performs m^3 multiply–adds on m^2 data. A `dot` has $q \approx 1$, and a triangular solve (n^2 flops on about $n^2/2$ entries) has $q \approx 2$; both are limited by bandwidth.

For naive $C \mathrel{+}= AB$ with $n \times n$ matrices, B is read n times (n^3 accesses) while A and C cost $3n^2$, so

$$q = \frac{2n^3}{n^3 + 3n^2} \approx 2, \tag{2.16}$$

no better than Level 2. For the tiled version (Section 2.5) the slow-memory traffic is $m = 2n^2 + 2Nn^2$, so

$$q \approx \frac{2n^3}{2Nn^2} = \frac{n}{N} = b. \tag{2.17}$$

Three blocks must fit in fast memory, so $3b^2 \le M$ and $q \approx b \le \sqrt{M/3}$. A lower bound of Hong and Kung shows that any matrix multiplication that only reorders the sums has $q = O(\sqrt{M})$, so tiling is asymptotically optimal. Real implementations tile at *several* levels (registers, L1, L2, and across nodes), each with its own b.

2.9.3 Blocked LU

The in-place LU algorithm of Lecture 3 spends its time in rank-one updates $A \leftarrow A - \vec{x}\vec{y}^*$. These read and write all of A for 2 flops per entry ($q \approx 1$), so naive Gaussian elimination is bandwidth-bound. The tiling idea fixes this [2, §2.6.3]. Choose a block width b and write

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad A_{11} \in \mathbb{K}^{b \times b}. \tag{2.18}$$

1. Factor the $m \times b$ panel $\begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix}$ by ordinary elimination, obtaining L_{11}, L_{21}, U_{11} (Level 2, but only an $O(b/m)$ fraction of the work).
2. Compute $U_{12} = L_{11}^{-1} A_{12}$, a triangular solve with many right-hand sides (Level 3).
3. Compute $\tilde{A}_{22} = A_{22} - L_{21}U_{12}$, a matrix multiplication (Level 3) that contains most of the flops.
4. Recurse on $\tilde{A}_{22}$.

Almost all the work is matrix multiplication with $q \approx b$. This is how LAPACK's `getrf` beats the older LINPACK: rewrite everything as matrix multiplication.

Lecture 3

Solving Linear Systems and Stability

Supplementary reading:

- [1] Lectures 20 and 21.
- [2] Section 2.3.

3.1 Recap: From Triangular Systems to Elimination

We begin with a recap of Lecture 2: matrices are stored column by column, memory access costs far more than arithmetic, and the loop order and tiling of an algorithm should match the storage. We then returned to linear systems. Triangular systems are solved by forward or back substitution in $O(m^2)$ flops, and Gaussian elimination reduces a general system to a triangular one: by Theorem 9, applying elementary lower triangular matrices $L_k = I - \vec{\ell}_k \vec{e}_k^*$ for $k = 1, \ldots, m-1$ gives

$$U \coloneqq L_{m-1} \cdots L_1 A \text{ upper triangular}, \qquad L \coloneqq (L_{m-1} \cdots L_1)^{-1}, \qquad A = LU. \tag{3.1}$$

This lecture answers the two questions left open: *is L lower triangular, and can we compute it cheaply?* Answering them requires a little algebra of lower triangular matrices.

3.2 The Algebra of Lower Triangular Matrices

Proposition 11 (Closure Properties)

The lower triangular matrices in $\mathbb{K}^{m \times m}$ are closed under

1. addition,
2. scalar multiplication,
3. matrix multiplication, and
4. inversion (of invertible ones).

Proof. Items 1 and 2 are clear. For item 3, let L and $\tilde{L}$ be lower triangular, so $(L\tilde{L})_{ij} = \sum_k L_{ik}\tilde{L}_{kj}$. A term $L_{ik}\tilde{L}_{kj}$ can be nonzero only if $i \geq k$ and $k \geq j$, hence $i \geq j$. So every term vanishes when $i < j$, and $L\tilde{L}$ is

lower triangular. Item 4 is proved below for the matrices arising in LU, using Theorem 12; the general case is in Section 3.8. □

Theorem 12 (Inverses and Products of Elementary Lower Triangular Matrices)

Let $L_k = I - \vec{\ell}_k \vec{e}_k^*$ be as in Theorem 9. Then:

1. $L_k^{-1} = I + \vec{\ell}_k \vec{e}_k^*$, i.e., inversion just *flips the signs* of the multipliers;
2. the product of the inverses, *in this order*, places every multiplier in its own position:

$$L_1^{-1} L_2^{-1} \cdots L_{m-1}^{-1} = \begin{bmatrix} 1 & & & \\ \ell_{21} & 1 & & \\ \vdots & \ddots & \ddots & \\ \ell_{m1} & \cdots & \ell_{m,m-1} & 1 \end{bmatrix}. \tag{3.2}$$

Trefethen and Bau call these two facts the "two strokes of luck" of Gaussian elimination [1, Lecture 20]. The order in item 2 matters. In the reverse order the multipliers mix: for example,

$$L_2^{-1} L_1^{-1} = (I + \vec{\ell}_2 \vec{e}_2^*)(I + \vec{\ell}_1 \vec{e}_1^*) = I + \vec{\ell}_1 \vec{e}_1^* + \vec{\ell}_2 \vec{e}_2^* + \ell_{21}\, \vec{\ell}_2 \vec{e}_1^*, \tag{3.3}$$

and the last term adds $\ell_{21}\ell_{i2}$ to the entries of column 1 below row 2.

Both items rest on one simple fact: $\vec{\ell}_k$ is zero in its first k entries. The number $\vec{e}_p^* \vec{\ell}_k$ is entry p of $\vec{\ell}_k$, so it is 0 for every $p \le k$. This is what makes the extra terms vanish in the proof below.

Proof. For item 1, since $\vec{e}_k^* \vec{\ell}_k = 0$,

$$(I + \vec{\ell}_k \vec{e}_k^*)(I - \vec{\ell}_k \vec{e}_k^*) = I + \vec{\ell}_k \vec{e}_k^* - \vec{\ell}_k \vec{e}_k^* - \vec{\ell}_k (\vec{e}_k^* \vec{\ell}_k) \vec{e}_k^* = I. \tag{3.4}$$

For item 2, we compute the product column by column. Column j of $M := L_1^{-1} \cdots L_{m-1}^{-1}$ is

$$M \vec{e}_j = (I + \vec{\ell}_1 \vec{e}_1^*) \cdots (I + \vec{\ell}_{m-1} \vec{e}_{m-1}^*)\, \vec{e}_j. \tag{3.5}$$

Here j is fixed: it is the column we compute. The factors are $L_p^{-1} = I + \vec{\ell}_p \vec{e}_p^*$ for $p = 1, \ldots, m-1$. We apply them to $\vec{e}_j$ from right to left, using $(I + \vec{\ell}_p \vec{e}_p^*)\vec{x} = \vec{x} + \vec{\ell}_p\, (\vec{e}_p^* \vec{x})$, where $\vec{e}_p^* \vec{x}$ is entry p of $\vec{x}$.

- *Factors with $p > j$*: entry p of $\vec{e}_j$ is 0, so these factors leave $\vec{e}_j$ unchanged.
- *The factor $p = j$*: entry j of $\vec{e}_j$ is 1, so it gives $\vec{e}_j + \vec{\ell}_j$.
- *Factors with $p < j$*: entry p of $\vec{e}_j + \vec{\ell}_j$ is 0, since $\vec{e}_j$ has its 1 in position j and $\vec{\ell}_j$ is zero in its first j entries. So these factors leave $\vec{e}_j + \vec{\ell}_j$ unchanged.

Hence column j of M is

$$M \vec{e}_j = \vec{e}_j + \vec{\ell}_j = \begin{bmatrix} 0 & \cdots & 0 & 1 & \ell_{j+1,j} & \cdots & \ell_{mj} \end{bmatrix}^\top, \tag{3.6}$$

with the 1 in position j. These are exactly the columns of the matrix in item 2. □

3.3 Answering the Two Questions

Now we can answer the two questions from the end of Lecture 2. Since the inverse of a product reverses the order of the factors, $L = (L_{m-1} \cdots L_1)^{-1} = L_1^{-1} L_2^{-1} \cdots L_{m-1}^{-1}$.

- *Is L lower triangular? Yes.* Each L_k^{-1} is lower triangular (item 1 of Theorem 12), and so is their product (item 3 of Proposition 11). Its diagonal is all ones: L is *unit* lower triangular.
- *Can we compute L cheaply? Yes, for free.* By item 2 of Theorem 12, L is the matrix (3.2), whose entries below the diagonal are the multipliers ℓ_{ik} that elimination computes anyway.

3.4 The In-Place Algorithm

Idea: transform A into U with Gaussian elimination while recording the multipliers. Both L and U are stored in place, in the memory of A:

- U goes on and above the diagonal;
- the multipliers ℓ_{ik} go below the diagonal, where elimination has just created zeros;
- the diagonal of L is all ones, so it need not be stored.

Hence we can overwrite A with L and U, without any extra memory.

Algorithm 5 LU factorization without pivoting, in place.

```
for k = 1, ..., m do
    A_{k+1:m, k} ← A_{k+1:m, k} / A_{kk}          ▷ multipliers ℓ_ik, stored in place
    for j = k + 1, ..., m do
        for i = k + 1, ..., m do
            A_ij ← A_ij − A_ik A_kj               ▷ = A_ij − ℓ_ik u_kj
```

When the algorithm finishes, A holds both factors: U on and above the diagonal, and the multipliers of L below it. The inner loop runs over i, i.e., down a column of A, which is the fast direction in column-major storage (Lecture 2).

Vector form. Step k computes the multipliers $\vec{\ell}_k$ and subtracts an outer product from the trailing block:

$$\vec{\ell}_k = A_{k+1:m,\,k} / A_{kk}, \qquad A_{k+1:m,\,k+1:m} \leftarrow A_{k+1:m,\,k+1:m} - \vec{\ell}_k \, A_{k,\,k+1:m}. \tag{3.7}$$

A column vector times a row vector has rank one, so each step is a *rank-one update* of the trailing block.

Block form. Write $A = \begin{bmatrix} a_{11} & \vec{w}^* \\ \vec{v} & B \end{bmatrix}$ with a scalar a_{11}. The first step of elimination is the factorization

$$A = \begin{bmatrix} 1 & 0 \\ \vec{v}/a_{11} & I \end{bmatrix} \begin{bmatrix} a_{11} & \vec{w}^* \\ 0 & S \end{bmatrix}, \qquad S = B - \frac{\vec{v}\vec{w}^*}{a_{11}}. \tag{3.8}$$

S is the *Schur complement* (Lecture 9). The algorithm then factors S in the same way, so LU is this one step applied recursively.

3.5 Zero Pivots

The algorithm divides by A_{kk}, the k-th *pivot*. If a pivot is zero, it cannot proceed.

Example 13 (An Invertible Matrix with No LU Factorization)**.** The matrix

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \tag{3.9}$$

is invertible, but $A_{11} = 0$ is a zero pivot, so the very first step divides by zero and $A = LU$ does not exist. *Invertibility does not imply the existence of an LU factorization.*

Theorem 14 (Zero Pivots and Leading Principal Submatrices)

If the k-th pivot of the LU factorization of A is zero, then the leading principal submatrix $A[1{:}k, 1{:}k]$ is singular. All pivots of the LU factorization are nonzero if and only if every leading principal submatrix of A is nonsingular.

Proof. Step 1: the top-left block. Take the top-left $k \times k$ block of $A = LU$. Entry (i, j) with $i, j \le k$ is $\sum_p L_{ip}U_{pj}$, and $L_{ip} = 0$ for $p > i$, so only $p \le k$ contributes. Hence

$$A[1{:}k, 1{:}k] = L[1{:}k, 1{:}k]\, U[1{:}k, 1{:}k]. \tag{3.10}$$

Step 2: determinants. $L[1{:}k, 1{:}k]$ is unit lower triangular, so its determinant is 1. $U[1{:}k, 1{:}k]$ is upper triangular, so its determinant is the product of its diagonal, i.e., of the first k pivots:

$$\det A[1{:}k, 1{:}k] = u_{11}u_{22}\cdots u_{kk}. \tag{3.11}$$

So $A[1{:}k, 1{:}k]$ is nonsingular if and only if the first k pivots are nonzero.

Step 3: a zero pivot stops the algorithm early. If $u_{kk} = 0$, the algorithm cannot continue past step k. But the first k rows and columns of L and U have already been computed, and Step 1 uses only those. So the argument still applies. □

3.6 Small Pivots Are Also Dangerous

Exactly zero pivots are unlikely in practice. Is it then enough that no pivot is exactly zero? Consider a small number $\varepsilon > 0$, much smaller than the machine epsilon (e.g., $\varepsilon = 10^{-20}$; here ε is *not* the machine epsilon):

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix}, \qquad L = \begin{bmatrix} 1 & 0 \\ \varepsilon^{-1} & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} \varepsilon & 1 \\ 0 & 1 - \varepsilon^{-1} \end{bmatrix}. \tag{3.12}$$

This is mathematically correct. In floating-point arithmetic, however, ε^{-1} is huge and $1 - \varepsilon^{-1}$ rounds to $-\varepsilon^{-1}$. The computed factors then give

$$\tilde{L}\tilde{U} = \begin{bmatrix} \varepsilon & 1 \\ 1 & \varepsilon^{-1} - \varepsilon^{-1} \end{bmatrix} = \begin{bmatrix} \varepsilon & 1 \\ 1 & 0 \end{bmatrix} \approx \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \tag{3.13}$$

which is nothing like A. The small pivot produces a huge multiplier, and rounding wipes out all information in A_{22}.

3.7 Error Analysis of the LU Factorization

Theorem 15 (Backward Error of LU)

Let $\varepsilon_{\text{mach}}$ be the machine epsilon. If the LU factorization runs to completion in floating-point arithmetic without encountering a zero pivot, the computed factors L, U satisfy, to first order in $\varepsilon_{\text{mach}}$,

$$A = LU + E, \qquad |E| \leq \varepsilon_{\text{mach}}\, m\, |L|\, |U|\,, \tag{3.14}$$

where $|\cdot|$ takes entrywise absolute values and the inequality holds entrywise.

The proof is given in Lecture 4. The theorem says that the computed factorization is the exact factorization of a nearby matrix, with a perturbation controlled by $|L|\,|U|$. If $|L|\,|U|$ is comparable to $|A|$, the error is at the level of machine precision. If L or U contains huge entries, the bound is useless. In the example above, L contains ε^{-1} and U contains $1 - \varepsilon^{-1}$, so $|L|\,|U|$ has entries of size ε^{-1} and the error can be as large as A itself.

3.8 (SUPPLEMENT) Further Topics

3.8.1 Lower Triangular Matrices

Inverses of general lower triangular matrices. Above, we proved item 4 of Proposition 11 only for matrices of the form $(L_{m-1} \cdots L_1)^{-1}$. It holds for every invertible lower triangular T:

1. $\det T$ is the product of the diagonal entries, so they are all nonzero. Let $D = \operatorname{diag}(T)$.
2. $N \coloneqq D^{-1}T$ divides row i of T by T_{ii}, so N is *unit* lower triangular.
3. By item 2 of Theorem 12, $N = L_1^{-1} \cdots L_{m-1}^{-1}$ if we take the entries of N below the diagonal as the multipliers. So $N^{-1} = L_{m-1} \cdots L_1$ is a product of lower triangular matrices, hence lower triangular.
4. $T = DN$, so $T^{-1} = N^{-1}D^{-1}$ is lower triangular as well.

A shorter proof of item 2 of Theorem 12. Following [1, Lecture 20]: for $k < j$ we have $\vec{e}_k^{\,*}\vec{\ell}_j = 0$, so multiplying two of the factors just adds their multipliers:

$$(I + \vec{\ell}_k\vec{e}_k^{\,*})(I + \vec{\ell}_j\vec{e}_j^{\,*}) = I + \vec{\ell}_k\vec{e}_k^{\,*} + \vec{\ell}_j\vec{e}_j^{\,*}. \tag{3.15}$$

Multiplying in the factors one at a time from the right, the same cancellation happens at every step, and we get $L_1^{-1} \cdots L_{m-1}^{-1} = I + \sum_{k=1}^{m-1} \vec{\ell}_k\vec{e}_k^{\,*}$.

3.8.2 Cost of LU and Solving Linear Systems

Cost of LU. Step k of Algorithm 5 updates the trailing $(m-k) \times (m-k)$ block, with one multiplication and one subtraction per entry. Adding up over all steps,

$$\sum_{k=1}^{m-1} 2(m-k)^2 = 2\sum_{j=1}^{m-1} j^2 \approx \frac{2}{3}m^3 \text{ flops.} \tag{3.16}$$

The divisions for the multipliers add only $O(m^2)$. The update is a rank-one update, a Level 2 operation with low computational intensity; Section 2.9.3 shows how blocking turns most of it into matrix multiplication.

Solving $A\vec{x} = \vec{b}$ with LU. We split the solve into three steps:

Step	Operation	Cost
1. factor	$A = LU$	$\frac{2}{3}m^3$
2. forward substitution	$L\vec{y} = \vec{b}$	m^2
3. back substitution	$U\vec{x} = \vec{y}$	m^2

The factorization is done once. Every further right-hand side costs only $2m^2$, the same as a multiplication by A^{-1}. Forming A^{-1} would cost about $2m^3$ up front, three times as much as LU, and is less accurate. So we never compute A^{-1} to solve a linear system.

3.8.3 Pivots, Minors, and Uniqueness

The pivots as ratios of minors. Following [2, §2.3]: Step 2 of the proof of Theorem 14 gives $\det A[1{:}k, 1{:}k] = u_{11} \cdots u_{kk}$ for each k. Dividing two consecutive ones,

$$u_{kk} = \frac{\det A[1{:}k, 1{:}k]}{\det A[1{:}k{-}1, 1{:}k{-}1]}. \tag{3.17}$$

So each pivot is determined by A alone, before any elimination is done. For example, for the matrix of the small-pivot example, $u_{11} = \varepsilon$ and $u_{22} = \det A/\varepsilon = 1 - \varepsilon^{-1}$, which matches the U computed there. Moreover, $A = LU$ with L unit lower triangular exists and is unique if and only if $A[1{:}k, 1{:}k]$ is nonsingular for every $k = 1, \ldots, m-1$.

Why the factorization is unique. Let A be nonsingular and suppose $A = L_1U_1 = L_2U_2$, with L_1, L_2 unit lower triangular and U_1, U_2 upper triangular. All four are invertible, and

$$L_2^{-1}L_1 = U_2U_1^{-1}. \tag{3.18}$$

The left side is unit lower triangular and the right side is upper triangular (Proposition 11). A matrix that is both must be I, so $L_1 = L_2$ and $U_1 = U_2$.

3.8.4 Pivoting Preview

Swapping rows. In the small-pivot example, swap the two rows first:

$$PA = \begin{bmatrix} 1 & 1 \\ \varepsilon & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \varepsilon & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1-\varepsilon \end{bmatrix}. \tag{3.19}$$

Now the only multiplier is ε, and every entry of L and U is at most 1 in size. So $|L|\,|U|$ is about as large as $|A|$, and by Theorem 15 the error is at the level of machine precision. This is the idea of *partial pivoting*: at each step, swap rows so that the entry of largest absolute value in the current column becomes the pivot. Then every multiplier satisfies $|\ell_{ik}| \le 1$, and we get $PA = LU$ for a permutation matrix P. This is the subject of Lecture 4.

In Julia. `lu(A)` computes $PA = LU$ with partial pivoting; `lu(A, NoPivot())` runs Algorithm 5 as written. For $A = \left[\begin{smallmatrix} 10^{-20} & 1 \\ 1 & 1 \end{smallmatrix}\right]$ the unpivoted factors satisfy $LU - A = \left[\begin{smallmatrix} 0 & 0 \\ 0 & -1 \end{smallmatrix}\right]$, exactly the failure of the small-pivot example, while the pivoted ones are exact.

```
F = lu(A)                 # F.L, F.U, F.p (row permutation)
x = F \ b                 # reuses the factorization: 2m^2 flops per b
G = lu(A, NoPivot())      # G.L * G.U - A == [0 0; 0 -1] for this A
```

Lecture 4

LU Factorization with Pivoting

Supplementary reading:

- [1] Lectures 21 and 22.
- [2] Sections 2.3 and 2.4.
- [7] Chapter 9.

4.1 Backward Error of the LU Factorization

Recall from Lecture 3 that Gaussian elimination computes $A = LU$ in place, with the multipliers stored below the diagonal and U on and above it, and that it fails on a zero pivot even when A is nonsingular (for example $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$). Lecture 3 ended with an example in which a tiny pivot destroys the LU factorization of the well-conditioned matrix

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix}, \qquad \kappa(A) \approx 2.6 \text{ as } \varepsilon \to 0. \tag{4.1}$$

Here ε is just a small number, much smaller than $\varepsilon_{\text{mach}}$ (e.g., 10^{-20}); it is not the machine epsilon. The problem is well-conditioned, so the failure lies with the algorithm. The following theorem explains exactly where it comes from.

Theorem 16 (Backward Error of LU)

Suppose the LU factorization of $A \in \mathbb{K}^{m\times m}$ runs to completion in floating-point arithmetic without encountering a zero pivot, producing $\tilde{L}$ and $\tilde{U}$. Then, to first order in $\varepsilon_{\text{mach}}$,

$$A = \tilde{L}\tilde{U} + E, \qquad |E| \le m\varepsilon_{\text{mach}} \left|\tilde{L}\right| \left|\tilde{U}\right|. \tag{4.2}$$

So $\tilde{L}$ and $\tilde{U}$ are the exact LU factors of a nearby matrix, $A - E$. If $\left|\tilde{L}\right| \left|\tilde{U}\right|$ is about as large as $|A|$, then E is tiny compared with A and the algorithm is backward stable. If $\left|\tilde{L}\right| \left|\tilde{U}\right|$ is much larger than $|A|$, then E can be as large as A. ("To first order" means we drop terms of size $\varepsilon_{\text{mach}}^2$.)

4.1.1 Proof of Theorem 16

Formulas for L and U. From $A = LU$ and $L_{ii} = 1$, solving for the entries of L and U gives (these are the numbers elimination computes, in a different order)

$$U_{ik} = A_{ik} - \sum_{j=1}^{i-1} L_{ij}U_{jk}, \qquad i \le k, \tag{4.3}$$

$$L_{ik} = \frac{1}{U_{kk}}\left(A_{ik} - \sum_{j=1}^{k-1} L_{ij}U_{jk}\right), \qquad i > k. \tag{4.4}$$

Rounding in inner products (Homework 2, Proposition 119): in floating point, every term of a sum gets its own small relative error,

$$\mathrm{fl}\Big(\sum_{i=1}^{m} x_i y_i\Big) = \sum_{i=1}^{m} x_i y_i(1+\delta_i), \qquad |\delta_i| \lesssim m\varepsilon_{\mathrm{mach}}. \tag{4.5}$$

Proof. The U part ($i \le k$). In floating point, (4.3) becomes

$$\tilde{U}_{ik} = \Big(A_{ik} - \sum_{j=1}^{i-1} \tilde{L}_{ij}\tilde{U}_{jk}(1+\delta_j)\Big)(1+\delta'), \tag{4.6}$$

where δ' is the error of the last subtraction. Solve for A_{ik}, use $\tilde{L}_{ii} = 1$, and write $1/(1+\delta') = 1+\delta_i$:

$$A_{ik} = \sum_{j=1}^{i} \tilde{L}_{ij}\tilde{U}_{jk}(1+\delta_j) = \sum_{j=1}^{i} \tilde{L}_{ij}\tilde{U}_{jk} + \underbrace{\sum_{j=1}^{i} \tilde{L}_{ij}\tilde{U}_{jk}\delta_j}_{=:E_{ik}}. \tag{4.7}$$

Every $|\delta_j| \lesssim m\varepsilon_{\mathrm{mach}}$, so $|E_{ik}| \le m\varepsilon_{\mathrm{mach}} \sum_j \left|\tilde{L}_{ij}\right|\left|\tilde{U}_{jk}\right| = m\varepsilon_{\mathrm{mach}}(\left|\tilde{L}\right|\left|\tilde{U}\right|)_{ik}$. (The terms with $j > i$ are zero because $\tilde{L}$ is lower triangular.)

The L part ($i > k$). The same argument applied to (4.4). The division by $\tilde{U}_{kk}$ adds one more rounding error, which is merged into δ_k in the same way, and the bound is the same. □

4.1.2 What the Bound Says

If $\left|\tilde{L}\right|\left|\tilde{U}\right|$ is of the same order as $|A|$, then E is a relative perturbation of size $O(m\varepsilon_{\mathrm{mach}})$ and the algorithm is backward stable. But $\left|\tilde{L}\right|\left|\tilde{U}\right|$ can be far larger than $|A|$. In the example above,

$$|L|\,|U| = \begin{bmatrix} 1 & 0 \\ \varepsilon^{-1} & 1 \end{bmatrix}\begin{bmatrix} \varepsilon & 1 \\ 0 & \varepsilon^{-1}-1 \end{bmatrix} = \begin{bmatrix} \varepsilon & 1 \\ 1 & 2\varepsilon^{-1}-1 \end{bmatrix}. \tag{4.8}$$

The bottom-right entry is about $2\varepsilon^{-1}$; multiplied by $\varepsilon_{\mathrm{mach}}$, the error can be $O(1)$ or larger, exactly as observed. The question becomes: *how can we keep $|L|\,|U|$ small?*

4.2 Pivots and the Growth of Products

Look at the storage after $k-1$ steps of elimination: the first $k-1$ rows already hold U, the first $k-1$ columns below the diagonal hold L, and the trailing block $\tilde{A}$ is still being processed.

$$\begin{bmatrix} U_{11} & \cdots & & & & \\ & \ddots & & & & \\ & & U_{k-1,k-1} & U_{k-1,k} & \cdots & U_{k-1,m} \\ & L & L_{k,k-1} & \tilde{A}_{kk} & \cdots & \tilde{A}_{km} \\ & & \vdots & \vdots & & \vdots \\ & & L_{m,k-1} & \tilde{A}_{mk} & \cdots & \tilde{A}_{mm} \end{bmatrix} \tag{4.9}$$

Step k turns row k into a row of U and column k below the diagonal into a column of L, and updates the trailing $(m-k)\times(m-k)$ block:

$$L_{ik} = \frac{\tilde{A}_{ik}}{\tilde{A}_{kk}}, \qquad U_{kj} = \tilde{A}_{kj}, \qquad \tilde{A}_{ij} \leftarrow \tilde{A}_{ij} - L_{ik}U_{kj}. \tag{4.10}$$

The entry $\tilde{A}_{kk}$ is the k-th *pivot*. The largest product created in step k, which enters both the update and $|L|\,|U|$, is

$$\max_{i,j} |L_{ik}U_{kj}| = \max_{i,j} \frac{\left|\tilde{A}_{ik}\right|\left|\tilde{A}_{kj}\right|}{\left|\tilde{A}_{kk}\right|}. \tag{4.11}$$

The numerator is determined by column k and row k. To keep this quantity small, we want the denominator, the pivot, to be as large as possible.

4.3 Partial Pivoting

We must pivot when $\tilde{A}_{kk} = 0$, since otherwise the algorithm cannot continue. Even when $\tilde{A}_{kk} \neq 0$, we want to avoid *small* pivots, which make the products $L_{ik}U_{kj}$ explode.

Definition 17 (Partial (Row) Pivoting)

At step k, *partial pivoting* chooses the pivot to be the entry of largest absolute value in column k, on or below the diagonal:

$$r = \operatorname*{argmax}_{i\geq k} \left|\tilde{A}_{ik}\right|. \tag{4.12}$$

It swaps rows r and k with a permutation matrix P_k and then eliminates with L_k. The sequence $A \xrightarrow{L_k} \cdots$ becomes $A \xrightarrow{P_k} \cdot \xrightarrow{L_k} \cdots$.

Because the pivot is the largest entry in its column, every multiplier satisfies

$$|L_{ik}| = \frac{\left|\tilde{A}_{ik}\right|}{\left|\tilde{A}_{kk}\right|} \leq 1, \tag{4.13}$$

so $|L_{ik}U_{kj}| \leq |U_{kj}|$: no update exceeds the size of the current pivot row.

4.3.1 Moving the Permutations Together

With row swaps, elimination becomes

$$L_{m-1}P_{m-1}\cdots L_2P_2L_1P_1A = U, \tag{4.14}$$

where L's and P's alternate, so we cannot directly write $PA = LU$. We need to move all permutations to the right.

Proposition 18 (Commuting a Permutation Past an Elimination)

Let $\ell > k$ and let P_ℓ swap rows ℓ and i for some $i \geq \ell$. Swapping twice undoes the swap, so $P_\ell^{-1} = P_\ell$. Then

$$P_\ell L_k = (P_\ell L_k P_\ell)P_\ell, \qquad P_\ell L_k P_\ell = I - (P_\ell \vec{\ell}_k)\vec{e}_k^*, \tag{4.15}$$

so $P_\ell L_k P_\ell$ is again an elementary lower triangular matrix, obtained from L_k by swapping two of its multipliers.

Proof. The first identity holds because $P_\ell P_\ell = I$. For the second, substitute $L_k = I - \vec{\ell}_k\vec{e}_k^*$ and expand:

$$P_\ell(I - \vec{\ell}_k\vec{e}_k^*)P_\ell = P_\ell P_\ell - (P_\ell\vec{\ell}_k)(\vec{e}_k^*P_\ell) = I - (P_\ell\vec{\ell}_k)(\vec{e}_k^*P_\ell). \tag{4.16}$$

Multiplying the row vector $\vec{e}_k^*$ on the right by P_ℓ swaps its entries ℓ and i. Both entries are 0, since $\ell, i > k$, so nothing changes: $\vec{e}_k^*P_\ell = \vec{e}_k^*$. This gives $P_\ell L_k P_\ell = I - (P_\ell\vec{\ell}_k)\vec{e}_k^*$.

Finally, $P_\ell\vec{\ell}_k$ is $\vec{\ell}_k$ with entries ℓ and i swapped. Its first k entries are still 0, so $P_\ell L_k P_\ell$ is again elementary lower triangular. □

Using Proposition 18 repeatedly to push every P to the right,

$$L_{m-1}P_{m-1}\cdots L_1P_1 = L'_{m-1}\cdots L'_1\,P_{m-1}\cdots P_1, \qquad L'_k \coloneqq P_{m-1}\cdots P_{k+1}L_kP_{k+1}\cdots P_{m-1}, \tag{4.17}$$

and each L'_k is still elementary lower triangular. With $P \coloneqq P_{m-1}\cdots P_1$ and $L \coloneqq (L'_{m-1}\cdots L'_1)^{-1}$, we obtain

$$PA = LU. \tag{4.18}$$

4.4 The In-Place Algorithm with Partial Pivoting

How do we compute this in place, up to the permutation? Suppose k steps are done:

$$L_k\cdots L_1PA = U_k, \tag{4.19}$$

where the L_j are already the "permutation-adjusted" versions and U_k is the current working matrix. In storage, U_k occupies the upper triangle and the trailing block, and the multipliers of $L_1, \ldots, L_k$ sit in the strictly lower part of the first k columns. Step $k+1$ swaps rows and then eliminates:

$$L_{k+1}P_{k+1}L_k\cdots L_1PA = L_{k+1}P_{k+1}U_k. \tag{4.20}$$

Moving P_{k+1} to the right,

$$L_{k+1}\underbrace{L'_k\cdots L'_1}_{\text{multipliers permuted by } P_{k+1}}\underbrace{P_{k+1}P}_{P'}A = L_{k+1}\underbrace{P_{k+1}U_k}_{U'_k} = U_{k+1}. \tag{4.21}$$

Reading term by term:

- $L'_j = P_{k+1} L_j P_{k+1}$ only permutes the stored multipliers;
- $P' = P_{k+1} P$ swaps the same two rows of the accumulated permutation;
- $U'_k = P_{k+1} U_k$ swaps the same two rows of the working matrix.

Since the multipliers and U_k live in the same array, *swapping entire rows (columns* 1 *through* m*)* performs all three updates at once, after which we eliminate as usual.

Algorithm 6 LU factorization with partial pivoting, in place.

$P \leftarrow I$
for $k = 1, \dots, m$ **do**
  $r \leftarrow \operatorname{argmax}_{i \geq k} |A_{ik}|$
  swap rows r and k of A (all columns $1, \dots, m$) and of P
  for $i = k+1, \dots, m$ **do**
    $A_{ik} \leftarrow A_{ik} / A_{kk}$
  for $j = k+1, \dots, m$ **do**
    for $i = k+1, \dots, m$ **do**
      $A_{ij} \leftarrow A_{ij} - A_{ik} A_{kj}$

Some remarks:

- In Julia, `argmax(abs.(A[k:end, k]))` returns an index into the subvector, so one adds $k-1$ to recover the global row index.
- The row swap runs over all columns $1, \dots, m$, not just $k, \dots, m$: the first $k-1$ columns hold multipliers, and swapping them is exactly $L'_j = P_{k+1} L_j P_{k+1}$.
- On exit the strictly lower part of A holds L, the upper part holds U, and $PA = LU$.

4.5 (SUPPLEMENT) Further Topics

4.5.1 Storing P and the Cost of Pivoting

In practice P is not stored as a matrix but as a permutation vector of length m (LAPACK's `ipiv`), so each swap exchanges two integers. The search for pivots costs only $O(m^2)$ comparisons on top of the $\frac{2}{3}m^3$ flops. In column-major storage, however, a row swap is a strided access, and its cost is not negligible.

4.5.2 Growth Factor

Definition 19 (Growth Factor)

The *growth factor* of Gaussian elimination on A is

$$\rho := \frac{\max_{i,j} |U_{ij}|}{\max_{i,j} |A_{ij}|}. \tag{4.22}$$

Why it matters. With partial pivoting $|L_{ik}| \le 1$, so each entry of $|L|\,|U|$ is a sum of at most m terms, each at most $\max|U_{ij}| = \rho \max|A_{ij}|$:

$$(|L|\,|U|)_{ij} \le m\rho \max_{i,j} |A_{ij}| \qquad \Longrightarrow \qquad |E_{ij}| \le m^2 \rho\, \varepsilon_{\text{mach}} \max_{i,j} |A_{ij}| \tag{4.23}$$

by Theorem 16. So partial pivoting is backward stable exactly when ρ stays small.

How large can ρ be? Each step updates $\tilde{A}_{ij} \leftarrow \tilde{A}_{ij} - L_{ik}U_{kj}$ with $|L_{ik}| \le 1$, so the largest entry can at most double. After $m-1$ steps,

$$\rho \le 2^{m-1}. \tag{4.24}$$

This bound is attained by the matrix with 1 on the diagonal, -1 below it, and 1 in the last column (Example 20, Homework 2): no rows are swapped, and the last column of U is $1, 2, 4, \ldots, 2^{m-1}$. For $m = 8$, Julia's `lu` gives $\rho = 128 = 2^7$.

In practice. Such matrices are extremely rare. For random matrices ρ grows slowly; with `randn` (median of 5 runs) we measured $\rho \approx 2, 4, 8, 19$ for $m = 16, 64, 256, 1024$. This is why partial pivoting is used as a stable algorithm in practice, although in theory it is not backward stable.

4.5.3 Complete Pivoting

The idea. At step k, *complete pivoting* chooses the pivot to be the entry of largest absolute value in the whole trailing block, $\max_{i,j\ge k} \left|\tilde{A}_{ij}\right|$, and moves it to position (k,k) by swapping two rows *and* two columns. This gives

$$PAQ = LU, \tag{4.25}$$

with a row permutation P and a column permutation Q. Swapping columns reorders the unknowns, so $A\vec{x} = \vec{b}$ is solved as $LU(Q^\top \vec{x}) = P\vec{b}$.

Benefit. Every entry of the trailing block is now at most the pivot, so the growth is much better controlled. Wilkinson showed a bound on ρ that grows much more slowly than 2^{m-1}, roughly like $m^{(\ln m)/4}$ [7, Chapter 9].

Cost. Step k searches all $(m-k+1)^2$ entries of the trailing block, about $m^3/3$ comparisons in total. That is the same order as the $\frac{2}{3}m^3$ flops, and the column swaps add strided memory access. Since partial pivoting is almost always good enough, complete pivoting is rarely used.

Example. We ran complete pivoting (a short Julia implementation) on the worst-case matrix above and on random matrices, and compared with partial pivoting:

Matrix	ρ, partial pivoting	ρ, complete pivoting
worst case, $m = 64$	$2^{63} \approx 9 \cdot 10^{18}$	2
`randn`, $m = 64$	4.2	1.9
`randn`, $m = 256$	7.8	3.6

On the worst-case matrix, complete pivoting removes the growth entirely. On random matrices both are small, which is why the extra cost of complete pivoting is rarely worth it.

A compromise: rook pivoting. Search alternately along a column and a row until the entry is the largest in both. This is cheap and controls growth much better [7, Chapter 9].

Lecture 5

Least Squares Problems, Gram–Schmidt, and QR

Supplementary reading:

- [1] Lectures 6, 7, 8, 11, and 22.
- [2] Sections 3.1–3.3.

5.1 Is Partial Pivoting Enough?

Lecture 4 bounded the backward error of LU by $m\varepsilon_{\text{mach}} \left|\tilde{L}\right| \left|\tilde{U}\right|$ and introduced partial pivoting to keep the multipliers bounded, $|L_{ij}| \leq 1$; commuting the permutations past the elimination matrices then gave $PA = LU$ and an in-place algorithm. Rows and columns can also be pivoted simultaneously (*complete pivoting*), giving $PA\tilde{P} = LU$ with a column permutation $\tilde{P}$. In practice, partial pivoting is usually enough. In theory, however, it can be *unstable* when m is large.

Example 20 (Exponential Growth Under Partial Pivoting)**.** Consider

$$A = \begin{bmatrix} 1 & & & 1 \\ -1 & 1 & & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & 1 \end{bmatrix}. \tag{5.1}$$

In every column the candidate pivots have the same absolute value, $|1| = |-1|$, and `argmax` returns the first, so partial pivoting never swaps rows. Tracking the in-place storage (multipliers below the diagonal),

$$A \rightsquigarrow \begin{bmatrix} 1 & & & 1 \\ -1 & 1 & & 2 \\ -1 & -1 & 1 & 2 \\ -1 & -1 & -1 & 2 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 1 & & & 1 \\ -1 & 1 & & 2 \\ -1 & -1 & 1 & 4 \\ -1 & -1 & -1 & 4 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 1 & & & 1 \\ -1 & 1 & & 2 \\ -1 & -1 & 1 & 4 \\ -1 & -1 & -1 & 8 \end{bmatrix}. \tag{5.2}$$

The multiplier at step k is -1, i.e., row k is added to every row below it. Row k is zero in columns $k+1$

through $m-1$, so the middle part never changes, but the last column doubles at every step. The result is

$$L = \begin{bmatrix} 1 & & & \\ -1 & 1 & & \\ -1 & -1 & 1 & \\ -1 & -1 & -1 & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} 1 & & & 1 \\ & 1 & & 2 \\ & & 1 & 4 \\ & & & 8 \end{bmatrix}. \tag{5.3}$$

For general m, $U_{mm} = 2^{m-1}$, so $|L|\,|U| \gtrsim 2^m$ and the error bound $m\varepsilon_{\text{mach}}\,|L|\,|U|$ can reach $m2^m\varepsilon_{\text{mach}}$. In double precision, $2^{m-1}\varepsilon_{\text{mach}} \approx 1$ already for $m \approx 53$, so all accuracy can be lost.

This is the worst case $\rho = 2^{m-1}$ of the growth factor mentioned in Lecture 4. Fortunately, such matrices are extremely rare in practice.

5.2 Non-Square Systems

So far we have only considered $A \in \mathbb{K}^{m\times n}$ with $m = n$. If $m \neq n$, there are two cases.

Overdetermined systems ($m > n$). A is tall: there are more equations than unknowns. Then $\mathcal{R}(A) \neq \mathbb{K}^m$, since n columns span at most an n-dimensional subspace. Hence for a general $\vec{b}$ the equation $A\vec{x} = \vec{b}$ *does not have a solution*. Such a system is called an *overdetermined linear system*.

Underdetermined systems ($m < n$). A is wide: there are more unknowns than equations. Then $\mathcal{N}(A) \neq \{\vec{0}\}$, since the rank is at most $m < n$. Hence even when a solution exists, it is *never unique*: adding any vector of the null space to a solution gives another solution. Such a system is called an *underdetermined linear system*.

5.3 Least Squares Problems

Overdetermined systems usually have no solution, and underdetermined systems have many. In both cases we replace the equation by a minimization problem:

$$\text{overdetermined:} \quad \min_{\vec{x}} \left\| A\vec{x} - \vec{b} \right\|, \qquad \text{underdetermined:} \quad \min_{\vec{x}\,:\,A\vec{x}=\vec{b}} \|\vec{x}\|. \tag{5.4}$$

In the overdetermined case we take the $\vec{x}$ that comes closest to satisfying the equations; in the underdetermined case we take the smallest of all solutions.

Which norm should we use? There are many choices, depending on the application. In linear regression, for example, the norm in the overdetermined case measures the *fitting error*, and in the underdetermined case it measures how *complex* the model is.

The key fact: the solution depends linearly on $\vec{b}$ *only if the norm comes from an inner product*, $\|\cdot\| = \sqrt{\langle\cdot,\cdot\rangle}$. This case is the *least squares problem*, and we can solve it with linear algebra.

Example 21 (Mean Versus Median)**.** Let $A = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^\top$. Then $\min_x \left\| Ax - \vec{b} \right\|$ looks for the single number x closest to all of b_1, b_2, b_3.

- In the 2-norm, the best x is the mean $\frac{1}{3}(b_1 + b_2 + b_3)$, which is linear in $\vec{b}$.
- In the 1-norm, the best x is the median, which is not linear in $\vec{b}$.

5.4 The Geometry of Least Squares

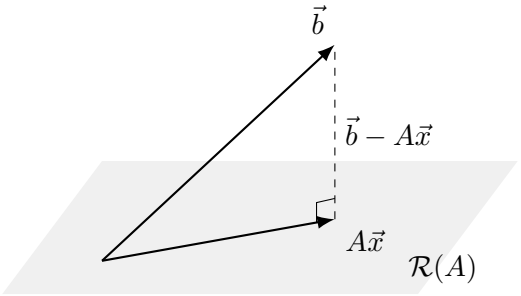
For $m = n$, solving $A\vec{x} = \vec{b}$ means finding a linear combination of the columns of A that equals $\vec{b}$ exactly:

$$A\vec{x} = x_1\vec{a}_1 + x_2\vec{a}_2 + \cdots + x_n\vec{a}_n = \vec{b}. \tag{5.5}$$

For $m > n$, consider

$$\min_{\vec{x}} \left\| A\vec{x} - \vec{b} \right\|_2^2. \tag{5.6}$$

The columns of A span only an n-dimensional subspace $\mathcal{R}(A)$ of $\mathbb{K}^m$, and $\vec{b}$ generally does not lie in it. As $\vec{x}$ varies, $A\vec{x}$ ranges over this subspace, and the point closest to $\vec{b}$ is the *orthogonal projection* of $\vec{b}$ onto $\mathcal{R}(A)$: dropping a perpendicular from $\vec{b}$ to the subspace, its foot is the optimal $A\vec{x}$, and the residual $A\vec{x} - \vec{b}$ is orthogonal to the whole subspace.



Writing "the residual is orthogonal to every column of A" as an equation gives the *normal equations*:

$$A^*(A\vec{x} - \vec{b}) = \vec{0} \quad \Longleftrightarrow \quad A^*A\vec{x} = A^*\vec{b}. \tag{5.7}$$

Orthonormal columns. If the columns of A are orthonormal, then $A^*A = I$ and the normal equations give the solution directly:

$$\vec{x} = A^*\vec{b}, \qquad \text{i.e.,} \qquad x_i = \left\langle \vec{a}_i, \vec{b} \right\rangle. \tag{5.8}$$

Since $\vec{a}_i$ is a unit vector, x_i is how much of $\vec{b}$ points along the i-th column. The columns are orthogonal, so each x_i is computed on its own, and $A\vec{x} = \sum_i x_i\vec{a}_i$ is the projection of $\vec{b}$ onto $\mathcal{R}(A)$. Without orthogonality the coefficients depend on each other, and we must solve $A^*A\vec{x} = A^*\vec{b}$.

Example 22 (Projection onto a Coordinate Plane)**.** For

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \tag{5.9}$$

we get $\vec{x} = (b_1, b_2)^\top$ and $A\vec{x} = (b_1, b_2, 0)^\top$, the closest point of the xy-plane to $\vec{b}$. The residual $(0, 0, b_3)^\top$ is orthogonal to the plane.

This motivates the next section: since orthonormal columns make least squares easy, we first *make* the columns orthonormal.

5.5 Orthogonalization: The QR Factorization

What if the columns of A are not orthonormal? The idea is to find an *orthonormal basis* of the space spanned by the columns of A and to transfer the problem to that basis. In matrix form,

$$A = QR, \tag{5.10}$$

where $Q \in \mathbb{K}^{m\times n}$ has orthonormal columns and $R \in \mathbb{K}^{n\times n}$ is invertible (it will turn out to be upper triangular). The least squares solution is then

$$\vec{x} = R^{-1}Q^*\vec{b}. \tag{5.11}$$

Both steps have a clear meaning:

- $Q^*\vec{b}$ expresses $\vec{b}$ in terms of the columns of Q (the coordinates of the projection, exactly as in the orthonormal case);
- $A = QR$ says that R expresses the columns of A as combinations of the columns of Q, so R^{-1} converts coefficients with respect to Q back into coefficients with respect to A.

Algebraically, in the normal equations (5.7) we have $A^*A = R^*Q^*QR = R^*R$ and $A^*\vec{b} = R^*Q^*\vec{b}$; cancelling the invertible R^* leaves $R\vec{x} = Q^*\vec{b}$.

5.6 Classical Gram–Schmidt

The idea. Process the columns of A one at a time. When we reach $\vec{a}_j$, the vectors $\vec{q}_1, \dots, \vec{q}_{j-1}$ are already orthonormal. To get $\vec{q}_j$:

1. subtract from $\vec{a}_j$ its components along $\vec{q}_1, \dots, \vec{q}_{j-1}$;
2. normalize what is left.

We first make $\vec{v}_j$ orthogonal to the previous vectors and normalize afterwards. The component of $\vec{a}_j$ along $\vec{v}_i$ is $\frac{\langle \vec{v}_i, \vec{a}_j\rangle}{\langle \vec{v}_i, \vec{v}_i\rangle}\vec{v}_i$, so

$$\begin{aligned}
\vec{v}_1 &= \vec{a}_1, \\
\vec{v}_2 &= \vec{a}_2 - \frac{\langle \vec{v}_1, \vec{a}_2\rangle}{\langle \vec{v}_1, \vec{v}_1\rangle}\vec{v}_1, \\
\vec{v}_3 &= \vec{a}_3 - \frac{\langle \vec{v}_1, \vec{a}_3\rangle}{\langle \vec{v}_1, \vec{v}_1\rangle}\vec{v}_1 - \frac{\langle \vec{v}_2, \vec{a}_3\rangle}{\langle \vec{v}_2, \vec{v}_2\rangle}\vec{v}_2, \\
&\vdots \\
\vec{v}_j &= \vec{a}_j - \sum_{i=1}^{j-1} \frac{\langle \vec{v}_i, \vec{a}_j\rangle}{\langle \vec{v}_i, \vec{v}_i\rangle}\vec{v}_i,
\end{aligned} \tag{5.12}$$

and finally $\vec{q}_j = \vec{v}_j / \|\vec{v}_j\|_2$. Since $\vec{q}_i = \vec{v}_i / \|\vec{v}_i\|_2$, each term equals $\langle \vec{q}_i, \vec{a}_j\rangle \vec{q}_i$, the component of $\vec{a}_j$ along $\vec{q}_i$.

Where R comes from. Write each term of (5.12) as $\langle \vec{q}_i, \vec{a}_j\rangle \vec{q}_i$ and name the coefficients:

$$r_{ij} = \langle \vec{q}_i, \vec{a}_j\rangle \quad (i < j), \qquad r_{jj} = \|\vec{v}_j\|_2, \qquad r_{ij} = 0 \quad (i > j). \tag{5.13}$$

Since $\vec{v}_j = r_{jj}\vec{q}_j$, solving (5.12) for $\vec{a}_j$ gives

$$\vec{a}_j = r_{1j}\vec{q}_1 + \cdots + r_{j-1,j}\vec{q}_{j-1} + r_{jj}\vec{q}_j = \sum_{i=1}^{j} r_{ij}\vec{q}_i, \tag{5.14}$$

that is, column j of A is Q times column j of R. For all columns at once,

$$\underbrace{\begin{bmatrix}\vec{a}_1 & \vec{a}_2 & \cdots & \vec{a}_n\end{bmatrix}}_{A} = \underbrace{\begin{bmatrix}\vec{q}_1 & \vec{q}_2 & \cdots & \vec{q}_n\end{bmatrix}}_{Q} \underbrace{\begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & & & r_{nn}\end{bmatrix}}_{R}. \tag{5.15}$$

R is upper triangular because $\vec{a}_j$ involves only $\vec{q}_1, \ldots, \vec{q}_j$, and invertible because $r_{jj} > 0$. Entry r_{ij} $(i < j)$ is the amount of $\vec{q}_i$ in $\vec{a}_j$; r_{jj} is the length of the new part of $\vec{a}_j$.

The algorithm. Computing the sum in (5.12) one term at a time:

Algorithm 7 Classical Gram–Schmidt (CGS).

for $j = 1, \ldots, n$ **do**
  $\vec{v}_j \leftarrow \vec{a}_j$
  for $i = 1, \ldots, j-1$ **do**
    $r_{ij} \leftarrow \langle \vec{q}_i, \vec{a}_j \rangle$ ▷ inner product with the *original* column
    $\vec{v}_j \leftarrow \vec{v}_j - r_{ij}\vec{q}_i$
  $r_{jj} \leftarrow \|\vec{v}_j\|_2$
  $\vec{q}_j \leftarrow \vec{v}_j / r_{jj}$

Theorem 23 (Gram–Schmidt)

Let $\vec{a}_1, \ldots, \vec{a}_n \in \mathbb{K}^m$ $(m \geq n)$ be linearly independent. Then the vectors $\vec{q}_1, \ldots, \vec{q}_n$ produced by Algorithm 7 form an orthonormal basis of $\operatorname{span}(\vec{a}_1, \ldots, \vec{a}_n)$, and $\operatorname{span}(\vec{q}_1, \ldots, \vec{q}_j) = \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_j)$ for every j.

Proof. By induction on j: $\vec{q}_1, \ldots, \vec{q}_j$ are orthonormal and span $\operatorname{span}(\vec{a}_1, \ldots, \vec{a}_j)$. *Base case:* $\vec{a}_1 \neq \vec{0}$ by linear independence, so $\vec{q}_1 = \vec{a}_1 / \|\vec{a}_1\|_2$ is a unit vector spanning the same line.

Inductive step. Assume the claim for j, and let $\vec{v}_{j+1} = \vec{a}_{j+1} - \sum_{i=1}^{j} \langle \vec{q}_i, \vec{a}_{j+1} \rangle \vec{q}_i$.

1. *Orthogonality.* For every $k \leq j$,

$$\langle \vec{q}_k, \vec{v}_{j+1} \rangle = \langle \vec{q}_k, \vec{a}_{j+1} \rangle - \sum_{i=1}^{j} \langle \vec{q}_i, \vec{a}_{j+1} \rangle \underbrace{\langle \vec{q}_k, \vec{q}_i \rangle}_{\delta_{ki}} = \langle \vec{q}_k, \vec{a}_{j+1} \rangle - \langle \vec{q}_k, \vec{a}_{j+1} \rangle = 0. \tag{5.16}$$

2. *Nonzero.* If $\vec{v}_{j+1} = \vec{0}$, then $\vec{a}_{j+1} \in \operatorname{span}(\vec{q}_1, \ldots, \vec{q}_j) = \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_j)$, contradicting linear independence. So $\vec{q}_{j+1} = \vec{v}_{j+1} / \|\vec{v}_{j+1}\|_2$ is defined, and $\vec{q}_1, \ldots, \vec{q}_{j+1}$ are orthonormal.

3. *Span.* $\vec{q}_{j+1} \in \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_{j+1})$, and $\vec{q}_1, \ldots, \vec{q}_{j+1}$ are $j+1$ orthonormal, hence independent, vectors in this $(j+1)$-dimensional space, so they span it. □

Example 24 (CGS Loses Orthogonality)**.** Consider the matrix

$$A = \begin{bmatrix} 1 & 1 & 1 \\ \varepsilon & 0 & 0 \\ 0 & \varepsilon & 0 \\ 0 & 0 & \varepsilon \end{bmatrix}, \tag{5.17}$$

with ε small enough that $1 + \varepsilon^2 = 1$ in floating point, i.e., $\varepsilon < \sqrt{u} \approx 10^{-8}$, for example $\varepsilon = 10^{-10}$.

Step 1. $\vec{v}_1 = \vec{a}_1 = (1, \varepsilon, 0, 0)^\top$ and $\vec{q}_1 = (1 + \varepsilon^2)^{-1/2}(1, \varepsilon, 0, 0)^\top$, which in floating point is $(1, \varepsilon, 0, 0)^\top$.

Step 2. $r_{12} = \langle \vec{q}_1, \vec{a}_2 \rangle = 1$, so

$$\vec{v}_2 = \vec{a}_2 - r_{12}\vec{q}_1 = \begin{bmatrix} 1 \\ 0 \\ \varepsilon \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ \varepsilon \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -\varepsilon \\ \varepsilon \\ 0 \end{bmatrix}, \qquad \vec{q}_2 = \frac{\vec{v}_2}{\sqrt{2}\,\varepsilon} = \begin{bmatrix} 0 \\ -1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix}. \tag{5.18}$$

Step 3. $r_{13} = \langle \vec{q}_1, \vec{a}_3 \rangle = 1$ and $r_{23} = \langle \vec{q}_2, \vec{a}_3 \rangle = 0$, so

$$\vec{v}_3 = \vec{a}_3 - r_{13}\vec{q}_1 - r_{23}\vec{q}_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \varepsilon \end{bmatrix} - \begin{bmatrix} 1 \\ \varepsilon \\ 0 \\ 0 \end{bmatrix} - 0 \cdot \vec{q}_2 = \begin{bmatrix} 0 \\ -\varepsilon \\ 0 \\ \varepsilon \end{bmatrix}, \qquad \vec{q}_3 = \frac{\vec{v}_3}{\sqrt{2}\,\varepsilon} = \begin{bmatrix} 0 \\ -1/\sqrt{2} \\ 0 \\ 1/\sqrt{2} \end{bmatrix}. \tag{5.19}$$

The result

$$Q = \begin{bmatrix} 1 & 0 & 0 \\ \varepsilon & -1/\sqrt{2} & -1/\sqrt{2} \\ 0 & 1/\sqrt{2} & 0 \\ 0 & 0 & 1/\sqrt{2} \end{bmatrix} \tag{5.20}$$

is *far from orthogonal*: $\langle \vec{q}_2, \vec{q}_3 \rangle = \frac{1}{2}$, so the angle between them is $60°$ rather than $90°$.

5.7 Modified Gram–Schmidt

What went wrong. Look at step 3 of Example 24.

- First CGS removes the $\vec{q}_1$-component: $\vec{a}_3 - r_{13}\vec{q}_1 = (0, -\varepsilon, 0, \varepsilon)^\top$. Two vectors of size 1 cancel, and only a vector of size ε is left. So any error of size ε is now as large as what is left.
- Such an error is there: the computed $\vec{q}_1$ and $\vec{q}_2$ are not exactly orthogonal, $\langle \vec{q}_1, \vec{q}_2 \rangle = -\varepsilon/\sqrt{2}$.
- Next CGS removes the $\vec{q}_2$-component, but it measures this component in the *original* column: $r_{23} = \langle \vec{q}_2, \vec{a}_3 \rangle = 0$. What we actually need to remove is the $\vec{q}_2$-component of *what is left*:

$$r_{23} = \langle \vec{q}_2, \vec{a}_3 - r_{13}\vec{q}_1 \rangle = \frac{\varepsilon}{\sqrt{2}} \neq 0. \tag{5.21}$$

The two values differ by $-r_{13}\langle \vec{q}_2, \vec{q}_1 \rangle$, which is exactly the orthogonality error. In exact arithmetic it is 0 and the two agree; in floating point it is not.

Notation. To write the fix, we name the intermediate values of $\vec{v}_j$ in Algorithm 7: $\vec{v}_j^{(i)}$ is $\vec{v}_j$ after i subtractions,

$$\vec{v}_j^{(0)} = \vec{a}_j, \qquad \vec{v}_j^{(i)} = \vec{v}_j^{(i-1)} - r_{ij}\vec{q}_i, \qquad \vec{v}_j^{(j-1)} = \vec{v}_j. \tag{5.22}$$

For example, $\vec{v}_3^{(1)} = \vec{a}_3 - r_{13}\vec{q}_1$ is the "what is left" above.

The fix. Change a single line: compute r_{ij} from the *partially orthogonalized* vector instead of the original column.

$$\text{CGS:} \quad r_{ij} = \left\langle \vec{q}_i, \vec{v}_j^{(0)} \right\rangle = \langle \vec{q}_i, \vec{a}_j \rangle, \qquad \text{MGS:} \quad r_{ij} = \left\langle \vec{q}_i, \vec{v}_j^{(i-1)} \right\rangle. \tag{5.23}$$

Everything else, including the update $\vec{v}_j^{(i)} = \vec{v}_j^{(i-1)} - r_{ij}\vec{q}_i$, stays the same.

To compare with CGS directly, we write MGS in the same unnormalized form. MGS subtracts one direction at a time, and each inner product uses the vector *as it is now*:

$$\begin{aligned}
\vec{v}_1 &= \vec{a}_1, \\
\vec{v}_2 &= \vec{a}_2 - \frac{\langle \vec{v}_1, \vec{a}_2 \rangle}{\langle \vec{v}_1, \vec{v}_1 \rangle}\vec{v}_1, \\
\vec{v}_3^{(1)} &= \vec{a}_3 - \frac{\langle \vec{v}_1, \vec{a}_3 \rangle}{\langle \vec{v}_1, \vec{v}_1 \rangle}\vec{v}_1, \\
\vec{v}_3 &= \vec{v}_3^{(1)} - \frac{\langle \vec{v}_2, \vec{v}_3^{(1)} \rangle}{\langle \vec{v}_2, \vec{v}_2 \rangle}\vec{v}_2, \\
&\vdots \\
\vec{v}_j^{(i)} &= \vec{v}_j^{(i-1)} - \frac{\langle \vec{v}_i, \vec{v}_j^{(i-1)} \rangle}{\langle \vec{v}_i, \vec{v}_i \rangle}\vec{v}_i, \qquad i = 1, \dots, j-1,
\end{aligned} \tag{5.24}$$

In terms of the normalized $\vec{q}_i$, this gives the following algorithm. It differs from Algorithm 7 only in line 4.

Algorithm 8 Modified Gram–Schmidt (MGS).

```
1: for j = 1, ..., n do
2:     v_j ← a_j
3:     for i = 1, ..., j − 1 do
4:         r_ij ← ⟨q_i, v_j⟩                ▷ inner product with the updated vector
5:         v_j ← v_j − r_ij q_i
6:     r_jj ← ‖v_j‖_2
7:     q_j ← v_j / r_jj
```

Same in exact arithmetic. Expanding (5.22),

$$\left\langle \vec{q}_i, \vec{v}_j^{(i-1)} \right\rangle = \langle \vec{q}_i, \vec{a}_j \rangle - \sum_{k=1}^{i-1} r_{kj} \langle \vec{q}_i, \vec{q}_k \rangle, \tag{5.25}$$

and the sum vanishes when the $\vec{q}$'s are orthonormal, so CGS and MGS compute the same r_{ij}. The point is that CGS *relies* on $\langle \vec{q}_i, \vec{q}_k \rangle = 0$, which holds only approximately in floating point; as soon as cancellation occurs, this small error becomes large. MGS does not rely on it: each r_{ij} is the $\vec{q}_i$-component of what is actually left, so errors made in earlier steps are projected out by later ones.

Example 25 (MGS on the Same Matrix)**.** For the matrix of Example 24, $\vec{q}_1$ and $\vec{q}_2$ are computed exactly as in CGS (with one previous vector there is nothing to modify). Step 3 now projects in two stages:

$$\vec{v}_3^{(1)} = \vec{a}_3 - r_{13}\vec{q}_1 = \begin{bmatrix}1\\0\\0\\\varepsilon\end{bmatrix} - \begin{bmatrix}1\\\varepsilon\\0\\0\end{bmatrix} = \begin{bmatrix}0\\-\varepsilon\\0\\\varepsilon\end{bmatrix}. \tag{5.26}$$

Now $r_{23} = \left\langle \vec{q}_2, \vec{v}_3^{(1)} \right\rangle = \varepsilon/\sqrt{2}$ is no longer zero, and

$$\vec{v}_3^{(2)} = \vec{v}_3^{(1)} - r_{23}\vec{q}_2 = \begin{bmatrix}0\\-\varepsilon\\0\\\varepsilon\end{bmatrix} - \frac{\varepsilon}{\sqrt{2}}\begin{bmatrix}0\\-1/\sqrt{2}\\1/\sqrt{2}\\0\end{bmatrix} = \begin{bmatrix}0\\-\varepsilon/2\\-\varepsilon/2\\\varepsilon\end{bmatrix}, \qquad \vec{q}_3 = \frac{\vec{v}_3^{(2)}}{\varepsilon\sqrt{3/2}} = \begin{bmatrix}0\\-1/\sqrt{6}\\-1/\sqrt{6}\\\sqrt{2/3}\end{bmatrix}. \tag{5.27}$$

The result is

$$Q = \begin{bmatrix}1 & 0 & 0\\ \varepsilon & -1/\sqrt{2} & -1/\sqrt{6}\\ 0 & 1/\sqrt{2} & -1/\sqrt{6}\\ 0 & 0 & \sqrt{2/3}\end{bmatrix}. \tag{5.28}$$

Now $\langle \vec{q}_2, \vec{q}_3\rangle = \frac{1}{\sqrt{12}} - \frac{1}{\sqrt{12}} = 0$, and the remaining inner products $\langle \vec{q}_1, \vec{q}_2\rangle = -\varepsilon/\sqrt{2}$ and $\langle \vec{q}_1, \vec{q}_3\rangle = -\varepsilon/\sqrt{6}$ are $O(\varepsilon)$: the columns are *nearly orthogonal*.

5.8 (SUPPLEMENT) Further Topics

5.8.1 The Projector View

Trefethen and Bau [1, Lecture 8] write both algorithms with projectors, which shows the difference in one line.

Two kinds of projectors. Let $Q_{j-1} = \begin{bmatrix}\vec{q}_1 & \cdots & \vec{q}_{j-1}\end{bmatrix}$.

- $P_i\vec{x} := \vec{x} - \langle \vec{q}_i, \vec{x}\rangle\, \vec{q}_i$ removes the $\vec{q}_i$-component of $\vec{x}$: *one* direction.
- $(I - Q_{j-1}Q_{j-1}^*)\vec{x} = \vec{x} - \sum_{i<j} \langle \vec{q}_i, \vec{x}\rangle\, \vec{q}_i$ removes the components along $\vec{q}_1, \ldots, \vec{q}_{j-1}$: *all* of them at once.

Then the two algorithms compute

$$\text{CGS:}\quad \vec{v}_j = (I - Q_{j-1}Q_{j-1}^*)\,\vec{a}_j, \qquad \text{all directions at once, using } \vec{a}_j, \tag{5.29}$$

$$\text{MGS:}\quad \vec{v}_j = P_{j-1}\cdots P_2P_1\,\vec{a}_j, \qquad \text{one direction at a time, using the current vector.} \tag{5.30}$$

If the $\vec{q}_i$ are exactly orthonormal, both give the same result.

Where they differ. Take $j = 3$ and apply P_1, then P_2:

$$P_1\vec{a}_3 = \vec{a}_3 - \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_1, \tag{5.31}$$

$$P_2P_1\vec{a}_3 = \vec{a}_3 - \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_1 - \langle \vec{q}_2, \vec{a}_3 - \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_1\rangle\, \vec{q}_2 \tag{5.32}$$

$$= \underbrace{\vec{a}_3 - \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_1 - \langle \vec{q}_2, \vec{a}_3\rangle\, \vec{q}_2}_{\text{CGS}} + \langle \vec{q}_2, \vec{q}_1\rangle\, \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_2. \tag{5.33}$$

So MGS equals CGS plus one extra term. That term is zero when $\vec{q}_1 \perp \vec{q}_2$. When rounding has made $\vec{q}_1$ and $\vec{q}_2$ slightly non-orthogonal, the extra term is exactly the correction that CGS misses.

Check on the example. In Example 24, $\langle \vec{q}_2, \vec{q}_1 \rangle = -\varepsilon/\sqrt{2}$ and $\langle \vec{q}_1, \vec{a}_3 \rangle = 1$, so the MGS vector should be the CGS vector plus $-\frac{\varepsilon}{\sqrt{2}}\vec{q}_2$:

$$\underbrace{\begin{bmatrix} 0 \\ -\varepsilon \\ 0 \\ \varepsilon \end{bmatrix}}_{\text{CGS}} - \frac{\varepsilon}{\sqrt{2}} \begin{bmatrix} 0 \\ -1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 \\ -\varepsilon/2 \\ -\varepsilon/2 \\ \varepsilon \end{bmatrix}}_{\text{MGS}}, \tag{5.34}$$

which matches Example 25.

5.8.2 How Much Orthogonality Is Lost?

The quantity $\|I - Q^*Q\|$ measures how far the computed Q is from having orthonormal columns. A result of Björck states:

- MGS: $\|I - Q^*Q\| \approx \varepsilon_{\text{mach}}\, \kappa(A)$;
- CGS: the loss can be as bad as $\varepsilon_{\text{mach}}\, \kappa(A)^2$.

The matrix of Example 24 has $\kappa(A) \approx \sqrt{3}/\varepsilon$. With $\varepsilon = 10^{-10}$, $\varepsilon_{\text{mach}}\kappa(A)^2 \approx 7 \cdot 10^4 \gg 1$, so CGS may lose orthogonality completely, while for MGS $\varepsilon_{\text{mach}}\kappa(A) \approx 4 \cdot 10^{-6}$. In Julia we measured $\|I - Q^*Q\| \approx 0.71$ for CGS (with $\langle \vec{q}_2, \vec{q}_3 \rangle = 0.5$) and $\approx 1.2 \cdot 10^{-10}$ for MGS.

5.8.3 Why Not Solve the Normal Equations?

Solving $A^*A\vec{x} = A^*\vec{b}$ looks most direct, but forming A^*A squares the condition number, $\kappa(A^*A) = \kappa(A)^2$ [1, Lecture 11]. For the matrix of Example 24,

$$A^*A = \begin{bmatrix} 1+\varepsilon^2 & 1 & 1 \\ 1 & 1+\varepsilon^2 & 1 \\ 1 & 1 & 1+\varepsilon^2 \end{bmatrix}, \tag{5.35}$$

and in floating point $1 + \varepsilon^2 = 1$. So the computed A^*A is the all-ones matrix, which has rank 1: the information is lost before we even start solving. QR works on A directly and avoids this [2, §3.3].

5.8.4 Householder QR and Reorthogonalization

Even with MGS, how orthogonal Q is depends on $\kappa(A)$. Libraries therefore use *Householder QR* (Lecture 6), whose Q is orthogonal to working precision whatever $\kappa(A)$ is. Another remedy is to *run CGS twice* ("twice is enough"); it parallelizes better than MGS, since the inner products of CGS form one matrix–vector product $Q_{j-1}^*\vec{a}_j$.

Lecture 6

Product View on Error Bounds, Householder QR

Supplementary reading:

- [1] Lectures 8, 10, and 16.
- [2] Section 3.4.

6.1 Why Write Algorithms as Matrix Products?

So far we described LU (Lecture 3) and Gram–Schmidt (Lecture 5) step by step, one row or one vector at a time, and we analyzed the error of LU entry by entry (Lecture 4). This has two drawbacks:

- the analysis has to be redone for every new algorithm;
- it does not show *why* some algorithms are stable and others are not.

In this lecture we write every factorization algorithm as a *product of matrices*: each step multiplies by one matrix. Then we only need to look at that matrix. It turns out that stability depends on one number: the condition number of the matrix used at each step.

6.2 The Product Form of Gram–Schmidt

The analogy with LU. LU *left*-multiplies by a sequence of lower triangular matrices in order to reach an *upper triangular* matrix:

$$L_{m-1} \cdots L_1 A = U. \tag{6.1}$$

Each L_k performs row operations that zero column k below the diagonal. Gram–Schmidt does the opposite: it *right*-multiplies by a sequence of upper triangular matrices in order to reach *orthonormal columns*:

$$A R_1 R_2 \cdots R_n = \hat{Q}. \tag{6.2}$$

Right multiplication performs column operations, and column operations are exactly what Gram–Schmidt does: normalize one column, then subtract its component from the other columns.

The step matrices. We first fix the notation.

- $\vec{a}_j^{(k-1)}$ is the j-th column after $k-1$ steps, i.e., $\vec{a}_j$ with its components along $\vec{q}_1, \dots, \vec{q}_{k-1}$ removed.
- $\vec{a}_j^{(0)} = \vec{a}_j$.

At the start of step k the matrix is

$$\begin{bmatrix} \vec{q}_1 & \cdots & \vec{q}_{k-1} & \vec{a}_k^{(k-1)} & \cdots & \vec{a}_n^{(k-1)} \end{bmatrix}. \tag{6.3}$$

- The first $k-1$ columns are orthonormal.
- The remaining columns are orthogonal to them, but not yet to each other.

Step k uses the numbers

$$r_{kk} \coloneqq \left\| \vec{a}_k^{(k-1)} \right\|, \qquad \vec{q}_k \coloneqq \vec{a}_k^{(k-1)} / r_{kk}, \qquad r_{kj} \coloneqq \left\langle \vec{q}_k, \vec{a}_j^{(k-1)} \right\rangle \quad (j > k). \tag{6.4}$$

The step matrix R_k equals the identity except in row k:

$$R_k = \begin{bmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & \frac{1}{r_{kk}} & -\frac{r_{k,k+1}}{r_{kk}} & \cdots & -\frac{r_{kn}}{r_{kk}} & \\ & & & 1 & & & \\ & & & & \ddots & & \\ & & & & & 1 & \end{bmatrix}. \tag{6.5}$$

Right multiplication by R_k performs step k:

$$\begin{bmatrix} \vec{q}_1 & \cdots & \vec{q}_{k-1} & \vec{a}_k^{(k-1)} & \cdots & \vec{a}_n^{(k-1)} \end{bmatrix} R_k = \begin{bmatrix} \vec{q}_1 & \cdots & \vec{q}_k & \vec{a}_{k+1}^{(k)} & \cdots & \vec{a}_n^{(k)} \end{bmatrix}. \tag{6.6}$$

To check this, look at the columns of the product.

- Columns $1, \dots, k-1$ are unchanged.
- Column k is
$$\vec{a}_k^{(k-1)} / r_{kk} = \vec{q}_k.$$
- Column $j > k$ is
$$\vec{a}_j^{(k-1)} - \frac{r_{kj}}{r_{kk}} \vec{a}_k^{(k-1)} = \vec{a}_j^{(k-1)} - r_{kj} \vec{q}_k = \vec{a}_j^{(k)}.$$

Note the factor $1/r_{kk}$ in the off-diagonal entries. It is there because the coefficient multiplies the *unnormalized* column $\vec{a}_k^{(k-1)}$, not $\vec{q}_k$. For example, for $k = 1$,

$$(R_1)_{1j} = -\frac{\langle \vec{q}_1, \vec{a}_j \rangle}{\|\vec{a}_1\|} = -\frac{\langle \vec{a}_1, \vec{a}_j \rangle}{\|\vec{a}_1\|^2}. \tag{6.7}$$

The product form is modified Gram–Schmidt. CGS and MGS differ only in which column the coefficient r_{kj} uses:

- CGS uses the original column: $r_{kj} = \langle \vec{q}_k, \vec{a}_j \rangle$.
- MGS uses the updated column: $r_{kj} = \left\langle \vec{q}_k, \vec{a}_j^{(k-1)} \right\rangle$.

The product form can only be MGS. R_k multiplies the *current* matrix, whose j-th column is already $\vec{a}_j^{(k-1)}$; the original $\vec{a}_j$ has been overwritten. So each step computes $\vec{q}_k$ and right away removes it from all later columns, which is exactly MGS.

The reduced QR factorization. Each R_k is upper triangular, and so is their product:

$$R_1 R_2 \cdots R_n = \hat{R}^{-1} \quad \Longrightarrow \quad A = \hat{Q}\hat{R}. \tag{6.8}$$

Here $\hat{Q} \in \mathbb{K}^{m \times n}$ is the factor of the *reduced QR factorization*: it has only n orthonormal columns, $\hat{Q}^* \hat{Q} = I_n$, but $\hat{Q}\hat{Q}^* \neq I_m$ (it is the orthogonal projector onto $\mathcal{R}(A)$). The factor $\hat{R} \in \mathbb{K}^{n \times n}$ is upper triangular and invertible.

Like R_k, the inverse R_k^{-1} differs from the identity only in row k, and that row has no reciprocals and no minus signs:

$$R_k^{-1} = \begin{bmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & r_{kk} & r_{k,k+1} & \cdots & r_{kn} & \\ & & & 1 & & & \\ & & & & \ddots & & \\ & & & & & 1 & \end{bmatrix}. \tag{6.9}$$

The same happens in LU, where L_k^{-1} is L_k with the minus signs removed.

Consequently

$$\hat{R} = R_n^{-1} \cdots R_1^{-1} = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & & & r_{nn} \end{bmatrix} \tag{6.10}$$

That is, row k of $\hat{R}$ is simply row k of R_k^{-1}. LU works the same way: in $L = L_1^{-1} \cdots L_{m-1}^{-1}$, column k of L is simply column k of L_k^{-1}, i.e., the multipliers of step k (Theorem 12).

6.3 Error Amplification in Products

Consider algorithms of the form

$$A_k = X_k A_{k-1}, \qquad A_0 = A. \tag{6.11}$$

LU ($X_k = L_k$) and Householder QR (X_k orthogonal, below) are of this type; Gram–Schmidt multiplies from the right and is analyzed symmetrically. Every step incurs rounding errors. How stable is such an algorithm?

Forward error of one step. One can show that in floating-point arithmetic

$$\mathrm{fl}(XA) = XA + E + O(\varepsilon_{\mathrm{mach}}^2), \qquad \text{for } \|E\|_2 \le O(\varepsilon_{\mathrm{mach}})\,\|X\|_2\,\|A\|_2\,. \tag{6.12}$$

Note that E is measured against $\|X\|\,\|A\|$, not $\|XA\|$. If XA is small because of cancellation, E can be large relative to the result. This is the same as in LU, where the error is governed by $|L|\,|U|$ rather than $|A|$.

The $O(\varepsilon_{\mathrm{mach}})$ hides a dimension-dependent constant, and $O(\varepsilon_{\mathrm{mach}}^2)$ means we only keep first-order terms.

Rewriting as a backward error. Thus, with $F := X^{-1}E$,

$$\mathrm{fl}(XA) = X(A+F) + O(\varepsilon_{\mathrm{mach}}^2). \tag{6.13}$$

So the computed result is the *exact* product with a perturbed input $A + F$.

The perturbation satisfies

$$\|F\|_2 = \|X^{-1}E\|_2 \le \|X^{-1}\|_2\|E\|_2 \le O(\varepsilon_{\mathrm{mach}})\,\kappa(X)\,\|A\|_2. \tag{6.14}$$

Dividing by $\|A\|_2$ gives the relative backward error:

$$\frac{\|F\|_2}{\|A\|_2} \le O(\varepsilon_{\mathrm{mach}})\,\kappa(X).$$

Theorem 26 (The Transformation Determines the Backward Error)

For one step $A \mapsto XA$ of a product-form algorithm with invertible X, the computed result equals $X(A+F)$ with

$$\frac{\|F\|_2}{\|A\|_2} \le O(\varepsilon_{\mathrm{mach}})\,\kappa(X). \tag{6.15}$$

Two condition numbers must be kept apart.

- $\kappa(X)$ belongs to the *algorithm*. It determines the backward error, i.e., whether the algorithm is stable, and the algorithm designer controls it.
- $\kappa(A)$ belongs to the *problem*. It is fixed by the input and turns backward error into forward error: forward error $\lesssim \kappa(A)\times$ backward error.

Design principle: choose transformations with small condition numbers. Orthogonal matrices have $\kappa = 1$, the best possible.

6.4 Lower Triangularization: The Case of LU

Apply the framework to LU. After k steps of elimination the accumulated transformation is $X = L_k \cdots L_1$, and its inverse is the unit lower triangular matrix of multipliers,

$$L = X^{-1} = \begin{bmatrix} 1 & & & \\ \ell_{21} & 1 & & \\ \vdots & \ddots & \ddots & \\ \ell_{m1} & \cdots & \ell_{m,m-1} & 1 \end{bmatrix}. \tag{6.16}$$

Since $\kappa(X) = \kappa(X^{-1}) = \kappa(L)$, Theorem 26 gives a backward error of about $O(\varepsilon_{\text{mach}})\kappa(L)\,\|A\|_2$. The stability of LU thus comes down to the condition number of L. Partial pivoting chooses the largest entry of each column as pivot and so guarantees $|\ell_{ij}| \le 1$.

$\|L\|$ is under control. Since $|\ell_{ij}| \le 1$,

$$\|L\|_2 \le \|L\|_F \le \sqrt{\frac{m(m+1)}{2}} \le m. \tag{6.17}$$

The middle step holds because L has m ones on the diagonal and $m(m-1)/2$ entries of absolute value at most 1 below it, so $\|L\|_F^2 \le m(m+1)/2$. Here $\|\cdot\|_F$ is the Frobenius norm (Section B.4).

The bound is loose. What matters is that it is polynomial in m, not exponential.

$\|L^{-1}\|$ is not. What about $\|L^{-1}\|_2$? To control $\kappa(L) = \|L\|_2 \|L^{-1}\|_2$, we also need $\|L^{-1}\|$ to be moderate. But $|\ell_{ij}| \le 1$ does *not* guarantee this.

The reason is forward substitution: each row uses the results of all previous rows. So values pile up, and the entries of L^{-1} can grow exponentially.

Example 27 (Exponential Growth of L^{-1})**.** The factor L of Example 20 has -1 everywhere below the diagonal, so $|\ell_{ij}| \le 1$, yet

$$L = \begin{bmatrix} 1 & & & \\ -1 & 1 & & \\ -1 & -1 & 1 & \\ -1 & -1 & -1 & 1 \end{bmatrix}, \qquad L^{-1} = \begin{bmatrix} 1 & & & \\ 1 & 1 & & \\ 2 & 1 & 1 & \\ 4 & 2 & 1 & 1 \end{bmatrix}. \tag{6.18}$$

For general m, $(L^{-1})_{ij} = 2^{i-j-1}$ for $i > j$, and the bottom-left entry is 2^{m-2}, so

$$\left\|L^{-1}\right\|_2 \ge 2^{m-2}, \qquad \kappa(L) \ge 2^{m-2}. \tag{6.19}$$

To see where the doubling comes from, note that computing L^{-1} means solving $L\vec{x} = \vec{b}$. By forward substitution, the i-th equation $x_i - x_1 - \cdots - x_{i-1} = b_i$ gives

$$x_i = b_i + (x_1 + \cdots + x_{i-1}). \tag{6.20}$$

For the first column, $\vec{b} = \vec{e}_1$: $x_1 = 1$, $x_2 = 1$, $x_3 = 1 + 1 = 2$, $x_4 = 1 + 1 + 2 = 4$, and so on. Each entry is the sum of all previous ones, so from the second entry on they double, $x_i = 2^{i-2}$. The other columns are the same pattern shifted down.

Large L^{-1} causes large U. From $A = LU$,

$$U = L^{-1}A, \tag{6.21}$$

so a large $\|L^{-1}\|$ can produce a large U: *large U comes from large L^{-1}*. This connects the condition-number viewpoint with the growth factor of Lectures 4 and 5; they describe the same thing. In Example 20 the last column of A is all ones, so the last column of U is L^{-1} times the all-ones vector, i.e., the row sums of L^{-1}:

$$1 + \sum_{j=1}^{i-1} 2^{i-j-1} = 1 + (2^{i-1} - 1) = 2^{i-1}, \tag{6.22}$$

which is exactly the last column $1, 2, 4, 8, \dots$ of U.

Partial pivoting thus solves only half of the problem: it controls $\|L\|$ but not $\|L^{-1}\|$, and in the worst case LU with partial pivoting can still be unstable. Such matrices are extremely rare, so in practice it is used as a stable algorithm; but to *guarantee* well-conditioned transformations we must switch to orthogonal ones.

6.5 Gram–Schmidt Is Also a Lower Triangularization

The upper triangular R_k is nontrivial only in row k, so its transpose $L_k^\top := R_k$ has L_k nontrivial only in column k: an elementary lower triangular matrix (with diagonal entry $1/r_{kk}$ instead of 1). Then $AR_1 \cdots R_n = \hat{Q}$ reads $AL_1^\top \cdots L_n^\top = \hat{Q}$, and transposing,

$$L_n \cdots L_1 A^\top = \hat{Q}^\top. \tag{6.23}$$

Gram–Schmidt on A is therefore the same as left-multiplying $A^\top$ by a sequence of lower triangular matrices. It too falls into the error-amplification framework, and its stability is likewise governed by the condition numbers of these triangular factors.

6.6 Householder Triangularization

Idea: multiply by orthogonal matrices. LU and Gram–Schmidt have the same defect: each step applies a triangular matrix, whose condition number is not under control. Following the design principle of Section 6.3, we make every step orthogonal instead.

We are looking for unitary matrices Q_k such that each step zeros one column below the diagonal:

$$\begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{Q_1} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{Q_2} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{Q_3} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}. \tag{6.24}$$

After n steps,

$$Q_n \cdots Q_1 A = R \quad \Longleftrightarrow \quad A = \underbrace{(Q_n \cdots Q_1)^{-1}}_{Q} R = Q_1^* \cdots Q_n^* R. \tag{6.25}$$

This is the *full QR factorization*: $Q \in \mathbb{K}^{m\times m}$ is a square unitary matrix, and $R \in \mathbb{K}^{m\times n}$ is upper triangular, with $m - n$ zero rows at the bottom. Taking the first n columns of Q and the first n rows of R recovers the reduced QR factorization.

Compare with the two earlier algorithms:

- like LU, Householder multiplies from the *left* and aims for an upper triangular matrix, but with unitary instead of triangular matrices;
- Gram–Schmidt uses triangular matrices to *produce* an orthogonal matrix; Householder uses orthogonal matrices to *produce* a triangular one.

How do we choose the Q_k?

The Householder reflector. Let us begin with the first step. Q_1 must map the first column $\vec{a}_1$ to a multiple of $\vec{e}_1$. Unitary maps preserve the 2-norm, so there are only two possible targets:

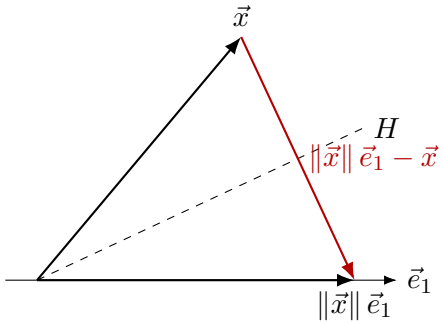
$$Q_1 \vec{a}_1 = \pm \|\vec{a}_1\| \, \vec{e}_1. \tag{6.26}$$

Take the $+$ sign first.

Idea: use a reflection. Since $\vec{a}_1$ and $\|\vec{a}_1\| \, \vec{e}_1$ have the same length, they are mirror images of each other. The mirror is the hyperplane through the origin with normal vector

$$\vec{v} \coloneqq \vec{a}_1 - \|\vec{a}_1\| \, \vec{e}_1. \tag{6.27}$$

Reflecting across it sends $\vec{a}_1$ to $\|\vec{a}_1\| \, \vec{e}_1$.



Definition 28 (Householder Reflector)

For a nonzero $\vec{v} \in \mathbb{K}^m$, the *Householder reflector* is

$$F = I - 2\frac{\vec{v}\vec{v}^*}{\vec{v}^*\vec{v}}. \tag{6.28}$$

F flips the component along $\vec{v}$ and keeps the rest:

- $\vec{u} \perp \vec{v} \implies F\vec{u} = \vec{u}$: vectors in the hyperplane stay fixed;
- $\vec{u} \parallel \vec{v} \implies F\vec{u} = -\vec{u}$: the normal direction is flipped.

F is Hermitian, and reflecting twice changes nothing, so $F^2 = I$. Hence $F^*F = F^2 = I$: F is unitary and is its own inverse.

We now check that $Q_1 = I - 2\vec{v}\vec{v}^*/(\vec{v}^*\vec{v})$ does what we want (real case). First,

$$\vec{v}^*\vec{v} = 2\|\vec{a}_1\|^2 - 2\|\vec{a}_1\| a_{11}, \qquad \vec{v}^*\vec{a}_1 = \|\vec{a}_1\|^2 - \|\vec{a}_1\| a_{11} = \tfrac{1}{2}\vec{v}^*\vec{v}, \tag{6.29}$$

so

$$Q_1\vec{a}_1 = \vec{a}_1 - 2\vec{v}\frac{\vec{v}^*\vec{a}_1}{\vec{v}^*\vec{v}} = \vec{a}_1 - \vec{v} = \|\vec{a}_1\|\vec{e}_1. \tag{6.30}$$

Choosing the sign. There are two possible reflections, onto $+\|\vec{a}_1\|\vec{e}_1$ and onto $-\|\vec{a}_1\|\vec{e}_1$. Which one should we choose? Mathematically, there is no preference. In finite precision, we choose the target *farther from* $\vec{a}_1$:

$$Q_1\vec{a}_1 = -\operatorname{sign}(a_{11})\|\vec{a}_1\|\vec{e}_1 \quad \Longrightarrow \quad \vec{v} \propto \operatorname{sign}(a_{11})\|\vec{a}_1\|\vec{e}_1 + \vec{a}_1. \tag{6.31}$$

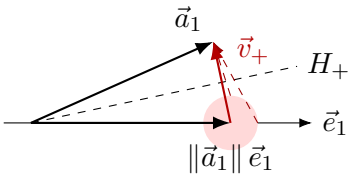
We take $\operatorname{sign}(0) = 1$.

Why? Suppose $\vec{a}_1$ is nearly parallel to $\vec{e}_1$, with $a_{11} > 0$. With the $+$ sign,

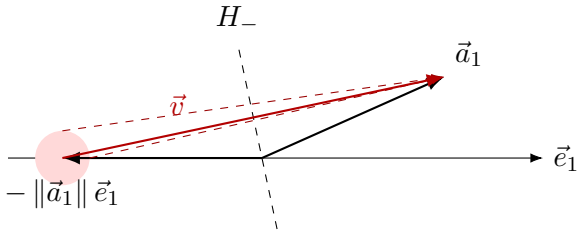
$$\|\vec{v}_+\| = \|\|\vec{a}_1\|\vec{e}_1 - \vec{a}_1\| \ll \|\vec{a}_1\|. \tag{6.32}$$

So $\vec{v}_+$ is the difference of two nearly equal vectors, and its first entry $\|\vec{a}_1\| - a_{11}$ suffers cancellation. The computed $\|\vec{a}_1\|$ has an absolute error of about $\varepsilon_{\text{mach}}\|\vec{a}_1\|$, which is large compared with the short $\vec{v}_+$. So the direction of $\vec{v}_+$, and with it the reflection, is inaccurate.

With $-\operatorname{sign}(a_{11})$, the first entry of $\vec{v}$ is $a_{11} + \operatorname{sign}(a_{11})\|\vec{a}_1\|$, a sum of two numbers of the same sign. There is no cancellation, and $|v_1| \geq \|\vec{a}_1\|$, so $\vec{v}$ is never short.



Reflecting onto $+\|\vec{a}_1\|\vec{e}_1$: $\vec{v}_+$ is short, so its direction is unreliable.



Reflecting onto $-\|\vec{a}_1\|\vec{e}_1$: $\vec{v}$ is long, so its direction is reliable.

Step k: the block form. What about Q_k for $k > 1$? The first $k-1$ rows and columns are finished and must not change. So we use

$$Q_k = \begin{bmatrix} I_{k-1} & 0 \\ 0 & F_k \end{bmatrix}, \tag{6.33}$$

where F_k is the $(m-k+1)\times(m-k+1)$ Householder reflector built as in step 1 from $\vec{x} = A[k{:}m, k]$, the first column of the trailing submatrix $A[k{:}m, k{:}n]$.

Earlier results are preserved:

- the block I_{k-1} leaves the first $k-1$ rows untouched;
- the first $k-1$ columns are already zero from row k down, and F_k maps zero to zero.

Q_k is itself a Householder reflector (with $\vec{v}$ padded by $k-1$ leading zeros), so it is unitary.

Algorithm 9 Householder QR, in place.

for $k = 1, \dots, n$ **do**
  $\vec{x} \leftarrow A[k{:}m, k]$
  $\vec{v} \leftarrow \operatorname{sign}(x_1) \|\vec{x}\|_2 \vec{e}_1 + \vec{x}$
  $\vec{v} \leftarrow \vec{v}/v_1$ ▷ scale so that $v_1 = 1$
  $A[k{:}m, k{:}n] \leftarrow A[k{:}m, k{:}n] - 2\vec{v}\left(\vec{v}^* A[k{:}m, k{:}n]\right)/(\vec{v}^*\vec{v})$
  $A[k{+}1{:}m, k] \leftarrow \vec{v}[2{:}\text{end}]$ ▷ store $\vec{v}$ below the diagonal
return A ▷ upper triangle: R; strictly lower triangle: the vectors $\vec{v}$

Some remarks on the implementation (see also [1, Lecture 10]):

- *F_k is never formed.* Following the parentheses, first compute the row vector $\vec{v}^* A[k{:}m, k{:}n]$, then perform a rank-one update. Each step costs $O((m-k)(n-k))$; forming $\vec{v}\vec{v}^*$ first would cost an order of magnitude more.
- *Scaling $\vec{v}$ by $1/v_1$* makes its first entry 1. The reflector depends only on the direction of $\vec{v}$, so scaling does not change F_k. Since $|v_1| \geq \|\vec{x}\| > 0$ (for $\vec{x} \neq \vec{0}$), this never divides by zero.
- *Storage.* After the update, $A[k{+}1{:}m, k]$ should be zero, which frees exactly enough room for the remaining $m-k$ entries of $\vec{v}$; the leading $v_1 = 1$ is implicit.
- On exit the upper triangle of A is R, with diagonal entries $r_{kk} = -\operatorname{sign}(x_1)\|\vec{x}\|$. The matrix Q is never formed explicitly; it is stored implicitly through the vectors $\vec{v}$ (see Lecture 7).
- $\operatorname{sign}(0)$ must be taken as 1. Otherwise $x_1 = 0$ gives $\vec{v} = \vec{x}$, and the reflection maps $\vec{x}$ to $-\vec{x}$ instead of a multiple of $\vec{e}_1$.

Back into the framework. Each $X_k = Q_k$ is unitary, so $\|Q_k\|_2 = \left\|Q_k^{-1}\right\|_2 = 1$ and $\kappa(Q_k) = 1$ (Appendix B). By Theorem 26,

$$\|F\|_2 \leq O(\varepsilon_{\text{mach}})\,\kappa(Q_k)\,\|A\|_2 = O(\varepsilon_{\text{mach}})\,\|A\|_2\,. \tag{6.34}$$

- The transformation does not amplify the error at all.
- Unitary matrices preserve the 2-norm, so every intermediate A_k has norm $\|A\|_2$. There is no element growth as in LU.
- Over all steps, the backward error grows only linearly with the number of steps (Section 6.7). So Householder QR is backward stable.

This presumes that each Q_k is computed accurately, i.e., that the direction of $\vec{v}$ is accurate. This is exactly what the sign choice ensures.

6.7 (SUPPLEMENT) Further Topics

6.7.1 Three Factorizations as Products

	Side	Transformation	Target	Condition of transformation
LU	left	lower triangular L_k	upper triangular U	can be large
Gram–Schmidt	right	upper triangular R_k	orthonormal $\hat{Q}$	depends on A
Householder	left	unitary Q_k	upper triangular R	1

Trefethen and Bau call Gram–Schmidt *triangular orthogonalization* and Householder *orthogonal triangularization* [1, Lectures 8 and 10].

6.7.2 The Condition of the Gram–Schmidt Transformation

The accumulated transformation of Gram–Schmidt is

$$R_1 \cdots R_n = \hat{R}^{-1}, \qquad \text{so} \qquad \kappa(R_1 \cdots R_n) = \kappa(\hat{R}).$$

This equals $\kappa(A)$. Since $A = \hat{Q}\hat{R}$ and $\hat{Q}$ has orthonormal columns,

$$\|A\vec{x}\|_2 = \left\|\hat{Q}\hat{R}\vec{x}\right\|_2 = \left\|\hat{R}\vec{x}\right\|_2 \quad \text{for all } \vec{x},$$

so A and $\hat{R}$ stretch every vector by the same factor. Their largest and smallest stretches agree, and hence $\kappa(\hat{R}) = \kappa(A)$.

So the triangular transformations of Gram–Schmidt are exactly as ill-conditioned as the problem: the more ill-conditioned A, the more the error is amplified. This matches the loss of orthogonality $\approx \varepsilon_{\text{mach}}\kappa(A)$ of MGS in Lecture 5.

6.7.3 Backward Error Over Many Steps

Theorem 26 covers one step. For many steps, write step k as $\tilde{A}_k = X_k(\tilde{A}_{k-1} + F_k)$. After two steps,

$$\tilde{A}_2 = X_2(X_1(A + F_1) + F_2) = X_2X_1(A + F_1 + X_1^{-1}F_2).$$

Continuing in the same way, every perturbation is pulled back to the original input:

$$\tilde{A}_n = X_n \cdots X_1 \left(A + \sum_{k=1}^{n} (X_{k-1} \cdots X_1)^{-1} F_k \right) + O(\varepsilon_{\text{mach}}^2). \tag{6.35}$$

For general X_k, the factors $(X_{k-1} \cdots X_1)^{-1}$ can amplify the F_k. When every X_k is unitary:

- $(X_{k-1} \cdots X_1)^{-1}$ is unitary, so it does not change the norm of F_k;
- $\left\|\tilde{A}_{k-1}\right\|_2 = \|A\|_2$ to first order, so $\|F_k\|_2 \le O(\varepsilon_{\text{mach}}) \|A\|_2$.

Adding up the n terms,

$$\|F_{\text{total}}\|_2 \lesssim O(n\varepsilon_{\text{mach}}) \|A\|_2 . \tag{6.36}$$

This is the argument behind Demmel's Lemma 3.1 and Theorem 3.5 [2, §3.4.3]. Trefethen and Bau state the result as $\tilde{Q}\tilde{R} = A + \delta A$ with $\|\delta A\| / \|A\| = O(\varepsilon_{\text{mach}})$ [1, Theorem 16.1], where $\tilde{Q}$ is the *exactly* unitary product of the reflectors defined by the computed vectors $\tilde{\vec{v}}_k$.

6.7.4 Orthogonality of Q

We measure the loss of orthogonality by $\|Q^*Q - I\|_2$.

- MGS computes $\hat{Q}$ by triangular column operations on A. Their condition is $\kappa(A)$, so $\left\|\hat{Q}^*\hat{Q} - I\right\|_2 \approx \varepsilon_{\text{mach}}\kappa(A)$.
- Householder never builds Q from A. Q is a product of reflectors, each unitary up to rounding in $\vec{v}_k$, so $\|Q^*Q - I\|_2 = O(\varepsilon_{\text{mach}})$, independently of $\kappa(A)$.

When an orthonormal basis of $\mathcal{R}(A)$ is needed explicitly, Householder is the safer choice.

6.7.5 Operation Counts

Step k of Algorithm 9 updates the $(m-k+1)\times(n-k+1)$ block $A[k{:}m, k{:}n]$. For each column $\vec{a}$ of this block, the inner product $\vec{v}^*\vec{a}$ and the update $\vec{a} - (2\vec{v}^*\vec{a}/\vec{v}^*\vec{v})\,\vec{v}$ each cost about $2(m-k+1)$ flops. So step k costs about $4(m-k+1)(n-k+1)$ flops, and in total

$$\sum_{k=1}^{n} 4(m-k+1)(n-k+1) \approx 4\int_0^n (m-t)(n-t)\,\mathrm{d}t = 2mn^2 - \frac{2}{3}n^3. \tag{6.37}$$

- For $m \gg n$ this is about $2mn^2$, the same as MGS.
- For $m = n$ it is $\frac{4}{3}n^3$, twice as much as LU ($\frac{2}{3}n^3$). This is the price of stability.

6.7.6 Using Q Implicitly for Least Squares

To solve $\min_{\vec{x}} \left\|A\vec{x} - \vec{b}\right\|_2$ we never form Q. Instead we apply the stored reflectors to $\vec{b}$ one by one, exactly as in step 5 of Algorithm 9:

$$\vec{c} \coloneqq Q^*\vec{b} = Q_n \cdots Q_1\vec{b}, \qquad \text{cost } O(mn).$$

Split $\vec{c}$ into $\vec{c}_1 = \vec{c}[1{:}n]$ and $\vec{c}_2 = \vec{c}[n{+}1{:}m]$. Since Q^* preserves the 2-norm,

$$\left\|A\vec{x} - \vec{b}\right\|_2^2 = \|R\vec{x} - \vec{c}\|_2^2 = \left\|\hat{R}\vec{x} - \vec{c}_1\right\|_2^2 + \|\vec{c}_2\|_2^2\,.$$

The first term vanishes when $\hat{R}\vec{x} = \vec{c}_1$, which we solve by back substitution. The second term does not depend on $\vec{x}$, so $\|\vec{c}_2\|_2$ is the minimal residual. Details are in Lecture 7.

6.7.7 The Complex Case

For complex data the real sign does not work: with the target $+\|\vec{x}\|\,\vec{e}_1$, the number $\vec{v}^*\vec{x}$ is in general not real, and the verification of $F\vec{x} = \|\vec{x}\|\,\vec{e}_1$ fails. Instead we take

$$\operatorname{sign}(x_1) \coloneqq x_1/\,|x_1| \quad (\text{and } 1 \text{ if } x_1 = 0), \qquad \vec{v} = \operatorname{sign}(x_1)\,\|\vec{x}\|\,\vec{e}_1 + \vec{x}.$$

- The first entry $v_1 = \operatorname{sign}(x_1)(\,\|\vec{x}\| + |x_1|\,)$ adds two numbers with the same phase, so there is again no cancellation.
- Since $\overline{\operatorname{sign}(x_1)}\,x_1 = |x_1|$,

$$\vec{v}^*\vec{x} = \|\vec{x}\|^2 + \|\vec{x}\|\,|x_1|\,, \qquad \vec{v}^*\vec{v} = 2\,\|\vec{x}\|^2 + 2\,\|\vec{x}\|\,|x_1| = 2\,\vec{v}^*\vec{x}.$$

- Hence, as in the real case, $F\vec{x} = \vec{x} - \vec{v} = -\operatorname{sign}(x_1)\,\|\vec{x}\|\,\vec{e}_1$.

Lecture 7

Modified Gram–Schmidt

Supplementary reading:

- [1] Lectures 7 and 10.
- [2] Sections 3.2.2 and 3.4.

7.1 Picking Up from Householder QR

At the end of Householder QR (Algorithm 9), the upper triangle of A holds R and the strictly lower triangle holds the Householder vectors $\vec{v}_k$; Q is never computed explicitly. This raises two questions:

- Applications need $Q^*\vec{b}$ (for least squares) or $Q\vec{x}$. How do we compute them from the $\vec{v}_k$ alone?
- Every Householder step processes a whole column. If A is already close to upper triangular and only a few entries need to be eliminated, is this wasteful?

The first leads to the implicit use of Q, the second to Givens rotations. Finally we return to theory: does a QR factorization always exist, and is it unique?

7.2 Using Q

Implicit storage. Householder QR does not store Q as a matrix, only the vectors $\vec{v}_k$ defining the reflectors

$$Q_k = I - 2\frac{\vec{v}_k\vec{v}_k^*}{\vec{v}_k^*\vec{v}_k}. \tag{7.1}$$

Here $\vec{v}_k$ is regarded as a vector of length m whose first $k-1$ entries are zero, so Q_k acts only on rows k through m; this is the same as the block form $Q_k = \operatorname{diag}(I_{k-1}, F_k)$ of Lecture 6. From $Q_n \cdots Q_1 A = R$, and since each Q_k is Hermitian and its own inverse ($Q_k^* = Q_k = Q_k^{-1}$),

$$Q = (Q_n \cdots Q_1)^{-1} = Q_1 Q_2 \cdots Q_n, \qquad Q^* = Q_n \cdots Q_1. \tag{7.2}$$

Two algorithms. In $Q^*\vec{b} = Q_n \cdots Q_1 \vec{b}$ the first factor to act is Q_1, so k runs upward from 1; in $Q\vec{x} = Q_1 \cdots Q_n \vec{x}$ the first factor to act is Q_n, so k runs downward from n.

Algorithm 10 Implicit computation of $Q^*\vec{b}$.

1: **for** $k = 1, \ldots, n$ **do**
2: $\quad \vec{b}[k{:}m] \leftarrow \vec{b}[k{:}m] - 2\vec{v}_k \left(\vec{v}_k^* \vec{b}[k{:}m]\right) / (\vec{v}_k^* \vec{v}_k)$

Algorithm 11 Implicit computation of $Q\vec{x}$.

1: **for** $k = n, n-1, \ldots, 1$ **do**
2: $\quad \vec{x}[k{:}m] \leftarrow \vec{x}[k{:}m] - 2\vec{v}_k \left(\vec{v}_k^* \vec{x}[k{:}m]\right) / (\vec{v}_k^* \vec{v}_k)$

As in Lecture 6, following the parentheses we first compute the scalar $\vec{v}_k^* \vec{b}[k{:}m]$ and then update a vector, so each step costs only $O(m-k)$ and the total is about $4mn - 2n^2$ flops, the same order as a matrix–vector product. One should never form $\vec{v}_k \vec{v}_k^*$. The two algorithms differ only in the direction of the loop, much like forward and back substitution: the order of the factors in the product determines the order in which they act.

Consequences of implicit storage.

In place. Scaling $\vec{v}_k$ so that its k-th entry is 1 makes that entry implicit, and the remaining $m-k$ entries fit into the positions below the diagonal of column k that were just zeroed. The whole factorization needs no extra matrix storage and simply overwrites A.

A full QR factorization from only n vectors. Q is $m \times m$, so storing it explicitly takes m^2 numbers; the implicit form takes only the roughly $mn - \frac{n^2}{2}$ positions below the diagonal of A. When $m \gg n$ the difference is large. If the reduced factor $\hat{Q}$ (the first n columns of Q) is really needed, compute $Q\vec{e}_1, \ldots, Q\vec{e}_n$ with Algorithm 11.

Q is orthogonal by definition. We stress this point. However much rounding error the computed $\vec{v}_k$ carry, $I - 2\vec{v}_k\vec{v}_k^*/(\vec{v}_k^*\vec{v}_k)$ is an exact reflector for *every* nonzero $\vec{v}_k$, hence exactly unitary. Rounding errors only affect *which* orthogonal matrix Q is, not *whether* it is orthogonal; they are absorbed into a backward perturbation of A. Contrast this with Gram–Schmidt, which computes every column of $\hat{Q}$ explicitly, so rounding errors directly destroy the orthogonality between columns, by about $\varepsilon_{\text{mach}}\kappa(A)$ for MGS and $\varepsilon_{\text{mach}}\kappa(A)^2$ for CGS. When the Householder Q is used in a computation (for example applied to $\vec{b}$ as above), the only error comes from applying exact reflectors in floating point, and this step is backward stable.

7.3 Givens Rotations

Motivation: Householder can be overkill. A Householder reflector processes a whole column at once: step k performs a rank-one update of the whole block $A[k{:}m, k{:}n]$, no matter how many nonzeros lie below the diagonal in column k. If the matrix is already *close to upper triangular*, with only a narrow band of nonzeros below the diagonal (for example an upper Hessenberg matrix, which has a single nonzero subdiagonal), each column needs only one or two entries eliminated and Householder is wasteful. We want an orthogonal transformation that zeros *one entry at a time*.

The 2×2 case. Consider the leading 2×2 block

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}. \tag{7.3}$$

We want to eliminate A_{21}. Geometrically, the first column $\vec{a}_1 = (A_{11}, A_{21})^\top$ is a vector in the plane, and it suffices to *rotate* the plane so that $\vec{a}_1$ lands on the horizontal axis; $\vec{a}_2$ rotates along with it.

Definition 29 (Givens Rotation)

With $\|\vec{a}_1\| = \sqrt{A_{11}^2 + A_{21}^2}$, the *Givens rotation* for $\vec{a}_1$ is

$$G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}, \qquad c = \frac{A_{11}}{\|\vec{a}_1\|}, \qquad s = \frac{A_{21}}{\|\vec{a}_1\|}. \tag{7.4}$$

A direct check gives

$$G \begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix} = \frac{1}{\|\vec{a}_1\|} \begin{bmatrix} A_{11}^2 + A_{21}^2 \\ -A_{21}A_{11} + A_{11}A_{21} \end{bmatrix} = \begin{bmatrix} \|\vec{a}_1\| \\ 0 \end{bmatrix}. \tag{7.5}$$

Since $c^2 + s^2 = 1$, we have $G^\top G = I$ and $\det G = 1$, so G is a rotation: if $\vec{a}_1$ makes the angle θ with the horizontal axis, then $c = \cos\theta$, $s = \sin\theta$, and G rotates clockwise by θ. Compare with Householder: a Householder reflector is a *reflection* ($\det = -1$) that maps a whole vector onto the direction of $\vec{e}_1$ at once, whereas a Givens rotation is a *rotation* ($\det = 1$) acting in a single two-dimensional plane and zeroing one entry.

QR by Givens rotations. In the $m \times m$ setting, the Givens rotation acting on rows i and j is the identity except in four positions:

$$G(i,j) = \begin{bmatrix} 1 & & & & \\ & c & & s & \\ & & \ddots & & \\ & -s & & c & \\ & & & & 1 \end{bmatrix}, \tag{7.6}$$

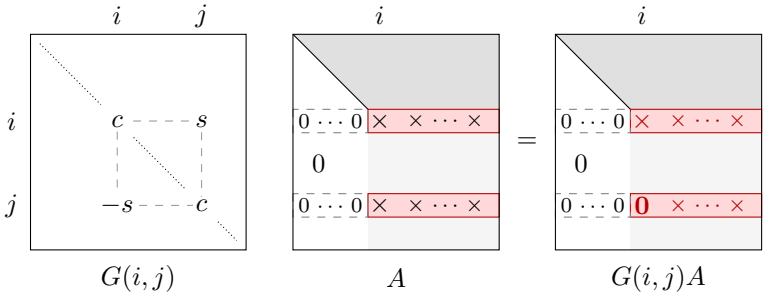
with c in positions (i,i) and (j,j), s in (i,j), and $-s$ in (j,i). Left multiplication by $G(i,j)$ changes only rows i and j: writing $\vec{r}_i^\top$ for the i-th row of A,

$$\vec{r}_i^\top \leftarrow c\,\vec{r}_i^\top + s\,\vec{r}_j^\top, \qquad \vec{r}_j^\top \leftarrow -s\,\vec{r}_i^\top + c\,\vec{r}_j^\top, \tag{7.7}$$

and every other row is left alone. To eliminate A_{ji} ($j > i$), take

$$c = \frac{A_{ii}}{\sqrt{A_{ii}^2 + A_{ji}^2}}, \qquad s = \frac{A_{ji}}{\sqrt{A_{ii}^2 + A_{ji}^2}}, \tag{7.8}$$

which is the 2×2 case transplanted to rows i and j. Suppose the first $i-1$ columns have already been reduced. Then one step gives the following picture.



The dark gray rows are finished rows of R, and the light gray block is still to be reduced. Only rows i and j (outlined) change; in the first $i-1$ columns both are zero, and in column i the entry A_{ji} is rotated into A_{ii}.

We see that existing zeros are preserved: in the first $i-1$ columns rows i and j are both zero, and a linear combination of two zeros is zero. In column i itself, row i becomes $\sqrt{A_{ii}^2 + A_{ji}^2}$, row j becomes 0, and no other row changes. So we may sweep through the subdiagonal nonzeros column by column, left to right, and within column i pair row i with $j = i+1, \dots, m$ in turn. For a 4×3 matrix this takes six rotations, one per subdiagonal entry. Below, $\times$ is a generic nonzero, red marks the entries that the current rotation changes, and the bold **0** is the entry it eliminates.

$$A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{G(1,2)} \begin{bmatrix} \times & \times & \times \\ \mathbf{0} & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{G(1,3)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ \mathbf{0} & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{G(1,4)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ \mathbf{0} & \times & \times \end{bmatrix}$$

$$\xrightarrow{G(2,3)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \mathbf{0} & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G(2,4)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & \mathbf{0} & \times \end{bmatrix} \xrightarrow{G(3,4)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \mathbf{0} \end{bmatrix} = R.$$

Notice that $G(1,3)$ does not touch row 2, so the zero created by $G(1,2)$ survives, and from $G(2,3)$ on, the zeros in column 1 are carried along as combinations of zeros. In general, labeling the rotations in the order they are applied gives

$$G_N \cdots G_1 A = R, \qquad Q = G_1^* \cdots G_N^*, \tag{7.9}$$

where N is the number of subdiagonal nonzeros; for a dense $m \times n$ matrix, $N = \sum_{i=1}^{n}(m-i) = mn - \frac{n(n+1)}{2}$. As with Householder, Q is not formed explicitly; one stores the pairs (c, s) of the rotations and applies them in sequence when needed, just as in Algorithms 10 and 11. Every G is orthogonal with $\kappa = 1$, so by the error-amplification framework of Lecture 6, Givens QR is backward stable as well.

Efficiency.

Dense matrices: slower than Householder. Each rotation acts on two rows, at 6 flops per column (4 multiplications and 2 additions). Eliminating column i takes $m-i$ rotations, each acting on rows of length about $n-i$, for a total of about

$$\sum_{i=1}^{n} 6(m-i)(n-i) \approx 3mn^2 - n^3 \text{ flops}, \tag{7.10}$$

about 1.5 times the cost of Householder ($2mn^2 - \frac{2}{3}n^3$).

Sparse or nearly triangular matrices: possibly faster. For an $n \times n$ upper Hessenberg matrix, only $n-1$ rotations are needed (one per subdiagonal entry), each costing $O(n)$, for a total of about $3n^2$ flops. Givens touches only the entries that must be eliminated and so exploits sparsity naturally.

7.4 Full and Reduced QR

Let $A \in \mathbb{K}^{m \times n}$ with $m \geq n$.

Definition 30 (Reduced and Full QR Factorizations)

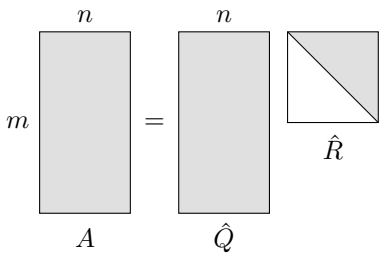
A *reduced QR factorization* of A is

$$A = \hat{Q}\hat{R}, \qquad \hat{Q} \in \mathbb{K}^{m\times n} \text{ with orthonormal columns}, \quad \hat{R} \in \mathbb{K}^{n\times n} \text{ upper triangular.} \tag{7.11}$$

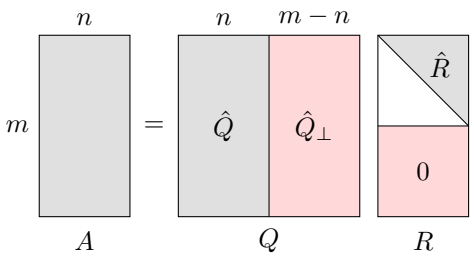
A *full QR factorization* of A is

$$A = QR, \qquad Q \in \mathbb{K}^{m\times m} \text{ unitary}, \quad R = \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} \in \mathbb{K}^{m\times n}, \ \hat{R} \text{ upper triangular.} \tag{7.12}$$

Gram–Schmidt produces a reduced factorization; Householder and Givens produce a full one. Drawing each factor to scale, this gives the following picture.



Reduced QR (Gram–Schmidt): $\hat{Q}$ has the same shape as A, and $\hat{R}$ is $n \times n$.



Full QR (Householder, Givens): Q is $m \times m$, and R is $\hat{R}$ padded with $m - n$ zero rows.

Splitting $Q = \begin{bmatrix} \hat{Q} & \hat{Q}_\perp \end{bmatrix}$ by columns,

$$QR = \begin{bmatrix} \hat{Q} & \hat{Q}_\perp \end{bmatrix} \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} = \hat{Q}\hat{R}. \tag{7.13}$$

In the picture, the extra $m - n$ columns $\hat{Q}_\perp$ (red) multiply only the zero block of R (also red) and do not contribute to the product. Their role is to complete $\hat{Q}$ to an orthonormal basis of $\mathbb{K}^m$; when A has full rank, the columns of $\hat{Q}_\perp$ span $\mathcal{R}(A)^\perp$.

7.5 Existence

Theorem 31 (Existence of QR)

Every $A \in \mathbb{K}^{m\times n}$ with $m \geq n$ has a full QR factorization and a reduced QR factorization.

The proof is constructive: run Gram–Schmidt, see where it can fail, and patch those cases.

Proof. A of full rank, reduced QR. The only place where Gram–Schmidt can fail is the normalization $\vec{q}_k = \hat{\vec{q}}_k / \left\|\hat{\vec{q}}_k\right\|$, which divides by zero if $\hat{\vec{q}}_k = \vec{0}$. But

$$\left\|\hat{\vec{q}}_k\right\| = 0 \iff \vec{a}_k \in \operatorname{span}(\vec{q}_1, \ldots, \vec{q}_{k-1}) = \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_{k-1}), \tag{7.14}$$

which contradicts full column rank. So Gram–Schmidt runs to completion, and all $r_{kk} = \left\|\hat{\vec{q}}_k\right\| > 0$.

A rank deficient, reduced QR. Run Gram–Schmidt as usual. Whenever $\hat{\vec{q}}_k = \vec{0}$, set $r_{kk} = 0$ and take for $\vec{q}_k$ any unit vector orthogonal to $\vec{q}_1, \dots, \vec{q}_{k-1}$; such a vector exists because $k - 1 < n \le m$. Then still

$$\vec{a}_k = \sum_{i<k} r_{ik}\vec{q}_i + 0 \cdot \vec{q}_k, \tag{7.15}$$

so $A = \hat{Q}\hat{R}$ holds, only with zeros on the diagonal of $\hat{R}$.

Full QR. Compute a reduced QR factorization, extend $\vec{q}_1, \dots, \vec{q}_n$ to an orthonormal basis of $\mathbb{K}^m$ by adding $m - n$ orthonormal vectors, and append $m - n$ zero rows to $\hat{R}$. □

7.6 Uniqueness

When QR is not unique.

Full QR with $m > n$ is not unique. $\hat{Q}_\perp$ can be any orthonormal basis of $\mathcal{R}(\hat{Q})^\perp$, because it multiplies a zero block. For instance, replacing $\hat{Q}_\perp$ by $\hat{Q}_\perp U$ with any unitary $U \in \mathbb{K}^{(m-n)\times(m-n)}$ leaves the product unchanged.

Rank-deficient A. When $\hat{\vec{q}}_k = \vec{0}$, the choice of $\vec{q}_k$ is arbitrary.

Example 32 (Non-Uniqueness for a Rank-Deficient Matrix)**.** Let

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \qquad \vec{q}_1 = \vec{e}_1, \qquad \hat{\vec{q}}_2 = \vec{a}_2 - \vec{e}_1 = \vec{0}. \tag{7.16}$$

Both $\vec{q}_2 = \vec{e}_2$ and $\vec{q}_2 = \vec{e}_3$ work, with the same $\hat{R} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$.

Phases. Even for full-rank A, given $|d_k| = 1$ we may multiply $\vec{q}_k$ by d_k and row k of $\hat{R}$ by $\bar{d}_k$ without changing the product; in the real case, each column may flip its sign. Uniqueness therefore requires a normalization.

Theorem 33 (Uniqueness of Reduced QR)

Let $A \in \mathbb{K}^{m\times n}$, $m \ge n$, have full column rank. Then the reduced QR $r_{jj} > 0$ for all j is unique.

Uniqueness of the reduced QR factorization.

Proof. Let $A = \hat{Q}\hat{R} = \tilde{Q}\tilde{R}$ be two such factorizations. We show $\tilde{\vec{q}}_k = \vec{q}_k$ column by column. Fix k, write $\vec{q} = \vec{q}_k$ and $\tilde{\vec{q}} = \tilde{\vec{q}}_k$, and use the inner product $\langle \vec{x}, \vec{y} \rangle = \vec{x}^* \vec{y}$.

First, what conditions do $\vec{q}$ and $\tilde{\vec{q}}$ satisfy? Full rank gives $r_{jj} \neq 0$, so $\hat{R}$ is upper triangular and invertible, and the first k columns of $\hat{Q} = A\hat{R}^{-1}$ span the same space as the first k columns of A:

$$\operatorname{span}(\vec{q}_1, \dots, \vec{q}_k) = \operatorname{span}(\vec{a}_1, \dots, \vec{a}_k) \qquad \text{for every } k. \tag{7.17}$$

Hence $\vec{q} \in \operatorname{span}(\vec{a}_1, \dots, \vec{a}_k)$ and $\vec{q}$ is orthogonal to $\vec{q}_1, \dots, \vec{q}_{k-1}$, hence to $\vec{a}_1, \dots, \vec{a}_{k-1}$. The same holds for $\tilde{\vec{q}}$.

Since $\mathrm{span}(\vec{a}_1,\dots,\vec{a}_k) = \mathrm{span}(\vec{a}_1,\dots,\vec{a}_{k-1},\vec{q})$, we can write

$$\tilde{\vec{q}} = \alpha\vec{q} + \sum_{j=1}^{k-1} \beta_j \vec{a}_j. \tag{7.18}$$

Taking inner products with $\vec{q}$ and with $\tilde{\vec{q}}$, and using $\vec{q} \perp \vec{a}_j$ and $\tilde{\vec{q}} \perp \vec{a}_j$ for $j < k$,

$$\left\langle \vec{q}, \tilde{\vec{q}} \right\rangle = \alpha, \qquad 1 = \left\langle \tilde{\vec{q}}, \tilde{\vec{q}} \right\rangle = \alpha \left\langle \tilde{\vec{q}}, \vec{q} \right\rangle = \alpha\bar{\alpha} = |\alpha|^2 . \tag{7.19}$$

Therefore

$$\left\| \tilde{\vec{q}} - \alpha\vec{q} \right\|^2 = \left\langle \tilde{\vec{q}}, \tilde{\vec{q}} \right\rangle - \alpha \left\langle \tilde{\vec{q}}, \vec{q} \right\rangle - \bar{\alpha} \left\langle \vec{q}, \tilde{\vec{q}} \right\rangle + |\alpha|^2 = 1 - 1 - 1 + 1 = 0, \tag{7.20}$$

so $\tilde{\vec{q}} = \alpha\vec{q}$ with $|\alpha| = 1$.

Finally, $r_{kk} > 0$ fixes α. From $A = \hat{Q}\hat{R}$ we get $r_{kk} = \langle \vec{q}_k, \vec{a}_k \rangle$, and likewise $\tilde{r}_{kk} = \left\langle \tilde{\vec{q}}_k, \vec{a}_k \right\rangle = \langle \alpha\vec{q}_k, \vec{a}_k \rangle = \bar{\alpha} r_{kk}$. Both are positive real numbers, so $\bar{\alpha} = \tilde{r}_{kk}/r_{kk} > 0$, and with $|\alpha| = 1$ this forces $\alpha = 1$.

Thus $\tilde{\vec{q}}_k = \vec{q}_k$ for every k, i.e., $\tilde{Q} = \hat{Q}$, and then $\tilde{R} = \hat{Q}^* A = \hat{R}$. □

Put differently: the part of $\mathrm{span}(\vec{a}_1,\dots,\vec{a}_k)$ orthogonal to $\mathrm{span}(\vec{a}_1,\dots,\vec{a}_{k-1})$ is one-dimensional, so the direction of $\vec{q}_k$ is completely determined by A up to a unimodular phase, and $r_{kk} > 0$ fixes exactly that phase.

7.7 (SUPPLEMENT) Further Topics

7.7.1 Three QR Algorithms

	Factorization	Form of Q	Orthogonality of Q	Dense flops
MGS	reduced	explicit $\hat{Q}$	loses $\sim \varepsilon_{\mathrm{mach}}\kappa(A)$	$2mn^2$
Householder	full	implicit (n vectors $\vec{v}_k$)	orthogonal by definition	$2mn^2 - \frac{2}{3}n^3$
Givens	full	implicit (pairs (c,s))	orthogonal by definition	$3mn^2 - n^3$

7.7.2 Householder as an Existence Proof

The Householder algorithm never breaks down: the only case needing care is $\vec{x} = A[k{:}m, k] = \vec{0}$, where one simply takes $F_k = I$. It therefore produces a full QR factorization of *any* A, rank-deficient or not, which is a shorter existence proof than the one based on Gram–Schmidt. Trefethen and Bau prove existence with Gram–Schmidt in Lecture 7, while the algorithm of their Lecture 10 implicitly gives the second proof [1].

7.7.3 Uniqueness via Cholesky

If $A = \hat{Q}\hat{R}$, then

$$A^*A = \hat{R}^*\hat{Q}^*\hat{Q}\hat{R} = \hat{R}^*\hat{R}. \tag{7.21}$$

For A of full column rank, A^*A is Hermitian positive definite, and its Cholesky factorization with positive diagonal is unique; hence $\hat{R}$ is unique, and so is $\hat{Q} = A\hat{R}^{-1}$. This route is not suitable for *computing* QR, since forming A^*A squares the condition number (Lecture 5).

7.7.4 Computing c and s Without Overflow

Evaluating $\sqrt{A_{ii}^2 + A_{ji}^2}$ directly can overflow for large entries and underflow for small ones. Implementations scale by the larger entry first: if $|A_{ji}| > |A_{ii}|$, set $\tau = A_{ii}/A_{ji}$, $s = 1/\sqrt{1+\tau^2}$, $c = s\tau$, and symmetrically otherwise. This gives (c, s) up to a common sign, which still zeros the entry. The function `hypot` available in many languages works this way. Demmel also describes how to store a rotation in a single number so that it fits into the zeroed entry [2, §3.4.2].

7.7.5 Other Advantages of Givens Rotations

- *Parallelism*: rotations acting on disjoint pairs of rows can be applied simultaneously.
- *Updating*: when a row is appended to an already factored A, the new row can be eliminated with n Givens rotations instead of refactoring, which is common in recursive least squares.
- QR of Hessenberg matrices is the core step of the QR eigenvalue algorithm [1, Lectures 28–29].

7.7.6 Correspondence with Trefethen and Bau

Full and reduced QR and their existence and uniqueness are Theorems 7.1 and 7.2 of [1]; the two implicit algorithms are Algorithms 10.2 and 10.3; Givens rotations appear in Exercise 10.4.

Lecture 8

Overdetermined Least Squares and the Normal Equations

Supplementary reading:

- [1] Lectures 2, 11, and 23.
- [2] Sections 2.7.1 and 3.2.

8.1 Recap and Motivation

Last lecture we finished the computation of the QR factorization.

- *How to use the factors.* With Householder QR, Q is never formed. To apply Q^* to a vector we run through the stored reflectors $\vec{v}_1, \ldots, \vec{v}_n$ forward, and to apply Q we run through them backward.
- *Givens rotations.* A rotation in the (i, j) plane with $c = A_{ii}/\sqrt{A_{ii}^2 + A_{ji}^2}$ and $s = A_{ji}/\sqrt{A_{ii}^2 + A_{ji}^2}$ zeros out one entry at a time. This gives Givens QR.
- *Full and reduced QR*, and the *existence and uniqueness* of QR.

Solving overdetermined least squares. We now return to the problem that motivated QR in the first place: given $A \in \mathbb{K}^{m \times n}$ with $m > n$ and $\vec{b} \in \mathbb{K}^m$, find $\vec{x}$ minimizing

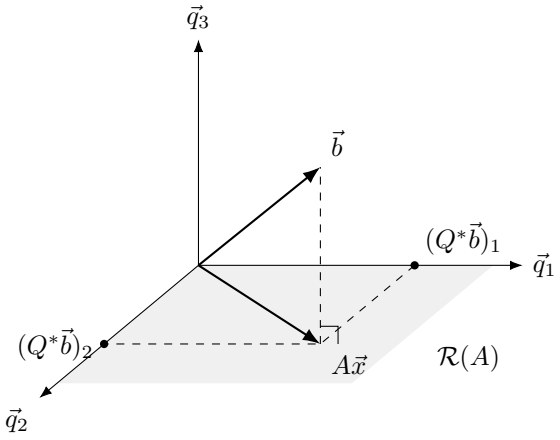
$$\left\| A\vec{x} - \vec{b} \right\|^2. \tag{8.1}$$

There are more equations than unknowns, so $A\vec{x} = \vec{b}$ generally has no solution, and we cannot simply solve it as in the square case.

We have already seen an "intuitive" explanation of why $\vec{x} = \hat{R}^{-1}\hat{Q}^*\vec{b}$ solves this problem for a reduced QR factorization $A = \hat{Q}\hat{R}$. Let us recall it. Extend $\vec{q}_1, \ldots, \vec{q}_n$ to an orthonormal basis $\vec{q}_1, \ldots, \vec{q}_m$ of $\mathbb{K}^m$. In this basis,

- $\vec{b}$ has the coordinates $Q^*\vec{b}$;
- $A\vec{x} = \hat{Q}(\hat{R}\vec{x})$ has the coordinates $\hat{R}\vec{x}$ in the first n directions and 0 in the remaining $m - n$.

Since $\hat{R}$ is invertible, we can match the first n coordinates of $\vec{b}$ exactly by solving $\hat{R}\vec{x} = \hat{Q}^*\vec{b}$, while nothing we do can affect the remaining ones. For $m = 3$ and $n = 2$, this gives the following picture.



The first two coordinates of $\vec{b}$ in the basis $\vec{q}_1, \vec{q}_2, \vec{q}_3$ can be matched by $A\vec{x}$; the third, $(Q^*\vec{b})_3$, is the residual that no $\vec{x}$ can remove.

This is convincing, but it leans on the picture. Why is matching coordinates optimal, and is it the only optimal choice? *How do we make this rigorous?* One route uses that Q^* preserves norms (Section B.2). This lecture takes a different route that needs no factorization at all.

Before using any factorization, we answer a more basic question: *what condition does a minimizer satisfy?* Such a condition is useful in two ways.

- It lets us *verify* that the $\vec{x}$ produced by QR really is optimal.
- It turns a minimization problem into a *system of equations*.

The main line of this lecture is

$$\vec{x} \text{ minimizes } \left\|A\vec{x} - \vec{b}\right\|^2 \quad \Longleftrightarrow \quad A\vec{x} - \vec{b} \perp \mathcal{R}(A) \quad \Longleftrightarrow \quad A^*A\vec{x} = A^*\vec{b}. \tag{8.2}$$

In Lecture 5 we saw this geometrically; here we prove it. We then study the coefficient matrix A^*A of the last system, which leads to Gram matrices and positive semidefinite matrices.

8.2 Conventions

As throughout, $\mathbb{K}$ is $\mathbb{R}$ or $\mathbb{C}$. Working over $\mathbb{K}$ lets one derivation cover both cases, at the price of writing it with the complex rules: conjugate transposes A^*, real parts Re, and a separate argument for the imaginary part. For real problems, read A^* as $A^\top$ and drop Re.

We use the inner product $\langle \vec{x}, \vec{y} \rangle = \vec{x}^*\vec{y}$, as in Lectures 5 and 7. It is linear in the second argument, conjugate linear in the first, and satisfies $\langle \vec{y}, \vec{x} \rangle = \overline{\langle \vec{x}, \vec{y} \rangle}$, so that

$$\langle \vec{x}, \vec{y} \rangle + \langle \vec{y}, \vec{x} \rangle = 2\,\mathrm{Re}\,\langle \vec{x}, \vec{y} \rangle\,. \tag{8.3}$$

The residual is $\vec{r} := A\vec{x} - \vec{b}$. Perturbation directions are denoted by $\vec{d}$.

One identity is used again and again in this lecture.

Lemma 34 (Moving a Matrix Across the Inner Product)

For any $M \in \mathbb{K}^{m\times n}$ (not necessarily square or unitary), $\vec{u} \in \mathbb{K}^n$, and $\vec{v} \in \mathbb{K}^m$,

$$\langle M\vec{u}, \vec{v}\rangle = \langle \vec{u}, M^*\vec{v}\rangle \,. \tag{8.4}$$

Proof. This is just associativity: $\langle M\vec{u}, \vec{v}\rangle = (M\vec{u})^*\vec{v} = \vec{u}^*(M^*\vec{v}) = \langle \vec{u}, M^*\vec{v}\rangle$. □

In words, a matrix can be moved from one side of the inner product to the other, and it becomes M^* on the way. Two remarks.

- This is *not* the same as multiplying both sides by M^*. For the $m \times n$ factor $\hat{Q}$ of a reduced QR factorization, we have $\hat{Q}^*\hat{Q} = I$ but $\hat{Q}\hat{Q}^* \neq I$, and in general $\left\langle \hat{Q}^*\vec{y}, \hat{Q}^*\vec{z}\right\rangle \neq \langle \vec{y}, \vec{z}\rangle$ (compare Section B.2).
- In Axler's *Linear Algebra Done Right* (Section 7.A), Lemma 34 is the definition of the adjoint.

8.3 The Optimality Condition

We want to find $\vec{x}$ that minimizes $\left\|A\vec{x} - \vec{b}\right\|^2$. The idea is simple: a minimizer cannot be improved by moving it. If $\vec{x}$ minimizes the objective, then for every $\vec{\varepsilon} \in \mathbb{K}^n$,

$$\left\|A(\vec{x} + \vec{\varepsilon}) - \vec{b}\right\|^2 \geq \left\|A\vec{x} - \vec{b}\right\|^2 . \tag{8.5}$$

So we first compute exactly how the objective changes under a perturbation.

Lemma 35 (Change of the Objective Under a Perturbation)

Let $A \in \mathbb{K}^{m\times n}$ and $\vec{b} \in \mathbb{K}^m$. For all $\vec{x}, \vec{\varepsilon} \in \mathbb{K}^n$, with $\vec{r} = A\vec{x} - \vec{b}$,

$$\left\|A(\vec{x} + \vec{\varepsilon}) - \vec{b}\right\|^2 - \left\|A\vec{x} - \vec{b}\right\|^2 = \|A\vec{\varepsilon}\|^2 + 2\,\mathrm{Re}\,\langle A\vec{\varepsilon}, \vec{r}\rangle \,. \tag{8.6}$$

Proof. Since $A(\vec{x} + \vec{\varepsilon}) - \vec{b} = A\vec{\varepsilon} + \vec{r}$, we have

$$\begin{aligned} \|A\vec{\varepsilon} + \vec{r}\|^2 - \|\vec{r}\|^2 &= \langle A\vec{\varepsilon} + \vec{r}, A\vec{\varepsilon} + \vec{r}\rangle - \langle \vec{r}, \vec{r}\rangle \\ &= \langle A\vec{\varepsilon}, A\vec{\varepsilon}\rangle + \langle A\vec{\varepsilon}, \vec{r}\rangle + \underbrace{\langle \vec{r}, A\vec{\varepsilon}\rangle}_{=\overline{\langle A\vec{\varepsilon}, \vec{r}\rangle}} \\ &= \|A\vec{\varepsilon}\|^2 + 2\,\mathrm{Re}\,\langle A\vec{\varepsilon}, \vec{r}\rangle \,, \end{aligned} \tag{8.7}$$

where the last step uses $z + \bar{z} = 2\,\mathrm{Re}\,z$. □

The lemma assumes nothing about $\vec{x}$. Let us look at its right-hand side.

- The quadratic term $\|A\vec{\varepsilon}\|^2$ is always ≥ 0, so it can never make the objective decrease.
- Only the cross term $2\,\mathrm{Re}\,\langle A\vec{\varepsilon}, \vec{r}\rangle$ can be negative.

This suggests that $\vec{x}$ is a minimizer exactly when the cross term vanishes for every $\vec{\varepsilon}$. That is the content of the following theorem.

Theorem 36 (Optimality Condition for Least Squares)

Let $A \in \mathbb{K}^{m\times n}$, $\vec{b} \in \mathbb{K}^m$, and $\vec{x} \in \mathbb{K}^n$. The following two statements are equivalent:

(P) $\vec{x}$ is a minimizer, i.e., $\left\|A\vec{x} - \vec{b}\right\|^2 \le \left\|A\vec{y} - \vec{b}\right\|^2$ for all $\vec{y} \in \mathbb{K}^n$.

(Q) $\left\langle A\vec{d}, A\vec{x} - \vec{b}\right\rangle = 0$ for all $\vec{d} \in \mathbb{K}^n$, i.e., $A\vec{x} - \vec{b} \perp \mathcal{R}(A)$.

Moreover, if $\vec{x}$ is a minimizer, then the set of all minimizers is $\vec{x} + \mathcal{N}(A)$. In particular, the minimizer is unique if and only if A has full column rank.

The plan of the proof: for (P) $\Rightarrow$ (Q), perturb $\vec{x}$ by a small step $\delta\vec{d}$ and let $\delta \to 0$, so that only the cross term survives; for (Q) $\Rightarrow$ (P), plug (Q) into Lemma 35 to kill the cross term.

Proof. Write $\vec{r} = A\vec{x} - \vec{b}$.

(P) $\Rightarrow$ (Q). First suppose that $\vec{x}$ is a minimizer. Fix any $\vec{d} \in \mathbb{K}^n$ and any $\delta > 0$. Applying (P) with $\vec{y} = \vec{x} + \delta\vec{d}$, and then Lemma 35 with $\vec{\varepsilon} = \delta\vec{d}$, we get

$$\begin{aligned} 0 &\le \left\|A(\vec{x} + \delta\vec{d}) - \vec{b}\right\|^2 - \left\|A\vec{x} - \vec{b}\right\|^2 \\ &= \delta^2 \left\|A\vec{d}\right\|^2 + 2\delta \operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle \\ \implies 0 &\le \delta \left\|A\vec{d}\right\|^2 + 2 \operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle, \end{aligned} \tag{8.8}$$

where we divided by $\delta > 0$; since δ is real, it comes out of the inner product without conjugation. This holds for every $\delta > 0$. Taking the limit $\delta \to 0^+$ on both sides, we get

$$\operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle \ge 0 \qquad \text{for all } \vec{d} \in \mathbb{K}^n. \tag{8.9}$$

We now upgrade this inequality to an equality in two steps.

- *The real part.* Since (8.9) holds for all $\vec{d}$, it also holds for $-\vec{d}$, which gives $-\operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle \ge 0$. Thus $\operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle = 0$ for all $\vec{d} \in \mathbb{K}^n$.
- *The imaginary part.* If $\mathbb{K} = \mathbb{C}$, we apply this to $i\vec{d}$. Since the first argument of the inner product is conjugated,
$$0 = \operatorname{Re}\left\langle iA\vec{d}, \vec{r}\right\rangle = \operatorname{Re}\left(-i\left\langle A\vec{d}, \vec{r}\right\rangle\right) = \operatorname{Im}\left\langle A\vec{d}, \vec{r}\right\rangle. \tag{8.10}$$

So both the real and the imaginary part of $\left\langle A\vec{d}, \vec{r}\right\rangle$ vanish, and $\left\langle A\vec{d}, \vec{r}\right\rangle = 0$ for all $\vec{d} \in \mathbb{K}^n$, as desired.

(Q) $\Rightarrow$ (P). Now suppose that (Q) holds, and fix any $\vec{y} \in \mathbb{K}^n$. Applying Lemma 35 with $\vec{\varepsilon} = \vec{y} - \vec{x}$, and then

(Q) with $\vec{d} = \vec{y} - \vec{x}$, we get

$$\begin{aligned} \left\|A\vec{y} - \vec{b}\right\|^2 &= \left\|A\vec{x} - \vec{b}\right\|^2 + \|A(\vec{y} - \vec{x})\|^2 + 2\,\mathrm{Re}\,\underbrace{\langle A(\vec{y} - \vec{x}), \vec{r}\rangle}_{=0 \text{ by (Q)}} \\ &= \left\|A\vec{x} - \vec{b}\right\|^2 + \underbrace{\|A(\vec{y} - \vec{x})\|^2}_{\geq 0} \\ &\geq \left\|A\vec{x} - \vec{b}\right\|^2. \end{aligned} \tag{8.11}$$

Since $\vec{y}$ was arbitrary, $\vec{x}$ is a minimizer.

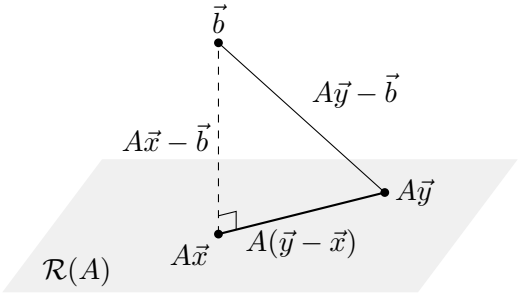
The set of minimizers. Finally, let $\vec{x}$ be a minimizer. By the first part, $\vec{x}$ satisfies (Q), so the second line of (8.11) holds for every $\vec{y}$. Hence $\vec{y}$ is also a minimizer if and only if $\|A(\vec{y} - \vec{x})\|^2 = 0$, i.e., $\vec{y} - \vec{x} \in \mathcal{N}(A)$. This set is the single point $\vec{x}$ if and only if $\mathcal{N}(A) = \{\vec{0}\}$, i.e., A has full column rank. □

Thus, if $\vec{x}$ minimizes the squared residual, then $\left\langle A\vec{d}, A\vec{x}\right\rangle = \left\langle A\vec{d}, \vec{b}\right\rangle$ for all $\vec{d}$. Let us reflect on the two directions.

- *Necessity*, (P) ⇒ (Q): orthogonality is *necessary* for optimality. Every minimizer satisfies it, so solving (Q) cannot miss a minimizer.
- *Sufficiency*, (Q) ⇒ (P): orthogonality is *sufficient*. To certify that a given $\vec{x}$ is a minimizer, it suffices to check (Q). This is how we will verify the QR solution in Section 8.4.

The two halves of the proof also have different characters.

- Necessity only compares $\vec{x}$ with nearby points $\vec{x} + \delta\vec{d}$. For small δ, the quadratic term $\delta^2 \left\|A\vec{d}\right\|^2$ is negligible, so the sign of the change is decided by the directional derivative $2\,\mathrm{Re}\left\langle A\vec{d}, \vec{r}\right\rangle$, and at a minimizer no directional derivative can be negative.
- Sufficiency compares $\vec{x}$ with an arbitrary $\vec{y}$ in one step, so the minimizer it certifies is global. Geometrically, (8.11) is the Pythagorean theorem, as in the following picture.



When the residual $A\vec{x} - \vec{b}$ is orthogonal to $\mathcal{R}(A)$, (8.11) is the Pythagorean theorem: every other point $A\vec{y}$ of $\mathcal{R}(A)$ is at least as far from $\vec{b}$.

Geometric meaning. The vectors $A\vec{d}$ for $\vec{d} \in \mathbb{K}^n$ are exactly the elements of $\mathcal{R}(A)$. So the optimality condition says that the residual is orthogonal to every vector in the column space, $A\vec{x} - \vec{b} \perp \mathcal{R}(A)$. Geometrically: drop a perpendicular from $\vec{b}$ to $\mathcal{R}(A)$; its foot is $A\vec{x}$, the orthogonal projection of $\vec{b}$ onto $\mathcal{R}(A)$. This is the picture of Lecture 5, now with a proof.

8.4 What the Optimality Condition Says

What do the optimality conditions mean in concrete cases? The key point is that (Q) tests the residual only against vectors of the form $A\vec{d}$. How much this tells us depends on how many vectors have that form.

Invertible square matrices: solving a linear system. Let $m = n$ and A be invertible (full rank), and let $\vec{x}$ be a minimizer. By necessity, it satisfies the orthogonality condition. The condition holds for *every* $\vec{d}$, so we may pick a special direction: $\vec{d} = A^{-1}(A\vec{x} - \vec{b})$, which has $A\vec{d} = A\vec{x} - \vec{b}$. Then

$$0 = \left\langle A\vec{d}, A\vec{x} - \vec{b} \right\rangle = \left\langle A\vec{x} - \vec{b}, A\vec{x} - \vec{b} \right\rangle = \left\| A\vec{x} - \vec{b} \right\|^2, \tag{8.12}$$

so $A\vec{x} = \vec{b}$. In words: for invertible square matrices, least squares is just solving the linear system, and the minimal residual is zero.

Singular or tall matrices: only some directions are tested. If A is singular or $m > n$, not every $\vec{y} \in \mathbb{K}^m$ can be written as $A\vec{d}$. Then $\mathcal{R}(A)$ is a proper subspace, and the optimality condition only checks some directions (some equations). The residual generally does not lie in $\mathcal{R}(A)$, so we cannot conclude $\vec{r} = \vec{0}$, only $\vec{r} \perp \mathcal{R}(A)$.

Verifying the QR solution. We can now make the intuitive QR argument rigorous. Let $A = \hat{Q}\hat{R}$ be a reduced QR factorization with $\hat{Q}^*\hat{Q} = I$ and $\hat{R}$ upper triangular and invertible (A of full column rank). Take $\vec{x} = \hat{R}^{-1}\hat{Q}^*\vec{b}$. We check (Q), moving $\hat{Q}$ to the right with Lemma 34:

$$\begin{aligned} \left\langle A\vec{d}, A\vec{x} - \vec{b} \right\rangle &= \left\langle \hat{Q}\hat{R}\vec{d}, \hat{Q}\hat{R}\hat{R}^{-1}\hat{Q}^*\vec{b} - \vec{b} \right\rangle \\ &= \left\langle \hat{R}\vec{d}, \hat{Q}^*\hat{Q}\hat{R}\hat{R}^{-1}\hat{Q}^*\vec{b} - \hat{Q}^*\vec{b} \right\rangle \\ &= \left\langle \hat{R}\vec{d}, \hat{Q}^*\vec{b} - \hat{Q}^*\vec{b} \right\rangle = 0. \end{aligned} \tag{8.13}$$

Note that $\hat{Q}^*$ acts on *every* term in the second argument, so $\vec{b}$ becomes $\hat{Q}^*\vec{b}$ as well.

Hence $\vec{x} = \hat{R}^{-1}\hat{Q}^*\vec{b}$ satisfies (Q), and by sufficiency it is a least squares solution; since A has full column rank, it is the only one. In practice we never form $\hat{R}^{-1}$; we solve the upper triangular system $\hat{R}\vec{x} = \hat{Q}^*\vec{b}$ by back substitution.

8.5 The Normal Equations

The optimality condition is a statement "for all $\vec{d}$". We now turn it into a single linear system. The trick is to use the transpose: apply Lemma 34 to both sides, moving A to the right.

$$\left\langle A\vec{d}, A\vec{x} \right\rangle = \left\langle A\vec{d}, \vec{b} \right\rangle \ \ \forall \vec{d} \in \mathbb{K}^n \quad \Longleftrightarrow \quad \left\langle \vec{d}, A^*A\vec{x} \right\rangle = \left\langle \vec{d}, A^*\vec{b} \right\rangle \ \ \forall \vec{d} \in \mathbb{K}^n \quad \Longleftrightarrow \quad A^*A\vec{x} = A^*\vec{b}. \tag{8.14}$$

Only the last step needs an argument. Let $\vec{w} = A^*A\vec{x} - A^*\vec{b}$. The middle condition says that $\vec{w}$ is orthogonal to every vector; taking $\vec{d} = \vec{w}$ gives $\|\vec{w}\|^2 = 0$, so $\vec{w} = \vec{0}$. The other direction is clear.

So (Q) is equivalent to the normal equations. Combining this with (P) $\Longleftrightarrow$ (Q) from Theorem 36 gives the following theorem.

Theorem 37 (The Normal Equations)

$\vec{x}$ minimizes $\left\|A\vec{x} - \vec{b}\right\|^2$ if and only if it solves the *normal equations*

$$A^*A\vec{x} = A^*\vec{b}. \tag{8.15}$$

We have turned our system into a square one! The overdetermined $m \times n$ system $A\vec{x} = \vec{b}$ has become an $n \times n$ system. The name "normal" means perpendicular: the equation $A^*(A\vec{x} - \vec{b}) = \vec{0}$ says exactly that the residual is perpendicular to the column space, as in (5.7).

The normal equations have clear advantages and one serious drawback.

- *Pro:* the system is square, and its matrix A^*A is Hermitian (symmetric in the real case); in fact it is positive semidefinite (Theorem 42). Such systems can be solved with the Cholesky factorization of Lecture 9 at half the cost of LU.
- *Con:* the condition number is squared, $\kappa_2(A^*A) = \kappa_2(A)^2$, so forming A^*A loses precision when A is ill-conditioned (see the supplement at the end of this lecture).

8.6 Gram Matrices and Positive Semidefinite Matrices

Solving the normal equations means solving a system whose matrix has the form B^*B. (We switch to the letter B so that A is free for the coefficient matrix.) So it pays to study the structure of such matrices. We begin with the larger class they belong to.

Hermitian and anti-Hermitian matrices.

Definition 38 (Hermitian and Anti-Hermitian Matrices)

A matrix $A \in \mathbb{K}^{m\times m}$ is *Hermitian* if $A^* = A$ and *anti-Hermitian* (or skew-
For $\mathbb{K} = \mathbb{R}$, these are called *symmetric* and *antisymmetric* (or skew-symmetri

Proposition 39 (Eigenvalues of (Anti-)Hermitian Matrices)

All eigenvalues of a Hermitian matrix are real, and all eigenvalues of an anti-Hermitian matrix are purely imaginary.

Proof. Let $A\vec{v} = \lambda\vec{v}$ with $\vec{v} \neq \vec{0}$, so that $\vec{v}^*A\vec{v} = \lambda \|\vec{v}\|^2$. Taking the complex conjugate of the scalar $\vec{v}^*A\vec{v}$,

$$\overline{\vec{v}^*A\vec{v}} = (\vec{v}^*A\vec{v})^* = \vec{v}^*A^*\vec{v} = \pm\vec{v}^*A\vec{v}, \tag{8.16}$$

with $+$ for Hermitian and $-$ for anti-Hermitian A. So $\vec{v}^*A\vec{v}$ is real in the first case and purely imaginary in the second, and so is $\lambda = \vec{v}^*A\vec{v}/\|\vec{v}\|^2$. □

These matrices behave a lot like complex numbers.

- Every square matrix splits uniquely into a Hermitian and an anti-Hermitian part, $A = \frac{1}{2}(A + A^*) + \frac{1}{2}(A - A^*)$, much like a complex number splits into real and imaginary parts.

- A second analogy, with $z = e^{i\theta} r$, $r \geq 0$, is the polar decomposition below. It uses the following notion, which we study in detail in the next subsection: a Hermitian matrix S is *positive semidefinite* if $\vec{v}^* S \vec{v} \geq 0$ for all $\vec{v}$.

Theorem 40 (Polar Decomposition)

For every $A \in \mathbb{K}^{m \times m}$ there exist a unitary U and a Hermitian positive semidefinite S such that

$$A = US. \tag{8.17}$$

Proof. Take an SVD $A = W\Sigma V^*$ and insert $V^*V = I$:

$$A = W\Sigma V^* = \underbrace{(WV^*)}_{U} \underbrace{(V\Sigma V^*)}_{S}. \tag{8.18}$$

We check the two factors.

- $U = WV^*$ is a product of unitary matrices, hence unitary.
- $S = V\Sigma V^*$ is Hermitian since Σ is real diagonal, and $\vec{v}^* S \vec{v} = \sum_i \sigma_i \left|(V^*\vec{v})_i\right|^2 \geq 0$. □

Geometrically, S stretches space along the orthogonal directions given by the columns of V, and U then rotates (or reflects) the result. The factor S is always determined by A: since $A^*A = SU^*US = S^2$, S is the positive semidefinite square root of A^*A. If A is invertible, so is S, and $U = AS^{-1}$ is determined as well.

Gram matrices. A matrix of the form B^*B is called a *Gram matrix* (or inner product matrix), because its entries are the inner products of the columns $B_{:,1}, \ldots, B_{:,n}$ of B with each other. Using $B\vec{e}_j = B_{:,j}$,

$$(B^*B)_{ij} = \vec{e}_i^* B^* B \vec{e}_j = \langle B\vec{e}_i, B\vec{e}_j \rangle = \langle B_{:,i}, B_{:,j} \rangle. \tag{8.19}$$

Two consequences are immediate.

- The diagonal entries $(B^*B)_{ii} = \|B_{:,i}\|^2$ are nonnegative.
- $B^*B = I$ exactly when the columns of B are orthonormal.

Definition 41 (Positive (Semi)definite Matrices)

A Hermitian matrix $A \in \mathbb{K}^{n \times n}$ is

- *positive semidefinite* if $\vec{x}^* A \vec{x} \geq 0$ for all $\vec{x} \in \mathbb{K}^n$;
- *positive definite* if $\vec{x}^* A \vec{x} > 0$ for all $\vec{x} \neq \vec{0}$.

The main result of this section says that these two notions describe the same matrices.

Theorem 42 (Gram Matrices Are Exactly the Positive Semidefinite Matrices)

1. $A \in \mathbb{K}^{n \times n}$ is Hermitian positive semidefinite if and only if it is a Gram matrix, that is, $A = B^*B$ for some matrix B.
2. For $B \in \mathbb{K}^{m \times n}$, the Gram matrix B^*B is positive definite if and only if B has full column rank.

Item 1 does not hold for all Hermitian matrices. For example, $\mathrm{diag}(1,-1)$ is symmetric but not a Gram matrix: its second diagonal entry would have to be $\|B_{:,2}\|^2 \geq 0$. Note that the rank condition in item 2 is on B, not on the Gram matrix $A = B^*B$, which is square.

Proof of item 1. Gram matrices are Hermitian positive semidefinite. We have

$$(B^*B)^* = B^*(B^*)^* = B^*B, \qquad \vec{x}^*B^*B\vec{x} = \|B\vec{x}\|^2 \geq 0. \tag{8.20}$$

Hermitian positive semidefinite matrices are Gram matrices. We take a square root of A in three steps.

1. *Diagonalize.* By the spectral theorem, a Hermitian matrix can be diagonalized by a unitary matrix: $A = U\Lambda U^*$, where U is unitary and Λ is diagonal with the eigenvalues of A on the diagonal.
2. *The eigenvalues are nonnegative.* They are real by Proposition 39. If $A\vec{v} = \lambda\vec{v}$ with $\vec{v} \neq \vec{0}$, then $\lambda\|\vec{v}\|^2 = \vec{v}^*A\vec{v} \geq 0$, so $\lambda \geq 0$. (If A is positive definite, the same argument gives $\lambda > 0$.)
3. *Take square roots.* So we can define $\sqrt{\Lambda} = \mathrm{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_n})$. Inserting $U^*U = I$ in the middle,

$$A = U\sqrt{\Lambda}\,U^*U\sqrt{\Lambda}\,U^* = \left(U\sqrt{\Lambda}\,U^*\right)^*\left(U\sqrt{\Lambda}\,U^*\right), \tag{8.21}$$

   where the last step uses that $\sqrt{\Lambda}$ is a real diagonal matrix and hence equal to its conjugate transpose.

Taking $B = U\sqrt{\Lambda}\,U^*$ gives $A = B^*B$. □

The matrix $B = U\sqrt{\Lambda}\,U^*$ is the *matrix square root* $A^{1/2}$. It is not the only choice: for any unitary V, $(VB)^*(VB) = B^*B$. In particular, taking a QR factorization $B = QR$ (Theorem 31) gives $A = R^*Q^*QR = R^*R$ with R upper triangular. This is the *Cholesky factorization* of A, the subject of the next lecture.

Proof of item 2. We prove the contrapositive by a chain of equivalences:

$$\begin{aligned} B \text{ is column rank deficient} &\iff \exists\,\vec{x} \in \mathbb{K}^n \setminus \{\vec{0}\} : B\vec{x} = \vec{0} \\ &\iff \exists\,\vec{x} \in \mathbb{K}^n \setminus \{\vec{0}\} : \|B\vec{x}\|^2 = 0 \\ &\iff \exists\,\vec{x} \in \mathbb{K}^n \setminus \{\vec{0}\} : \vec{x}^*B^*B\vec{x} = 0. \end{aligned} \tag{8.22}$$

By item 1, B^*B is positive semidefinite, so $\vec{x}^*B^*B\vec{x} \geq 0$ always holds. Hence B^*B fails to be positive definite exactly when $\vec{x}^*B^*B\vec{x} = 0$ for some $\vec{x} \neq \vec{0}$. So B is column rank deficient if and only if B^*B is not positive definite; negating both sides gives item 2. □

Consequences for least squares. Putting Theorems 37 and 42 together,

$$\begin{aligned} A \text{ has full column rank} &\iff A^*A \text{ is positive definite} \\ &\iff \text{the normal equations have the unique solution } \vec{x} = (A^*A)^{-1}A^*\vec{b}, \end{aligned} \tag{8.23}$$

consistent with the uniqueness statement of Theorem 36. A positive definite system can be solved with the Cholesky factorization $A^*A = R^*R$, which costs about $\frac{1}{3}n^3$ flops, half of LU, and needs no pivoting.

8.7 (SUPPLEMENT) Further Topics

8.7.1 Relation to the Projection Argument

Introductory linear algebra texts (for example Lay's) derive the normal equations differently. Let $\hat{\vec{b}}$ be the orthogonal projection of $\vec{b}$ onto $\mathcal{R}(A)$. By the properties of the projection, $\vec{b} - A\hat{\vec{x}} \perp \mathcal{R}(A)$, that is, $\vec{b} - A\hat{\vec{x}} \in \mathcal{N}(A^*)$, which rearranges to the normal equations. Here $\hat{\vec{b}} \in \mathbb{K}^m$ is the projected point, while $\hat{\vec{x}} \in \mathbb{K}^n$ holds the coefficients that combine the columns of A into it, $\hat{\vec{b}} = A\hat{\vec{x}}$. The point $\hat{\vec{b}}$ is always unique; $\hat{\vec{x}}$ is unique if and only if A has full column rank.

This derivation is shorter, but it relies on the *best approximation theorem*: if W is a subspace and $\hat{\vec{b}}$ is the orthogonal projection of $\vec{b}$ onto W, then $\left\|\vec{b} - \hat{\vec{b}}\right\| < \left\|\vec{b} - \vec{w}\right\|$ for all $\vec{w} \in W$ with $\vec{w} \neq \hat{\vec{b}}$. Its proof takes two steps.

1. Choose an orthonormal basis $\vec{u}_1, \ldots, \vec{u}_k$ of W and set $\hat{\vec{b}} = \sum_i \left\langle \vec{u}_i, \vec{b} \right\rangle \vec{u}_i$. A direct computation gives $\left\langle \vec{u}_j, \vec{b} - \hat{\vec{b}} \right\rangle = 0$, so $\vec{b} - \hat{\vec{b}} \perp W$.

2. Split $\vec{b} - \vec{w} = (\vec{b} - \hat{\vec{b}}) + (\hat{\vec{b}} - \vec{w})$, where the first part is orthogonal to W and the second lies in W. By the Pythagorean theorem,

$$\left\|\vec{b} - \vec{w}\right\|^2 = \left\|\vec{b} - \hat{\vec{b}}\right\|^2 + \left\|\hat{\vec{b}} - \vec{w}\right\|^2 > \left\|\vec{b} - \hat{\vec{b}}\right\|^2. \tag{8.24}$$

So the best approximation theorem is "orthogonal ⇒ closest", our sufficiency, while our perturbation argument is "closest ⇒ orthogonal", our necessity. Our proof is self-contained: it needs no basis, no Gram–Schmidt, and no existence result for projections.

8.7.2 Why the Perturbation Argument

Its structure, "at a minimizer, the first-order change in every direction is nonnegative", generalizes to settings that the projection argument does not reach:

- first-order conditions for general differentiable functions (the gradient vanishes);
- constrained problems, where $\vec{d}$ can no longer be replaced by $-\vec{d}$, so the conclusion stays an inequality; this leads to variational inequalities and the KKT conditions;
- infinite-dimensional problems, such as the Galerkin orthogonality of finite elements, "$\langle v, \text{residual} \rangle = 0$ for every test function v", which has the same structure as "for all $\vec{d}$" here.

8.7.3 A Numerical Warning About the Normal Equations

The normal equations are elegant, but by Theorem 114 the singular values of A^*A are the squares of those of A, so

$$\kappa_2(A^*A) = \kappa_2(A)^2. \tag{8.25}$$

Forming and solving $A^*A\vec{x} = A^*\vec{b}$ therefore loses about twice as many digits as necessary when A is ill-conditioned. In terms of cost, forming A^*A takes about mn^2 flops and Cholesky another $\frac{1}{3}n^3$, roughly

half of Householder QR when $m \gg n$. But the speed is worth it only when A is well-conditioned. This is why in practice one solves $\hat{R}\vec{x} = \hat{Q}^*\vec{b}$ with a QR factorization instead [1, Lecture 11].

Lecture 9

Cholesky Factorization

Supplementary reading:

- [1] Lecture 23.
- [2] Section 2.7.1.
- [7] Chapter 10.

9.1 Recap and Motivation

Last lecture we solved the least squares problem $\min_{\vec{x}} \left\|A\vec{x} - \vec{b}\right\|^2$ in two ways.

- *By QR.* With a full QR factorization, $\left\|A\vec{x} - \vec{b}\right\|^2 = \left\|\hat{R}\vec{x} - (Q^*\vec{b})_{1:n}\right\|^2 + \left\|(Q^*\vec{b})_{n+1:m}\right\|^2$. We make the first term zero by solving $\hat{R}\vec{x} = (Q^*\vec{b})_{1:n}$. The second term does not depend on $\vec{x}$; it is the squared distance from $\vec{b}$ to $\mathcal{R}(A)$.
- *By the optimality condition.* At the minimizer, the residual is orthogonal to $\mathcal{R}(A)$: $\left\langle A\vec{d}, A\vec{x}\right\rangle = \left\langle A\vec{d}, \vec{b}\right\rangle$ for all $\vec{d}$ (Theorem 36). Written as a linear system, these are the normal equations $A^*A\vec{x} = A^*\vec{b}$ (Theorem 37).

The matrix A^*A of the normal equations is a Gram matrix, and we saw that such matrices are exactly the positive semidefinite ones: M is Hermitian positive semidefinite if and only if $M = B^*B$ for some B (Theorem 42).

The factor B in $M = B^*B$ is far from unique. For any unitary U, $(UB)^*(UB) = B^*U^*UB = B^*B$. With this much freedom, a natural question is:

If $A = B^*B$ is positive definite, can we choose B to be upper triangular?

If yes, then $A = R^*R$ with R upper triangular, and solving $A\vec{x} = \vec{c}$ takes only two triangular solves: first $R^*\vec{y} = \vec{c}$, then $R\vec{x} = \vec{y}$. The answer of this lecture is yes, and the resulting factorization is called the *Cholesky factorization*. Even better, we will see that it needs no pivoting and is always backward stable.

Here is our plan.

1. Derive Cholesky from LU by making elimination *symmetric*, and turn it into an algorithm.
2. Prove that the algorithm never breaks down (existence) and that it does not amplify rounding errors (stability).
3. Look at the elimination in blocks, which leads to *Schur complements*.
4. Show that Cholesky and QR compute the same triangular factor.

Conventions. In this lecture $A \in \mathbb{K}^{m\times m}$ is a square matrix; it has nothing to do with the $m \times n$ matrix of least squares. We say that A is *s.p.d.* if it is Hermitian positive definite, i.e., $A = A^*$ and $\vec{x}^* A\vec{x} > 0$ for all $\vec{x} \neq \vec{0}$. (In the real case this means symmetric positive definite, hence the name.) We write the Cholesky factorization as $A = LL^*$ with L lower triangular; this is the same as $A = R^*R$ with $R = L^*$.

Since A is Hermitian, its first row is the conjugate transpose of its first column. So we can split off the first row and column as

$$A = \begin{bmatrix} \alpha & \vec{a}^* \\ \vec{a} & A_{22} \end{bmatrix}, \qquad \alpha = A_{11} \in \mathbb{R}, \quad \vec{a} = A[2{:}m, 1] \in \mathbb{K}^{m-1}, \quad A_{22} = A[2{:}m, 2{:}m]. \tag{9.1}$$

In slice notation, $\vec{a} = A_{2:,1}$, $\vec{a}^* = A_{1,2:}$, and $A_{22} = A_{2:,2:}$.

9.2 Deriving Cholesky by Symmetric Elimination

The idea is simple. We already know how to factor a matrix by Gaussian elimination. We will run elimination on A, but at every step we do the *same* operation on the rows and on the columns, so that the matrix stays Hermitian. Let us go through one step.

Step 1: one step of LU. The first step of LU subtracts a_i/α times row 1 from row $i+1$, for every row below the first. In matrix form, this is multiplication from the left by a unit lower triangular L_1:

$$L_1 = \begin{bmatrix} 1 & 0 \\ -\vec{a}/\alpha & I \end{bmatrix}, \qquad L_1 A = \begin{bmatrix} \alpha & \vec{a}^* \\ 0 & A_{22} - \dfrac{\vec{a}\vec{a}^*}{\alpha} \end{bmatrix}. \tag{9.2}$$

The first column is now zero below the diagonal, and the lower right block has been updated to

$$S := A_{22} - \frac{\vec{a}\vec{a}^*}{\alpha}, \tag{9.3}$$

which is $A_{2:,2:} - A_{2:,1}A_{1,2:}/A_{11}$ in slice notation. The matrix S is called a *Schur complement*; we will study it in Section 9.6. Note that $\vec{a}\vec{a}^*$ is an outer product, so the update is a rank-one change of A_{22}.

Can we divide by α? Yes: $\alpha = A_{11} = \vec{e}_1^* A\vec{e}_1 > 0$ by positive definiteness.

Step 2: add the matching column operation. If we continued with LU on S, we would get $A = LU$ where L and U have nothing to do with each other. To end up with $A = LL^*$, we keep the matrix symmetric: after the row operation L_1 on the left, we apply the matching column operation L_1^* on the right. The result is

$$L_1 A L_1^* = \begin{bmatrix} \alpha & 0 \\ 0 & S \end{bmatrix}. \tag{9.4}$$

Let us check this. Right multiplication by L_1^* subtracts $\overline{a_j}/\alpha$ times column 1 from column $j+1$.

- It does not change the lower right block, since column 1 of $L_1 A$ is zero below the diagonal.
- It zeros out the first row. The $(1, j+1)$ entry of $L_1 A$ is $\overline{a_j}$, because A is Hermitian, so it becomes $\overline{a_j} - (\overline{a_j}/\alpha)\alpha = 0$.

This is exactly where symmetry helps: the first row is the conjugate of the first column, so the column operation that clears the row uses the same multipliers as the row operation that cleared the column. One matrix L_1 handles both sides.

Step 3: normalize by $\sqrt{\alpha}$. It remains to scale the top left entry to 1. Divide the first row and the first column by $\sqrt{\alpha}$, i.e., replace L_1 by $\bar{L}_1 = \mathrm{diag}(1/\sqrt{\alpha}, 1, \ldots, 1) L_1$. Then

$$\bar{L}_1 A \bar{L}_1^* = \begin{bmatrix} 1 & 0 \\ 0 & S \end{bmatrix}, \qquad \bar{L}_1^{-1} = \begin{bmatrix} \sqrt{\alpha} & 0 \\ \vec{a}/\sqrt{\alpha} & I \end{bmatrix}. \tag{9.5}$$

(To invert $\bar{L}_1$, undo the scaling and then flip the sign of the multipliers.) Moving $\bar{L}_1$ and $\bar{L}_1^*$ to the other side,

$$A = \begin{bmatrix} \sqrt{\alpha} & 0 \\ \vec{a}/\sqrt{\alpha} & I \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & S \end{bmatrix} \begin{bmatrix} \sqrt{\alpha} & \vec{a}^*/\sqrt{\alpha} \\ 0 & I \end{bmatrix}. \tag{9.6}$$

Now suppose that S has a Cholesky factorization $S = L_S L_S^*$. Substituting it into (9.6) and multiplying out, we get

$$A = LL^*, \qquad L = \begin{bmatrix} \sqrt{\alpha} & 0 \\ \vec{a}/\sqrt{\alpha} & L_S \end{bmatrix}. \tag{9.7}$$

In words: *the first column of L is the first column of A divided by $\sqrt{\alpha}$, and the rest of L is the Cholesky factor of the Schur complement S.* So Cholesky is a recursion: compute one column, update the remaining block to S, and repeat on S. The only thing that could go wrong is that some later S is not s.p.d., so that its top left entry is not positive. We will rule this out in Section 9.4.

Problem Solving Strategy 43 (Computing a Cholesky Factorization by Hand)**.** *Repeat for $k = 1, \ldots, m$: let d_k be the top left entry of the current matrix. Column k of L is the current first column divided by $\sqrt{d_k}$. Then replace the current matrix by its Schur complement, i.e., delete the first row and column and subtract $\vec{a}\vec{a}^*/d_k$ from what is left.*

Example 44 (A 3×3 Example)**.** Let

$$A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 3 \\ 2 & 3 & 6 \end{bmatrix}. \tag{9.8}$$

Step 1. $d_1 = 4$, so the first column of L is $(4, 2, 2)^\top / 2 = (2, 1, 1)^\top$. The Schur complement is

$$\begin{bmatrix} 5 & 3 \\ 3 & 6 \end{bmatrix} - \frac{1}{4} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \begin{bmatrix} 2 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}. \tag{9.9}$$

Step 2. $d_2 = 4$, so the next column is $(4, 2)^\top / 2 = (2, 1)^\top$, and the Schur complement is $5 - 2 \cdot 2/4 = 4$.

Step 3. $d_3 = 4$, so the last entry is $\sqrt{4} = 2$. Altogether,

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 1 & 2 \end{bmatrix}, \tag{9.10}$$

and one can check that $LL^\top = A$.

Why the square root? LU and Cholesky do the same elimination; the only difference is how the pivots are distributed between the two factors. Suppose we eliminate symmetrically but skip the normalization. Then we get

$$A = \tilde{L} D \tilde{L}^*, \tag{9.11}$$

where $\tilde{L}$ is unit lower triangular and $D = \operatorname{diag}(d_1, \dots, d_m)$ holds the pivots. To turn this into LL^*, we split D evenly between the two sides:

$$A = (\tilde{L} D^{1/2})(D^{1/2} \tilde{L}^*) = LL^*, \qquad L = \tilde{L} D^{1/2}. \tag{9.12}$$

- LU puts the whole pivot d_k into U, so column k of L is divided by d_k.
- Cholesky puts half of the pivot on each side, so column k of L is divided by $\sqrt{d_k}$.

For the last step to work, every d_k must be positive: then $\sqrt{d_k}$ is real, $(D^{1/2})^* = D^{1/2}$, and indeed $(\tilde{L} D^{1/2})^* = D^{1/2} \tilde{L}^*$. If some $d_k < 0$, then $\sqrt{d_k}$ is imaginary, and $LL^* = \tilde{L} \, |D| \, \tilde{L}^* \neq A$.

9.3 The Cholesky Algorithm

The right-looking algorithm. We now turn the recursion into an in-place algorithm. Step k computes column k of L and then applies the Schur complement update to the lower right block. Since every Schur complement is Hermitian, we only store and update the lower triangle.

Algorithm 12 Cholesky factorization, right-looking, in place (lower triangle).

1: **for** $k = 1, \dots, m$ **do**
2: $\quad A_{kk} \leftarrow \sqrt{A_{kk}}$
3: $\quad$ **for** $i = k+1, \dots, m$ **do**
4: $\quad\quad A_{ik} \leftarrow A_{ik} / A_{kk}$ ▷ column k of L
5: $\quad$ **for** $j = k+1, \dots, m$ **do**
6: $\quad\quad$ **for** $i = j, \dots, m$ **do**
7: $\quad\quad\quad A_{ij} \leftarrow A_{ij} - A_{ik} \overline{A_{jk}}$ ▷ Schur complement update

A few remarks on this algorithm.

- In line 2, A_{kk} is *not* an entry of the original matrix. After $k-1$ updates it is the top left entry of the current Schur complement, i.e., the pivot d_k.
- Lines 2–4 divide the current column k by $\sqrt{d_k}$.
- In the update, both A_{ik} and A_{jk} have already been divided by $\sqrt{d_k}$, so their product is $a_i \overline{a_j} / d_k$, which is exactly the (i, j) entry of $\vec{a}\vec{a}^*/\alpha$.
- On exit, the lower triangle of A holds L.

In compact form, the algorithm reads `A[:,k] /= sqrt(A[k,k])` followed by `A[i,j] -= A[i,k]*A[k,j];` Algorithm 12 spells out the index ranges and the conjugate.

Comparison with LU. Compared with LU, Cholesky differs in three ways.

- *Normalization.* Columns are divided by the square root of the pivot, as explained above.

- *Half the work and storage.* Every Schur complement is Hermitian, so we only need to keep track of one triangle; the other one is its conjugate transpose. The cost drops from about $\frac{2}{3}m^3$ flops for LU to about $\frac{1}{3}m^3$.
- *No pivoting.* We prove in Sections 9.4 and 9.5 that Cholesky without pivoting never breaks down and is stable.

Where positive definiteness enters. The algorithm uses positive definiteness in exactly one place: the square root $\sqrt{A_{kk}}$ in line 2. Every *updated* pivot must be positive, both to have a real square root and to divide by it. Positive definiteness of A directly tells us only that the *original* diagonal entries are positive. That the updated pivots are positive too needs an argument, which is the existence theorem below. The following example shows that something does need to be proved.

Example 45 (A Symmetric Matrix That Is Not Positive Definite)**.** Run Algorithm 12 on

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}. \tag{9.13}$$

For $k = 1$, $\sqrt{1} = 1$, the first column becomes $(1, 2)^\top$, and the update gives $A_{22} = 1 - 2 \cdot 2 = -3$. For $k = 2$, we need $\sqrt{-3}$, and the algorithm fails. Both original diagonal entries are 1; the trouble is the updated pivot.

Algorithm 12 assumes a positive definite input and does no checking. In real arithmetic, `sqrt` of a negative number returns `NaN`, which then contaminates everything after it. LAPACK's `potrf` checks whether $A_{kk} \le 0$ before taking the square root, and if so stops and reports the index at which it failed.

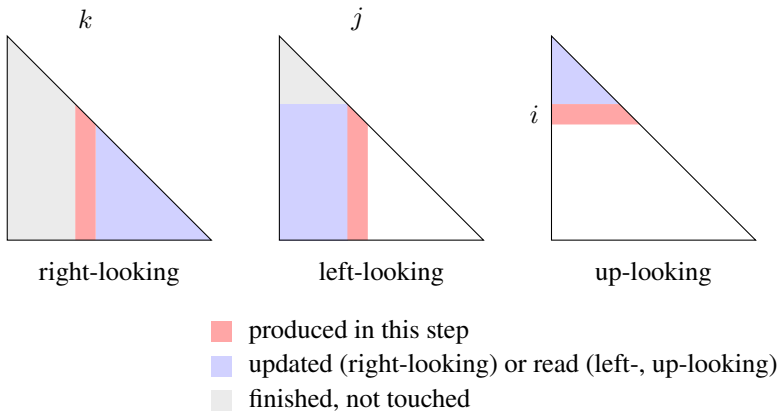
Left-looking and up-looking variants. The right-looking algorithm is not the only way to organize the computation. Cholesky can be implemented with especially nice memory access patterns. To see the options, compare entries on both sides of $A = LL^*$ for $i \ge j$:

$$L_{jj} = \sqrt{A_{jj} - \sum_{k<j} |L_{jk}|^2}, \qquad L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k<j} L_{ik} \overline{L_{jk}} \right) \quad (i > j). \tag{9.14}$$

Any loop order that evaluates these formulas, using only entries of L that are already finished, gives a valid algorithm. All variants below do exactly the same floating-point operations and, in exact arithmetic, produce the same L. They differ only in the order of the three nested loops over i, j, k.

Variant	Outer loop	Produces per step	Reads	Writes
right-looking	k	column k of L	column k	whole trailing Schur complement
left-looking	j	column j of L	columns $1, \dots, j-1$	column j only
up-looking	i	row i of L	rows $1, \dots, i-1$	row i only

The picture below shows one step of each variant.



Left-looking. The right-looking algorithm is *eager*: as soon as a column is finished, it pushes its contribution into the whole trailing block. The left-looking algorithm is *lazy*: when it reaches column j, it pulls in all the updates owed by the finished columns to its left at once, and then normalizes the column. As a result, each entry is written in its final form exactly once, whereas in the right-looking variant the entries of the Schur complement are read and written $O(m)$ times. The innermost loop runs down a column, which suits column-major languages (Julia, Fortran, MATLAB).

Algorithm 13 Cholesky factorization, left-looking, in place (lower triangle).

1: **for** $j = 1, \dots, m$ **do**
2: **for** $k = 1, \dots, j-1$ **do**
3: **for** $i = j, \dots, m$ **do**
4: $A_{ij} \leftarrow A_{ij} - A_{ik}\overline{A_{jk}}$
5: $s \leftarrow \sqrt{A_{jj}}$ ▷ save it: line 7 overwrites A_{jj}
6: **for** $i = j, \dots, m$ **do**
7: $A_{ij} \leftarrow A_{ij}/s$

Up-looking. Here L is computed row by row. Row i depends only on its own earlier entries and on the finished rows above it. Since row i of L is column i of $R = L^*$, a column-major code stores the upper triangular factor R instead; then walking along a row of L becomes walking down a column of R. In *packed storage*, the upper triangle is stored column by column, with column i (of length i) right after column $i-1$:

$$\begin{bmatrix} 1 & 2 & 4 & 7 & 11 \\ & 3 & 5 & 8 & 12 \\ & & 6 & 9 & 13 \\ & & & 10 & 14 \\ & & & & 15 \end{bmatrix}. \tag{9.15}$$

Computing column i uses only earlier data. So when A grows by one row and one column, the factor grows by appending one segment at the end, and nothing computed before has to change.

Algorithm 14 Cholesky factorization, up-looking, in place (lower triangle).

for $i = 1, \dots, m$ **do**
  for $j = 1, \dots, i$ **do**
    for $k = 1, \dots, j-1$ **do**
      $A_{ij} \leftarrow A_{ij} - A_{ik}\overline{A_{jk}}$
    if $j < i$ **then**
      $A_{ij} \leftarrow A_{ij}/A_{jj}$ ▷ A_{jj} already holds L_{jj}
    else
      $A_{ii} \leftarrow \sqrt{A_{ii}}$

In Algorithms 13 and 14 it is tempting to divide by $\sqrt{A_{jj}}$ in every case, but this is wrong: in the left-looking loop, A_{jj} itself is overwritten when $i = j$, and in the up-looking loop, for $j < i$ the entry A_{jj} already holds L_{jj}. This is why the algorithms above save s and divide by A_{jj}, respectively.

9.4 Existence: Cholesky Never Breaks Down

We have seen that LU without pivoting can meet a zero pivot; for example, $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ fails at the very first step. Cholesky asks for even more: the pivots must be *real and positive*, because we take their square roots. In this section we show that for s.p.d. matrices this is always the case.

The idea of the proof is as follows. After k steps of symmetric elimination, the matrix we are working on is A multiplied by a nonsingular matrix on the right and by its conjugate transpose on the left. We first show that such a "sandwich" of an s.p.d. matrix is again s.p.d.; then its diagonal entries, and in particular the next pivot, are positive.

Lemma 46 (Congruence Preserves Positive Definiteness)

Let $A \in \mathbb{K}^{m\times m}$ be s.p.d. and $B \in \mathbb{K}^{m\times m}$ be nonsingular. Then B^*AB is s.p.d.

Proof. First, B^*AB is Hermitian, since $(B^*AB)^* = B^*A^*B = B^*AB$. For positive definiteness, let $\vec{x} \neq \vec{0}$. Since B is nonsingular, $B\vec{x} \neq \vec{0}$, and hence

$$\vec{x}^* B^* AB\vec{x} = (B\vec{x})^* A(B\vec{x}) > 0, \tag{9.16}$$

as desired. □

The map $A \mapsto B^*AB$ is called a *congruence*. The assumption that B is nonsingular is needed: if $B\vec{x} = \vec{0}$ for some $\vec{x} \neq \vec{0}$, then $\vec{x}^*B^*AB\vec{x} = 0$, and we only get positive semidefiniteness.

Theorem 47 (Existence of the Cholesky Factorization)

Let $A \in \mathbb{K}^{m\times m}$ be s.p.d. Then Cholesky without pivoting encounters only real, positive pivots $d_1, \dots, d_m$, and hence produces $A = LL^*$ with L lower triangular with positive diagonal. Moreover, the pivots are bounded below by

$$d_k \geq \frac{1}{\|A^{-1}\|_2} = \lambda_{\min}(A), \qquad k = 1, \dots, m. \tag{9.17}$$

Proof. We show by induction on $k = 0, \dots, m-1$ that the pivot d_{k+1} is real and positive.

Suppose that the first k pivots are positive, so that the elimination matrices $L_1, \dots, L_k$ of the first k steps are defined. (For $k = 0$ there is nothing to assume.) After k symmetric steps, we have

$$S \coloneqq L_k \cdots L_1 A L_1^* \cdots L_k^* = \begin{bmatrix} \operatorname{diag}(d_1, \dots, d_k) & 0 \\ 0 & S^{(k)} \end{bmatrix}, \tag{9.18}$$

where $S^{(k)}$ is the current Schur complement. Its top left entry is the next pivot, $d_{k+1} = S_{k+1,k+1}$.

Let $B = L_1^* \cdots L_k^*$, so that $S = B^* A B$. Each L_i is unit lower triangular and therefore has determinant 1, so B is nonsingular. By Lemma 46, S is s.p.d. Now:

- *The pivot is real.* S is Hermitian, so $S_{k+1,k+1} = \overline{S_{k+1,k+1}}$.
- *The pivot is positive.* A standard basis vector picks out a diagonal entry, $\vec{e}_j^* S \vec{e}_j = S_{jj}$. So by positive definiteness,
$$d_{k+1} = S_{k+1,k+1} = \vec{e}_{k+1}^* S \vec{e}_{k+1} > 0. \tag{9.19}$$

This completes the induction. With all pivots positive, the construction of the previous section gives $A = LL^*$ with $L_{kk} = \sqrt{d_k} > 0$.

The lower bound (9.17) needs a fact about Schur complements, and we prove it in Section 9.6. □

Note that "real" and "positive" come from different hypotheses: realness from the Hermitian property, positivity from definiteness. Along the way we also proved a fact worth remembering: *every diagonal entry of a positive definite matrix is positive*. The lower bound (9.17) says even more than positivity: the pivots stay a fixed distance away from zero, independently of k, so tiny pivots cannot occur.

Corollary 48 (Uniqueness of the Cholesky Factorization)**.** *An s.p.d. matrix has exactly one factorization $A = LL^*$ with L lower triangular with positive diagonal.*

Proof. Existence is Theorem 47. Suppose $A = LL^* = MM^*$ with both factors of this form. Then $M^{-1}L = M^* L^{-*}$. The left side is lower triangular and the right side is upper triangular, so both equal a diagonal matrix Δ, whose entries L_{kk}/M_{kk} are positive. Moreover,

$$\Delta\Delta^* = (M^{-1}L)(M^{-1}L)^* = M^{-1}LL^*M^{-*} = M^{-1}MM^*M^{-*} = I, \tag{9.20}$$

so $|\Delta_{kk}| = 1$. A positive number of absolute value 1 is 1, so $\Delta = I$ and $L = M$. □

9.5 Rounding Errors

Existence tells us that the algorithm runs to completion in exact arithmetic. We now ask what happens in floating point.

The key quantity for LU. Recall from the error analysis of LU (Theorem 16) that the computed factors satisfy $\tilde{L}\tilde{U} = A + \delta A$ with

$$|\delta A| \le cm\varepsilon_{\text{mach}} \left|\tilde{L}\right| \left|\tilde{U}\right| \quad \text{(entrywise)}. \tag{9.21}$$

So the key quantity is $|L|\,|U|$. If it is of the same size as $|A|$, the algorithm is backward stable; if it is much larger, errors are amplified. Without pivoting, $|L|\,|U|$ can be arbitrarily large: for $A = \left[\begin{smallmatrix} \delta & 1 \\ 1 & 1 \end{smallmatrix}\right]$ with small δ, L contains the entry $1/\delta$.

No element growth in Cholesky. For Cholesky, $U = L^*$, so the key quantity is $|L|\,|L^*|$. We will show that it is never larger than the largest entry of A. First we need to know where the large entries of an s.p.d. matrix are.

Lemma 49 (The Largest Entry Is on the Diagonal)

Let A be s.p.d. Then $|A_{ij}| < \sqrt{A_{ii}A_{jj}}$ for all $i \neq j$. In particular, $\max_{i,j} |A_{ij}| = \max_i A_{ii}$.

Proof. Fix $i \neq j$. The 2×2 principal submatrix

$$\begin{bmatrix} A_{ii} & A_{ij} \\ A_{ji} & A_{jj} \end{bmatrix} \tag{9.22}$$

is again s.p.d.: apply the definition of positive definiteness to vectors that are zero outside positions i and j. So its eigenvalues are positive, and so is its determinant, $A_{ii}A_{jj} - |A_{ij}|^2 > 0$. Hence $|A_{ij}| < \sqrt{A_{ii}A_{jj}} \leq \max(A_{ii}, A_{jj})$. □

Proposition 50 (No Growth in the Cholesky Factor)

Let $A = LL^*$ be s.p.d. Then for all i, j,

$$|L_{ij}| \leq \sqrt{A_{ii}}, \qquad (|L|\,|L^*|)_{ij} \leq \sqrt{A_{ii}A_{jj}} \leq \max_{k,l} |A_{kl}|\,. \tag{9.23}$$

Proof. The diagonal entries of $A = LL^*$ are $A_{ii} = \sum_k |L_{ik}|^2$, i.e., row i of L has squared length A_{ii}. In particular $|L_{ij}|^2 \leq A_{ii}$. For the second claim, we use the Cauchy–Schwarz inequality:

$$(|L|\,|L^*|)_{ij} = \sum_k |L_{ik}|\,|L_{jk}| \leq \left(\sum_k |L_{ik}|^2\right)^{1/2} \left(\sum_k |L_{jk}|^2\right)^{1/2} = \sqrt{A_{ii}}\sqrt{A_{jj}} \leq \max_{k,l} |A_{kl}|\,, \tag{9.24}$$

where the last step uses Lemma 49. □

Here $\max_{k,l} |A_{kl}| = \|\mathrm{vec}(A)\|_\infty$ is the largest entry of A, where $\mathrm{vec}(A) \in \mathbb{K}^{m^2}$ stacks the columns of A; it is *not* the induced ∞-norm (maximum row sum) of Proposition 104. In words: the growth factor of Cholesky is 1. Plugging this into the backward error bound gives the main result.

Theorem 51 (Backward Stability of Cholesky)

Let $A \in \mathbb{K}^{m\times m}$ be s.p.d., and suppose Cholesky runs to completion in floating-point arithmetic, producing $\tilde{L}$. Then, to first order in $\varepsilon_{\text{mach}}$,

$$\tilde{L}\tilde{L}^* = A + \delta A, \qquad |\delta A_{ij}| \leq cm\varepsilon_{\text{mach}} \max_{k,l} |A_{kl}| \tag{9.25}$$

for a modest constant c. That is, Cholesky without pivoting is backward stable.

Proof sketch. The analysis of Theorem 16 applies with $\tilde{U} = \tilde{L}^*$ and gives $|\delta A| \leq cm\varepsilon_{\text{mach}} \left|\tilde{L}\right| \left|\tilde{L}^*\right|$ [7, Theorem 10.3]. Strictly speaking, Proposition 50 is about the exact factor. The computed one satisfies

$\sum_k \left|\tilde{L}_{ik}\right|^2 = A_{ii} + \delta A_{ii}$, which differs by a term of order $\varepsilon_{\text{mach}}$ and changes the bound only at second order. Hence $\left|\tilde{L}\right|\left|\tilde{L}^*\right| \le \max_{k,l} |A_{kl}|$ to first order, as desired. □

So Theorems 47 and 51 answer two different questions: the first says that the algorithm runs to completion in exact arithmetic, the second that in floating point its answer is accurate. In LU, pivoting is what fixes both problems; for s.p.d. matrices, both hold automatically. (In floating point, completion itself is guaranteed as long as $\kappa_2(A)$ is not too close to $1/\varepsilon_{\text{mach}}$ [7, Theorem 10.7].)

Symmetric indefinite matrices. What if A is Hermitian but not positive definite? A Hermitian matrix is *indefinite* if $\vec{x}^* A\vec{x}$ takes both positive and negative values, equivalently if it has both positive and negative eigenvalues. Then pivots can be zero or negative, and Cholesky fails. Instead, one computes

$$PAP^\top = LDL^\top, \tag{9.26}$$

where P is a permutation and D is block diagonal with 1×1 and 2×2 blocks (LDL^* in the complex case).

Why the symmetric permutation $PAP^\top$ and not PA as in LU? The product $PAP^\top$ swaps rows and columns in the same way, so it keeps the matrix symmetric and we can still work on one half; PA swaps only rows and destroys symmetry. But a symmetric permutation only moves diagonal entries to other diagonal positions; it can never bring an off-diagonal entry onto the diagonal. Consider for instance

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{9.27}$$

Its diagonal is zero, so every 1×1 pivot is zero no matter how we permute. The way out is to take the whole 2×2 block as a single pivot: it is invertible, so it becomes one block of D, with $L = I$.

In general, each step moves an invertible 1×1 or 2×2 block E to the top left and factors

$$PAP^\top = \begin{bmatrix} E & C^* \\ C & B \end{bmatrix} = \begin{bmatrix} I & 0 \\ CE^{-1} & I \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & B - CE^{-1}C^* \end{bmatrix} \begin{bmatrix} I & E^{-1}C^* \\ 0 & I \end{bmatrix}, \tag{9.28}$$

then recurses on the Schur complement $B - CE^{-1}C^*$, exactly as in the derivation of Cholesky. The practical rule for choosing the blocks is *Bunch–Kaufman pivoting* (LAPACK `sytrf`): when a diagonal entry is too small compared with the off-diagonal entries in its column, a 2×2 block is used instead, which avoids small pivots. Its stability is comparable to LU with partial pivoting, and it costs about $\frac{1}{3}m^3$ flops.

9.6 Schur Complements

So far we eliminated one row and column at a time. Let us now do the same thing with blocks. This leads to the Schur complement, which shows up all over numerical linear algebra (and in statistics and optimization).

Block Cholesky. Partition $A = LL^*$ into 2×2 blocks, with L_{11} and L_{22} square lower triangular:

$$L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix}, \qquad A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad LL^* = \begin{bmatrix} L_{11}L_{11}^* & L_{11}L_{21}^* \\ L_{21}L_{11}^* & L_{21}L_{21}^* + L_{22}L_{22}^* \end{bmatrix}. \tag{9.29}$$

Comparing blocks, we have

$$L_{11}L_{11}^* = A_{11}, \qquad L_{21}L_{11}^* = A_{21}, \qquad L_{21}L_{21}^* + L_{22}L_{22}^* = A_{22}. \tag{9.30}$$

(The top right block is the conjugate transpose of the second equation and gives nothing new.) We want to solve for $L_{22}L_{22}^*$. The second equation gives $L_{21} = A_{21}L_{11}^{-*}$, so, using $A_{21}^* = A_{12}$,

$$L_{21}L_{21}^* = A_{21}L_{11}^{-*}L_{11}^{-1}A_{21}^* = A_{21}(L_{11}L_{11}^*)^{-1}A_{21}^* = A_{21}A_{11}^{-1}A_{12}. \tag{9.31}$$

Substituting this into the third equation, we get

$$L_{22}L_{22}^* = A_{22} - A_{21}A_{11}^{-1}A_{12}. \tag{9.32}$$

The matrix on the right has a name.

Definition 52 (Schur Complement)

Let $A = \left[\begin{smallmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{smallmatrix}\right]$ with A_{11} square and invertible. The *Schur complement* of A_{11} in A is

$$A/A_{11} \coloneqq A_{22} - A_{21}A_{11}^{-1}A_{12}. \tag{9.33}$$

The Schur complement S of Section 9.4 is the special case where $A_{11} = \alpha$ is 1×1, so that A_{11}^{-1} is just division by a scalar. (Strictly, the Schur complement is $A_{22} - A_{21}A_{11}^{-1}A_{12}$; as the block equations show, it happens to equal $L_{22}L_{22}^*$.) The block equations tell us two things.

1. L_{22} is the Cholesky factor of the Schur complement. So the algorithm can recurse on blocks instead of single columns.
2. The Schur complement equals $L_{22}L_{22}^*$ with L_{22} invertible, so it is s.p.d. This is a second proof, besides the congruence argument, that the recursion never breaks down.

The Schur complement is also exactly what the column-by-column elimination of Section 9.4 computes. If A_{11} is the leading $k \times k$ block, then after k steps of symmetric elimination,

$$L_k \cdots L_1 A L_1^* \cdots L_k^* = \begin{bmatrix} \operatorname{diag}(d_1, \ldots, d_k) & 0 \\ 0 & A/A_{11} \end{bmatrix}. \tag{9.34}$$

Indeed, both are obtained by eliminating the first k rows and columns of A, and the block equations above show that what is left over in the lower right is $L_{22}L_{22}^* = A/A_{11}$. In other words, *$k$ steps of elimination produce the Schur complement of the leading $k \times k$ block.*

The inverse and the Schur complement. The following lemma is the key tool for the pivot bound. It relates the Schur complement to a block of the inverse.

Lemma 53 (Blocks of the Inverse)

Let $A = \left[\begin{smallmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{smallmatrix}\right]$ be invertible with A_{11} invertible, let $S = A_{22} - A_{21}A_{11}^{-1}A_{12}$ be invertible, and partition $\Theta \coloneqq A^{-1}$ in the same way. Then

$$\Theta_{22} = S^{-1}. \tag{9.35}$$

Proof. The idea is to compute a block LU factorization of A and invert it. Block elimination gives

$$A = \begin{bmatrix} I & 0 \\ A_{21}A_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{11} & 0 \\ 0 & S \end{bmatrix} \begin{bmatrix} I & A_{11}^{-1}A_{12} \\ 0 & I \end{bmatrix}. \tag{9.36}$$

(Multiply it out to check.) A unit block triangular matrix is inverted by negating its off-diagonal block, so

$$A^{-1} = \begin{bmatrix} I & -A_{11}^{-1}A_{12} \\ 0 & I \end{bmatrix} \begin{bmatrix} A_{11}^{-1} & 0 \\ 0 & S^{-1} \end{bmatrix} \begin{bmatrix} I & 0 \\ -A_{21}A_{11}^{-1} & I \end{bmatrix}. \tag{9.37}$$

To read off the lower right block of the product, we only need the second block row of the left factor, which is $\begin{bmatrix} 0 & I \end{bmatrix}$, and the second block column of the right factor, which is $\begin{bmatrix} 0 & I \end{bmatrix}^\top$. So the lower right block of A^{-1} is S^{-1}. □

The proof uses only that A_{11} and S are invertible; neither symmetry nor definiteness is needed. In words, *to get the Schur complement, invert, take the submatrix, and invert again*:

$$A/A_{11} = \left((A^{-1})_{22}\right)^{-1}. \tag{9.38}$$

Note the inverse: it is easy to misremember the lemma as $\Theta_{22} = A_{22} - A_{21}A_{11}^{-1}A_{12}$, but the block of the inverse is S^{-1}, not S. The slogan "invert, take submatrix, invert" keeps the correct version in mind.

Proof of the pivot bound. We can now finish the proof of Theorem 47. The plan: the k-th pivot is a diagonal entry of a Schur complement S'; a diagonal entry is at least $\lambda_{\min}(S')$; and by Lemma 53, $(S')^{-1}$ is a submatrix of A^{-1}, so it cannot be larger than A^{-1}.

Proof of (9.17). Fix k, and let S' be the Schur complement of the leading $(k-1)\times(k-1)$ block of A (for $k=1$, $S'=A$). By the previous subsection, S' is the matrix left over after $k-1$ steps of elimination, so the k-th pivot is its top left entry, $d_k = \vec{e}_1^* S' \vec{e}_1$. We bound d_k in three steps.

1. *A diagonal entry is at least the smallest eigenvalue.* S' is s.p.d., so by the Rayleigh quotient bound $\vec{x}^* S' \vec{x} \geq \lambda_{\min}(S') \|\vec{x}\|^2$,
$$d_k = \vec{e}_1^* S' \vec{e}_1 \geq \lambda_{\min}(S') = \frac{1}{\|(S')^{-1}\|_2}, \tag{9.39}$$
where the last equality holds because the singular values of an s.p.d. matrix are its eigenvalues (Theorem 114).

2. *The inverse of the Schur complement is a submatrix of* A^{-1}. By Lemma 53, $(S')^{-1} = \Theta[k{:}m, k{:}m]$ with $\Theta = A^{-1}$.

3. *A submatrix is no larger than the whole matrix.* $\Theta[k{:}m, k{:}m] = P^\top \Theta P$ for the matrix $P = I[:, k{:}m]$ with orthonormal columns, so $\|(S')^{-1}\|_2 \leq \|A^{-1}\|_2$.

Putting the three steps together,

$$d_k \geq \frac{1}{\|(S')^{-1}\|_2} \geq \frac{1}{\|A^{-1}\|_2} = \lambda_{\min}(A), \tag{9.40}$$

where the last equality again uses that A is s.p.d. □

9.7 Cholesky and QR

We end with a connection that ties this lecture back to least squares: the Cholesky factorization and the QR factorization are two views of the same object. Let $B \in \mathbb{K}^{m\times n}$ have full column rank, so that B^*B is s.p.d. (Theorem 42). If $B = QR$, then

$$B^*B = R^*Q^*QR = R^*R. \tag{9.41}$$

That is, the R of QR is a Cholesky factor of B^*B. The following theorem makes this precise and also gives the converse.

Theorem 54 (QR of B Is Cholesky of B^*B)

Let $B \in \mathbb{K}^{m\times n}$ have full column rank.

1. If $B = \hat{Q}\hat{R}$ is a reduced QR factorization with $r_{jj} > 0$, then $B^*B = \hat{R}^*\hat{R}$ is the Cholesky factorization of B^*B.
2. Conversely, if $B^*B = R^*R$ is the Cholesky factorization, then $B = (BR^{-1})R$ is a reduced QR factorization: BR^{-1} has orthonormal columns.

Proof. For item 1, $B^*B = \hat{R}^*\hat{Q}^*\hat{Q}\hat{R} = \hat{R}^*\hat{R}$, and $\hat{R}^*$ is lower triangular with positive diagonal. By Corollary 48, this is *the* Cholesky factorization. For item 2, R is invertible because its diagonal is positive, and

$$(BR^{-1})^*(BR^{-1}) = R^{-*}B^*BR^{-1} = R^{-*}R^*RR^{-1} = I, \tag{9.42}$$

as desired. □

So QR of B and Cholesky of B^*B compute the same triangular factor. This has three consequences.

- *Uniqueness of QR.* The uniqueness of Cholesky (Corollary 48) gives the uniqueness of reduced QR with $r_{jj} > 0$ (Theorem 33): $\hat{R}$ must be the Cholesky factor of B^*B, and then $\hat{Q} = B\hat{R}^{-1}$.
- *Least squares.* Solving the normal equations with Cholesky computes the same R as QR, but by way of B^*B, whose condition number is $\kappa_2(B)^2$. The two routes agree in exact arithmetic but not in floating point.
- *CholeskyQR.* Item 2 is itself an algorithm: form B^*B, factor it, and set $\hat{Q} = BR^{-1}$. It is fast and consists mostly of matrix multiplications, but for the same reason the computed $\hat{Q}$ loses orthogonality like $\varepsilon_{\text{mach}}\kappa_2(B)^2$, worse than modified Gram–Schmidt.

9.8 (SUPPLEMENT) Further Topics

9.8.1 The Roles of Symmetry and Definiteness

Cholesky needs both conditions, but they do different jobs. Symmetry lets the row and the column eliminations share the same multipliers, which produces the shape LDL^*. Definiteness makes every pivot positive (so no pivoting is needed), keeps the recursion going (every Schur complement is again positive definite), allows the square root at the end, and brings stability.

Case	Outcome
symmetric, positive definite	Cholesky $A = R^*R$, no pivoting, stable
symmetric, indefinite	pivots may vanish; even if LDL^* exists, D has negative entries; use Bunch–Kaufman
positive definite, not symmetric (real)	pivots still positive and LU without pivoting completes, but no R^*R
symmetric positive semidefinite	pivots may vanish; R^*R exists but is not unique; use pivoted Cholesky

In the semidefinite case, $A_{11} = 0$ forces the whole first row and column to vanish (the argument of Lemma 49, applied to a semidefinite matrix, gives $|A_{1j}|^2 \le A_{11}A_{jj}$), so that step can simply be skipped.

9.8.2 Over $\mathbb{R}$, "Positive Definite" Does Not Imply Symmetric

If $A = B^*B$ with B real, then A is automatically symmetric. But if we only ask that $\vec{x}^\top A\vec{x} > 0$ for all real $\vec{x} \neq \vec{0}$, then A need not be symmetric. For example, $A = \left[\begin{smallmatrix} 1 & 1 \\ -1 & 1 \end{smallmatrix}\right]$ has $\vec{x}^\top A\vec{x} = x_1^2 + x_2^2$. The reason is that every real square matrix is the sum of a symmetric part and a skew-symmetric part K, and $\vec{x}^\top K\vec{x} = 0$, so the quadratic form only sees the symmetric part. This is why, over $\mathbb{R}$, symmetry is stated as a separate condition in "s.p.d.". Over $\mathbb{C}$ it is automatic: if $\vec{x}^*A\vec{x}$ is real for all complex $\vec{x}$, then $\vec{x}^*(A - A^*)\vec{x} = 0$ for all $\vec{x}$, and substituting $\vec{x} = \vec{e}_j + \vec{e}_k$ and $\vec{x} = \vec{e}_j + i\vec{e}_k$ (polarization) gives $A = A^*$. The example above, at $\vec{x} = (1, i)^\top$, gives $2 - 2i$, so it is not positive definite over $\mathbb{C}$.

9.8.3 Positive Definiteness Is Necessary for Cholesky

If $A = R^*R$ with R invertible, then $\vec{x}^*A\vec{x} = \|R\vec{x}\|^2 > 0$ for $\vec{x} \neq \vec{0}$, so A must be positive definite. So when A is not positive definite, the algorithm is not failing for lack of cleverness: the factorization it seeks does not exist. Moreover, unit lower triangular congruences do not change the determinants of the leading principal submatrices, so $\det A[1{:}k, 1{:}k] = d_1 \cdots d_k$, and the pivots are ratios of consecutive leading principal minors,

$$d_k = \frac{\det A[1{:}k, 1{:}k]}{\det A[1{:}k-1, 1{:}k-1]}. \tag{9.43}$$

Combined with Sylvester's criterion (a Hermitian matrix is positive definite if and only if all its leading principal minors are positive), this gives

$$\begin{aligned} A \text{ is positive definite} &\iff \text{all leading principal minors are } > 0 \\ &\iff \text{all pivots are } > 0 \iff \text{Cholesky completes.} \end{aligned} \tag{9.44}$$

Cholesky is therefore the standard test for positive definiteness, and much cheaper than computing eigenvalues; `numpy.linalg.cholesky` raises `LinAlgError` on a non-positive-definite input exactly at this point.

9.8.4 Pivots Are Not Eigenvalues

The diagonal entries of D in $A = \tilde{L}D\tilde{L}^*$ are the pivots, which in general differ from the eigenvalues. For example, $\left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$ has eigenvalues $1, 3$ and pivots $2, \frac{3}{2}$; only the products agree, both equal to $\det A = 3$. The reason is that $\tilde{L}^* \neq \tilde{L}^{-1}$: this is a *congruence*, not a similarity (compare $A = Q\Lambda Q^*$ with $Q^* = Q^{-1}$). What a congruence preserves is the *inertia* (Sylvester's law of inertia): the numbers of positive, negative, and zero eigenvalues. So the number of positive pivots equals the number of positive eigenvalues. For instance, $\left[\begin{smallmatrix} 1 & 2 \\ 2 & 1 \end{smallmatrix}\right]$ has eigenvalues $3, -1$ and pivots $1, -3$: different values, but one positive and one negative each. The equivalence "all pivots $> 0 \iff$ positive definite" rests on exactly this, and the block diagonal D of Bunch–Kaufman reveals the inertia of A in the same way.

9.8.5 Definiteness at a Glance

"Definite" means that the sign is determined. A Hermitian matrix is positive (semi)definite if $\vec{x}^*A\vec{x} > 0$ (≥ 0) for all $\vec{x} \neq \vec{0}$, equivalently all eigenvalues are > 0 (≥ 0); negative (semi)definite is defined likewise; and it is indefinite if $\vec{x}^*A\vec{x}$ takes both signs, equivalently there are both positive and negative eigenvalues. For example, $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ has the quadratic form $2x_1x_2$, positive along $(1, 1)$ and negative along $(1, -1)$: its graph is a saddle. In optimization, an indefinite Hessian signals a saddle point. A matrix with eigenvalues $1, 0, -1$ is also indefinite.

9.8.6 Blocked Cholesky and Hardware

Libraries (LAPACK and cuSOLVER `potrf`) use a blocked version, as for LU in Section 2.9.3. Take A_{11} to be a block of size, say, 256×256 and factor it. Then $L_{21} = A_{21} L_{11}^{-*}$ is a triangular solve with many right-hand sides (TRSM), and the Schur complement update $A_{22} - L_{21} L_{21}^{*}$ is a matrix multiplication (SYRK/GEMM). Almost all flops land in Level 3 BLAS and run close to peak. This continues the discussion of memory access in the left- and up-looking variants.

9.8.7 A Statistical Interpretation

If A is the covariance matrix of a Gaussian vector $(\vec{x}_1, \vec{x}_2)$, then the Schur complement $A_{22} - A_{21} A_{11}^{-1} A_{12}$ is exactly the conditional covariance of $\vec{x}_2$ given $\vec{x}_1$. Lemma 53 then says that the corresponding block of the precision matrix A^{-1} is the inverse of the conditional covariance. Much of sparse Cholesky for Gaussian processes and kernel methods builds on this correspondence.

9.8.8 The Normal Equations, Again

Backward stability of Cholesky means that the computed $\tilde{R}$ is the exact Cholesky factor of $A^* A + \delta$ with a small δ. When least squares is solved through the normal equations, however, the matrix being factored is $A^* A$, and the error in $\vec{x}$ is amplified by $\kappa(A^* A) = \kappa(A)^2$. So a stable Cholesky does not make "normal equations plus Cholesky" a stable least squares solver; for ill-conditioned A, QR is still the method of choice (Lecture 8).

Lecture 10

Eigen-like Decompositions, Computing a Single Eigenvalue

Supplementary reading:

- [1] Lectures 24, 25, and 27.
- [2] Sections 4.4.1–4.4.2 and 5.3.2.
- [5] Sections 7.3.1 and 8.2.

10.1 Recap and Motivation

Last lecture we factored an s.p.d. matrix as $A = LL^*$ by symmetric elimination $A \mapsto L_1 A L_1^*$. We showed that no pivoting is needed, that there is no element growth, and that the steps of elimination compute Schur complements (Lemma 53). Recall the precise statement, which is easy to get backwards: for $\Theta = A^{-1}$, the Schur complement $A_{22} - A_{21}A_{11}^{-1}A_{12}$ equals Θ_{22}^{-1}, not Θ_{22}.

From elimination to similarity transformations. So far we have seen LU, Cholesky, and QR. All of them are based on *elimination* or *projection*.

- LU and Cholesky are Gaussian elimination: lower triangular matrices zero out the entries below the pivots, column by column.
- QR uses projection: Gram–Schmidt subtracts from each column its projections onto the previous ones, and Householder reflects each column onto the direction of $\vec{e}_1$.

These factorizations have two things in common.

1. Their purpose is to *solve equations*. The transformations they use are one-sided ($A \mapsto LA$, $A \mapsto Q^*A$) or congruences ($A \mapsto LAL^*$), and none of these preserve eigenvalues. This is why the pivots of Cholesky are not the eigenvalues: a congruence is not a similarity.
2. They finish in finitely many steps, about $O(m^3)$, and are exact if there is no rounding.

Starting with this lecture we study a second family: the eigen, polar, Schur, and singular value decompositions.

These are all closely tied to the *eigenpairs* $(\lambda_i, \vec{x}_i)$ of a matrix. To expose the structure of A as a linear map, they use *similarity transformations* $A \mapsto X^{-1}AX$, which preserve eigenvalues.

Factorization	Form	Exists for	Transformation
Eigen	$A = X\Lambda X^{-1}$	diagonalizable (nondefective) A	general X, possibly ill-conditioned
Polar	$A = QP$, $P = (A^*A)^{1/2} \succeq 0$	every square A	Q unitary
Schur	$A = QTQ^*$, T upper triangular	every square A	Q unitary
SVD	$A = U\Sigma V^*$	every A	U, V unitary

A few remarks on the table.

- Only the eigendecomposition may fail to exist. The other three exist for every (square) matrix and use only unitary transformations, which have condition number 1 and do not amplify errors.
- A *normal* matrix ($A^*A = AA^*$, which includes Hermitian matrices) can be diagonalized by a unitary matrix, $A = Q\Lambda Q^*$.
- The polar decomposition (Theorem 40) is the matrix version of $z = e^{i\theta}r$: Q is a "rotation or reflection" and P is a "stretch". It follows directly from the SVD,

$$A = U\Sigma V^* = (UV^*)(V\Sigma V^*), \qquad Q = UV^*, \quad P = V\Sigma V^*. \tag{10.1}$$

  The factor Q is the unitary matrix closest to A in the Frobenius norm (the orthogonal Procrustes problem).
- The Schur *decomposition* has nothing directly to do with the Schur *complement* of Lecture 9.

We start with the eigendecomposition of Hermitian matrices. Here is our plan.

1. Recall the basic facts about eigenpairs of Hermitian matrices and introduce the Rayleigh quotient.
2. Show that eigenvalues *cannot* be computed exactly in finitely many steps, so every eigenvalue algorithm must be iterative.
3. Develop three iterative methods, each improving on the previous one: power iteration, inverse iteration, and Rayleigh quotient iteration.

10.2 Eigenpairs of Hermitian Matrices

Problem. Given a Hermitian $A \in \mathbb{K}^{m\times m}$, find $\lambda_1, \dots, \lambda_m \in \mathbb{R}$ and $\vec{x}_1, \dots, \vec{x}_m \in \mathbb{K}^m$ with

$$A\vec{x}_i = \lambda_i \vec{x}_i, \qquad i = 1, \dots, m. \tag{10.2}$$

Recall that $\langle \vec{x}, \vec{y} \rangle = \vec{x}^*\vec{y}$ is conjugate-linear in its first argument.

Real eigenvalues and the Rayleigh quotient. Proposition 39 showed that the eigenvalues of a Hermitian matrix are real. Written with inner products, the argument is the chain

$$\lambda_i = \frac{\langle \vec{x}_i, A\vec{x}_i \rangle}{\|\vec{x}_i\|^2} = \frac{\langle A\vec{x}_i, \vec{x}_i \rangle}{\|\vec{x}_i\|^2} = \overline{\lambda_i}. \tag{10.3}$$

Let us justify each step. Two facts are used: the inner product $\langle \vec{x}, \vec{y} \rangle = \vec{x}^*\vec{y}$ is linear in its second argument and conjugate-linear in its first, and $A = A^*$.

- *The first equality.* Substitute $A\vec{x}_i = \lambda_i \vec{x}_i$ into the second argument, where the scalar comes out unchanged:
$$\langle \vec{x}_i, A\vec{x}_i \rangle = \vec{x}_i^*(\lambda_i \vec{x}_i) = \lambda_i \, \vec{x}_i^* \vec{x}_i = \lambda_i \, \|\vec{x}_i\|^2 . \tag{10.4}$$
- *The second equality.* Move A to the other side of the inner product, using $(A\vec{x})^* = \vec{x}^* A^*$ and $A = A^*$:
$$\langle \vec{x}_i, A\vec{x}_i \rangle = \vec{x}_i^* A \vec{x}_i = \vec{x}_i^* A^* \vec{x}_i = (A\vec{x}_i)^* \vec{x}_i = \langle A\vec{x}_i, \vec{x}_i \rangle . \tag{10.5}$$
In general $\langle \vec{x}, A\vec{y} \rangle = \langle A^*\vec{x}, \vec{y} \rangle$; being Hermitian means that A can be moved freely between the two arguments.
- *The third equality.* Substitute $A\vec{x}_i = \lambda_i \vec{x}_i$ again, but now into the first argument, where the conjugate transpose turns λ_i into $\overline{\lambda_i}$:
$$\langle A\vec{x}_i, \vec{x}_i \rangle = (\lambda_i \vec{x}_i)^* \vec{x}_i = \overline{\lambda_i} \, \vec{x}_i^* \vec{x}_i = \overline{\lambda_i} \, \|\vec{x}_i\|^2 . \tag{10.6}$$

Thus $\lambda_i = \overline{\lambda_i}$, i.e., $\lambda_i \in \mathbb{R}$. More compactly, $\vec{x}^* A \vec{x}$ is a 1×1 matrix equal to its own conjugate transpose, $(\vec{x}^* A \vec{x})^* = \vec{x}^* A^* \vec{x} = \vec{x}^* A \vec{x}$, so it is real for *every* $\vec{x}$; dividing by the real number $\|\vec{x}_i\|^2 > 0$ gives $\lambda_i \in \mathbb{R}$.

Note that even for a real symmetric A the argument must allow $\vec{x}_i \in \mathbb{C}^m$: we do not know in advance that λ_i is real, so we cannot assume that its eigenvector is real. Restricting to $\vec{x}_i \in \mathbb{R}^m$ from the start would assume what we want to prove. Notice that the first expression makes sense for *any* nonzero vector, not just for eigenvectors. This motivates the following definition, which will be our main tool for estimating eigenvalues.

Definition 55 (Rayleigh Quotient)

Let $A \in \mathbb{K}^{m \times m}$. The *Rayleigh quotient* of a nonzero $\vec{x} \in \mathbb{K}^m$ is
$$r(\vec{x}) := \frac{\vec{x}^* A \vec{x}}{\vec{x}^* \vec{x}}. \tag{10.7}$$

If A is Hermitian, the Rayleigh quotient is always real, and at an eigenvector it equals the eigenvalue. So if we have a good approximate eigenvector, its Rayleigh quotient is a natural guess for the eigenvalue. We will see that this guess is surprisingly accurate.

Proposition 56 (Orthogonal Eigenvectors)

Let A be Hermitian, $A\vec{x}_i = \lambda_i \vec{x}_i$ and $A\vec{x}_j = \lambda_j \vec{x}_j$ with $\lambda_i \neq \lambda_j$. Then $\vec{x}_i \perp \vec{x}_j$.

Orthogonality of eigenvectors.

Proof. We have
$$\lambda_i \langle \vec{x}_i, \vec{x}_j \rangle = \langle A\vec{x}_i, \vec{x}_j \rangle = \langle \vec{x}_i, A\vec{x}_j \rangle = \lambda_j \langle \vec{x}_i, \vec{x}_j \rangle . \tag{10.8}$$
The first equality is $\langle \lambda_i \vec{x}_i, \vec{x}_j \rangle = \overline{\lambda_i} \langle \vec{x}_i, \vec{x}_j \rangle$ together with $\overline{\lambda_i} = \lambda_i$ (Proposition 39); the second uses $A = A^*$. Thus $(\lambda_i - \lambda_j) \langle \vec{x}_i, \vec{x}_j \rangle = 0$, and since $\lambda_i \neq \lambda_j$, we get $\langle \vec{x}_i, \vec{x}_j \rangle = 0$, as desired. □

Note that the proof uses that the eigenvalues are real, so the two facts must be proved in this order.

Within the eigenspace of a repeated eigenvalue we can orthonormalize by Gram–Schmidt, so orthogonality is never the problem. What we have *not* yet proved is *existence*: why a Hermitian matrix has m linearly independent eigenvectors, i.e., the spectral theorem $A = Q\Lambda Q^*$. The standard route is to prove the Schur decomposition $A = QTQ^*$ for every square matrix, and then note that $T = Q^* A Q$ is both Hermitian and upper triangular, hence diagonal. We will do this in Lecture 11.

10.3 A Negative Result: Eigenvalues Cannot Be Computed in Finitely Many Steps

LU and its relatives can be computed exactly (up to rounding) in $O(m^3)$ arithmetic operations. Can we do the same for eigenvalues? The answer is no, and the obstruction comes from algebra.

Theorem 57 (Abel–Ruffini)

For every $m \geq 5$, there is a polynomial of degree m with rational coefficients whose roots cannot be expressed in terms of its coefficients using finitely many operations $+$, $-$, $\times$, $\div$, and k-th roots.

Why is this a problem for eigenvalues? Because we can turn any polynomial into a matrix whose eigenvalues are its roots. If we could compute eigenvalues exactly, we could then find the roots of any polynomial exactly, contradicting Abel. Let us make this precise.

Lemma 58 (Companion Matrix)

Let $p(z) = z^m + a_{m-1}z^{m-1} + \cdots + a_1 z + a_0$ be monic, and let

$$A = \begin{bmatrix} 0 & 1 & & & \\ & 0 & 1 & & \\ & & \ddots & \ddots & \\ & & & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{m-2} & -a_{m-1} \end{bmatrix}, \qquad \vec{x} = \begin{bmatrix} 1 \\ z \\ z^2 \\ \vdots \\ z^{m-1} \end{bmatrix}. \tag{10.9}$$

If $p(z) = 0$, then $A\vec{x} = z\vec{x}$, so z is an eigenvalue of A with eigenvector $\vec{x} \neq \vec{0}$.

Proof. We check $A\vec{x} = z\vec{x}$ row by row.

- *The first $m-1$ rows.* Row i of A picks out the next entry of $\vec{x}$: $(A\vec{x})_i = x_{i+1} = z^i = z \cdot z^{i-1} = zx_i$.
- *The last row.* Using $p(z) = 0$,

$$(A\vec{x})_m = -\sum_{i=0}^{m-1} a_i z^i = z^m - \underbrace{p(z)}_{=0} = z \cdot z^{m-1} = zx_m. \tag{10.10}$$

Finally $\vec{x} \neq \vec{0}$ since $x_1 = 1$. □

Expanding $\det(zI - A)$ along the last row shows more: $\det(zI - A) = p(z)$, so the eigenvalues of A are *exactly* the roots of p, counted with multiplicity. If p is not monic, divide by the leading coefficient a_m first; the last row is then $-a_i/a_m$.

Theorem 59 (Eigenvalue Algorithms Must Be Iterative)

For $m \geq 5$, there is no algorithm that computes the eigenvalues of every $m \times m$ matrix exactly in finitely many operations $+$, $-$, $\times$, $\div$, and k-th roots.

Proof. Suppose such an algorithm exists. Given any monic polynomial p of degree m, form its companion matrix. This needs no arithmetic at all, since its entries are 0, 1, and the coefficients $-a_i$. By Lemma 58 and the remark after it, the algorithm applied to this matrix returns the roots of p in finitely many operations. This contradicts Theorem 57. □

So every eigenvalue algorithm must be *iterative*: it can only produce a sequence that converges to the eigenvalues.

The direction of the reduction. It is worth being careful about which way the argument goes. The familiar direction "matrix → characteristic polynomial → roots" only shows that the eigenvalue problem is *no harder* than root finding; it proves no impossibility. To show that the eigenvalue problem is *at least as hard* as root finding, we need the opposite direction: every polynomial can be disguised as an eigenvalue problem. Moreover, the disguise must be free. The companion matrix is built from 0, 1, and the coefficients without any computation; using $\mathrm{diag}(z_1, \ldots, z_m)$ instead would require knowing the roots, which is circular.

Where does the companion matrix come from? It is the matrix of the linear map "multiply by z" on the quotient space $\mathbb{C}[z]/(p)$, in the basis $1, z, \ldots, z^{m-1}$. The superdiagonal ones move each basis vector to the next, and the last row uses $p(z) = 0$ to rewrite the overflowing z^m in terms of lower powers.

The conclusion also holds for Hermitian matrices: for a polynomial whose roots are all real, a Sturm sequence (built with arithmetic operations and square roots) gives a real symmetric tridiagonal matrix with that characteristic polynomial.

Iterative does not mean slow. The standard approach has two phases.

1. Finitely many unitary similarity transformations reduce A to Hessenberg form (tridiagonal if A is Hermitian). This phase is a direct method.
2. An iteration runs on the reduced matrix and converges very fast.

The total cost is still $O(m^3)$. In the opposite direction, `roots` in MATLAB and NumPy finds the roots of a polynomial by forming its companion matrix and calling an eigenvalue solver: the reduction above is used in practice.

10.4 Power Iteration

We cannot hope for exact computation. So how can we *approximate* the eigenpairs of A? The simplest idea is to multiply a vector by A over and over. To see why this works, let us first look at what multiplication by A does in the eigenbasis.

Multiplying by A in the eigenbasis. Let $A = X\Lambda X^{-1}$ be diagonalizable. Write any vector as a combination of eigenvectors, $\vec{z} = X\vec{y} = \sum_i y_i \vec{x}_i$, where $\vec{y} = X^{-1}\vec{z}$ holds the coordinates of $\vec{z}$ in the eigenbasis. Then

$$A\vec{z} = X\Lambda\vec{y}, \qquad A^2\vec{z} = X\Lambda^2\vec{y}, \qquad A^k\vec{z} = X\Lambda^k\vec{y} = \sum_{i=1}^{m} \lambda_i^k y_i \vec{x}_i. \tag{10.11}$$

In words: *in the eigenbasis, multiplication by A multiplies each coordinate by its own eigenvalue, and the directions do not interact.*

Now assume that the largest eigenvalue strictly dominates, $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_m|$. Factoring out λ_1^k,

$$A^k \vec{z} = \lambda_1^k \left(y_1 \vec{x}_1 + \sum_{i=2}^{m} \left(\frac{\lambda_i}{\lambda_1} \right)^k y_i \vec{x}_i \right). \tag{10.12}$$

Every ratio $|\lambda_i/\lambda_1|$ with $i \geq 2$ is less than 1, so $(\lambda_i/\lambda_1)^k \to 0$, and the *direction* of $A^k \vec{z}$ approaches $\vec{x}_1$. Informally, $\Lambda^k \approx \lambda_1^k \operatorname{diag}(1, 0, \ldots, 0)$ and $A^k \vec{z} \approx \lambda_1^k y_1 \vec{x}_1$. These approximations hold in the relative sense, $\Lambda^k / \lambda_1^k \to \operatorname{diag}(1, 0, \ldots, 0)$; the vector $A^k \vec{z}$ itself may blow up or vanish.

The condition $y_1 \neq 0$. All of this needs $y_1 \neq 0$: if the starting vector has no component along $\vec{x}_1$, multiplying by A will never create one. What does $y_1 \neq 0$ mean concretely?

The coordinate y_1 is the inner product of the first row of X^{-1} with $\vec{z}$. From $X^{-1}A = \Lambda X^{-1}$, the i-th row $\vec{w}_i^*$ of X^{-1} satisfies $\vec{w}_i^* A = \lambda_i \vec{w}_i^*$; it is a *left eigenvector*. Hence

$$y_1 \neq 0 \iff \langle \vec{w}_1, \vec{z} \rangle \neq 0, \tag{10.13}$$

i.e., the starting vector must not be orthogonal to the left eigenvector of λ_1. If A is Hermitian, then $X^{-1} = Q^*$, the left and right eigenvectors coincide, and the condition simply says that $\vec{z}$ is not orthogonal to $\vec{x}_1$.

What if $y_1 = 0$ anyway? In exact arithmetic the direction $\vec{x}_1$ never appears, and the iteration converges to $\vec{x}_j$ for the first j with $y_j \neq 0$, without noticing. In practice this is not a concern, for two reasons.

- A random starting vector lies on the hyperplane $\vec{w}_1^{\perp}$ with probability 0.
- Even if it does, rounding errors introduce an $O(\varepsilon_{\text{mach}})$ component along $\vec{x}_1$, which is then amplified step by step.

The algorithm. The idea is to compute $\vec{z}_k = A^k \vec{z} / \left\| A^k \vec{z} \right\|$ as an approximation to $\vec{x}_1$, and to use the Rayleigh quotient $\langle \vec{z}_k, A\vec{z}_k \rangle$ as an approximation to λ_1. Choose $\vec{z} \sim N(0, I)$ to ensure $y_1 \neq 0$, and normalize at every step to avoid overflow.

Algorithm 15 Power iteration.

1: $\vec{q} \leftarrow$ vector with i.i.d. standard normal entries; $\vec{q} \leftarrow \vec{q} / \|\vec{q}\|$
2: **for** $k = 1, 2, \ldots$ **do**
3: $\quad \vec{z} \leftarrow A\vec{q}$
4: $\quad \lambda \leftarrow \vec{q}^* \vec{z}$ ▷ Rayleigh quotient, reuses $\vec{z}$
5: $\quad$ **if** $\|\vec{z} - \lambda \vec{q}\| \leq \text{tol} \cdot |\lambda|$ **then**
6: $\quad\quad$ **return** $\lambda, \vec{q}$
7: $\quad \vec{q} \leftarrow \vec{z} / \|\vec{z}\|$

Some details of the algorithm.

- *The random start.* A standard normal vector has a rotation-invariant distribution. After normalization it is uniformly distributed on the unit sphere, and satisfies $y_1 \neq 0$ almost surely.
- *The order of lines 4 and 8.* Line 4 must come before line 8, because $\vec{z} = A\vec{q}$ belongs to the old $\vec{q}$.
- *The order in $\vec{q}^* \vec{z}$.* In the complex case, $\vec{z}^* \vec{q} = \overline{\vec{q}^* A \vec{q}}$, which agrees with $\vec{q}^* \vec{z}$ only when $\vec{q}^* A \vec{q}$ is real. For real A either order works.

- *Cost.* Each step costs one matrix-vector product: $O(m^2)$ for a dense matrix and $O(\mathrm{nnz})$ for a sparse one. So power iteration suits large matrices that we can only multiply by (PageRank, for example).

Why normalize. The length $\left\|A^k\vec{z}\right\|$ changes like $|\lambda_1|^k$, while double precision covers only about $10^{\pm 308}$. With $|\lambda_1| = 10$ the iterates overflow to `Inf` after about 300 steps; with $|\lambda_1| = 0.1$ they underflow to 0. The number of steps needed for convergence depends on $|\lambda_2/\lambda_1|$, not on $|\lambda_1|$, so overflow or underflow would almost always come first, and normalizing only at the end is too late.

Normalizing does not change the result. Since $A(c\vec{v}) = c \cdot A\vec{v}$, scaling at one step does not affect the directions at later steps, and in exact arithmetic $\vec{q}^{(k)} = A^k\vec{q}^{(0)} / \left\|A^k\vec{q}^{(0)}\right\|$. As a bonus, the Rayleigh quotient needs no division by $\|\vec{q}\|^2$, and the stopping test is not affected by scaling.

Why the residual. The stopping test uses the residual $\|A\vec{q} - \lambda\vec{q}\|$, not the change $\left\|\vec{q}^{(k)} - \vec{q}^{(k-1)}\right\|$. The reason: if $\lambda_1 < 0$ (or is complex), $\vec{q}$ changes sign (is multiplied by a phase) at every step, so the *direction* converges but the vector does not. For Hermitian matrices the residual also bounds the eigenvalue error directly.

Proposition 60 (Residual Bound)

Let A be Hermitian, $\vec{q}$ a unit vector, and $\lambda \in \mathbb{R}$. Then A has an eigenvalue λ_j with

$$|\lambda_j - \lambda| \le \|A\vec{q} - \lambda\vec{q}\| . \tag{10.14}$$

Proof. Expand $\vec{q} = \sum_j c_j\vec{x}_j$ in an orthonormal eigenbasis, so that $\sum_j |c_j|^2 = 1$. Then $A\vec{q} - \lambda\vec{q} = \sum_j(\lambda_j - \lambda)c_j\vec{x}_j$, and by orthonormality

$$\|A\vec{q} - \lambda\vec{q}\|^2 = \sum_j |\lambda_j - \lambda|^2 |c_j|^2 \ge \min_j |\lambda_j - \lambda|^2 \sum_j |c_j|^2 = \min_j |\lambda_j - \lambda|^2 , \tag{10.15}$$

as desired. □

Rate of convergence. We now analyze how fast power iteration converges, for Hermitian A. We need to track two errors: the error in the vector and the error in the eigenvalue. The key observation is that the second is much smaller than the first, because the Rayleigh quotient of a Hermitian matrix is *quadratically* accurate.

Lemma 61 (Quadratic Accuracy of the Rayleigh Quotient)

Let A be Hermitian with orthonormal eigenvectors $\vec{x}_1, \dots, \vec{x}_m$, and let $\vec{q}$ be a unit vector. Then for every J,

$$r(\vec{q}) - \lambda_J = \sum_{j \ne J} (\lambda_j - \lambda_J) |c_j|^2 , \qquad c_j = \langle \vec{x}_j, \vec{q} \rangle , \tag{10.16}$$

and hence $|r(\vec{q}) - \lambda_J| \le (\lambda_{\max} - \lambda_{\min}) \|\vec{q} - \vec{x}_J\|^2$.

Proof. Since $\vec{q} = \sum_j c_j\vec{x}_j$ and $\sum_j |c_j|^2 = 1$,

$$r(\vec{q}) = \vec{q}^* A\vec{q} = \sum_j \lambda_j |c_j|^2 = \lambda_J \underbrace{\sum_j |c_j|^2}_{=1} + \sum_{j \ne J} (\lambda_j - \lambda_J) |c_j|^2 . \tag{10.17}$$

For the bound, we use two facts. First, $|\lambda_j - \lambda_J| \le \lambda_{\max} - \lambda_{\min}$. Second,

$$\sum_{j \neq J} |c_j|^2 = \|\vec{q} - c_J \vec{x}_J\|^2 \le \|\vec{q} - \vec{x}_J\|^2 , \tag{10.18}$$

since $c_J \vec{x}_J$ is the orthogonal projection of $\vec{q}$ onto $\operatorname{span}(\vec{x}_J)$, i.e., the closest point of that line to $\vec{q}$. □

Why "quadratic"? The coefficients c_j, $j \neq J$, are the vector error, and the right side is quadratic in them. Another way to see it: the Rayleigh quotient of a Hermitian matrix is *stationary* at eigenvectors. In the real case, $\nabla r(\vec{x}) = \frac{2}{\vec{x}^\top \vec{x}}(A\vec{x} - r(\vec{x})\vec{x})$, which vanishes exactly at eigenvectors. So an ε error in the vector moves the Rayleigh quotient only by $O(\varepsilon^2)$. We return to this picture in Section 10.7.1.

Theorem 62 (Convergence of Power Iteration)

Let A be Hermitian with orthonormal eigenvectors $\vec{x}_1, \dots, \vec{x}_m$ and eigenvalues $|\lambda_1| > |\lambda_2| \ge \cdots \ge |\lambda_m|$, and let the unit starting vector satisfy $c_1 = \left\langle \vec{x}_1, \vec{q}^{(0)} \right\rangle \neq 0$. Then there are unimodular scalars s_k ($s_k = \pm 1$ in the real case) such that the iterates of Algorithm 15 satisfy

$$\left\| \vec{q}^{(k)} - s_k \vec{x}_1 \right\| = O\left(\left| \frac{\lambda_2}{\lambda_1} \right|^k \right), \qquad \left| r(\vec{q}^{(k)}) - \lambda_1 \right| = O\left(\left| \frac{\lambda_2}{\lambda_1} \right|^{2k} \right). \tag{10.19}$$

The plan of the proof is: write $A^k \vec{q}^{(0)}$ as $\lambda_1^k c_1$ times "$\vec{x}_1$ plus a small error", show that normalizing keeps the error small, and then square it with Lemma 61.

Proof. Step 1: the unnormalized iterate. By (10.12) with $y_j = c_j$, we have $A^k \vec{q}^{(0)} = \lambda_1^k c_1 (\vec{x}_1 + \vec{e}_k)$, where

$$\vec{e}_k = \sum_{j=2}^{m} \left(\frac{\lambda_j}{\lambda_1} \right)^k \frac{c_j}{c_1} \vec{x}_j, \qquad \|\vec{e}_k\|^2 = \sum_{j=2}^{m} \left| \frac{\lambda_j}{\lambda_1} \right|^{2k} \frac{|c_j|^2}{|c_1|^2} \le \left| \frac{\lambda_2}{\lambda_1} \right|^{2k} \frac{1}{|c_1|^2}, \tag{10.20}$$

using $\sum_j |c_j|^2 = 1$.

Step 2: normalization. Let $s_k = \lambda_1^k c_1 / \left| \lambda_1^k c_1 \right|$ and $\vec{u} = \vec{x}_1 + \vec{e}_k$, so that $\vec{q}^{(k)} = s_k \vec{u} / \|\vec{u}\|$. Since $\|\vec{x}_1\| = 1$, the triangle inequality gives

$$\left\| \frac{\vec{u}}{\|\vec{u}\|} - \vec{x}_1 \right\| \le \left\| \frac{\vec{u}}{\|\vec{u}\|} - \vec{u} \right\| + \|\vec{u} - \vec{x}_1\| = \underbrace{|1 - \|\vec{u}\||}_{\le \|\vec{u} - \vec{x}_1\|} + \|\vec{u} - \vec{x}_1\| \le 2 \|\vec{e}_k\| , \tag{10.21}$$

so $\left\| \vec{q}^{(k)} - s_k \vec{x}_1 \right\| \le 2 \left| \lambda_2 / \lambda_1 \right|^k / |c_1|$.

Step 3: the eigenvalue. $s_k \vec{x}_1$ is a unit eigenvector for λ_1, so Lemma 61 gives

$$\left| r(\vec{q}^{(k)}) - \lambda_1 \right| \le (\lambda_{\max} - \lambda_{\min}) \left\| \vec{q}^{(k)} - s_k \vec{x}_1 \right\|^2 \le \frac{4(\lambda_{\max} - \lambda_{\min})}{|c_1|^2} \left| \frac{\lambda_2}{\lambda_1} \right|^{2k}, \tag{10.22}$$

as desired. □

The eigenvalue error is often quoted as $O(|\lambda_2/\lambda_1|^k)$, without the factor 2 in the exponent. That is the right rate for a general diagonalizable matrix. For non-Hermitian A, the first estimate still holds, with a constant that also depends on the condition number of X, but the second does not: the eigenvectors are no longer

orthogonal, the expansion of $r(\vec{q})$ contains cross terms linear in the c_j, and the eigenvalue error is only $O(|\lambda_2/\lambda_1|^k)$. For Hermitian A we get the square.

How many steps does this take? Linear convergence to an error ε takes about $k \approx \log\varepsilon / \log|\lambda_2/\lambda_1|$ steps. For a vector accuracy of 10^{-8}, this is about 27 steps when the ratio is 0.5, 175 when it is 0.9, and 1800 when it is 0.99. So the convergence speed depends on λ_2/λ_1, and the natural question is what happens when this ratio is close to 1 in modulus. There are two very different cases: $\lambda_2 \approx \lambda_1$ and $\lambda_2 \approx -\lambda_1$.

When $\lambda_2 \approx \lambda_1$: the eigenvalue still converges. In short, in this case *the convergence in λ is not bad, but the eigenvector converges slowly.*

Suppose $|\lambda_1| \sim |\lambda_2| \gg |\lambda_3|$. In the space spanned by $\vec{x}_1, \vec{x}_2, \vec{x}_3$, the iterates move in two phases.

1. They first fall quickly into the plane $\operatorname{span}(\vec{x}_1, \vec{x}_2)$, at the rate $|\lambda_3/\lambda_1|^k$.
2. Then they turn slowly toward $\vec{x}_1$ within the plane, at the rate $|\lambda_2/\lambda_1|^k$.

After the first phase, $\vec{q} \approx c_1\vec{x}_1 + c_2\vec{x}_2$, and the Rayleigh quotient $\lambda_1 |c_1|^2 + \lambda_2 |c_2|^2$ is a weighted average of λ_1 and λ_2. Its error is at most $|\lambda_1 - \lambda_2|$, which is already small.

In terms of the formula $\sum_{j\geq 2}(\lambda_j - \lambda_1)|c_j|^2$ of Lemma 61: directions far from λ_1 have $|c_j|^2$ decaying quickly, while directions close to λ_1 decay slowly but have a small lever $\lambda_j - \lambda_1$. One of the two factors is always small.

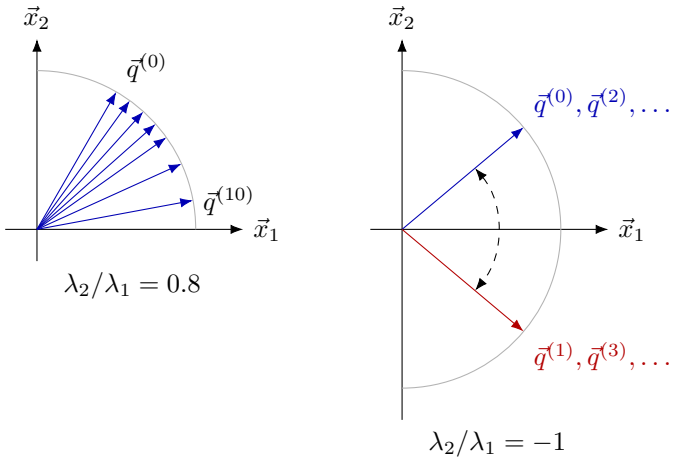
In the extreme case $\lambda_2 = \lambda_1$, $\vec{q}$ converges to some vector in $\operatorname{span}(\vec{x}_1, \vec{x}_2)$ determined by the starting vector. Every nonzero vector in this eigenspace is an eigenvector for λ_1, so the algorithm in fact succeeds, and the eigenvalue converges at the rate $|\lambda_3/\lambda_1|$.

This matches the conditioning of the problem. Eigenvalues of Hermitian matrices are always well-conditioned (Weyl: $|\delta\lambda| \leq \|\delta A\|_2$), whereas the sensitivity of an eigenvector is about $\|\delta A\| / \mathrm{gap}$. When the gap is small, $\vec{x}_1$ itself is ill-conditioned, so we should not expect to compute it quickly.

When $\lambda_2 \approx -\lambda_1$: the real failure. Now only the plane matters: $\vec{q} = c_1\vec{x}_1 + c_2\vec{x}_2$, and one multiplication by A gives $\lambda_1(c_1\vec{x}_1 - c_2\vec{x}_2)$. Geometrically this is a *reflection across the $\vec{x}_1$ axis*. The iterates jump back and forth between two directions symmetric about $\vec{x}_1$. If $\lambda_2 = -\lambda_1$ exactly, $|c_2/c_1|$ never changes and there is no convergence at all. The Rayleigh quotient

$$\vec{q}^* A\vec{q} \approx \lambda_1 \left(|c_1|^2 - |c_2|^2\right) \tag{10.23}$$

is a value determined by the starting vector, which may lie anywhere in $[-\lambda_1, \lambda_1]$.



Left: with $\lambda_2/\lambda_1 = 0.8$ the iterates turn slowly but steadily toward $\vec{x}_1$. Right: with $\lambda_2 = -\lambda_1$ each step reflects $\vec{q}$ across the $\vec{x}_1$ axis, and the iteration never converges.

So although the rate depends only on $|\lambda_2/\lambda_1|$, what is truly fatal is eigenvalues of *similar modulus but far-apart values*; close values are harmless. The cure is to change the spectrum, which is the subject of the next section.

10.5 Matrix Functions: Manipulating the Spectrum

Power iteration finds the eigenvalue of largest modulus, and it is fast when that eigenvalue stands out. So the question becomes: *how can we manipulate the eigenvalues of a matrix, so that the one we want stands out?* The answer is to apply a function to the matrix. For every polynomial p, $p(A)\vec{x}_i = p(\lambda_i)\vec{x}_i$, which gives the following table.

Transformation	Eigenvalues	Eigenvectors
$A \to A^k$	$\lambda \to \lambda^k$	unchanged
$A \to \alpha A$	$\lambda \to \alpha\lambda$	unchanged
$A \to A + \beta I$	$\lambda \to \lambda + \beta$	unchanged
$A \to p(A)$	$\lambda \to p(\lambda)$	unchanged

The important column is the last one: power iteration on $p(A)$ produces eigenvectors of A, and from these the Rayleigh quotient of A reads off the eigenvalues of A.

From this point of view, power iteration itself filters the spectrum with $p(z) = z^k$: it amplifies eigenvalues of large modulus and damps small ones. This is called *polynomial filtering*, and a different p filters differently. Scaling by α does not change the ratios and acts like normalization; the useful operation is the shift.

Shifting. As an example, suppose a cluster of eigenvalues sits on the negative axis and an isolated eigenvalue on the positive axis, with similar moduli. Then the ratio for plain power iteration is close to 1. Iterating with $A + I$ shifts the whole spectrum to the right: the cluster moves toward the origin and the isolated eigenvalue moves farther out, so the ratio drops.

How should we choose the shift in general? Let A have real eigenvalues $\lambda_1 > \lambda_2 \geq \cdots \geq \lambda_m$, ordered by value, and let λ_1 be the one we want. Power iteration on $A + \beta I$ converges at the rate

$$\max_{j \geq 2} \frac{|\lambda_j + \beta|}{|\lambda_1 + \beta|}. \tag{10.24}$$

The best shift moves the midpoint of $[\lambda_m, \lambda_2]$ to the origin, $\beta = -\frac{\lambda_2+\lambda_m}{2}$, and gives the ratio

$$\frac{(\lambda_2 - \lambda_m)/2}{\lambda_1 - \frac{\lambda_2+\lambda_m}{2}}. \tag{10.25}$$

To see that this is optimal: for larger β the ratio is $(\lambda_2 + \beta)/(\lambda_1 + \beta)$, which increases with β; for smaller β it is $-(\lambda_m + \beta)/(\lambda_1 + \beta)$, which decreases with β. The bad case $\lambda_2 \approx -\lambda_1$ of the previous section is solved in the same way: with $\lambda_1 > 0$ and any $\beta > 0$, the ratio becomes $|\beta - \lambda_1| / (\beta + \lambda_1) < 1$.

A shift can only translate the spectrum, and choosing it requires rough knowledge of where the spectrum lies. Two generalizations go further.

- Higher-degree polynomials p can be designed to be large at λ_1 and uniformly small on the interval containing the rest of the spectrum (Chebyshev acceleration).
- Replacing polynomials by rational functions leads to inverse iteration, the subject of the next section.

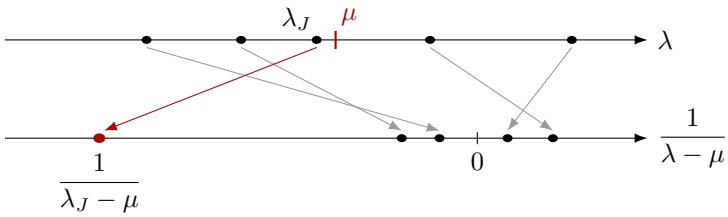
10.6 Inverse Iteration

Power iteration can only find eigenvalues at the edge of the spectrum. What if we want to target an eigenvalue in the *middle*, the one closest to a given $\mu \in \mathbb{R}$? The idea is to use $(A - \mu I)^{-1}$, i.e., the function

$$f(z) = \frac{1}{z - \mu}, \qquad (A - \mu I)^{-1}\vec{x}_j = \frac{1}{\lambda_j - \mu}\vec{x}_j. \tag{10.26}$$

Let us see what this does to the spectrum.

- The eigenvalue λ_J closest to μ is mapped to $1/(\lambda_J - \mu)$, which has the largest modulus.
- The second closest is mapped a bit closer to the origin.
- The far ones are all squeezed toward the origin: those with $\lambda_j < \mu$ onto the negative axis, those with $\lambda_j > \mu$ onto the positive axis.



The map $\lambda \mapsto 1/(\lambda - \mu)$ sends the eigenvalue closest to μ far away and squeezes all the others toward 0.

So power iteration on $(A - \mu I)^{-1}$ finds $\vec{x}_J$. Let λ_K be the second closest eigenvalue to μ.

Theorem 63 (Convergence of Inverse Iteration)

Let A be Hermitian, let λ_J be the eigenvalue closest to μ and λ_K the second closest, with $|\lambda_J - \mu| < |\lambda_K - \mu|$, and let $\left\langle \vec{x}_J, \vec{q}^{(0)} \right\rangle \neq 0$. Then there are unimodular s_k such that the iterates of Algorithm 16 satisfy

$$\left\| \vec{q}^{(k)} - s_k \vec{x}_J \right\| = O\left(\left| \frac{\lambda_J - \mu}{\lambda_K - \mu} \right|^k \right), \qquad \left| \lambda^{(k)} - \lambda_J \right| = O\left(\left| \frac{\lambda_J - \mu}{\lambda_K - \mu} \right|^{2k} \right). \tag{10.27}$$

Proof. The matrix $(A-\mu I)^{-1}$ is Hermitian, with the same orthonormal eigenvectors as A and eigenvalues $1/(\lambda_j-\mu)$. The largest in modulus is $1/(\lambda_J-\mu)$ and the second largest is $1/(\lambda_K-\mu)$. Up to normalization, the iterates of Algorithm 16 are those of Algorithm 15 applied to $(A-\mu I)^{-1}$. So the first estimate is Theorem 62 with the ratio

$$\left|\frac{(\lambda_K-\mu)^{-1}}{(\lambda_J-\mu)^{-1}}\right| = \left|\frac{\lambda_J-\mu}{\lambda_K-\mu}\right|. \tag{10.28}$$

The eigenvalue estimate $\lambda^{(k)} = r(\vec{q}^{(k)})$ is the Rayleigh quotient of A itself, and Lemma 61 applied to A squares the vector error. □

The closer μ is to λ_J, the smaller the ratio. In other words, *the rate of convergence is now controlled by our choice of* μ, not by the intrinsic spectrum of the matrix.

Algorithm 16 Inverse iteration with a fixed shift μ.

1: factor $A-\mu I = PLU$ ▷ once, $O(m^3)$
2: $\vec{q} \leftarrow$ random unit vector
3: **for** $k = 1, 2, \ldots$ **do**
4: solve $(A-\mu I)\vec{z} = \vec{q}$ with the factorization ▷ $O(m^2)$
5: $\vec{q} \leftarrow \vec{z}/\|\vec{z}\|$
6: $\vec{w} \leftarrow A\vec{q}, \quad \lambda \leftarrow \vec{q}^*\vec{w}$ ▷ Rayleigh quotient of A
7: **if** $\|\vec{w}-\lambda\vec{q}\| \le \text{tol}\cdot|\lambda|$ **then**
8: **return** $\lambda, \vec{q}$

In MATLAB notation, line 4 is `z = (A - mu*I) \ q`. Some remarks on implementing it.

- *Factor once.* Since μ is fixed, the factorization is computed once, outside the loop. Solving from scratch in every step would cost $O(m^3)$ per step instead of $O(m^2)$.
- *Which factorization.* For Hermitian A we can use a symmetric indefinite factorization LDL^* (Bunch–Kaufman, Section 9.5), but not Cholesky: with μ in the middle of the spectrum, $A-\mu I$ is indefinite.
- *The eigenvalue estimate.* We use the Rayleigh quotient of A itself, which gives λ_J directly; there is no need to recover it from $1/(\lambda_J-\mu)$.
- *Normalization* is especially important here, since $\|\vec{z}\| \approx 1/|\lambda_J-\mu|$ can be huge.

Ill-conditioning is harmless. The closer μ is to λ_J, the closer $A-\mu I$ is to singular, and the larger the error in the solve. Is this a problem? No: this error lies almost entirely along $\vec{x}_J$, which is exactly the direction we want, and normalization removes it. An exact zero pivot, when μ is exactly an eigenvalue, almost never happens in floating point; if it does, perturbing μ by about $\varepsilon_{\text{mach}}$ suffices.

10.7 Rayleigh Quotient Iteration

In some problems we know a good μ a priori. But what if we don't? And even with a fixed good μ, inverse iteration still converges only linearly. The idea of Rayleigh quotient iteration (RQI) is to *choose* μ *to be our present guess of the eigenvalue*: the Rayleigh quotient of the current vector.

Algorithm 17 Rayleigh quotient iteration.

$\vec{q} \leftarrow$ unit starting vector; $\mu \leftarrow \vec{q}^* A\vec{q}$ (or a given guess)
for $k = 1, 2, \ldots$ **do**
    solve $(A - \mu I)\vec{z} = \vec{q}$
    $\vec{q} \leftarrow \vec{z} / \|\vec{z}\|$
    $\mu \leftarrow \vec{q}^* A\vec{q}$

This sets up a positive feedback loop:

- a more accurate μ gives a smaller inverse iteration ratio, which improves $\vec{q}$ more;
- a better $\vec{q}$ makes the Rayleigh quotient μ more accurate.

In short, the Rayleigh quotient turns good vectors into better eigenvalues, and inverse iteration turns good eigenvalues into good vectors.

10.7.1 The Rayleigh Quotient Near an Eigenvector

To analyze RQI, we first need to know exactly how close $r(\vec{q})$ is to λ_J when $\vec{q}$ is close to $\vec{x}_J$. We already know the answer from Lemma 61, but it can also be derived in a different way, with calculus, which gives a nice picture.

Take $A \in \mathbb{R}^{m\times m}$ symmetric (the complex case is covered by Lemma 61), and view $r(\vec{x}) = \vec{x}^\top A\vec{x}/\vec{x}^\top\vec{x}$ as a function on $\mathbb{R}^m \setminus \{\vec{0}\}$. In inverse iteration and RQI the eigenvalue estimate is $\mu_k = r(\vec{q}^{(k)})$. For an eigenpair, $r(\vec{x}_J) = \lambda_J$, so $\vec{q} \approx \vec{x}_J$ implies $r(\vec{q}) \approx \lambda_J$. How close? Let us compute the Taylor expansion around $\vec{x}_J$, with $\vec{e} = \vec{y} - \vec{x}_J$:

$$r(\vec{y}) = r(\vec{x}_J) + \nabla r(\vec{x}_J)^\top \vec{e} + \tfrac{1}{2}\vec{e}^\top H(\vec{x}_J)\vec{e} + O(\|\vec{e}\|^3), \tag{10.29}$$

where H is the Hessian of r. By the quotient rule, the gradient is

$$\nabla r(\vec{x}) = \frac{2A\vec{x}\,(\vec{x}^\top\vec{x}) - 2\vec{x}\,(\vec{x}^\top A\vec{x})}{(\vec{x}^\top\vec{x})^2} = \frac{2}{\vec{x}^\top\vec{x}}(A\vec{x} - r(\vec{x})\vec{x}). \tag{10.30}$$

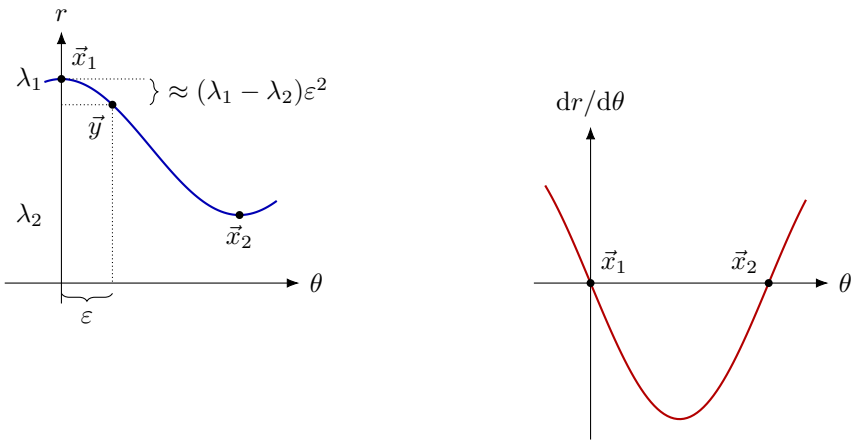
So $\nabla r(\vec{x}) = \vec{0}$ if and only if $\vec{x}$ is an eigenvector: *the eigenvectors are exactly the critical points of* r. The linear term in the Taylor expansion drops out, and

$$r(\vec{y}) = \lambda_J + O(\|\vec{y} - \vec{x}_J\|^2). \tag{10.31}$$

We can go one step further and compute the quadratic term. Differentiating ∇r once more and using $\nabla r(\vec{x}_J) = \vec{0}$ gives, for $\|\vec{x}_J\| = 1$,

$$H(\vec{x}_J) = 2(A - \lambda_J I), \qquad \tfrac{1}{2}\vec{e}^\top H(\vec{x}_J)\vec{e} = \vec{e}^\top(A - \lambda_J I)\vec{e} = \sum_{j\neq J}(\lambda_j - \lambda_J)c_j^2, \quad c_j = \vec{x}_j^\top\vec{e}, \tag{10.32}$$

which is the expression of Lemma 61 again. The Hessian is indefinite unless λ_J is the largest or the smallest eigenvalue. So $\vec{x}_{\max}$ and $\vec{x}_{\min}$ are the maximum and minimum of r, and every other eigenvector is a saddle point.



The Rayleigh quotient on the unit circle $\vec{q} = \cos\theta\,\vec{x}_1 + \sin\theta\,\vec{x}_2$ for $\lambda_1 = 3$, $\lambda_2 = 1$: $r(\theta) = \lambda_2 + (\lambda_1 - \lambda_2)\cos^2\theta$. The derivative vanishes at the eigenvectors, so an angle error ε moves r by only $(\lambda_1 - \lambda_2)\sin^2\varepsilon \approx (\lambda_1 - \lambda_2)\varepsilon^2$.

In RQI the next vector is $\vec{q}^{(k+1)} \propto (A - r(\vec{q}^{(k)})I)^{-1}\vec{q}^{(k)}$. The good news is that *getting $\vec{q}$ close to $\vec{x}_J$ means getting μ even closer to λ_J.* A vector error ε gives an eigenvalue error $O(\varepsilon^2)$.

Analogy with Newton's method. RQI resembles Newton's method, and the comparison can be made precise. An eigenpair is a root of

$$F(\vec{x}, \lambda) = \begin{bmatrix} (A - \lambda I)\vec{x} \\ \frac{1}{2}(1 - \vec{x}^\top \vec{x}) \end{bmatrix}, \qquad F'(\vec{x}, \lambda) = \begin{bmatrix} A - \lambda I & -\vec{x} \\ -\vec{x}^\top & 0 \end{bmatrix}. \tag{10.33}$$

The first block row of the Newton step $F'(\vec{x}, \lambda)(\Delta\vec{x}, \Delta\lambda) = -F(\vec{x}, \lambda)$ reads $(A - \lambda I)\Delta\vec{x} - \Delta\lambda\,\vec{x} = -(A - \lambda I)\vec{x}$, i.e.,

$$\vec{x} + \Delta\vec{x} = \Delta\lambda\,(A - \lambda I)^{-1}\vec{x}. \tag{10.34}$$

So the Newton update of the vector is parallel to one step of inverse iteration with shift λ. RQI takes this direction, normalizes it, and then replaces Newton's $\lambda + \Delta\lambda$ by the Rayleigh quotient of the new vector. Newton's method converges quadratically because the error of the linearization is quadratic; for symmetric A, the stationarity of r adds one more order.

10.7.2 Cubic Convergence

Theorem 64 (Convergence of Rayleigh Quotient Iteration [1, Thm. 27.3])

Let A be Hermitian. If RQI converges to an eigenpair $(\lambda_J, \vec{x}_J)$, then for $\vec{q}^{(k)}$ sufficiently close to $\pm\vec{x}_J$ (in the complex case, to a unimodular multiple of $\vec{x}_J$),

$$\left\|\vec{q}^{(k+1)} - (\pm\vec{x}_J)\right\| = O\left(\left\|\vec{q}^{(k)} - (\pm\vec{x}_J)\right\|^3\right), \qquad \left|\mu^{(k+1)} - \lambda_J\right| = O\left(\left|\mu^{(k)} - \lambda_J\right|^3\right). \tag{10.35}$$

For a general (non-Hermitian) matrix, the convergence is quadratic instead.

Why cubic: the heuristic. Before the proof, let us see where the exponent 3 comes from. Suppose $\left\|\vec{q}^{(k)} - \vec{x}_J\right\| = \varepsilon$.

1. One step of inverse iteration multiplies the vector error by $|\lambda_J - \mu| \,/\, |\lambda_K - \mu|$. The denominator is roughly the spectral gap and stays bounded away from 0, so the new error is about $\varepsilon\,|\lambda_J - \mu|$.

2. For a general matrix the Rayleigh quotient is only first-order accurate, $|\mu - \lambda_J| = O(\varepsilon)$, so the new error is $O(\varepsilon^2)$: quadratic convergence.
3. For Hermitian A, Lemma 61 gives $|\mu - \lambda_J| = O(\varepsilon^2)$, so the new error is $O(\varepsilon^3)$: cubic convergence.

In the cubic case the number of correct digits triples at every step, $10^{-2} \to 10^{-6} \to 10^{-18}$. The following proposition makes this precise.

Proposition 65 (One Step of RQI)

Let A be Hermitian with orthonormal eigenvectors $\vec{x}_1, \dots, \vec{x}_m$, let λ_J be simple, and let

$$g := \min_{j \neq J} |\lambda_j - \lambda_J| > 0, \qquad s_A := \lambda_{\max} - \lambda_{\min}. \tag{10.36}$$

Let $\vec{q}$ be a unit vector with $\|\vec{q} - \vec{x}_J\| \leq \varepsilon$, where $\varepsilon \leq \frac{1}{2}$ and $s_A \varepsilon^2 \leq g/2$, let $\mu = r(\vec{q}) \neq \lambda_J$, and let $\vec{q}^+ = (A - \mu I)^{-1}\vec{q} / \|(A - \mu I)^{-1}\vec{q}\|$. Then there is a unimodular s with

$$\left\|\vec{q}^+ - s\vec{x}_J\right\| \leq \frac{4 s_A}{g} \varepsilon^3. \tag{10.37}$$

Proof. *Step 1: split $\vec{q}$.* Write $\vec{q} = \alpha \vec{x}_J + \beta \vec{v}$ with $\alpha = \langle \vec{x}_J, \vec{q} \rangle$, $\vec{v} \perp \vec{x}_J$, and $\|\vec{v}\| = 1$. Since $\alpha \vec{x}_J$ is the closest point to $\vec{q}$ on $\operatorname{span}(\vec{x}_J)$, we have $|\beta| = \|\vec{q} - \alpha \vec{x}_J\| \leq \|\vec{q} - \vec{x}_J\| \leq \varepsilon$. Also $|\alpha - 1| = |\langle \vec{x}_J, \vec{q} - \vec{x}_J \rangle| \leq \varepsilon$, so $|\alpha| \geq \frac{1}{2}$.

Step 2: apply $(A - \mu I)^{-1}$. The vector $\vec{x}_J$ is an eigenvector of $(A - \mu I)^{-1}$ with eigenvalue $1/(\lambda_J - \mu)$, so

$$(A - \mu I)^{-1}\vec{q} = \frac{\alpha}{\lambda_J - \mu} \vec{x}_J + \beta (A - \mu I)^{-1} \vec{v}. \tag{10.38}$$

We bound the second term. The subspace $\operatorname{span}\{\vec{x}_j : j \neq J\}$ contains $\vec{v}$ and is invariant under $(A - \mu I)^{-1}$, so $(A - \mu I)^{-1}\vec{v} \perp \vec{x}_J$ and $\|(A - \mu I)^{-1}\vec{v}\| \leq \max_{j \neq J} 1/|\lambda_j - \mu|$. By Lemma 61, $|\mu - \lambda_J| \leq s_A \varepsilon^2 \leq g/2$, hence for $j \neq J$

$$|\lambda_j - \mu| \geq |\lambda_j - \lambda_J| - |\lambda_J - \mu| \geq g/2. \tag{10.39}$$

Step 3: factor out the main term. With $c = \alpha/(\lambda_J - \mu)$,

$$(A - \mu I)^{-1}\vec{q} = c(\vec{x}_J + \vec{d}), \qquad \vec{d} = \frac{\beta(\lambda_J - \mu)}{\alpha} (A - \mu I)^{-1}\vec{v} \perp \vec{x}_J, \tag{10.40}$$

and

$$\left\|\vec{d}\right\| \leq \frac{|\beta|\,|\lambda_J - \mu|}{|\alpha|} \cdot \frac{2}{g} \leq \frac{4}{g} \varepsilon\,|\lambda_J - \mu| \leq \frac{4 s_A}{g} \varepsilon^3. \tag{10.41}$$

In O-notation, the middle bound is $O(\varepsilon\,|\lambda_J - \mu|)$, and the last step uses $|\lambda_J - \mu| = O(\varepsilon^2)$.

Step 4: normalize. With $s = c/|c|$, we have $\vec{q}^+ = s\vec{u}$, where $\vec{u} = (\vec{x}_J + \vec{d}) / \left\|\vec{x}_J + \vec{d}\right\|$. Let $t = \left\|\vec{d}\right\|$. Since $\vec{d} \perp \vec{x}_J$, $\left\|\vec{x}_J + \vec{d}\right\| = \sqrt{1 + t^2}$, and

$$\|\vec{u} - \vec{x}_J\|^2 = \left(1 - \frac{1}{\sqrt{1 + t^2}}\right)^2 + \frac{t^2}{1 + t^2} = 2 - \frac{2}{\sqrt{1 + t^2}} \leq t^2, \tag{10.42}$$

using $1/\sqrt{1 + t^2} \geq 1 - t^2/2$. Hence $\left\|\vec{q}^+ - s\vec{x}_J\right\| = \|\vec{u} - \vec{x}_J\| \leq t$, as desired. □

Step 4 is the rigorous form of the estimate $\frac{1}{a+\varepsilon} - \frac{1}{a} \approx -\frac{\varepsilon}{a^2}$: dividing by a length that is perturbed by a small amount changes the result only by a perturbation of the same order, so normalizing does not destroy the $O(\varepsilon^3)$ bound.

From one step to the theorem. Since $s\vec{x}_J$ is again a unit eigenvector for λ_J, the proposition can be applied at every step. With $\varepsilon_k = \left\|\vec{q}^{(k)} - s_k\vec{x}_J\right\|$ it gives $\varepsilon_{k+1} \le C\varepsilon_k^3$ with $C = 4s_A/g$, and the bound $\left|\mu^{(k)} - \lambda_J\right| \le s_A\varepsilon_k^2$ on the eigenvalue error is cubed as well. This proves the Hermitian case of Theorem 64. Here "sufficiently close" means $\varepsilon \le \frac{1}{2}$ and $s_A\varepsilon^2 \le g/2$, together with $C\varepsilon^2 < 1$ so that the errors decrease. If $\mu = \lambda_J$ exactly, the iteration has already converged.

Note that this is a *local* statement: it applies only once $\vec{q}^{(k)}$ is already close enough to some eigenvector.

Cost and limitations.

- *Cost.* Since μ changes at every step, the factorization cannot be reused, and each step costs $O(m^3)$ for a dense matrix. In practice A is first reduced to tridiagonal (Hermitian) or Hessenberg form, after which each solve costs only $O(m)$ or $O(m^2)$. The shifted QR algorithm is essentially RQI in disguise.
- *Near-singular systems.* As RQI converges, $A - \mu I$ becomes ever closer to singular. As for inverse iteration, this is ill-conditioning but harmless; an exact zero pivot means that the iteration has converged, and we can stop.
- *Which eigenvalue.* Which eigenvalue RQI converges to depends on the starting vector and cannot be prescribed in advance. For Hermitian matrices it can be shown to converge for almost every starting vector. A common practice is to run a few steps of inverse iteration with a fixed shift to steer the vector toward the target, and then switch to RQI to accelerate.

10.8 (SUPPLEMENT) Further Topics

10.8.1 The Jordan Normal Form

Diagonalizability requires m linearly independent eigenvectors, and this can fail. A counterexample is

$$J = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}, \tag{10.43}$$

whose eigenvalue λ has algebraic multiplicity 2, but $\mathcal{N}(J - \lambda I) = \operatorname{span}(\vec{e}_1)$, so its geometric multiplicity is 1. A matrix with an eigenvalue whose geometric multiplicity is smaller than its algebraic multiplicity is called *defective*.

Every square matrix is similar to a Jordan normal form $A = XJX^{-1}$, $J = \operatorname{diag}(J_1, \dots, J_p)$, where each Jordan block is $J_i = \lambda_i I + N$ with N the nilpotent matrix with ones on the superdiagonal; J is unique up to the order of the blocks.

- The algebraic multiplicity of an eigenvalue is the total size of its blocks, and the geometric multiplicity is the number of its blocks. So A is diagonalizable if and only if all blocks are 1×1.
- The columns of X belonging to a $k \times k$ block satisfy $(A - \lambda I)\vec{x}_j = \vec{x}_{j-1}$: $\vec{x}_1$ is an eigenvector and the rest are *generalized eigenvectors*.

In theory, the Jordan form explains the transient growth of defective matrices: $J_i^n = \sum_{j=0}^{k-1} \binom{n}{j} \lambda^{n-j} N^j$, so even with $|\lambda| < 1$, $\|A^n\|$ may first grow like $n^{k-1}\,|\lambda|^n$. The factor t in the solutions of ODEs with repeated roots has the same origin.

Numerically, however, the Jordan form is not used, because it is discontinuous under perturbation. For example, $\left[\begin{smallmatrix} \lambda & 1 \\ \varepsilon & \lambda \end{smallmatrix}\right]$ has eigenvalues $\lambda \pm \sqrt{\varepsilon}$ and is immediately diagonalizable, and a $k \times k$ block amplifies perturbations like $\varepsilon^{1/k}$. In floating point, whether a matrix is defective and how large its blocks are cannot be decided, and $\kappa(X) \to \infty$ as a matrix approaches a defective one. This is why we give up on diagonal form and settle for the upper triangular Schur form.

10.8.2 Why Not Go Through the Characteristic Polynomial

Even in low degrees where root formulas exist, we do not compute $\det(zI - A)$ and then its roots. The reason is that the step from a matrix to the coefficients of its polynomial is extremely ill-conditioned. For the Wilkinson polynomial $\prod_{i=1}^{20}(z-i)$, changing the coefficient of z^{19} by about 10^{-10} sends several roots into the complex plane, moving them by $O(1)$; yet the eigenvalues of $\mathrm{diag}(1, \ldots, 20)$ are perfectly well-conditioned under perturbations of the matrix. So all iterative methods of this lecture act directly on the matrix, using only matrix-vector products or linear solves. They exploit $p(A)\vec{x}_i = p(\lambda_i)\vec{x}_i$, rather than approximating roots of a polynomial.

10.8.3 A Gap-Independent Eigenvalue Bound

Let A be s.p.d. with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots > 0$ and orthonormal eigenvectors, and let $t_j = \lambda_j/\lambda_1 \in (0, 1]$. The Rayleigh quotient of power iteration satisfies

$$\frac{\lambda_1 - r(\vec{q}^{(k)})}{\lambda_1} = \frac{\sum_{j\geq 2}(1-t_j)t_j^{2k}\,|y_j|^2}{\sum_j t_j^{2k}\,|y_j|^2} \leq \sum_{j\geq 2}(1-t_j)t_j^{2k}\frac{|y_j|^2}{|y_1|^2} \leq \frac{1}{2k+1}\cdot\frac{\|\vec{y}\|^2}{|y_1|^2}. \tag{10.44}$$

The last step holds because the maximum of $(1-t)t^{2k}$ on $[0,1]$, attained at $t = \frac{2k}{2k+1}$, is at most $\frac{1}{2k+1}$. This bound does not depend on λ_2/λ_1: however small the gap, the relative eigenvalue error decreases at least like $O(1/k)$.

10.8.4 Matrix Functions in General

A matrix function does not act entrywise (in Julia, `exp(A)` is the matrix exponential, while `exp.(A)` acts entrywise). Polynomials extend to power series, such as $e^A = \sum_k A^k/k!$, and for diagonalizable A

$$f(A) = X f(\Lambda) X^{-1}, \qquad f(\Lambda) = \mathrm{diag}(f(\lambda_1), \ldots, f(\lambda_m)) \tag{10.45}$$

can serve as the definition for a general f. For example, an s.p.d. matrix has the square root $\sqrt{A} = Q\,\mathrm{diag}(\sqrt{\lambda_i})Q^*$, which is how $P = (A^*A)^{1/2}$ in the polar decomposition is defined. For a Jordan block, $f(J)$ is upper triangular, with $f'(\lambda), f''(\lambda)/2!, \ldots$ on the successive superdiagonals. The methods of this lecture are all choices of f:

$f(z)$	Amplified eigenvalue	Method
z	largest modulus	power iteration
$z + \sigma$	breaks symmetry of moduli	shifted power iteration
$(z-\mu)^{-1}$	closest to μ	inverse iteration
$(z-\mu_k)^{-1}$, μ_k updated	adaptive	Rayleigh quotient iteration
well-designed polynomial $p_k(z)$ of degree k	small on the rest of the spectrum	Chebyshev acceleration, Krylov methods

The last row is the natural generalization of power iteration: $A^k\vec{q}$ is just one of many polynomials of degree k in A applied to $\vec{q}$, and searching for the best one in $\operatorname{span}(\vec{q}, A\vec{q}, \ldots, A^k\vec{q})$ is what Krylov subspace methods (Lanczos, Arnoldi) do.

Numerically, when X is ill-conditioned, the error of $Xf(\Lambda)X^{-1}$ is amplified by $\kappa(X)$. For Hermitian A, $X = Q$ is unitary and the formula is safe. For general matrices one uses the Schur decomposition $f(A) = Qf(T)Q^*$ and a recurrence on the triangular T (Schur–Parlett); e^A is computed by scaling and squaring, $e^A = (e^{A/2^s})^{2^s}$, with a Padé approximant on the small matrix. See Higham [8].

10.8.5 Why Iterative Approximation Is Reliable

Eigenvalues cannot be computed exactly in finitely many steps, yet iterative approximation is numerically on equal footing with direct methods. There are three layers to this.

- *Why the vector converges.* Every vector is a combination of eigenvectors, and in $f(A)^k\vec{q} = \sum_j f(\lambda_j)^k y_j \vec{x}_j$ each direction is independently multiplied by $f(\lambda_j)^k$. Exponential growth lets the direction with the largest $|f(\lambda)|$ dominate, with geometric convergence. The three algorithms differ only in the choice of f.
- *Why the eigenvalue can be read off.* The Rayleigh quotient equals the eigenvalue at an eigenvector and (for Hermitian matrices) is stationary there, so a roughly correct vector already gives a quadratically accurate eigenvalue.
- *Why an approximation is good enough.* For any unit vector $\vec{q}$ and number μ, let $\vec{r} = A\vec{q} - \mu\vec{q}$ and $\delta A = -\vec{r}\vec{q}^*$. Then

$$(A + \delta A)\vec{q} = A\vec{q} - \vec{r} = \mu\vec{q}, \qquad \|\delta A\|_2 = \|\vec{r}\|\,\|\vec{q}\| = \|\vec{r}\|\,. \tag{10.46}$$

  So an approximate eigenpair with residual $\|\vec{r}\|$ is an *exact* eigenpair of a matrix that differs from A by $\|\vec{r}\|$. The input A already carries errors of order $\varepsilon_{\text{mach}}$, so once $\|\vec{r}\| \approx \varepsilon_{\text{mach}}\|A\|$, the result cannot be distinguished from an exact computation on a matrix indistinguishable from A. This is exactly backward stability (Definition 120), the same standard as for LU and QR. Combined with the well-conditioning of Hermitian eigenvalues (Weyl), a small backward error guarantees a small forward error. This is the fundamental reason for using the residual as the stopping criterion.

10.8.6 A Preview of Subspace Iteration

In the example $|\lambda_1| \sim |\lambda_2| \gg |\lambda_3|$, what converges quickly is not a vector but the subspace $\operatorname{span}(\vec{x}_1, \vec{x}_2)$, at the rate $|\lambda_3/\lambda_2|$, regardless of how close λ_1 and λ_2 are. Iterating several vectors at once and keeping them orthonormal by a QR factorization at each step quickly produces an invariant subspace; solving a small eigenvalue problem inside it then separates the eigenpairs. This is simultaneous (subspace) iteration, the precursor of the QR algorithm. When $\lambda_2 \approx -\lambda_1$, two consecutive iterates $\vec{q}^{(k)}, \vec{q}^{(k+1)}$ also span this invariant subspace, and the same approach separates $\pm\lambda_1$.

Lecture 11

Schur Factorization

Supplementary reading:

- [1] Lecture 24.
- [2] Sections 4.2–4.3.
- [4] Sections 5A, 5C, 6B, 7B, and 8A–8C (linear algebra background).

11.1 Recap and Motivation

Last lecture we started on eigenvalue problems.

- *A negative result.* Eigenvalues cannot be computed in finitely many steps (Theorem 59): the companion matrix turns every polynomial into an eigenvalue problem, and Abel rules out a finite formula for the roots. So every eigenvalue algorithm is iterative.
- *Power iteration.* Expanding the starting vector in the eigenbasis as $\vec{z} = X\vec{y}$ gives $A^k\vec{z} = X\Lambda^k\vec{y}$, and after normalization Λ^k keeps only the dominant eigenvalue.
- *Matrix functions.* We can move the spectrum around: a shift $A + \sigma I$ can make the wanted eigenvalue dominant, and inverse iteration and Rayleigh quotient iteration use $f(z) = (z - \mu)^{-1}$.

All of these methods compute *one* eigenpair. This lecture asks two questions.

1. How do we get *all* the eigenpairs, efficiently?
2. What object should "all eigenpairs" be? We will see that the obvious answer, the eigenvalue decomposition, does not always exist and is numerically unstable. The right answer is the *Schur factorization*.

11.2 Deflation

We have a fast way to compute *one* eigenpair. What if we want all of them? Suppose we have already computed the largest eigenpair $(\lambda_1, \vec{x}_1)$. If we start power iteration again from a new random vector, it converges to $\vec{x}_1$ again. How do we avoid recomputing the same thing?

The idea is to work with a modified matrix from which λ_1 has been removed: same eigenpairs, except that what was λ_1 is now 0. For Hermitian A this is a rank-one update, called Hotelling deflation.

Proposition 66 (Hotelling Deflation)

Let $A \in \mathbb{K}^{m\times m}$ be Hermitian with eigenpairs $(\lambda_i, \vec{x}_i)$, where the $\vec{x}_i$ are pairwise orthogonal. Let

$$B \coloneqq A - \lambda_1 \frac{\vec{x}_1 \vec{x}_1^*}{\vec{x}_1^* \vec{x}_1}. \tag{11.1}$$

Then $B\vec{x}_1 = \vec{0}$ and $B\vec{x}_i = \lambda_i \vec{x}_i$ for $i = 2, \ldots, m$. In particular, the eigenvalues of B are $0, \lambda_2, \ldots, \lambda_m$.

Proof. We check the two cases.

- For $i = 1$,
$$B\vec{x}_1 = A\vec{x}_1 - \lambda_1 \vec{x}_1 \frac{\vec{x}_1^* \vec{x}_1}{\vec{x}_1^* \vec{x}_1} = \lambda_1 \vec{x}_1 - \lambda_1 \vec{x}_1 = \vec{0}. \tag{11.2}$$
- For $i \geq 2$, orthogonality gives $\vec{x}_1^* \vec{x}_i = 0$, so the correction term vanishes and $B\vec{x}_i = A\vec{x}_i = \lambda_i \vec{x}_i$. □

The only property of Hermitian matrices used here is orthogonality of the eigenvectors. For distinct eigenvalues it is automatic (Proposition 56); within the eigenspace of a repeated eigenvalue the eigenvectors can be chosen orthogonal by Gram–Schmidt. Note also that $\vec{x}_1 \vec{x}_1^* / (\vec{x}_1^* \vec{x}_1)$ is the orthogonal projector onto $\operatorname{span}(\vec{x}_1)$, which is why $\vec{x}_1$ need not be normalized.

Power iteration applied to B then converges to the eigenpair of the next largest modulus. Repeating this, we can compute the eigenpairs one by one.

Implementation. We never form B: if A is sparse, B is dense. Instead we compute

$$B\vec{z} = A\vec{z} - \lambda_1 \vec{x}_1 \frac{\vec{x}_1^* \vec{z}}{\vec{x}_1^* \vec{x}_1}, \tag{11.3}$$

which costs one product with A plus $O(m)$ work.

There is another way to see what B does. With $P \coloneqq I - \vec{x}_1 \vec{x}_1^* / (\vec{x}_1^* \vec{x}_1)$, one checks that $B = AP = PA$. So power iteration on B is power iteration on A with the $\vec{x}_1$ component projected out. Could we just project the starting vector once and then iterate with A?

- In exact arithmetic, yes.
- In floating point, no. Rounding reintroduces a component of size $O(\varepsilon_{\text{mach}})$ along $\vec{x}_1$, which then grows like $|\lambda_1/\lambda_2|^k$ and eventually takes over.

So the projection must be applied at every step, which is exactly what multiplying by $B = AP$ does.

Limitations.

- The computed $(\lambda_1, \vec{x}_1)$ is only approximate, and its error is inherited by B and hence by every eigenpair computed after it. Deflating many times in a row accumulates these errors.
- For non-Hermitian A, the eigenvectors are not orthogonal and the proof above fails. See Section 11.9 for a version that works, and Section 11.7 for the stable, unitary way of deflating.

11.3 Subspace Iteration: Motivation

So far we can compute individual eigenvalues, and in principle, with deflation, we can compute all eigenpairs one by one. The problem is cost. Surprisingly, the issue is not the number of flops but memory traffic.

- Computing N eigenpairs needs at least N matrix-vector products, and in practice many more, since each eigenpair needs many iterations.
- These matrix-vector products happen *serially*. The $(k+1)$-st eigenpair cannot be started until the k-th has converged and been deflated.
- Each matrix-vector product must load the whole matrix from RAM. It performs $2m^2$ flops on m^2 entries of A, a computational intensity $q \approx 2$ (Section 2.9.2). It is a Level 2 operation, limited by memory bandwidth.

So we load the matrix from RAM at least N times, one after another.

The alternative is to run power iteration on *multiple vectors simultaneously*. With $Z \in \mathbb{K}^{m\times n}$, the product AZ performs $2m^2n$ flops while reading A only once, so its intensity grows proportionally to n. Matrix-vector products become matrix-matrix products (Level 3), which run near peak speed with the tiling of Lecture 2. This is *subspace iteration*.

Simply iterating $Z \mapsto AZ$ is not enough, though. Every column is then an independent power iteration, so every column converges to the same direction $\vec{x}_1$. The columns must be kept linearly independent, for example by orthonormalizing them with a QR factorization after each multiplication. The algorithm and its convergence come in a later lecture; the end of Lecture 10 gave a preview.

This lecture answers a prior question: *when we iterate a whole set of vectors, what object does the iteration converge to?* The answer (Section 11.7) is an orthonormal basis of nested invariant subspaces, i.e., a Schur factorization. We build up to it in the rest of this chapter.

11.4 Eigenvalue Problems as Matrix Factorizations

To ask for "all eigenpairs at once" we need to write them as one object. The first candidate is the eigenvalue decomposition $A = X\Lambda X^{-1}$. When does it exist? We start with the simplest sufficient condition: distinct eigenvalues.

Lemma 67 (Eigenvectors for Distinct Eigenvalues)

Eigenvectors $\vec{x}_1, \ldots, \vec{x}_k$ of A corresponding to distinct eigenvalues $\lambda_1, \ldots, \lambda_k$ are linearly independent [4, 5.11].

Proof. Suppose not, and choose a shortest nontrivial dependency $\sum_{i\in S} c_i\vec{x}_i = \vec{0}$, where all $c_i \neq 0$. Since eigenvectors are nonzero, S has at least two elements. The idea is to kill one term by applying $A - \lambda_j I$. Fix some $j \in S$; then

$$\sum_{i\in S\setminus\{j\}} c_i(\lambda_i - \lambda_j)\vec{x}_i = \vec{0}. \tag{11.4}$$

All coefficients $c_i(\lambda_i - \lambda_j)$ are nonzero because the eigenvalues are distinct, so this is a shorter nontrivial dependency, a contradiction. □

Proposition 68 (Distinct Eigenvalues Give an Eigenvalue Decomposition)

If $A \in \mathbb{K}^{m\times m}$ has m distinct eigenvalues $\lambda_1, \dots, \lambda_m$ with eigenvectors $\vec{x}_1, \dots, \vec{x}_m$, then

$$A = X\Lambda X^{-1}, \qquad X = \begin{bmatrix} \vec{x}_1 & \cdots & \vec{x}_m \end{bmatrix}, \quad \Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_m). \tag{11.5}$$

Proof. The m equations $A\vec{x}_i = \lambda_i \vec{x}_i$, written column by column, are $AX = X\Lambda$. By Lemma 67, X has linearly independent columns, so it is invertible, and we can multiply by X^{-1} on the right. □

Distinct eigenvalues are sufficient but not necessary: I has the single eigenvalue 1 and is diagonal. The exact condition is given in Theorem 73.

For self-adjoint matrices, we can say more: the eigenvectors can be chosen orthonormal. Let us first recall the term.

Definition 69 (Adjoint and Self-Adjoint)

Let $\langle \vec{x}, \vec{y} \rangle = \vec{x}^* \vec{y}$ be the standard inner product on $\mathbb{K}^m$. The *adjoint* of $A \in \mathbb{K}^{m\times m}$ is the unique matrix A^* with

$$\langle A\vec{x}, \vec{y} \rangle = \langle \vec{x}, A^*\vec{y} \rangle \quad \text{for all } \vec{x}, \vec{y} \in \mathbb{K}^m. \tag{11.6}$$

A is *self-adjoint* if $A^* = A$.

Since $\langle A\vec{x}, \vec{y} \rangle = (A\vec{x})^*\vec{y} = \vec{x}^*(\overline{A}^\top \vec{y})$, the adjoint for the standard inner product is the conjugate transpose. So self-adjoint means *symmetric* for $\mathbb{K} = \mathbb{R}$ and *Hermitian* for $\mathbb{K} = \mathbb{C}$; for example, $\left[\begin{smallmatrix} 2 & i \\ -i & 3 \end{smallmatrix}\right]$ is Hermitian but not symmetric. Why use a new word? The name refers only to the inner product, not to a matrix representation, so it also applies to operators on infinite-dimensional spaces (for example $-\mathrm{d}^2/\mathrm{d}x^2$ with zero boundary values), and it covers the real and complex cases at once. We keep saying "Hermitian" below, as in Lecture 10.

Theorem 70 (Spectral Theorem)

If $A \in \mathbb{K}^{m\times m}$ is self-adjoint, there is a unitary X with

$$A = X\Lambda X^*, \tag{11.7}$$

and Λ is real [1, Thm. 24.7], [4, 7.29, 7.31].

We postpone the proof to Corollary 80, where it follows from the Schur factorization in two lines. A few remarks.

- The theorem says X *can be chosen* orthogonal. If the eigenvalues are distinct, the eigenvectors are automatically orthogonal (Proposition 56) and only need to be normalized. If an eigenvalue is repeated, its eigenspace also contains non-orthogonal eigenvectors, and one must pick an orthonormal basis of it.
- Compared with Proposition 68, inverting X is free ($X^{-1} = X^*$), and $\kappa_2(X) = 1$, so changing to and from eigenvector coordinates does not amplify errors.

11.5 Defective Matrices and Invariant Subspaces

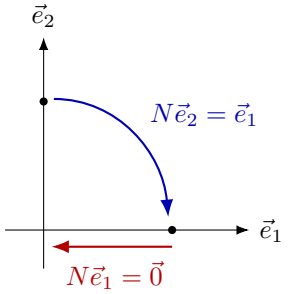
Are all matrices diagonalizable? Consider

$$J = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}. \tag{11.8}$$

What are its eigenpairs? Its only eigenvalue is λ, and $(J - \lambda I)\vec{x} = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]\vec{x} = \vec{0}$ forces $x_2 = 0$. So $(\lambda, \vec{e}_1)$ is the only eigenpair, up to scaling. There is no second independent eigenvector, and hence no invertible X with $J = X\Lambda X^{-1}$. Such matrices are called *defective*.

To see what is going on, take $\lambda = 0$, so that $J = N := \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$. Then $N\vec{e}_2 = \vec{e}_1$ and $N\vec{e}_1 = \vec{0}$: the matrix pushes $\vec{e}_2$ onto $\vec{e}_1$ and $\vec{e}_1$ onto $\vec{0}$, a chain $\vec{e}_2 \mapsto \vec{e}_1 \mapsto \vec{0}$. Hence $N^2 = 0$ although $N \neq 0$.

This gives another short proof that N is not diagonalizable. If $N = X\Lambda X^{-1}$, the diagonal of Λ holds the eigenvalues of N, which are all 0. So $\Lambda = 0$ and $N = 0$, a contradiction.



The nilpotent matrix N maps $\vec{e}_2$ to $\vec{e}_1$ and $\vec{e}_1$ to $\vec{0}$. Only $\mathrm{span}(\vec{e}_1)$ is an eigenspace.

11.5.1 Algebraic and Geometric Multiplicity

We follow the terminology of [1, Lecture 24]. The characteristic polynomial $p_A(z) := \det(zI - A)$ is monic of degree m, and its roots are the eigenvalues. Similar matrices have the same characteristic polynomial, since

$$\det(zI - X^{-1}AX) = \det(X^{-1}(zI - A)X) = p_A(z). \tag{11.9}$$

There are two ways to count how often an eigenvalue occurs, and defectiveness is exactly the gap between them.

Definition 71 (Multiplicities, Defective Matrices)

Let λ be an eigenvalue of A.

- The *eigenspace* is $E_\lambda := \mathcal{N}(A - \lambda I)$, and the *geometric multiplicity* of λ is $\dim E_\lambda$, the largest number of linearly independent eigenvectors for λ.
- The *algebraic multiplicity* of λ is its multiplicity as a root of p_A. The eigenvalue is *simple* if its algebraic multiplicity is 1.
- λ is *defective* if its algebraic multiplicity exceeds its geometric multiplicity, and A is *defective* if it has a defective eigenvalue.

For the matrix J above, $p_J(z) = (z - \lambda)^2$, so the algebraic multiplicity is 2, but $E_\lambda = \mathrm{span}(\vec{e}_1)$, so the geometric multiplicity is 1. The gap can only go one way.

Theorem 72 (Algebraic Multiplicity Bounds Geometric Multiplicity)

For every eigenvalue, the algebraic multiplicity is at least the geometric multiplicity [1, Thm. 24.4].

Proof. The idea is to change to a basis that starts with the eigenspace. Let $n = \dim E_\lambda$, let the columns of $\hat{V} \in \mathbb{K}^{m\times n}$ be an orthonormal basis of E_λ, and extend $\hat{V}$ to a unitary $V = [\hat{V}\ \tilde{V}]$. Since $A\hat{V} = \lambda\hat{V}$ and $\tilde{V}^*\hat{V} = 0$,

$$V^*AV = \begin{bmatrix} \lambda I_n & C \\ 0 & D \end{bmatrix}, \qquad C = \hat{V}^*A\tilde{V}, \quad D = \tilde{V}^*A\tilde{V}. \tag{11.10}$$

The determinant of a block triangular matrix is the product of the determinants of its diagonal blocks, so $p_A(z) = (z-\lambda)^n \det(zI - D)$. Thus λ is a root of multiplicity at least n. □

Theorem 73 (Diagonalizable Means Nondefective)

$A \in \mathbb{K}^{m\times m}$ has an eigenvalue decomposition $A = X\Lambda X^{-1}$ if and only if A is nondefective [1, Thm. 24.5].

Proof. ($\Rightarrow$) If $A = X\Lambda X^{-1}$, then A and Λ have the same eigenvalues with the same algebraic multiplicities, and X maps each eigenspace of Λ onto the corresponding eigenspace of A, so the geometric multiplicities agree too. A diagonal matrix is clearly nondefective.

($\Leftarrow$) If A is nondefective, choose a basis of each eigenspace. The geometric multiplicities add up to the algebraic ones, which add up to m, so we have m vectors. They are linearly independent: in a relation $\sum_\lambda \vec{u}_\lambda = \vec{0}$ with $\vec{u}_\lambda \in E_\lambda$, the nonzero $\vec{u}_\lambda$ would be eigenvectors for distinct eigenvalues, contradicting Lemma 67. So every $\vec{u}_\lambda = \vec{0}$, and then all coefficients vanish because each basis is independent. These m vectors form an invertible X with $AX = X\Lambda$. □

11.5.2 Invariant Subspaces

A defective matrix does not have enough eigenvectors to fill $\mathbb{K}^m$. But intuitively, all vectors still show "λ-growth" under J. Our idea is to generalize the λ-eigenspace to an *invariant subspace with λ-growth*: a subspace that A maps into itself, but on which A need not act as a multiple of the identity. For J, the whole plane $\operatorname{span}(\vec{e}_1, \vec{e}_2)$ is mapped into itself, and on it J acts as $\lambda I + N$ rather than as λI.

Definition 74 (Invariant Subspace)

Let $X = [\vec{x}_1\ \cdots\ \vec{x}_n] \in \mathbb{K}^{m\times n}$ have linearly independent columns. The space $\mathcal{R}(X)$ spanned by its columns is *A-invariant* if and only if there is $B \in \mathbb{K}^{n\times n}$ with

$$AX = XB. \tag{11.11}$$

Let us unpack the definition.

- Read column by column, $A\vec{x}_j = X\vec{b}_j$: A maps every basis vector into $\mathcal{R}(X)$. So the definition is equivalent to $A\,\mathcal{R}(X) \subseteq \mathcal{R}(X)$, the basis-free definition in [4, 5.2] and [2, Def. 4.4].
- The matrix B is the matrix of A restricted to $\mathcal{R}(X)$, in the basis X. Since X has independent columns, B is unique.

The following table shows that the definition covers everything we have seen so far.

Situation	X	B
eigenpair	$\vec{x}_1$ $(n = 1)$	λ_1
eigenspace	a basis of E_λ	λI
diagonalizable A	all eigenvectors $(n = m)$	Λ
the matrix J above	I_2	J

So $AX = XB$ generalizes both $A\vec{x} = \lambda\vec{x}$ and $AX = X\Lambda$, without requiring B to be diagonal. The eigenvalues of B are still eigenvalues of A.

Proposition 75 (Eigenvalues on an Invariant Subspace)

Let $AX = XB$ with X of full column rank. Then every eigenvalue of B is an eigenvalue of A. If the columns of $X = Q$ are orthonormal, then $B = Q^*AQ$ [2, Prop. 4.3].

Proof. If $B\vec{v} = \lambda\vec{v}$ with $\vec{v} \neq \vec{0}$, then $A(X\vec{v}) = XB\vec{v} = \lambda X\vec{v}$, and $X\vec{v} \neq \vec{0}$ because X has independent columns. If $Q^*Q = I$, multiplying $AQ = QB$ by Q^* on the left gives $B = Q^*AQ$. □

For $n = 1$ and $\|\vec{q}\| = 1$, $Q^*AQ = \vec{q}^*A\vec{q}$ is the Rayleigh quotient (Definition 55). So Q^*AQ is its block version.

The meaning of "λ-growth". Iterating $AX = XB$ gives $A^kX = XB^k$. On an invariant subspace, the powers of A (and hence power iteration) behave like the powers of the small matrix B, and their growth is governed by the eigenvalues of B. For the 2×2 block,

$$J^k = \begin{bmatrix} \lambda^k & k\lambda^{k-1} \\ 0 & \lambda^k \end{bmatrix}, \tag{11.12}$$

which grows like $|\lambda|^k$ up to a factor polynomial in k. Note that even for $|\lambda| < 1$, $\left\|J^k\right\|$ can first increase before it decays (see the supplement of Lecture 10).

11.6 The Jordan Normal Form

If we collect invariant subspaces on which A acts like a single eigenvalue, we arrive at the Jordan normal form. It decomposes the whole space into A-invariant subspaces.

Theorem 76 (Jordan Normal Form)

For every $A \in \mathbb{K}^{m\times m}$ there is a nonsingular $S \in \mathbb{C}^{m\times m}$ with

$$A = SJS^{-1}, \qquad J = \begin{bmatrix} J_1 & & \\ & \ddots & \\ & & J_k \end{bmatrix}, \qquad J_\ell = \begin{bmatrix} \lambda_\ell & 1 & & \\ & \lambda_\ell & \ddots & \\ & & \ddots & 1 \\ & & & \lambda_\ell \end{bmatrix} \in \mathbb{C}^{n_\ell \times n_\ell}. \tag{11.13}$$

J is unique up to the order of the blocks [2, Thm. 4.1], [4, 8.46].

Reading the theorem.

- S and J are complex in general, even for real A, because the eigenvalues may be complex.
- The λ_ℓ need not be distinct; one eigenvalue may own several blocks. For each eigenvalue, its algebraic multiplicity is the total size of its blocks, and its geometric multiplicity is the number of its blocks (by Proposition 77 below).
- A is diagonalizable if and only if all $n_\ell = 1$. Then $J = \Lambda$ and $S = X$.
- Splitting $S = [S_1 \ \cdots \ S_k]$ conformally with J gives $AS_\ell = S_\ell J_\ell$. So each block is an invariant subspace in the sense of Definition 74, with $B = J_\ell$.

Where the proof comes from. The proof is pure linear algebra, and we do not give it in full; see [4, Chapter 8]. Here is the outline.

- A nonzero $\vec{v}$ with $(A - \lambda I)^p \vec{v} = \vec{0}$ for some $p \geq 1$ is a *generalized eigenvector* [4, 8.8].
- The space $\mathbb{K}^m$ (over $\mathbb{C}$) is the direct sum of the generalized eigenspaces $\mathcal{N}(A - \lambda I)^m$, and on each of them $A - \lambda I$ is nilpotent [4, 8.14], i.e., some power of it is zero.
- The columns of S belonging to one block form a *Jordan chain* $(A - \lambda I)\vec{s}_j = \vec{s}_{j-1}$, $(A - \lambda I)\vec{s}_1 = \vec{0}$. This is the chain $\vec{e}_2 \mapsto \vec{e}_1 \mapsto \vec{0}$ of N above. Only $\vec{s}_1$ is an eigenvector.

Proposition 77 (One Eigenvector per Jordan Block)

Every Jordan block J_ℓ has, up to scaling, only one eigenvector, $\vec{e}_1$ [2, Prop. 4.2].

Proof. The only eigenvalue of J_ℓ is λ_ℓ. The matrix $J_\ell - \lambda_\ell I$ has ones on the superdiagonal and zeros elsewhere, so it shifts entries up: $(J_\ell - \lambda_\ell I)\vec{x} = (x_2, \ldots, x_{n_\ell}, 0)^\top$. This vanishes if and only if $x_2 = \cdots = x_{n_\ell} = 0$, i.e., $\vec{x} \in \operatorname{span}(\vec{e}_1)$. □

11.6.1 The Jordan Form Is Not Numerically Stable

The Jordan form exists for every matrix, so why not compute it? The problem is that it is not (numerically) stable, in two ways: it is discontinuous in the input, and the transformation S can be arbitrarily ill-conditioned.

Example 78 (A Tiny Perturbation Changes the Jordan Form)**.** Perturb $N = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ to

$$A_\varepsilon = \begin{bmatrix} \varepsilon & 1 \\ 0 & 2\varepsilon \end{bmatrix}, \qquad \varepsilon > 0. \tag{11.14}$$

A_ε is upper triangular with distinct eigenvalues ε and 2ε, so it is diagonalizable. Let us compute its eigenvectors.

- For ε, the eigenvector is $\vec{e}_1$.
- For 2ε, $(A_\varepsilon - 2\varepsilon I)\vec{x} = \left[\begin{smallmatrix} -\varepsilon & 1 \\ 0 & 0 \end{smallmatrix}\right] \vec{x} = \vec{0}$ gives the eigenvector $(1, \varepsilon)^\top$.

Hence

$$A_\varepsilon = S_\varepsilon \begin{bmatrix} \varepsilon & 0 \\ 0 & 2\varepsilon \end{bmatrix} S_\varepsilon^{-1}, \qquad S_\varepsilon = \begin{bmatrix} 1 & 1 \\ 0 & \varepsilon \end{bmatrix}. \tag{11.15}$$

As $\varepsilon \to 0$, $A_\varepsilon \to N$, but the Jordan form $\operatorname{diag}(\varepsilon, 2\varepsilon)$ tends to the zero matrix, while the Jordan form of N is $N \neq 0$. So the Jordan form is a *discontinuous* function of A. More generally, adding $\operatorname{diag}(\varepsilon, 2\varepsilon, \ldots, n\varepsilon)$ to an $n \times n$ Jordan block with eigenvalue 0 turns its Jordan form into $\operatorname{diag}(\varepsilon, \ldots, n\varepsilon)$ [2, Example 4.2].

In floating point, perturbations of size $\varepsilon_{\text{mach}} \|A\|$ are always present. So the question "does A have a nontrivial Jordan block?" cannot be answered by computation.

S can be arbitrarily ill-conditioned. The two columns of S_ε are nearly parallel: the angle θ between them satisfies $\tan\theta = \varepsilon$. With both columns normalized to unit length, the singular values are $\sqrt{1 \pm \cos\theta}$, so

$$\kappa_2(S_\varepsilon) = \sqrt{\frac{1+\cos\theta}{1-\cos\theta}} = \cot\frac{\theta}{2} \approx \frac{2}{\varepsilon}. \tag{11.16}$$

Could a better scaling of the columns help? No. Let $\vec{u}, \vec{v}$ be the unit columns and $SD = [a\vec{u}\; b\vec{v}]$ with $a, b \neq 0$.

- For $\vec{w} = (1/a, -1/b)^\top$ we get $SD\vec{w} = \vec{u} - \vec{v}$ and $\|\vec{w}\| \geq 1/\min(|a|, |b|)$, so $\sigma_{\min}(SD) \leq \min(|a|, |b|)\, \|\vec{u} - \vec{v}\|$.
- Also $\sigma_{\max}(SD) \geq \max(|a|, |b|)$, the norm of the larger column.

Hence $\kappa_2(SD) \geq 1/\|\vec{u} - \vec{v}\| = 1/(2\sin(\theta/2)) \approx 1/\varepsilon$. Since A_ε is diagonalizable, this also shows that the eigenvector matrix X of a diagonalizable matrix can be arbitrarily ill-conditioned when the matrix is close to a defective one.

Why an ill-conditioned S is fatal. Suppose we were lucky enough to compute S exactly and J with a tiny error δJ, $\|\delta J\| = O(\varepsilon_{\text{mach}})\, \|A\|$. The backward error δA with $S^{-1}(A + \delta A)S = J + \delta J$ is $\delta A = S\, \delta J\, S^{-1}$, and all we can say is

$$\|\delta A\| \leq \kappa(S)\, \|\delta J\| = O(\varepsilon_{\text{mach}})\, \kappa(S)\, \|A\|, \tag{11.17}$$

which can be far larger than $\varepsilon_{\text{mach}} \|A\|$. So the Jordan form cannot be computed in a backward stable way [2, Example 4.3]. The same factor $\kappa(S)$ multiplies the error of any computation of the form $f(A) = Sf(J)S^{-1}$.

The conclusion is to restrict the similarity transformation to be *unitary*, so that $\kappa_2 = 1$. We then cannot reach a form as simple as J, but we get something almost as good.

11.7 The Schur Factorization

Theorem 79 (Schur Factorization)

For every $A \in \mathbb{K}^{m \times m}$ there exist a unitary Q and an upper triangular T with

$$A = QTQ^*. \tag{11.18}$$

The eigenvalues of A are the diagonal entries of T [1, Thm. 24.9], [2, Thm. 4.2], [4, 6.38].

The idea of the proof: use the fact that every matrix has at least one eigenpair, rotate it into the first column by a unitary similarity, and then recurse on the bottom right part.

Proof. We use induction on m; for $m = 1$, take $Q = [1]$ and $T = A$. Let $m \geq 2$.

Step 1: find one eigenpair. By the fundamental theorem of algebra, p_A has a root $\lambda \in \mathbb{C}$, so there is $\vec{u}$ with $A\vec{u} = \lambda\vec{u}$, normalized to $\|\vec{u}\|_2 = 1$.

Step 2: rotate it into the first column. Extend $\vec{u}$ to a square unitary matrix $U = [\vec{u}\ \tilde{U}]$, $\tilde{U} \in \mathbb{C}^{m\times(m-1)}$; for example, take a Householder reflector that maps $\vec{e}_1$ to a multiple of $\vec{u}$ (Lecture 6). Then

$$U^*AU = \begin{bmatrix} \vec{u}^*A\vec{u} & \vec{u}^*A\tilde{U} \\ \tilde{U}^*A\vec{u} & \tilde{U}^*A\tilde{U} \end{bmatrix} = \begin{bmatrix} \lambda & \vec{u}^*A\tilde{U} \\ \vec{0} & A_2 \end{bmatrix}, \qquad A_2 \coloneqq \tilde{U}^*A\tilde{U}, \tag{11.19}$$

because $\vec{u}^*A\vec{u} = \lambda\vec{u}^*\vec{u} = \lambda$ and $\tilde{U}^*A\vec{u} = \lambda\tilde{U}^*\vec{u} = \vec{0}$ by orthogonality.

Step 3: recurse on the bottom right part. By the induction hypothesis, $A_2 = \tilde{Q}\tilde{T}\tilde{Q}^*$ with $\tilde{Q}$ unitary and $\tilde{T}$ upper triangular. Set

$$Q \coloneqq U \begin{bmatrix} 1 & \\ & \tilde{Q} \end{bmatrix}, \tag{11.20}$$

a product of unitary matrices. Then

$$Q^*AQ = \begin{bmatrix} 1 & \\ & \tilde{Q}^* \end{bmatrix} \begin{bmatrix} \lambda & \vec{u}^*A\tilde{U} \\ \vec{0} & A_2 \end{bmatrix} \begin{bmatrix} 1 & \\ & \tilde{Q} \end{bmatrix} = \begin{bmatrix} \lambda & \vec{u}^*A\tilde{U}\tilde{Q} \\ \vec{0} & \tilde{T} \end{bmatrix} \tag{11.21}$$

is upper triangular; call it T.

Step 4: the eigenvalues. A and T are similar, so they have the same characteristic polynomial, and $p_T(z) = \prod_i (z - t_{ii})$ since T is triangular. □

The fundamental theorem of algebra in Step 1 is where $\mathbb{C}$ enters. Over $\mathbb{R}$, a matrix may have no eigenvalue at all (a rotation of the plane), so Q and T are complex in general even for real A.

Remarks.

- *Comparison with Jordan.* Both forms exist for every matrix. Jordan uses a possibly ill-conditioned S and is discontinuous; Schur uses a unitary Q with $\kappa_2(Q) = 1$. The price is that T is only triangular, with a possibly dense strictly upper part.
- *Non-uniqueness.* The eigenvalue picked in Step 1 is arbitrary, so the eigenvalues can appear on the diagonal of T in any order.
- *Nested invariant subspaces.* Writing $AQ = QT$ and keeping the first k columns gives $AQ_{:,1:k} = Q_{:,1:k}T_{1:k,1:k}$, because T is upper triangular. So $\operatorname{span}(\vec{q}_1) \subset \operatorname{span}(\vec{q}_1, \vec{q}_2) \subset \cdots$ is a nested sequence of invariant subspaces (Definition 74) with orthonormal bases. Conversely, a basis whose leading spans are all invariant gives an upper triangular matrix [4, 5.39]. This is what subspace iteration with orthonormalization aims at (Section 11.3).
- *The proof is deflation.* Each step isolates one eigenvalue in the top left corner by a unitary similarity, and the remaining eigenvalues are those of A_2. This answers the question of Section 11.2 for all matrices, Hermitian or not, using only unitary transformations. It is not yet an algorithm, since each step needs an eigenpair, which itself must be computed iteratively.

As promised, the spectral theorem now follows in two lines.

Corollary 80 (Normal Matrices and the Spectral Theorem)**.** *If A is normal ($A^*A = AA^*$), then the triangular factor T of any Schur factorization of A is diagonal, so $A = Q\Lambda Q^*$. If A is Hermitian, Λ is moreover real. In particular, Theorem 70 holds [1, Thm. 24.8].*

Proof. Hermitian case. $T = Q^*AQ$ satisfies $T^* = Q^*A^*Q = T$. A Hermitian upper triangular matrix is diagonal with real diagonal.

Normal case. This is the forward direction of Theorem 85, proved in Lecture 12. □

11.7.1 The Real Schur Form

When working with real matrices, we prefer to stay with real numbers, for two reasons.

- *Cost.* A complex multiplication costs four real multiplications and two additions, and complex storage is twice as large.
- *Structure.* A real matrix has its non-real eigenvalues in conjugate pairs $\lambda, \overline{\lambda}$. Complex arithmetic with rounding would destroy this exact symmetry, while real arithmetic preserves it automatically.

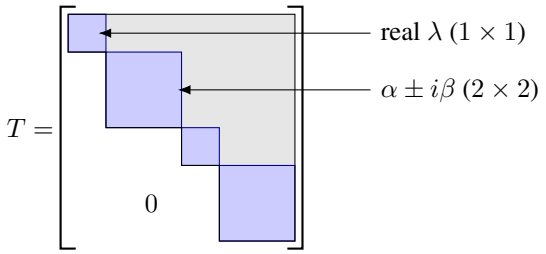
Theorem 81 (Real Schur Form)

If $A \in \mathbb{R}^{m\times m}$, there exist an orthogonal Q and a block upper triangular T with 1×1 or 2×2 diagonal blocks such that

$$A = QTQ^\top. \tag{11.22}$$

The 1×1 blocks are the real eigenvalues of A, and each 2×2 block has a pair of complex conjugate eigenvalues of A [2, Thm. 4.3].

Such a T is called *quasi-upper triangular*.



The quasi-upper triangular factor of the real Schur form. Shaded entries may be nonzero; each diagonal block holds a real eigenvalue or a conjugate pair.

The key idea is that complex eigenvalues of a real matrix appear in conjugate pairs, and together they span a *real* two-dimensional invariant subspace.

Lemma 82 (A Conjugate Pair Gives a Real Invariant Plane)

Let $A \in \mathbb{R}^{m\times m}$, $A\vec{x} = \lambda\vec{x}$ with $\lambda = \alpha + i\beta$, $\beta \neq 0$, and $\vec{x} = \vec{a} + i\vec{b}$ with $\vec{a}, \vec{b} \in \mathbb{R}^m$. Then $\vec{a}, \vec{b}$ are linearly independent and

$$A\begin{bmatrix}\vec{a} & \vec{b}\end{bmatrix} = \begin{bmatrix}\vec{a} & \vec{b}\end{bmatrix}\begin{bmatrix}\alpha & \beta\\ -\beta & \alpha\end{bmatrix}, \tag{11.23}$$

where the 2×2 matrix has eigenvalues $\alpha \pm i\beta$.

Proof. The identity. Since A is real, comparing real and imaginary parts of $A(\vec{a} + i\vec{b}) = (\alpha + i\beta)(\vec{a} + i\vec{b})$ gives

$$A\vec{a} = \alpha\vec{a} - \beta\vec{b}, \qquad A\vec{b} = \beta\vec{a} + \alpha\vec{b}, \tag{11.24}$$

which is the matrix identity, column by column.

Independence. If $\vec{a}, \vec{b}$ were dependent, then $\vec{x}$ would be a complex multiple of a single real vector $\vec{r} \neq \vec{0}$, so $A\vec{r} = \lambda\vec{r}$. The left side is real, which forces λ to be real, a contradiction.

The eigenvalues. The eigenvalues of $\left[\begin{smallmatrix} \alpha & \beta \\ -\beta & \alpha \end{smallmatrix}\right]$ are the roots of $(z-\alpha)^2 + \beta^2$, i.e., $\alpha \pm i\beta$. □

The 2×2 matrix is the standard real representation of the complex number $\alpha + i\beta$ acting by multiplication.

Proof of Theorem 81. Induct as in Theorem 79, but with real arithmetic. There are two cases.

A real eigenvalue. It has a real eigenvector, and the step is the same as before, with a 1×1 block.

No real eigenvalue. Take $\lambda = \alpha + i\beta$ with $\beta \neq 0$ and form $[\vec{a}\ \vec{b}]$ as in Lemma 82. Compute its QR factorization $[\vec{a}\ \vec{b}] = \hat{Q}R$, $\hat{Q} \in \mathbb{R}^{m \times 2}$, R invertible. Then $\mathcal{R}(\hat{Q}) = \operatorname{span}(\vec{a}, \vec{b})$ is invariant, $A\hat{Q} = \hat{Q}\hat{B}$ with $\hat{B} = R\left[\begin{smallmatrix} \alpha & \beta \\ -\beta & \alpha \end{smallmatrix}\right]R^{-1}$, which still has eigenvalues $\alpha \pm i\beta$. Extend $\hat{Q}$ to an orthogonal $U = [\hat{Q}\ \tilde{Q}]$. By Proposition 75, $\hat{Q}^\top A\hat{Q} = \hat{B}$, and $\tilde{Q}^\top A\hat{Q} = \tilde{Q}^\top\hat{Q}\hat{B} = 0$, so

$$U^\top AU = \begin{bmatrix} \hat{B} & \hat{Q}^\top A\tilde{Q} \\ 0 & \tilde{Q}^\top A\tilde{Q} \end{bmatrix}, \tag{11.25}$$

and we recurse on $\tilde{Q}^\top A\tilde{Q}$. □

A 2×2 block with a non-real conjugate pair cannot be triangularized further over $\mathbb{R}$, since its triangular form would show the non-real eigenvalues on a real diagonal.

11.8 Computing Eigenvectors from the Schur Form

The Schur factorization gives us the eigenvalues. What about the eigenvectors? Suppose a Schur factorization $A = QTQ^*$ is known. Then

$$T\vec{x} = \lambda\vec{x} \iff (QTQ^*)(Q\vec{x}) = \lambda Q\vec{x}, \tag{11.26}$$

so $\vec{x}$ is an eigenvector of T if and only if $Q\vec{x}$ is an eigenvector of A. So we need to find eigenvectors of T, a triangular matrix, which turns out to be just a triangular solve.

Proposition 83 (Eigenvectors of a Triangular Matrix)

Let T be upper triangular and let $\lambda = t_{kk}$ be a simple eigenvalue. Partition

$$T - \lambda I = \begin{bmatrix} T_{11} - \lambda I & \vec{t}_{12} & T_{13} \\ & 0 & \vec{t}_{23}^{\,*} \\ & & T_{33} - \lambda I \end{bmatrix}, \tag{11.27}$$

with T_{11} of size $(k-1) \times (k-1)$, a 1×1 middle block, and T_{33} of size $(m-k) \times (m-k)$. Then

$$\vec{x} = \begin{bmatrix} (\lambda I - T_{11})^{-1}\vec{t}_{12} \\ 1 \\ \vec{0} \end{bmatrix} \tag{11.28}$$

is an eigenvector of T for λ, and $Q\vec{x}$ is an eigenvector of A [2, §4.2.1].

Proof. Partition $\vec{x} = (\vec{x}_1, x_2, \vec{x}_3)$ conformally. Then $(T - \lambda I)\vec{x} = \vec{0}$ reads

$$\begin{bmatrix} (T_{11} - \lambda I)\vec{x}_1 + \vec{t}_{12}x_2 + T_{13}\vec{x}_3 \\ \vec{t}_{23}^{*}\vec{x}_3 \\ (T_{33} - \lambda I)\vec{x}_3 \end{bmatrix} = \vec{0}. \tag{11.29}$$

Since λ is simple, it does not appear on the diagonal of T_{11} or T_{33}, so $T_{11} - \lambda I$ and $T_{33} - \lambda I$ are nonsingular triangular matrices. We solve from the bottom up.

- The third row gives $\vec{x}_3 = \vec{0}$, and then the second row holds automatically.
- The scalar x_2 is free, and we choose $x_2 = 1$.
- The first row becomes $(T_{11} - \lambda I)\vec{x}_1 = -\vec{t}_{12}$, i.e., $\vec{x}_1 = (\lambda I - T_{11})^{-1}\vec{t}_{12}$. □

Cost. Computing $\vec{x}_1$ is one triangular solve of size $k-1$, about $(k-1)^2$ flops, and $Q\vec{x}$ uses only the first k columns of Q, about $2mk$ flops. Summing over $k = 1, \ldots, m$ gives roughly $\frac{1}{3}m^3 + m^3$ flops for all eigenvectors, the same order $O(m^3)$ as computing the Schur form itself. For $k = 1$, $\vec{x} = \vec{e}_1$ and $Q\vec{x} = \vec{q}_1$: the first Schur vector is always an eigenvector, namely the $\vec{u}$ chosen in Step 1 of the proof of Theorem 79.

Remarks.

- *Repeated eigenvalues.* If $t_{jj} = \lambda$ for some $j < k$, then $T_{11} - \lambda I$ is singular, and an eigenvector with $x_k = 1$ may not exist. This is the defective situation of Section 11.5 showing up again.
- *Sensitivity.* The Schur form can be computed stably, but if λ is close to some other diagonal entry of T_{11}, then $(\lambda I - T_{11})^{-1}$ is large and $\vec{x}$ is very sensitive to perturbations. This is not a flaw of the method: the eigenvector itself is ill-conditioned, as in Example 78, where two eigenvectors become nearly parallel.
- *Real Schur form.* From the real Schur form one solves a quasi-triangular system instead. Computing complex eigenvectors in real arithmetic needs a little more care; LAPACK does it in the routine `xTREVC` [2, §4.2.1].

11.9 (SUPPLEMENT) Further Topics

11.9.1 Deflation for Non-Hermitian Matrices

Hotelling deflation (Proposition 66) uses orthogonal eigenvectors. For general A with $A\vec{x}_1 = \lambda_1\vec{x}_1$, let $\vec{w} \in \mathbb{K}^m$ be arbitrary. Then the eigenvalues of $A - \vec{x}_1\vec{w}^*$ are $\lambda_1 - \vec{w}^*\vec{x}_1, \lambda_2, \ldots, \lambda_m$, counted with multiplicity (Brauer's theorem). The proof is one step of the Schur proof: with $\hat{\vec{x}}_1 = \vec{x}_1 / \|\vec{x}_1\|$ and a unitary $U = [\hat{\vec{x}}_1\ \tilde{U}]$,

$$U^*(A - \vec{x}_1\vec{w}^*)U = \begin{bmatrix} \lambda_1 & * \\ \vec{0} & A_2 \end{bmatrix} - \begin{bmatrix} \|\vec{x}_1\| \\ \vec{0} \end{bmatrix} \vec{w}^*U = \begin{bmatrix} \lambda_1 - \vec{w}^*\vec{x}_1 & * \\ \vec{0} & A_2 \end{bmatrix}, \tag{11.30}$$

since $U^*\vec{x}_1 = \|\vec{x}_1\|\,\vec{e}_1$ and the $(1,1)$ entry of the correction is $\|\vec{x}_1\|\,\vec{w}^*\hat{\vec{x}}_1 = \vec{w}^*\vec{x}_1$. Choosing $\vec{w} = \lambda_1\vec{v}$ with $\vec{v}^*\vec{x}_1 = 1$ moves λ_1 to 0 and keeps the rest of the spectrum (Wielandt deflation); $\vec{v} = \vec{x}_1 / \|\vec{x}_1\|^2$ recovers Hotelling's formula. Unlike the Hermitian case, the eigenvectors change: if $B\vec{w}_i = \lambda_i\vec{w}_i$ for $B = A - \lambda_1\vec{x}_1\vec{v}^*$ and $\lambda_i \neq \lambda_1$, a direct computation shows that $\vec{x}_i = \vec{w}_i + c\,\vec{x}_1$ with $c = \lambda_1\vec{v}^*\vec{w}_i/(\lambda_i - \lambda_1)$ is an eigenvector of A.

11.9.2 How Ill-Conditioned Eigenvalues Near a Jordan Block Are

In Example 78 the eigenvalues moved by only $O(\varepsilon)$ while the Jordan structure and S broke down. A different perturbation moves the eigenvalues themselves much more. Put ε in the bottom left corner of an $n \times n$ Jordan block with eigenvalue 0. The result is the companion matrix (Lemma 58) of $z^n - \varepsilon$, whose eigenvalues are the n complex n-th roots of ε, all of modulus $\varepsilon^{1/n}$ [2, Example 4.4]. For $n = 10$ and $\varepsilon = 10^{-10}$, a perturbation of size 10^{-10} moves the eigenvalues by 0.1. A multiple eigenvalue in a nontrivial Jordan block has infinite condition number.

11.9.3 The Bauer–Fike Theorem

The role of $\kappa(X)$ can be made quantitative. Let $A = X\Lambda X^{-1}$ be diagonalizable and let μ be an eigenvalue of $A + E$. Then

$$\min_i |\mu - \lambda_i| \le \kappa_2(X)\, \|E\|_2 \,. \tag{11.31}$$

To see this, assume μ is not an eigenvalue of A. Since $A + E - \mu I$ is singular, so is $X^{-1}(A + E - \mu I)X = (\Lambda - \mu I) + X^{-1}EX$, and hence so is $I + (\Lambda - \mu I)^{-1}X^{-1}EX$. A matrix $I + M$ can only be singular if $\|M\| \ge 1$, so

$$1 \le \left\|(\Lambda - \mu I)^{-1}\right\| \left\|X^{-1}\right\| \|E\| \, \|X\| = \frac{\kappa_2(X)\, \|E\|_2}{\min_i |\lambda_i - \mu|}. \tag{11.32}$$

For normal matrices, X can be unitary, $\kappa_2(X) = 1$, and the eigenvalues move by at most $\|E\|_2$. This is the eigenvalue analogue of the remark after Theorem 70. When A approaches a defective matrix, $\kappa_2(X) \to \infty$ and the bound becomes useless, consistent with the $\varepsilon^{1/n}$ behavior above. See [2, §4.3].

11.9.4 From the Schur Form to the QR Algorithm

The Schur factorization is what practical eigenvalue solvers compute. Subspace iteration with $n = m$ vectors and a QR factorization after each multiplication by A produces, under suitable conditions, a $Q^{(k)}$ whose leading columns converge to the nested invariant subspaces of Section 11.7, so that $(Q^{(k)})^* A Q^{(k)}$ converges to an upper triangular T. Rewritten in terms of $(Q^{(k)})^* A Q^{(k)}$ alone, this becomes the QR algorithm. Combined with an initial reduction to Hessenberg form and with shifts (the Rayleigh quotient idea of Lecture 10), it computes the Schur form in $O(m^3)$ flops, and the real version produces the real Schur form in real arithmetic. See [1, Lectures 25 and 28].

Lecture 12

Orthogonal Iteration

Supplementary reading:

- [1] Lectures 24 and 28.
- [2] Section 4.4.3.

12.1 Recap

Last lecture (Lecture 11) went from computing one eigenpair to asking what "all eigenpairs" should mean as a single object.

- *Deflation* (Section 11.2). Once $(\lambda_1, \vec{x}_1)$ is known, power iteration on $A - \lambda_1 \vec{x}_1 \vec{x}_1^* / (\vec{x}_1^* \vec{x}_1)$ finds the next eigenpair (Hermitian A, Proposition 66).
- *The need for subspace iteration* (Section 11.3). Deflation computes eigenpairs one after another, each with many Level 2 matrix-vector products. Iterating on a block of vectors at once turns them into Level 3 matrix-matrix products.
- *Eigenvalues as matrix factorizations* (Section 11.4). All eigenpairs together form the eigenvalue decomposition $A = X \Lambda X^{-1}$, but it need not exist: $N = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ sends $\vec{e}_2 \mapsto \vec{e}_1 \mapsto \vec{0}$ and has only one eigenvector.
- *Defective matrices and Jordan blocks* (Sections 11.5 and 11.6). Every matrix has a Jordan form $A = SJS^{-1}$ with $J = \operatorname{diag}(J_1, \ldots, J_k)$ and $J_\ell \in \mathbb{C}^{n_\ell \times n_\ell}$ having λ_ℓ on the diagonal and ones on the superdiagonal. It is discontinuous in A, and S can be arbitrarily ill-conditioned (Example 78).
- *Schur factorization* (Section 11.7). Every matrix can be written $A = QTQ^*$ with Q unitary and T upper triangular (Theorem 79). This is the stable replacement for the Jordan form, and it is what eigenvalue algorithms compute.

12.2 From Schur Factorization to Diagonalization

The Schur factorization $A = QTQ^*$ always exists, but T is only upper triangular. When is it actually a diagonalization, i.e., when is T diagonal, so that $A = Q \Lambda Q^*$ with Q unitary?

Definition 84 (Normal Matrix)

$A \in \mathbb{K}^{m\times m}$ is *normal* if $AA^* = A^*A$.

Hermitian ($A^* = A$), skew-Hermitian ($A^* = -A$), and unitary ($A^* = A^{-1}$) matrices are normal, since in each case A^* is a function of A and therefore commutes with it. The nilpotent $N = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ is not:

$$NN^* = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \neq \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = N^*N. \tag{12.1}$$

Theorem 85 (Normal Matrices Are Unitarily Diagonalizable)

$A \in \mathbb{K}^{m\times m}$ is normal if and only if it has an orthogonal eigenvalue decomposition $A = Q\Lambda Q^*$ with Q unitary and Λ diagonal. In this case the triangular factor T of every Schur factorization $A = QTQ^*$ is diagonal.

Over $\mathbb{C}$, an "orthogonal" eigenvalue decomposition means that the eigenvector matrix Q is unitary.

Proof. ($\Leftarrow$) If $A = Q\Lambda Q^*$, then $A^* = Q\overline{\Lambda}Q^*$, and since diagonal matrices commute,

$$AA^* = Q\Lambda\overline{\Lambda}Q^* = Q\overline{\Lambda}\Lambda Q^* = A^*A. \tag{12.2}$$

($\Rightarrow$) Take a Schur factorization $A = QTQ^*$ (Theorem 79). Then

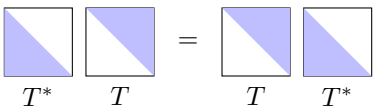
$$QTT^*Q^* = AA^* = A^*A = QT^*TQ^*, \tag{12.3}$$

and multiplying by Q^* on the left and Q on the right gives $TT^* = T^*T$: the triangular factor is normal too. Now compare the $(1,1)$ entries. Since T is upper triangular, its first column is $t_{11}\vec{e}_1$, while its first row is full:

$$(T^*T)_{11} = \sum_{i=1}^{m} |t_{i1}|^2 = |t_{11}|^2, \qquad (TT^*)_{11} = \sum_{i=1}^{m} |t_{1i}|^2. \tag{12.4}$$

Equating them gives $\sum_{i=2}^{m} |t_{1i}|^2 = 0$, so $t_{1i} = 0$ for $i \geq 2$, and $T = \operatorname{diag}(t_{11}, T_2)$ with T_2 upper triangular. The lower right blocks of $TT^* = T^*T$ give $T_2T_2^* = T_2^*T_2$, so T_2 is again normal, and induction on m shows that T is diagonal. Hence $A = QTQ^*$ is an orthogonal eigenvalue decomposition. □

Pictorially, in terms of the shapes of the factors, the key identity says that lower times upper equals upper times lower.



The $(1,1)$ entry of the left side sees only the first column of T, which is just t_{11}; the $(1,1)$ entry of the right side sees the whole first row. For the two to agree, the rest of the first row must vanish.

12.3 Orthogonal Iteration

We now return to the question from Section 11.3: how do we run power iteration on several vectors at once without all of them collapsing onto $\vec{x}_1$? Throughout this section we assume that

$$A \in \mathbb{R}^{m\times m} \text{ is diagonalizable}, \qquad |\lambda_1| > |\lambda_2| > \cdots > |\lambda_m|\,. \tag{12.5}$$

Since non-real eigenvalues of a real matrix come in conjugate pairs of equal modulus, the strict ordering forces all eigenvalues, and hence all eigenvectors, to be real.

Suppose we want to compute $r = 2$ eigenpairs at once. We start by applying power iteration to two vectors simultaneously.

- *Problem.* Run independently, both vectors converge to the same vector $\vec{x}_1$ (up to sign), since almost every starting vector has a component along $\vec{x}_1$.
- *Idea.* After each multiplication, project $\vec{q}_2$ onto the orthogonal complement of $\vec{q}_1$. Then $\vec{q}_2$ stays orthogonal to $\vec{q}_1$ and cannot converge to $\vec{x}_1$ as well.

Algorithm 18 Orthogonal iteration with two vectors.

1: $\vec{q}_1, \vec{q}_2 \leftarrow$ random vectors
2: **while** not converged **do**
3: $\quad \vec{q}_1 \leftarrow A\vec{q}_1, \quad \vec{q}_2 \leftarrow A\vec{q}_2$
4: $\quad \vec{q}_2 \leftarrow \left(I - \dfrac{\vec{q}_1\vec{q}_1^*}{\vec{q}_1^*\vec{q}_1}\right)\vec{q}_2$ ▷ project out $\vec{q}_1$
5: $\quad \vec{q}_1 \leftarrow \vec{q}_1/\left\|\vec{q}_1\right\|, \quad \vec{q}_2 \leftarrow \vec{q}_2/\left\|\vec{q}_2\right\|$

The last two lines are one step of Gram–Schmidt on $[A\vec{q}_1 \;\; A\vec{q}_2]$, i.e., a reduced QR factorization of a two-column matrix (Lecture 5). The first column is plain power iteration, so $\vec{q}_1 \to \pm\vec{x}_1/\left\|\vec{x}_1\right\|$. Compare this with deflation (Section 11.2): there we projected out the *converged* $\vec{x}_1$ at every step, here we project out the current approximation $\vec{q}_1$, so both vectors are iterated simultaneously.

12.3.1 What Do the Two Vectors Converge To?

Write $\vec{q}_i^{(k)}$ for the iterates after k steps. The first vector is never touched by the projection, so

$$\vec{q}_1^{(k)} = \frac{A^k\vec{q}_1^{(0)}}{\left\|A^k\vec{q}_1^{(0)}\right\|}. \tag{12.6}$$

That is, $\vec{q}_1$ performs plain power iteration (Algorithm 15) and converges to $\vec{x}_1$, normalized and up to sign, at the rate $|\lambda_2/\lambda_1|$.

For $\vec{q}_2$, assume that $\vec{q}_1 \approx \vec{x}_1$ already, with $\|\vec{x}_1\| = 1$. Then the projection in Algorithm 18 is (approximately) a fixed matrix, and up to normalization

$$\vec{q}_2^{(k)} \approx (I - \vec{x}_1\vec{x}_1^*)A\vec{q}_2^{(k-1)} = (I - \vec{x}_1\vec{x}_1^*)A(I - \vec{x}_1\vec{x}_1^*)\vec{q}_2^{(k-1)}. \tag{12.7}$$

The equality holds because $\vec{q}_2^{(k-1)}$ was itself projected onto the orthogonal complement of $\vec{x}_1$ in the previous step, so inserting the projector once more changes nothing. So once $\vec{q}_1$ has converged, $\vec{q}_2$ performs power iteration with the matrix $(I - \vec{x}_1\vec{x}_1^*)A(I - \vec{x}_1\vec{x}_1^*)$.

The normal case. For normal A this matrix is exactly the Hotelling deflation of Proposition 66:

$$(I - \vec{x}_1\vec{x}_1^*)A(I - \vec{x}_1\vec{x}_1^*) = A - 2\lambda_1\vec{x}_1\vec{x}_1^* + \lambda_1\vec{x}_1\vec{x}_1^* = A - \lambda_1\vec{x}_1\vec{x}_1^*. \tag{12.8}$$

To see the first equality, expand the product into four terms:

$$A - \vec{x}_1(\vec{x}_1^*A) - (A\vec{x}_1)\vec{x}_1^* + \vec{x}_1(\vec{x}_1^*A\vec{x}_1)\vec{x}_1^*. \tag{12.9}$$

We have $A\vec{x}_1 = \lambda_1\vec{x}_1$ and $\vec{x}_1^*A\vec{x}_1 = \lambda_1$. The remaining term needs normality: $\vec{x}_1$ is then also a *left* eigenvector, $\vec{x}_1^*A = \lambda_1\vec{x}_1^*$. Indeed, $B \coloneqq A - \lambda_1 I$ is normal along with A, and for any normal B and any $\vec{v}$,

$$\|B\vec{v}\|^2 = \vec{v}^*B^*B\vec{v} = \vec{v}^*BB^*\vec{v} = \|B^*\vec{v}\|^2, \tag{12.10}$$

so $B\vec{x}_1 = \vec{0}$ implies $B^*\vec{x}_1 = \vec{0}$, i.e., $A^*\vec{x}_1 = \overline{\lambda_1}\vec{x}_1$; taking the conjugate transpose gives $\vec{x}_1^*A = \lambda_1\vec{x}_1^*$. Thus three of the four terms equal $\pm\lambda_1\vec{x}_1\vec{x}_1^*$.

So for normal A, $\vec{q}_2$ performs power iteration on the deflated matrix $A - \lambda_1\vec{x}_1\vec{x}_1^*$, whose eigenvalues are $0, \lambda_2, \ldots, \lambda_m$. It converges to $\vec{x}_2$ at the rate $|\lambda_3/\lambda_2|$. Orthogonal iteration does deflation automatically, without waiting for $\vec{x}_1$ to converge first.

The general case. Without normality, $\vec{x}_1^*A \neq \lambda_1\vec{x}_1^*$ in general, and the deflated matrix is no longer $A - \lambda_1\vec{x}_1\vec{x}_1^*$. We can still find its eigenpairs by applying it to the eigenvectors of A. Write $P \coloneqq I - \vec{x}_1\vec{x}_1^*$. For every i,

$$PAP\vec{x}_i = P(\lambda_i\vec{x}_i - \lambda_1\vec{x}_1(\vec{x}_1^*\vec{x}_i)) = \lambda_i P\vec{x}_i, \tag{12.11}$$

since $P\vec{x}_1 = \vec{0}$. Grouping the left side as $[PA](P\vec{x}_i)$, the operator PA applied to the vector $P\vec{x}_i$, the identity says $PA(P\vec{x}_i) = \lambda_i(P\vec{x}_i)$ directly. (Read as PAP applied to $\vec{x}_i$, one uses $P^2 = P$ to get $PAP(P\vec{x}_i) = \lambda_i(P\vec{x}_i)$ as well.) Hence the eigenpairs of $PA = (I - \vec{x}_1\vec{x}_1^*)A$ (and of PAP) are

$$(0, \vec{x}_1) \qquad \text{and} \qquad (\lambda_i, P\vec{x}_i), \quad i = 2, \ldots, m. \tag{12.12}$$

Here $P\vec{x}_i = \vec{x}_i - (\vec{x}_1^*\vec{x}_i)\vec{x}_1 \neq \vec{0}$ for $i \geq 2$, because the $\vec{x}_i$ are linearly independent. In words: the projection keeps the eigenvalues $\lambda_2, \ldots, \lambda_m$, but replaces each eigenvector $\vec{x}_i$ by its component orthogonal to $\vec{x}_1$. In the normal case $\vec{x}_i \perp \vec{x}_1$, so $P\vec{x}_i = \vec{x}_i$, consistent with Equation (12.8).

Conclusion. Once $\vec{q}_1$ has converged, $\vec{q}_2$ performs power iteration with PA, whose dominant eigenpair is $(\lambda_2, P\vec{x}_2)$ and whose next largest eigenvalue in modulus is λ_3. Hence, normalized and up to sign,

$$\vec{q}_2^{(k)} \longrightarrow (I - \vec{x}_1\vec{x}_1^*)\vec{x}_2, \tag{12.13}$$

at the rate $|\lambda_3/\lambda_2|$. This is *not* $\vec{x}_2$ unless $\vec{x}_1 \perp \vec{x}_2$, as for normal A. The two vectors cannot both be eigenvectors in general, since $\vec{q}_1 \perp \vec{q}_2$ by construction while $\vec{x}_1, \vec{x}_2$ need not be orthogonal.

What does converge is the subspace: $P\vec{x}_2 = \vec{x}_2 - (\vec{x}_1^*\vec{x}_2)\vec{x}_1$, so

$$\operatorname{span}(\vec{q}_1, \vec{q}_2) \longrightarrow \operatorname{span}(\vec{x}_1, \vec{x}_2), \tag{12.14}$$

an invariant subspace of A, and $\vec{q}_1, \vec{q}_2$ is an orthonormal basis of it whose first vector is an eigenvector. These are exactly the first two columns of a Schur factorization (Section 11.7): $A\vec{q}_1 = \lambda_1\vec{q}_1$ and $A\vec{q}_2 \in \operatorname{span}(\vec{q}_1, \vec{q}_2)$, so T is upper triangular in its first two columns.

12.3.2 Extracting the Eigenvalues

Orthogonal iteration gives an orthonormal basis of $\operatorname{span}(\vec{x}_1, \vec{x}_2)$, but $\vec{q}_2$ is not an eigenvector. How do we find the eigenvalues? Collect the iterates into a tall matrix with orthonormal columns and compress A onto their span:

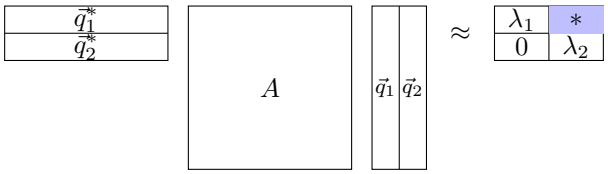
$$Q_k \coloneqq \begin{bmatrix} \vec{q}_1^{(k)} & \vec{q}_2^{(k)} \end{bmatrix} \in \mathbb{K}^{m\times 2}, \qquad Q_k^* A Q_k \in \mathbb{K}^{2\times 2}. \tag{12.15}$$

Proposition 86 (Projected Matrix)

As $k \to \infty$,

$$Q_k^* A Q_k = \begin{bmatrix} \vec{q}_1^* A \vec{q}_1 & \vec{q}_1^* A \vec{q}_2 \\ \vec{q}_2^* A \vec{q}_1 & \vec{q}_2^* A \vec{q}_2 \end{bmatrix} \longrightarrow \begin{bmatrix} \lambda_1 & * \\ 0 & \lambda_2 \end{bmatrix}. \tag{12.16}$$

In terms of shapes: a $2 \times m$ matrix times an $m \times m$ matrix times an $m \times 2$ matrix gives a 2×2 upper triangular matrix with the two dominant eigenvalues on its diagonal.



Proof sketch. Write $Q = Q_k$ and assume k is large, so that $\vec{q}_1 \approx \vec{x}_1$ and $\operatorname{span}(\vec{q}_1, \vec{q}_2) \approx \operatorname{span}(\vec{x}_1, \vec{x}_2)$.

The $(2,1)$ entry vanishes. Using $\vec{q}_1 \approx \vec{x}_1$ twice,

$$\vec{q}_2^* A \vec{q}_1 \approx \vec{q}_2^* A \vec{x}_1 = \lambda_1 \vec{q}_2^* \vec{x}_1 \approx \lambda_1 \vec{q}_2^* \vec{q}_1 = 0. \tag{12.17}$$

The eigenvalues are λ_1, λ_2. For $i = 1, 2$, the eigenvector $\vec{x}_i$ lies (approximately) in $\mathcal{R}(Q)$, so there is $\vec{v}_i \in \mathbb{K}^2$ with $Q\vec{v}_i = \vec{x}_i$. Then, using $Q^*Q = I_2$,

$$Q^* A Q \vec{v}_i = Q^* A \vec{x}_i = \lambda_i Q^* \vec{x}_i \approx \lambda_i Q^* Q \vec{v}_i = \lambda_i \vec{v}_i. \tag{12.18}$$

So $(\lambda_i, \vec{v}_i)$ are the eigenpairs of the 2×2 matrix Q^*AQ.

Order on the diagonal. The $(1,1)$ entry $\vec{q}_1^* A \vec{q}_1$ is the Rayleigh quotient of $\vec{q}_1 \approx \vec{x}_1$, so it tends to λ_1. A triangular matrix has its eigenvalues on the diagonal, so the $(2,2)$ entry tends to λ_2. □

The proof also shows how to get eigenvectors back. Solve the small eigenvalue problem $Q_k^* A Q_k \vec{v}_i = \lambda_i \vec{v}_i$ and lift: $\vec{x}_i \approx Q_k \vec{v}_i$. The $(1,2)$ entry $*$ is in general nonzero: by the computation $A\vec{q}_2 = \lambda_2 \vec{q}_2 + \beta \vec{q}_1$, it equals $\beta = (\lambda_2 - \lambda_1)(\vec{x}_1^* \vec{x}_2) / \|P\vec{x}_2\|$ in the limit, which vanishes exactly when $\vec{x}_1 \perp \vec{x}_2$.

12.4 Orthogonal Iteration in General

To compute $r > 2$ eigenpairs at once, start with r random vectors, the columns of $Q \in \mathbb{K}^{m\times r}$. Multiplying by A is one matrix-matrix product, and the Gram–Schmidt step of Algorithm 18 becomes a reduced QR factorization, which orthonormalizes all r columns at once.

Algorithm 19 Orthogonal iteration.

Input: $A \in \mathbb{K}^{m\times m}$, starting $Q \in \mathbb{K}^{m\times r}$ (e.g., random)

1: **for** $k = 1, \dots,$ iters **do**
2: $\quad Z \leftarrow AQ$ ▷ Level 3: A is read once for all r columns
3: $\quad QR \leftarrow Z$ ▷ reduced QR, $Q \in \mathbb{K}^{m\times r}$

Column j of the QR factorization is $A\vec{q}_j$ with its components along $\vec{q}_1, \dots, \vec{q}_{j-1}$ removed, so the first two columns do exactly what Algorithm 18 does, and column j is power iteration projected onto the orthogonal complement of the first $j-1$ columns.

Remark 87 (Julia)**.** The naive loop body `Q = A*Q; Q, R = qr(Q)` goes wrong in Julia: `qr` returns Q in the implicit form of Lecture 7, which acts as the full $m \times m$ matrix, so from the second iteration on `A*Q` is $m \times m$. Ask for the thin factor explicitly:

```
for k = 1:iters
    Q = Matrix(qr(A*Q).Q)    # m x r
end
```

12.5 Convergence of Eigenvector Estimates

Run Algorithm 19 with r columns and write $Q_k = [\vec{q}_1^{(k)} \cdots \vec{q}_r^{(k)}]$ for the iterate after k steps. The first i columns of a QR factorization depend only on the first i columns of the factored matrix, so the first i columns of Q_k are exactly what orthogonal iteration with i vectors would produce. Running Algorithm 19 with r columns therefore runs it with $1, 2, \dots, r$ columns at the same time, and the argument of Section 12.3 generalizes to

$$\operatorname{span}(\vec{q}_1^{(k)}, \dots, \vec{q}_i^{(k)}) \longrightarrow \operatorname{span}(\vec{x}_1, \dots, \vec{x}_i), \qquad i = 1, \dots, r. \tag{12.19}$$

A proof via $\operatorname{span}(Q_k) = \operatorname{span}(A^k Q_0)$ is given in Section 13.3.

Eigenvalues. By the same argument as in the proof of Proposition 86, for $i \le r$ the eigenvector $\vec{x}_i$ lies (approximately) in $\mathcal{R}(Q_k)$, so $Q_k\vec{v}_i \approx \vec{x}_i$ for some $\vec{v}_i \in \mathbb{K}^r$, and

$$Q_k^* A Q_k \vec{v}_i \approx Q_k^* A \vec{x}_i = \lambda_i Q_k^* \vec{x}_i \approx \lambda_i Q_k^* Q_k \vec{v}_i = \lambda_i \vec{v}_i. \tag{12.20}$$

So the eigenvalues of the $r \times r$ matrix $Q_k^* A Q_k$ converge to $\lambda_1, \dots, \lambda_r$, and $Q_k\vec{v}_i$ approximates $\vec{x}_i$. What about the columns of Q_k themselves?

The symmetric (normal) case. If A is symmetric, or more generally normal, then as before $\vec{q}_1^{(k)} \to \vec{x}_1$ and $\vec{q}_2^{(k)} \to \vec{x}_2$, up to sign. The same argument applies to every column. Once $\vec{q}_1, \dots, \vec{q}_{i-1}$ have converged to the orthonormal eigenvectors $\vec{x}_1, \dots, \vec{x}_{i-1}$, column i performs power iteration with

$$P_{i-1}A, \qquad P_{i-1} \coloneqq I - \sum_{j<i} \vec{x}_j \vec{x}_j^*, \tag{12.21}$$

and for normal $A = \sum_j \lambda_j \vec{x}_j \vec{x}_j^*$ this is $\sum_{j\ge i} \lambda_j \vec{x}_j \vec{x}_j^*$, with dominant eigenpair $(\lambda_i, \vec{x}_i)$. Hence

$$\vec{q}_i^{(k)} \longrightarrow \pm\vec{x}_i, \qquad 1 \le i \le r, \tag{12.22}$$

at the rate $|\lambda_{i+1}/\lambda_i|$. For normal A, orthogonal iteration delivers the eigenvectors directly.

The non-normal case. Now let $A = X\Lambda X^{-1}$ be diagonalizable, but not normal. The eigenvectors are no longer orthogonal, while the columns of Q_k are, so they cannot converge to the $\vec{x}_i$. To identify their limit, take a reduced QR factorization of the first r eigenvectors,

$$[\vec{x}_1 \cdots \vec{x}_r] =: Q^X R^X, \qquad Q^X = [\vec{q}_1^X \cdots \vec{q}_r^X] \in \mathbb{K}^{m\times r}, \quad R^X \in \mathbb{K}^{r\times r} \text{ upper triangular.} \tag{12.23}$$

Since the $\vec{x}_i$ are linearly independent, R^X is invertible, and as in Equation (13.6),

$$\operatorname{span}(\vec{q}_1^X, \ldots, \vec{q}_i^X) = \operatorname{span}(\vec{x}_1, \ldots, \vec{x}_i), \qquad i = 1, \ldots, r. \tag{12.24}$$

In other words, Q^X is the result of Gram–Schmidt applied to $\vec{x}_1, \ldots, \vec{x}_r$. For example, $\vec{q}_2^X$ is $(I - \vec{q}_1^X \vec{q}_1^{X*})\vec{x}_2$ normalized, which is exactly the limit of $\vec{q}_2^{(k)}$ found in Section 12.3.

Proposition 88 (Limit of the Orthogonal Iterates)

Let $A \in \mathbb{R}^{m\times m}$ be diagonalizable with $|\lambda_1| > \cdots > |\lambda_m|$, and assume that Equation (12.19) holds. Then $Q_k \to Q^X$ up to the signs of the columns: $\vec{q}_i^{(k)} \mp \vec{q}_i^X \to \vec{0}$ for $i = 1, \ldots, r$, with the sign chosen at each step k.

Proof sketch. We check column by column, writing $\vec{q}_i$ for the limit of $\vec{q}_i^{(k)}$ (up to sign). For the first column, $\vec{q}_1^{(k)}$ performs power iteration, so $\vec{q}_1 = \pm\vec{x}_1/\|\vec{x}_1\| = \pm\vec{q}_1^X$.

For the second column, Equation (12.19) gives

$$\operatorname{span}(\vec{x}_1, \vec{x}_2) = \operatorname{span}(\vec{q}_1, \vec{q}_2) = \operatorname{span}(\vec{q}_1^X, \vec{q}_2^X) =: S_2. \tag{12.25}$$

Now $\vec{q}_2$ is a unit vector in the orthogonal complement of $\vec{q}_1$ in S_2, and $\vec{q}_2^X$ is a unit vector in the orthogonal complement of $\vec{q}_1^X = \pm\vec{q}_1$ in S_2. This complement is one-dimensional, since S_2 is two-dimensional. A real one-dimensional space contains exactly two unit vectors, so $\vec{q}_2 = \pm\vec{q}_2^X$.

Repeating the argument with

$$\operatorname{span}(\vec{x}_1, \ldots, \vec{x}_i) = \operatorname{span}(\vec{q}_1, \ldots, \vec{q}_i) = \operatorname{span}(\vec{q}_1^X, \ldots, \vec{q}_i^X) =: S_i, \tag{12.26}$$

both $\vec{q}_i$ and $\vec{q}_i^X$ are unit vectors in the orthogonal complement of $\operatorname{span}(\vec{q}_1, \ldots, \vec{q}_{i-1}) = \operatorname{span}(\vec{q}_1^X, \ldots, \vec{q}_{i-1}^X)$ in S_i, which is one-dimensional. Hence $\vec{q}_i = \pm\vec{q}_i^X$ for all $i \le r$. □

Over $\mathbb{C}$ the same argument determines each column up to a factor $e^{i\theta}$ instead of a sign. For normal A the normalized eigenvectors are orthonormal, so Q^X consists of the normalized $\vec{x}_i$ themselves (up to sign), and Proposition 88 reduces to $\vec{q}_i^{(k)} \to \pm\vec{x}_i$, as above.

12.6 Convergence to Schur Factorization

Now take $r = m$, so that Q_k and Q^X are square and orthogonal, and consider the projected matrix

$$T_k := Q_k^* A Q_k \longrightarrow Q^{X*} A Q^X, \tag{12.27}$$

where the limit holds up to signs, as discussed below. To evaluate the limit, use Equation (12.23), now with $X = [\vec{x}_1 \cdots \vec{x}_m]$:

$$Q^X R^X = X \quad \Longleftrightarrow \quad Q^X = X(R^X)^{-1} \quad \Longleftrightarrow \quad Q^{X*} = (Q^X)^{-1} = R^X X^{-1}. \tag{12.28}$$

Hence

$$Q^{X*}AQ^X = R^X X^{-1} A X (R^X)^{-1} = R^X X^{-1} X \Lambda (R^X)^{-1} = \underbrace{R^X \Lambda (R^X)^{-1}}_{\text{upper triangular}}. \tag{12.29}$$

The product of upper triangular matrices is upper triangular, and its diagonal is $r_{ii}\lambda_i r_{ii}^{-1} = \lambda_i$. So $A = Q^X (R^X \Lambda (R^X)^{-1}) Q^{X*}$ is a Schur factorization of A (Theorem 79), with the eigenvalues on the diagonal in decreasing order of modulus. The eigenpairs of the triangular factor are

$$(\lambda_i, R^X \vec{e}_i), \qquad \text{since} \quad R^X \Lambda (R^X)^{-1} R^X \vec{e}_i = R^X \Lambda \vec{e}_i = \lambda_i R^X \vec{e}_i, \tag{12.30}$$

and lifting them back gives the eigenvectors of A, $Q^X R^X \vec{e}_i = X \vec{e}_i = \vec{x}_i$. (The eigenvector is $R^X \vec{e}_i$, the ith column of R^X, and not $(R^X)^{-1}\vec{e}_i$; only the former is mapped to $\vec{x}_i$ by Q^X.)

The signs do not affect this picture. If $Q_k \approx Q^X D$ with $D = \operatorname{diag}(\pm 1)$, then $T_k \approx D\, R^X \Lambda (R^X)^{-1} D$, which is still upper triangular with the same diagonal; only some entries above the diagonal change sign.

In summary: for every k, $A = Q_k T_k Q_k^*$ exactly, with Q_k orthogonal, and the factor $T_k = Q_k^* A Q_k$ converges to an upper triangular matrix. So

$$Q_k, \quad Q_k^* A Q_k, \quad Q_k^* \tag{12.31}$$

converge to the Schur factorization of A, and we can read off the eigenvalues from the diagonal of $Q_k^* A Q_k$. The precise conditions on the starting matrix, and a proof that does not presuppose Equation (12.19), are in Theorem 90.

Computing T_{k+1} this way requires Q_k and a matrix-matrix product with A in every step. Section 13.5 derives an iteration $T_k \to T_{k+1}$ that works with T_k alone, the QR algorithm, and Lecture 14 makes each of its steps cheap by first reducing A to Hessenberg form.

12.7 (SUPPLEMENT) Further Topics

12.7.1 More Consequences of Normality

Let $A = Q\Lambda Q^*$ be normal and write $\lambda_i = |\lambda_i|\, e^{\mathrm{i}\theta_i}$. Then $A = \left(Q \operatorname{diag}(e^{\mathrm{i}\theta_i})\right) |\Lambda|\, Q^*$ is an SVD up to the ordering of the entries, so the singular values of a normal matrix are the moduli of its eigenvalues [1, Lecture 24]. In particular $\|A\|_2 = \max_i |\lambda_i|$ and $\left\|A^k\right\|_2 = \max_i |\lambda_i|^k$: powers of a normal matrix show none of the transient growth of a Jordan block (Section 11.6). The Schur form also measures how far a general A is from normal. Write the triangular factor as $T = \Lambda + N$ with N strictly upper triangular. Since the Frobenius norm is unitarily invariant,

$$\|A\|_F^2 = \|T\|_F^2 = \sum_{i=1}^{m} |\lambda_i|^2 + \|N\|_F^2 . \tag{12.32}$$

Hence $\sum_i |\lambda_i|^2 \le \|A\|_F^2$, with equality if and only if $N = 0$, i.e., if and only if A is normal (Theorem 85). The quantity $\|N\|_F$, the *departure from normality*, does not depend on which Schur factorization is chosen, since $\|A\|_F$ and the eigenvalues do not.

12.7.2 The Rayleigh–Ritz Procedure

The projected matrix $H \coloneqq Q^* A Q$ of Proposition 86, for any $Q \in \mathbb{K}^{m \times r}$ with orthonormal columns, generalizes the Rayleigh quotient from one vector to a subspace. Its eigenvalues θ_i are called *Ritz values*, and

the lifted eigenvectors $Q\vec{v}_i$ *Ritz vectors*. The choice $R = H$ minimizes the residual $\|AQ - QR\|_2$ over all $R \in \mathbb{K}^{r\times r}$: with $E \coloneqq AQ - QH$ we have $Q^*E = 0$, so for every $\vec{y}$,

$$\|(AQ - QR)\vec{y}\|^2 = \|E\vec{y} + Q(H - R)\vec{y}\|^2 = \|E\vec{y}\|^2 + \|(H - R)\vec{y}\|^2 \geq \|E\vec{y}\|^2 . \tag{12.33}$$

For Hermitian A there are r eigenvalues of A, not necessarily the largest ones, each within $\|E\|_2$ of a different Ritz value [2, §7.2]. In subspace iteration for Hermitian matrices one therefore often replaces Q_k by its Ritz vectors Q_kV, where $Q_k^*AQ_k = V\Theta V^*$. These depend only on $\operatorname{span}(Q_k) = \operatorname{span}(A^kQ_0)$, in which the components along $\vec{x}_j$, $j > r$, are damped relative to $\vec{x}_i$ by roughly $|\lambda_j/\lambda_i|^k \leq |\lambda_{r+1}/\lambda_i|^k$. So the ith Ritz vector converges like $|\lambda_{r+1}/\lambda_i|^k$ rather than $|\lambda_{i+1}/\lambda_i|^k$, and iterating with a few more columns than eigenpairs wanted speeds up convergence. Without the Rayleigh–Ritz step, extra columns do not help the first ones, since the leading columns of Algorithm 19 never see the later ones.

12.7.3 Convergence of the Individual Entries

The leading p columns span a subspace converging at the rate $|\lambda_{p+1}/\lambda_p|$, so the block $(T_k)_{p+1:m,\,1:p}$ decays like $|\lambda_{p+1}/\lambda_p|^k$. The diagonal entry $(T_k)_{pp}$ is affected by the blocks on both sides of it, and converges to λ_p like

$$\max\left(\left|\frac{\lambda_{p+1}}{\lambda_p}\right|, \left|\frac{\lambda_p}{\lambda_{p-1}}\right|\right)^k , \tag{12.34}$$

where only the first ratio is present for $p = 1$ and only the second for $p = m$ [2, §4.4.3]. A pair of eigenvalues of nearly equal modulus thus slows down only the part of T_k near their position, while the rest of the Schur form converges quickly. For symmetric A, all columns of Q_k converge to eigenvectors linearly with constant $\max_{1\leq j\leq r} |\lambda_{j+1}/\lambda_j|$ [1, Thm. 28.1].

12.7.4 Why Orthonormalize at Every Step

Mathematically, $\operatorname{span}(Q_k) = \operatorname{span}(A^kQ_0)$, so we could compute $V_k = A^kQ_0$ and orthonormalize once at the end ("unnormalized simultaneous iteration"). In floating point this fails: every column of A^kQ_0 converges to a multiple of $\vec{x}_1$, so the columns become nearly parallel, and the information about $\vec{x}_2, \ldots, \vec{x}_r$ sits in components that are smaller by factors $|\lambda_i/\lambda_1|^k$ and are lost to rounding errors [1, Lecture 28]. Orthonormalizing after every multiplication keeps a well-conditioned basis of the same subspace. With $Q_0 = I$ and $r = m$, the accumulated triangular factors give the QR factorization of A^k itself; this identity is the starting point of the QR algorithm in Lecture 13.

12.7.5 Beyond Subspace Iteration

Each step of orthogonal iteration costs one product AQ, about $2m^2r$ flops for dense A and far less for sparse A, plus $O(mr^2)$ for the QR factorization, so it is attractive when only a few dominant eigenpairs of a large sparse matrix are needed. Its convergence is still linear with the ratio $|\lambda_{r+1}/\lambda_r|$. Krylov methods do better with the same products: instead of keeping only A^kQ_0, they apply Rayleigh–Ritz to the whole Krylov subspace $\operatorname{span}(\vec{q}, A\vec{q}, \ldots, A^{k-1}\vec{q})$, which contains all earlier power iterates. This leads to the Arnoldi iteration and, for Hermitian A, the Lanczos iteration [1, Lectures 33, 34, and 36], [2, Chapter 7].

Lecture 13

QR without Shifts

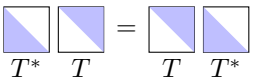
Supplementary reading:

- [1] Lecture 28.
- [2] Sections 4.4.3 and 4.4.4.

13.1 Recap

Last lecture (Lecture 12) characterized the matrices whose Schur factorization is a diagonalization, and began computing several eigenpairs at once.

- *From Schur factorization to diagonalization* (Section 12.2). A is normal, $A^*A = AA^*$, if and only if $A = Q\Lambda Q^*$ with Q unitary (Theorem 85). For the Schur factorization $A = QTQ^*$, normality of A gives $T^*T = TT^*$, and comparing $(1,1)$ entries gives $|t_{11}|^2 = \|T_{1,:}\|^2$: the first row of T is zero off the diagonal. Induction makes T diagonal. In terms of shapes, $T^*T = TT^*$ says that lower times upper equals upper times lower:



  The $(1,1)$ entry on the left sees only the first column of T, the one on the right the whole first row.

- *Orthogonal iteration for two vectors* (Section 12.3). Power iteration on $\vec{q}_1, \vec{q}_2$, projecting $\vec{q}_1$ out of $\vec{q}_2$ after each step (Algorithm 18). Then $\vec{q}_1 \to \vec{x}_1$, and once it has converged,

$$\vec{q}_2^{(k)} = (I - \vec{x}_1\vec{x}_1^*)A(I - \vec{x}_1\vec{x}_1^*)\vec{q}_2^{(k-1)} \tag{13.1}$$

  up to normalization. This matrix has the eigenpairs $(\lambda_i, (I - \vec{x}_1\vec{x}_1^*)\vec{x}_i)$ for $i \geq 2$:

$$(I - \vec{x}_1\vec{x}_1^*)A(I - \vec{x}_1\vec{x}_1^*)\vec{x}_i = \lambda_i(I - \vec{x}_1\vec{x}_1^*)\vec{x}_i. \tag{13.2}$$

  Hence $\vec{q}_2^{(k)} \to (I - \vec{x}_1\vec{x}_1^*)\vec{x}_2$, which differs from $\vec{x}_2$ unless $\vec{x}_1 \perp \vec{x}_2$ (as for normal A). What converges is the subspace, $\operatorname{span}(\vec{q}_1, \vec{q}_2) \to \operatorname{span}(\vec{x}_1, \vec{x}_2)$.

- *Orthogonal iteration for a general number of vectors* (Section 12.4). With r columns, the projection step becomes a reduced QR factorization (Algorithm 19):

```
for k = 1:iterations
    Q = A*Q
    Q, R = qr(Q)
end
```

13.2 Schur and Eigendecomposition

Throughout this lecture we consider

$$A = X\Lambda X^{-1}, \qquad |\lambda_1| > |\lambda_2| > \cdots > |\lambda_m|\,, \tag{13.3}$$

i.e., A is diagonalizable and its eigenvalues have distinct moduli.

This assumption is *generic*: it holds for all matrices except a set that can be removed by an arbitrarily small perturbation. The reason is that every A has a Schur factorization $A = QTQ^*$ with the eigenvalues on the diagonal of T (Theorem 79). We can add a small perturbation to the diagonal of T, and hence to A, such that

$$\tilde{A} \coloneqq Q(T + \operatorname{diag}(\varepsilon_1, \ldots, \varepsilon_m))Q^* = A + Q\operatorname{diag}(\varepsilon_1, \ldots, \varepsilon_m)Q^* \tag{13.4}$$

has eigenvalues of distinct magnitude. Indeed, $\tilde{A}$ is in Schur form with diagonal entries $t_{ii} + \varepsilon_i$, these are its eigenvalues, and the ε_i can be chosen to make their moduli distinct. The perturbation is small, $\left\|\tilde{A} - A\right\| = \max_i |\varepsilon_i|$, since Q is unitary. Distinct eigenvalues imply diagonalizability, so $\tilde{A}$ satisfies Equation (13.3). Conversely, eigenvalues depend continuously on the matrix, so the strict inequalities survive small perturbations of $\tilde{A}$.

Remark 89 (Real Matrices (SUPPLEMENT))**.** The perturbation above is complex in general. For real A, non-real eigenvalues come in conjugate pairs $\lambda, \overline{\lambda}$ with $|\lambda| = \left|\overline{\lambda}\right|$, and a small real perturbation keeps them a conjugate pair. So among real matrices, Equation (13.3) is not generic; it fails on an open set, e.g., near every rotation of $\mathbb{R}^2$ by an angle $\theta \notin \{0, \pi\}$.

Now, given $A = X\Lambda X^{-1}$, how do we compute its Schur form? Take a QR factorization of the eigenvector matrix, $X = QR$. Since X is invertible, so is R, and

$$X = QR \quad \Longrightarrow \quad A = QR\Lambda R^{-1}Q^* = Q\underbrace{(R\Lambda R^{-1})}_{\text{upper triangular}}Q^*. \tag{13.5}$$

Indeed, R^{-1} is upper triangular, products of upper triangular matrices are upper triangular, and the diagonal of $R\Lambda R^{-1}$ is $r_{ii}\lambda_i r_{ii}^{-1} = \lambda_i$. So Equation (13.5) is a Schur factorization of A, with the eigenvalues on the diagonal of $T = R\Lambda R^{-1}$ in the same order as in Λ.

Since R is upper triangular, $\vec{x}_j = \sum_{i \le j} r_{ij}\vec{q}_i$ only involves the first j columns of Q, and since R is invertible the converse holds as well. Hence

$$\operatorname{span}(\vec{q}_1, \ldots, \vec{q}_j) = \operatorname{span}(\vec{x}_1, \ldots, \vec{x}_j), \qquad j = 1, \ldots, m : \tag{13.6}$$

the Schur vectors span the same nested invariant subspaces as the eigenvectors. This is not an algorithm, since it needs X, but it identifies the target: in the rest of this chapter we show that orthogonal iteration converges to these subspaces without knowing X.

13.3 Convergence of Orthogonal Iteration

How should we think about the convergence of orthogonal iteration (Algorithm 19)? Let $Q_k \in \mathbb{K}^{m\times p}$ be the iterate after k steps, starting from Q_0 with p orthonormal columns. Let $A = X\Lambda X^{-1}$ be diagonalizable with its eigenvalues ordered by modulus, $|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_m|$. For this section we need less than Equation (13.3): it suffices that there is a gap after the pth eigenvalue,

$$|\lambda_p| > |\lambda_{p+1}|\,. \tag{13.7}$$

As in Lecture 12, track the subspace $\operatorname{span}(Q_k)$, the span of the columns, rather than the individual columns.

The QR step does not change the column space, so $\operatorname{span}(Q_{k+1}) = \operatorname{span}(AQ_k)$, and unrolling the recursion,

$$\operatorname{span}(Q_{k+1}) = \operatorname{span}(AQ_k) = \cdots = \operatorname{span}(A^{k+1}Q_0) = \operatorname{span}(X\Lambda^{k+1}X^{-1}Q_0). \tag{13.8}$$

Pull out the factor λ_p^{k+1}, which does not change the span (note $\lambda_p \neq 0$, since $|\lambda_p| > |\lambda_{p+1}| \geq 0$ by Equation (13.7)):

$$\operatorname{span}(Q_{k+1}) = \operatorname{span}\left(\lambda_p^{k+1} X \begin{bmatrix} (\lambda_1/\lambda_p)^{k+1} & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & & \ddots & \\ & & & & (\lambda_m/\lambda_p)^{k+1} \end{bmatrix} X^{-1}Q_0\right), \tag{13.9}$$

with the 1 in position (p,p). By the ordering of the eigenvalues and the gap Equation (13.7),

$$|\lambda_i/\lambda_p| \geq 1 \quad \text{for } i \leq p, \qquad |\lambda_i/\lambda_p| < 1 \quad \text{for } i > p. \tag{13.10}$$

So the first p rows of $(\Lambda/\lambda_p)^{k+1}X^{-1}Q_0$ do not decay, while the last $m-p$ rows are scaled by factors that tend to zero. Split accordingly:

$$(\Lambda/\lambda_p)^{k+1}\, X^{-1}Q_0 = \begin{bmatrix} V_k \\ W_k \end{bmatrix}, \qquad V_k \in \mathbb{K}^{p\times p}, \quad W_k \in \mathbb{K}^{(m-p)\times p}, \qquad W_k \xrightarrow{k\to\infty} 0, \tag{13.11}$$

where W_k decays like $|\lambda_{p+1}/\lambda_p|^{k+1}$. Splitting the columns of X in the same way, $X = [X_p\ \tilde{X}_p]$ with $X_p = [\vec{x}_1 \cdots \vec{x}_p]$ the first p eigenvectors, gives

$$\operatorname{span}(Q_{k+1}) = \operatorname{span}(X\Lambda^{k+1}X^{-1}Q_0) = \operatorname{span}(X_pV_k + \tilde{X}_pW_k) \xrightarrow{k\to\infty} \operatorname{span}(X_p). \tag{13.12}$$

So $\operatorname{span}(Q_k)$ converges to $\operatorname{span}(\vec{x}_1,\ldots,\vec{x}_p)$, the invariant subspace of the p dominant eigenvectors.

To make the limit precise (SUPPLEMENT), write $B \coloneqq X^{-1}Q_0 = \left[\begin{smallmatrix} B_1 \\ B_2 \end{smallmatrix}\right]$, the coordinates of Q_0 in the eigenvector basis, with $B_1 \in \mathbb{K}^{p\times p}$, and $\Lambda = \operatorname{diag}(\Lambda_1, \Lambda_2)$ with $\Lambda_1 = \operatorname{diag}(\lambda_1,\ldots,\lambda_p)$. Then

$$V_k = (\Lambda_1/\lambda_p)^{k+1}B_1, \qquad W_k = (\Lambda_2/\lambda_p)^{k+1}B_2. \tag{13.13}$$

Assume B_1 is invertible, i.e., Q_0 has a nonzero component in every direction $\vec{x}_1,\ldots,\vec{x}_p$; this is the analogue of $\vec{x}_1^*\vec{q}^{(0)} \neq 0$ for power iteration and holds with probability one for random Q_0. Then V_k is invertible, and multiplying by V_k^{-1} on the right does not change the span:

$$\operatorname{span}(X_pV_k + \tilde{X}_pW_k) = \operatorname{span}\big(X_p + \tilde{X}_pW_kV_k^{-1}\big). \tag{13.14}$$

Since $V_k^{-1} = B_1^{-1}(\lambda_p \Lambda_1^{-1})^{k+1}$ and $|\lambda_p/\lambda_i| \le 1$ for $i \le p$, we get $\left\|V_k^{-1}\right\| \le \left\|B_1^{-1}\right\|$, so

$$\left\|\tilde{X}_p W_k V_k^{-1}\right\| \le \left\|\tilde{X}_p\right\| \|B_2\| \left\|B_1^{-1}\right\| \left|\frac{\lambda_{p+1}}{\lambda_p}\right|^{k+1} \longrightarrow 0. \tag{13.15}$$

The subspace converges at the rate $|\lambda_{p+1}/\lambda_p|$, the gap between the last eigenvalue we keep and the first one we discard.

13.4 Convergence to Schur Factorization

Now run orthogonal iteration with $p = m$, i.e., on a full orthonormal basis, starting from $Q_0 = I$.

Theorem 90 (Orthogonal Iteration Converges to Schur Form)

Consider orthogonal iteration (Algorithm 19) on $A = X\Lambda X^{-1}$ with $p = m$ and $Q_0 = I$. If all eigenvalues of A have distinct absolute values, $|\lambda_1| > \cdots > |\lambda_m|$, and the leading blocks $(X^{-1})_{1:j,1:j}$ have full rank for $j = 1, \ldots, m$, then

$$T_k := Q_k^* A Q_k \tag{13.16}$$

converges to the Schur form of A: its strictly lower triangular part tends to zero and its diagonal tends to $(\lambda_1, \ldots, \lambda_m)$.

Note that the condition is on the leading blocks of X^{-1}, not of X; the two are not equivalent, although the condition is sometimes stated with $X_{1:j,1:j}$ (e.g., [2, Thm. 4.8]). The argument of Section 13.3 needs a condition on $X^{-1}Q_0 = X^{-1}$: for j columns, the block B_1 consists of the first j rows and columns, $(X^{-1})_{1:j,1:j}$. Requiring instead that $X_{1:j,1:j}$ have full rank for all j is not enough. For

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \qquad X^{-1} = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}, \tag{13.17}$$

$X_{1:1,1:1} = 1$ and X are nonsingular, but $(X^{-1})_{11} = 0$: the starting vector $\vec{e}_1 = \vec{x}_2$ is itself an eigenvector, has no component along $\vec{x}_1$, and in exact arithmetic $\vec{q}_1^{(k)} = \vec{x}_2$ forever. By Jacobi's identity for complementary minors, $\det(X^{-1})_{1:j,1:j} = \det X_{j+1:m,j+1:m} / \det X$, so the correct condition can equally be stated as: the trailing blocks $X_{j+1:m,j+1:m}$ are nonsingular. Equivalently, X^{-1} has an LU factorization without pivoting. For normal $A = Q\Lambda Q^*$ the distinction disappears: $X = Q$ is unitary, $X^{-1} = Q^*$, and $(Q^*)_{1:j,1:j} = (Q_{1:j,1:j})^*$, so the leading blocks of X^{-1} are nonsingular exactly when those of X are. This is why Trefethen and Bau, who treat real symmetric $A = Q\Lambda Q^\top$, can state the condition as "all leading principal submatrices of Q are nonsingular" [1, Thm. 28.4]. For non-normal A the example above shows that only the condition on X^{-1} is correct.

Proof sketch. Fix $p \in \{1, \ldots, m-1\}$ and split the columns of Q_k after the pth (here p is a split index; Q_k has all m columns):

$$Q_k = [\bar{Q}_k \; \tilde{Q}_k], \qquad \bar{Q}_k \in \mathbb{K}^{m \times p}. \tag{13.18}$$

Then

$$T_k = Q_k^* A Q_k = \begin{bmatrix} \bar{Q}_k^* A \bar{Q}_k & \bar{Q}_k^* A \tilde{Q}_k \\ \tilde{Q}_k^* A \bar{Q}_k & \tilde{Q}_k^* A \tilde{Q}_k \end{bmatrix}. \tag{13.19}$$

Column j of a QR factorization depends only on the first j columns of the factored matrix, so $\bar{Q}_k$ is exactly the iterate of orthogonal iteration with p columns started from $\bar{Q}_0 = I_{:,1:p}$. By Section 13.3,

$$\operatorname{span}(\bar{Q}_k) \longrightarrow \operatorname{span}(\vec{x}_1, \ldots, \vec{x}_p). \tag{13.20}$$

Applying A,

$$\operatorname{span}(A\bar{Q}_k) \longrightarrow \operatorname{span}(A[\vec{x}_1 \cdots \vec{x}_p]) = \operatorname{span}([\vec{x}_1 \cdots \vec{x}_p]), \tag{13.21}$$

where the equality holds because $A[\vec{x}_1 \cdots \vec{x}_p] = [\vec{x}_1 \cdots \vec{x}_p] \operatorname{diag}(\lambda_1, \ldots, \lambda_p)$ and $\lambda_1, \ldots, \lambda_p \neq 0$ (as $|\lambda_p| > |\lambda_{p+1}| \geq 0$): the span of the dominant eigenvectors is invariant under A. So the columns of $A\bar{Q}_k$ lie (asymptotically) in $\operatorname{span}(\bar{Q}_k)$, while the columns of $\tilde{Q}_k$ are orthogonal to $\operatorname{span}(\bar{Q}_k)$. Hence

$$\tilde{Q}_k^* A \bar{Q}_k \longrightarrow 0. \tag{13.22}$$

More precisely, $\tilde{Q}_k^* \bar{Q}_k = 0$ gives $\tilde{Q}_k^* A \bar{Q}_k = \tilde{Q}_k^* (I - \bar{Q}_k \bar{Q}_k^*) A \bar{Q}_k$, and $(I - \bar{Q}_k \bar{Q}_k^*) A \bar{Q}_k$, the part of $A\bar{Q}_k$ outside $\operatorname{span}(\bar{Q}_k)$, tends to zero.

The lower left block of the split after column p contains the entries (i, j) with $i > p \geq j$. Every strictly lower triangular entry (i, j), $i > j$, lies in this block for $p = j$, so the argument holds for all subdiagonal blocks, and letting p range over $1, \ldots, m-1$ shows that

$$T_k = Q_k^* A Q_k \longrightarrow \text{upper triangular.} \tag{13.23}$$

For the diagonal (SUPPLEMENT): $\bar{Q}_k^* A \bar{Q}_k$ is the matrix of A restricted to $\operatorname{span}(\bar{Q}_k) \to \operatorname{span}(\vec{x}_1, \ldots, \vec{x}_p)$, so its eigenvalues tend to $\lambda_1, \ldots, \lambda_p$, as in Proposition 86. Since this block becomes upper triangular, its diagonal holds these eigenvalues, and comparing p with $p-1$ shows that the pth diagonal entry of T_k tends to λ_p. □

13.5 Basic QR Iteration

Orthogonal iteration allows computing

$$T_k \coloneqq Q_k^* A Q_k, \tag{13.24}$$

which converges to the triangular factor of a Schur factorization (Theorem 90).

Computing T_{k+1} this way requires Q_k and a matrix-matrix product with A. Can we instead derive an iteration $T_k \to T_{k+1}$ that works with the T's alone? Observe that

$$T_k = Q_k^* A Q_k, \qquad T_{k+1} = Q_{k+1}^* A Q_{k+1}. \tag{13.25}$$

One step of orthogonal iteration is the QR factorization $Q_{k+1} R_{k+1} = A Q_k$. Multiplying by Q_{k+1}^* on the left and by Q_k^* on the right (here $p = m$, so Q_k is square and $Q_k Q_k^* = I$),

$$Q_{k+1} R_{k+1} = A Q_k \quad \Longrightarrow \quad R_{k+1} Q_k^* = Q_{k+1}^* A. \tag{13.26}$$

Substituting the first form into T_k and the second into T_{k+1},

$$T_k = Q_k^* (Q_{k+1} R_{k+1}), \qquad T_{k+1} = (R_{k+1} Q_k^*) Q_{k+1}. \tag{13.27}$$

Both expressions contain the same product $Q_k^* Q_{k+1}$. Naming it,

$$U_{k+1} \coloneqq Q_k^* Q_{k+1} : \qquad T_k = U_{k+1} R_{k+1}, \qquad T_{k+1} = R_{k+1} U_{k+1}. \tag{13.28}$$

Here U_{k+1} is unitary, since $U_{k+1}^* U_{k+1} = Q_{k+1}^* Q_k Q_k^* Q_{k+1} = I$, and R_{k+1} is upper triangular. So $T_k = U_{k+1} R_{k+1}$ is a QR factorization of T_k, and T_{k+1} is obtained by multiplying its factors in reverse order:

$$T_k = U_{k+1} R_{k+1} \quad \longrightarrow \quad T_{k+1} = R_{k+1} U_{k+1}. \tag{13.29}$$

In short: factor as QR, multiply back as RQ. Neither A nor Q_k appears. Note that $T_{k+1} = R_{k+1} U_{k+1} = U_{k+1}^* (U_{k+1} R_{k+1}) U_{k+1} = U_{k+1}^* T_k U_{k+1}$ is unitarily similar to T_k, so every T_k has the eigenvalues of A.

Starting orthogonal iteration from $Q_0 = I$ gives $T_0 = Q_0^* A Q_0 = A$, so the iteration starts from A itself.

Algorithm 20 Basic QR iteration (QR algorithm without shifts).

Input: $A \in \mathbb{K}^{m \times m}$

1: $T \leftarrow A$
2: **for** $k = 1, \ldots,$ maxiter **do**
3: $\quad UR \leftarrow T$ ▷ QR factorization
4: $\quad T \leftarrow RU$
5: **return** $\operatorname{diag}(T)$ ▷ approximate eigenvalues $\lambda_1, \ldots, \lambda_m$

By Theorem 90, T converges to upper triangular form with the eigenvalues on its diagonal, ordered by decreasing modulus. A library QR routine need not return the same factors as orthogonal iteration: for invertible T, QR factors are unique only up to signs (phases) of the columns of U and rows of R (Lecture 7). Any choice gives $T_{k+1} = U^* T_k U$, and different choices produce iterates that differ by a diagonal unitary similarity $D^* T_k D$, which leaves the diagonal and the moduli of all entries unchanged.

How to obtain Q? Algorithm 20 returns only the eigenvalues; the Schur vectors Q_k of orthogonal iteration never appear. They can be recovered from the factors U, since by definition

$$U_{k+1} = Q_k^* Q_{k+1}. \tag{13.30}$$

The product of the U's telescopes: since $Q_k Q_k^* = I$ and $Q_0 = I$,

$$U_1 U_2 \cdots U_{k+1} = (Q_0^* Q_1)(Q_1^* Q_2) \cdots (Q_k^* Q_{k+1}) = Q_0^* Q_{k+1} = Q_{k+1}. \tag{13.31}$$

So to obtain the Schur vectors, start from $Q \leftarrow I$ and accumulate $Q \leftarrow QU$ after each QR factorization in Algorithm 20; on return, $A \approx QTQ^*$.

The factors U_k have a simple interpretation. Close to convergence, $Q_{k+1} \approx Q_k$ up to the phases of the columns, so $U_{k+1} = Q_k^* Q_{k+1}$ is a small rotation: it is close to a diagonal unitary matrix D. This can also be seen from $T_k = U_{k+1} R_{k+1}$. Once T_k is nearly upper triangular, $U_{k+1} = T_k R_{k+1}^{-1}$ is nearly upper triangular and unitary, hence nearly diagonal. With the normalization $r_{ii} > 0$ we get $D \approx \operatorname{diag}(\lambda_1/|\lambda_1|, \ldots, \lambda_m/|\lambda_m|)$, so $D = I$ only when all eigenvalues are positive. A negative eigenvalue flips the sign of the corresponding column of Q_k in every step, exactly as in power iteration. Up to these phases, the U_k form a sequence of rotational corrections that transform $Q_0 = I$ into the Schur vectors Q, with large corrections in the first steps and ever smaller ones as T_k approaches triangular form.

Remark 91 (Julia)**.** For square T the implicit Q returned by `qr` (Lecture 7) can be multiplied directly, so the pitfall of Lecture 12 does not arise:

```
T = copy(A); Q = Matrix(1.0I, m, m)
for k = 1:maxiter
```

```
    F = qr(T)
    T = F.R * F.Q
    Q = Q * F.Q
end
```

13.6 (SUPPLEMENT) Further Topics

13.6.1 The QR Algorithm Factors the Powers of A

Let $\underline{R}_k \coloneqq R_k R_{k-1} \cdots R_1$, the product of the triangular factors of orthogonal iteration started from $Q_0 = I$. Then

$$A^k = Q_k \underline{R}_k, \qquad T_k = Q_k^* A Q_k, \qquad Q_k = U_1 U_2 \cdots U_k. \tag{13.32}$$

The first identity follows by induction, $A^k = A\,Q_{k-1}\underline{R}_{k-1} = Q_k R_k \underline{R}_{k-1} = Q_k \underline{R}_k$, and $\underline{R}_k$ is upper triangular as a product of upper triangular matrices. So the QR algorithm computes a QR factorization of A^k without ever forming A^k, whose columns would all be nearly parallel to $\vec{x}_1$. The two identities in Equation (13.32) explain the whole algorithm [1, Lecture 28]. The first says that the leading j columns of Q_k are an orthonormal basis of the span of the leading j columns of A^k: this is simultaneous power iteration on $\vec{e}_1, \ldots, \vec{e}_m$. The second says that the diagonal entries $(T_k)_{jj} = \vec{q}_j^{(k)*} A \vec{q}_j^{(k)}$ are Rayleigh quotients. For Hermitian A, they therefore converge twice as fast as the columns of Q_k (Lecture 10).

13.6.2 Simultaneous Inverse Iteration

If A is invertible, inverting and taking the adjoint of $A^k = Q_k \underline{R}_k$ gives

$$(A^*)^{-k} = \left(\underline{R}_k^{-1} Q_k^*\right)^* = Q_k \underline{R}_k^{-*}, \tag{13.33}$$

with $\underline{R}_k^{-*}$ lower triangular. Let P be the permutation that reverses the order of the rows, $P^2 = I$. Reversing both the rows and the columns of a lower triangular matrix makes it upper triangular, so

$$(A^*)^{-k} P = (Q_k P)(P \underline{R}_k^{-*} P) \tag{13.34}$$

is a QR factorization of $(A^*)^{-k}P$. Hence $Q_k P$, i.e., Q_k with its columns in reverse order, is what orthogonal iteration with $(A^*)^{-1}$ produces from $Q_0 = P$. In particular, the last column of Q_k is the result of k steps of inverse iteration (Algorithm 16) with A^*, started from $\vec{e}_m$. It converges at the rate $|\lambda_m/\lambda_{m-1}|$ to an eigenvector $\vec{y}_m$ of A^* for $\overline{\lambda_m}$. This is a left eigenvector of A, $\vec{y}_m^* A = \lambda_m \vec{y}_m^*$, and it is orthogonal to $\vec{x}_1, \ldots, \vec{x}_{m-1}$, consistent with Equation (13.6). So the QR algorithm runs power iteration in its first column and inverse iteration in its last column at the same time. For real symmetric A, $A^* = A$; see [1, Lecture 29].

13.6.3 Convergence Rates of Individual Entries

By Section 13.3, the block $(T_k)_{p+1:m,1:p}$ below and to the left of the split after column p decays like $|\lambda_{p+1}/\lambda_p|^k$. In particular, the subdiagonal entry $(T_k)_{j+1,j}$ decays like $|\lambda_{j+1}/\lambda_j|^k$, and the diagonal entry $(T_k)_{pp}$ converges to λ_p like $\max(|\lambda_{p+1}/\lambda_p|^k, |\lambda_p/\lambda_{p-1}|^k)$ [2, §4.4.3]. The overall rate is the worst ratio $\max_j |\lambda_{j+1}/\lambda_j|$ [1, Thm. 28.4], so two eigenvalues of nearly equal modulus make the unshifted algorithm arbitrarily slow. For example, for $\Lambda = \mathrm{diag}(30, 6, 2, 1)$ and a random X with $\kappa(X) \approx 20$, after 19 steps the subdiagonal entries are about $7 \cdot 10^{-13}$, $1 \cdot 10^{-9}$, and $5 \cdot 10^{-6}$ [2, Example 4.5], in line with $(1/5)^{19} \approx 5 \cdot 10^{-14}$, $(1/3)^{19} \approx 9 \cdot 10^{-10}$, and $(1/2)^{19} \approx 2 \cdot 10^{-6}$ up to constants. The most important block is the last row, $(T_k)_{m,1:m-1}$, which decays like $|\lambda_m/\lambda_{m-1}|^k$. Once it is below a tolerance such as $\varepsilon_{\text{mach}}\,\|A\|$, we accept $(T_k)_{mm}$ as an eigenvalue and continue on the leading $(m-1) \times (m-1)$ block (deflation).

13.6.4 Eigenvalues of Equal Modulus

Without a gap $|\lambda_p| > |\lambda_{p+1}|$, the block at the split after column p has no reason to decay. In the extreme case of a unitary A, all eigenvalues have modulus 1. Its QR factorization is $A = A \cdot I$, so $T_1 = I \cdot A = A$, and the iteration does not move at all [2, Example 4.6]. An example is $A = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$, with eigenvalues ± 1. The generic case for real matrices is a pair of complex conjugate eigenvalues $\lambda, \overline{\lambda}$ (see the remark in Section 13.2). For real A, all U_k, R_k, and T_k are real, so T_k cannot converge to a triangular matrix with non-real diagonal entries. The splits between eigenvalues of different moduli still have a gap, however, so the corresponding blocks still decay (under the analogous rank conditions). If the only ties in modulus come from conjugate pairs, T_k converges to a block upper triangular matrix with 1×1 and 2×2 diagonal blocks, i.e., to the real Schur form (Theorem 81). The entries of each 2×2 block keep changing from step to step, but its eigenvalues converge to $\lambda, \overline{\lambda}$.

13.6.5 Why Shifts Help

The shifted matrix $A - \sigma I = Q(T - \sigma I)Q^*$ has the same Schur vectors as A and the eigenvalues $\lambda_i - \sigma$. Running the QR algorithm on it therefore converges at the rates $|\lambda_{j+1} - \sigma| \,/\, |\lambda_j - \sigma|$, with the eigenvalues now ordered by their distance from σ. If σ is close to an eigenvalue λ, the last row decays at the rate $|\lambda - \sigma| \,/\, \min_{\lambda_j \neq \lambda} |\lambda_j - \sigma|$, which is nearly zero. The shift may change from step to step, and it need not be undone:

$$T_k - \sigma_k I = U_{k+1} R_{k+1}, \qquad T_{k+1} = R_{k+1} U_{k+1} + \sigma_k I = U_{k+1}^* T_k U_{k+1}, \tag{13.35}$$

so T_{k+1} is still unitarily similar to A [2, §4.4.4]. By the previous paragraphs, the last column of Q_k then performs shifted inverse iteration. The natural choice $\sigma_k = (T_k)_{mm}$, the Rayleigh quotient of that column, turns it into a version of Rayleigh quotient iteration (Algorithm 17), with cubic convergence for real symmetric A [1, Lecture 29]. Shifts also break ties in modulus: for $A = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ and real $\sigma \neq 0$, $|1 - \sigma| \neq |-1 - \sigma|$. There, however, $(T_k)_{mm} = 0$ gives no shift at all, which is why practical codes use a more robust shift. Fast convergence alone is not enough, since each step of Algorithm 20 still costs $O(m^3)$. Reducing this cost with a Hessenberg reduction, and adding shifts, is the subject of Lecture 14.

Lecture 14

Hessenberg QR

Supplementary reading:

- [1] Lecture 26.
- [2] Sections 4.4.5, 4.4.6, and 4.4.8.

14.1 Recap: From Orthogonal Iteration to the QR Algorithm

We begin with a recap of Lecture 13. If $A = X\Lambda X^{-1}$ and $X = QR$, then $A = Q(R\Lambda R^{-1})Q^*$ with $R\Lambda R^{-1}$ upper triangular, so the Schur vectors span the same nested subspaces as the eigenvectors (Section 13.2). Orthogonal iteration finds these subspaces without knowing X: since $\operatorname{span}(Q_k) = \operatorname{span}(A^k Q_0)$, the span of the first p columns converges to $\operatorname{span}(\vec{x}_1, \ldots, \vec{x}_p)$ at the rate $|\lambda_{p+1}/\lambda_p|$ (Section 13.3). Splitting $Q_k = [\bar{Q}_k \; \tilde{Q}_k]$ after every column p shows that $\tilde{Q}_k^* A \bar{Q}_k \to 0$, so $T_k \coloneqq Q_k^* A Q_k$ converges to the triangular factor of a Schur factorization (Theorem 90). Finally, with $U_{k+1} \coloneqq Q_k^* Q_{k+1}$ the iteration can be written in terms of T_k alone,

$$T_k = U_{k+1} R_{k+1} \quad \longrightarrow \quad T_{k+1} = R_{k+1} U_{k+1}, \tag{14.1}$$

starting from $T_0 = A$. This is the basic QR iteration (Algorithm 20).

This lecture asks: *how expensive is the QR iteration, and how can it be made cheaper?*

14.2 The Cost of the QR Iteration

Each step of Algorithm 20 computes the QR factorization of a dense $m \times m$ matrix, at a cost of $O(m^3)$, and we possibly need many such steps. Even if every eigenvector converged in $O(1)$ iterations, there are m of them, so we would still need $O(m)$ factorizations and a total cost of $O(m^4)$. Speeding up convergence alone is therefore not enough; the cost of each step must come down as well.

The idea is to *preprocess A once* so that every subsequent QR factorization becomes simple. The preprocessing must not change the eigenvalues, so it has to be a similarity transformation, $A \mapsto Q^* A Q$, rather than a one-sided product as in the QR factorization. We take Q unitary, which does not worsen the conditioning (Theorem 111). Reducing A all the way to triangular form is impossible, since that would compute a Schur factorization in finitely many steps (Section 10.3). The best we can do is one diagonal short of that.

Definition 92 (Hessenberg Matrix)

A matrix $H \in \mathbb{K}^{m\times m}$ is *upper Hessenberg* if $h_{ij} = 0$ for $i > j + 1$, i.e., it is upper triangular except for one additional subdiagonal:

$$H = \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix}. \tag{14.2}$$

14.3 Reduction to Hessenberg Form

We compute Q using Householder reflections (Lecture 6). Recall that a Householder reflector is Hermitian as well as unitary, so it is its own inverse and its own adjoint.

Let $P_1 \in \mathbb{K}^{(m-1)\times(m-1)}$ be the Householder reflector that maps

$$\vec{v} \coloneqq A_{2:m,1} \quad \text{to} \quad \|\vec{v}\|\,\vec{e}_1, \qquad \text{and set} \qquad Q_1 \coloneqq \begin{bmatrix} 1 & 0 \\ 0 & P_1 \end{bmatrix}. \tag{14.3}$$

Multiplying by Q_1 on the left leaves the first row of A unchanged and zeros out the first column below the subdiagonal:

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}}_{A} \xrightarrow{Q_1 \cdot} \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix}}_{Q_1 A}. \tag{14.4}$$

To complete the similarity we multiply by $Q_1^* = Q_1$ on the right. This acts only on columns $2, \ldots, m$, so the first column of $Q_1 A$, together with its new zeros, is left unchanged:

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix}}_{Q_1 A} \xrightarrow{\cdot Q_1^*} \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix}}_{Q_1 A Q_1^*}. \tag{14.5}$$

This is why the reflector starts at row 2 and leaves a_{21} in place. A reflector acting on all rows, as in the first step of Householder QR, would zero out a_{21} as well, but multiplying by its adjoint on the right would then mix all columns and fill the first column back in.

Now repeat the same step on the trailing $(m-1)\times(m-1)$ block. Step k uses the reflector P_k that maps the part of column k below the subdiagonal, $\vec{v} = (\cdot)_{k+1:m,k}$, to $\|\vec{v}\|\,\vec{e}_1$, embedded as $Q_k = \operatorname{diag}(I_k, P_k)$. Left multiplication by Q_k touches only rows $k+1, \ldots, m$ and right multiplication only columns $k+1, \ldots, m$, so the zeros created in columns $1, \ldots, k-1$ survive. After $m-2$ steps (the last column has nothing below the subdiagonal) we gather the reflectors into

$$Q \coloneqq Q_1 Q_2 \cdots Q_{m-2}, \qquad Q^* A Q = Q_{m-2} \cdots Q_1\, A\, Q_1 \cdots Q_{m-2} = H. \tag{14.6}$$

The order of the factors in Q matters: each Q_k is Hermitian, but $Q^* = Q_{m-2} \cdots Q_1$ has them in reverse order. (The board gathers the reflectors as "$Q_m \cdots Q_1 = Q$". Since every Q_k is Hermitian, that product is

Q^* in the notation used here, and only $m-2$ reflectors are needed because the last two columns have nothing below the subdiagonal.)

As in Householder QR, neither the Q_k nor Q is ever formed; we store the reflection vectors and apply them from both sides.

Algorithm 21 Householder reduction to Hessenberg form [1, Algorithm 26.1].

Input: $A \in \mathbb{K}^{m\times m}$, overwritten by $H = Q^*AQ$

1: **for** $k = 1, \dots, m-2$ **do**
2: $\quad \vec{x} \leftarrow A_{k+1:m,k}$
3: $\quad \vec{v}_k \leftarrow \operatorname{sign}(x_1)\,\|\vec{x}\|\,\vec{e}_1 + \vec{x}, \quad \vec{v}_k \leftarrow \vec{v}_k / \|\vec{v}_k\|$
4: $\quad A_{k+1:m,k:m} \leftarrow A_{k+1:m,k:m} - 2\vec{v}_k(\vec{v}_k^* A_{k+1:m,k:m})$ ▷ from the left: rows $k+1, \dots, m$
5: $\quad A_{1:m,k+1:m} \leftarrow A_{1:m,k+1:m} - 2(A_{1:m,k+1:m}\vec{v}_k)\vec{v}_k^*$ ▷ from the right: columns $k+1, \dots, m$

The reflector here maps $\vec{x}$ to $-\operatorname{sign}(x_1)\,\|\vec{x}\|\,\vec{e}_1$ rather than $\|\vec{x}\|\,\vec{e}_1$, which avoids cancellation exactly as in Lecture 6; either choice gives a Hessenberg matrix. The left updates cost about $\frac{4}{3}m^3$ flops and the right updates, which act on full columns, about $2m^3$, for a total of

$$\text{work for Hessenberg reduction} \sim \tfrac{10}{3}m^3 \text{ flops } [1, (26.1)], \tag{14.7}$$

paid once before the QR iteration starts.

14.4 The QR Iteration on Hessenberg Matrices

We now run Algorithm 20 on $T_0 = H$ instead of A. The first half of each step is a QR factorization $U_kR_k = T_{k-1}$. If T_{k-1} is upper Hessenberg, we have seen in Section 7.3 that $O(m)$ Givens rotations suffice: only the $m-1$ subdiagonal entries need to be eliminated, and they can be removed one at a time from the top. Multiplying by G_1^* on the left acts on rows 1 and 2 and zeros out the $(2,1)$ entry, G_2^* acts on rows 2 and 3 and zeros out the $(3,2)$ entry, and so on:

$$\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_1^*} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_2^*} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_3^*} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix}. \tag{14.8}$$

Each rotation changes only two rows, at a cost of $O(m)$, so the whole factorization

$$R_k = G_{m-1}^* \cdots G_1^* T_{k-1}, \qquad U_k = G_1 G_2 \cdots G_{m-1}, \tag{14.9}$$

costs $O(m^2)$ instead of $O(m^3)$. We never form U_k; we only store the $m-1$ rotations.

The second half of each step computes $T_k = R_kU_k$. With U_k in factored form,

$$T_k = R_k G_1 G_2 \cdots G_{m-1}, \tag{14.10}$$

and multiplying by G_j on the right changes only columns j and $j+1$, again at a cost of $O(m)$. So the second half also costs $O(m^2)$.

For this to repeat, T_k must again be upper Hessenberg. It is, because right multiplication by G_j replaces columns j and $j+1$ by linear combinations of these two columns. Column $j+1$ of $R_kG_1 \cdots G_{j-1}$ has

nonzeros down to row $j+1$, so mixing it into column j creates a single new nonzero at position $(j+1, j)$, on the subdiagonal, and nothing below it:

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix}}_{R_k} \xrightarrow{\cdot\, G_1} \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix} \xrightarrow{\cdot\, G_2} \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix} \xrightarrow{\cdot\, G_3} \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix}}_{T_k}. \tag{14.11}$$

Hence $T_k = R_k G_1 \cdots G_{m-1}$ is upper Hessenberg, and every step of the QR iteration costs $O(m^2)$. The total cost of m iterations is $O(m^3)$, the same order as the one-time reduction to Hessenberg form, compared with $O(m^4)$ for the QR iteration on the dense matrix A.

14.5 Bulge Chasing

So far, one step of the QR iteration on a Hessenberg matrix consists of two sweeps. The first sweep applies $G_1^*, \ldots, G_{m-1}^*$ from the left and turns T_{k-1} into the upper triangular R_k; the second applies $G_1, \ldots, G_{m-1}$ from the right and turns R_k back into the Hessenberg matrix T_k:

$$T_k = \left(G_{m-1}^* \cdots G_1^* T_{k-1}\right) G_1 \cdots G_{m-1}. \tag{14.12}$$

Since matrix multiplication is associative, nothing forces this order. *Can we apply the left and right rotations simultaneously*, that is, compute

$$T_k = G_{m-1}^* \Big(\cdots G_2^* (G_1^* T_{k-1} G_1) G_2 \cdots \Big) G_{m-1}, \tag{14.13}$$

completing each similarity $G_j^*(\cdot)G_j$ before moving on to the next rotation?

The difficulty is that the rotations are not known in advance. G_2 is defined by the requirement that G_2^* zeros out the $(3,2)$ entry of $G_1^* T_{k-1}$, so computing it needs the matrix $G_1^* T_{k-1}$. If we apply G_1 on the right immediately, we only have $G_1^* T_{k-1} G_1$ at hand, whose second column has been mixed with the first. *How can we compute G_2 from $G_1^* T_{k-1} G_1$?*

To answer this, look at what happens when we compute $G_1^* T_{k-1} G_1$. Left multiplication by G_1^* only combines rows 1 and 2 and keeps the Hessenberg form (it zeros out the $(2,1)$ entry). Right multiplication by G_1 then combines columns 1 and 2. Column 2 has a nonzero in row 3, so this creates a nonzero at position $(3,1)$, one below the subdiagonal:

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{bmatrix}}_{T_{k-1}} \xrightarrow{G_1^* \cdot G_1} \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ + & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{bmatrix}. \tag{14.14}$$

This entry, marked $+$, is called the *bulge*. The idea is to *choose G_2 such that it removes the bulge*: G_2^* acts on rows 2 and 3 and zeros out the $(3,1)$ entry. The subsequent right multiplication by G_2 combines columns 2

and 3 and creates a new bulge at $(4, 2)$, so the bulge moves one step further down the subdiagonal:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ + & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_2^* \cdot G_2} \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & + & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_3^* \cdot G_3} \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & + & \times & \times \end{bmatrix} \xrightarrow{G_4^* \cdot G_4} \text{Hessenberg.} \tag{14.15}$$

The last rotation chases the bulge off the bottom of the matrix, and we are left with an upper Hessenberg matrix. Every rotation is applied from both sides as soon as it is known, so the step is a single sweep through the matrix. This is *bulge chasing*. We still need to show that this leads to the same T_k as the two-sweep algorithm (up to signs). This follows from the implicit Q theorem, stated in Section 14.6 and proved at the start of the next lecture (Section 15.2).

Algorithm 22 One step of the QR iteration on an upper Hessenberg matrix by bulge chasing.

Input: upper Hessenberg $T \in \mathbb{K}^{m \times m}$, overwritten by the next iterate

choose G_1 acting on rows $1, 2$ such that $(G_1^* T)_{21} = 0$
$T \leftarrow G_1^* T G_1$ ▷ creates the bulge at $(3, 1)$
for $j = 2, \ldots, m-1$ **do**
    choose G_j acting on rows $j, j+1$ such that $(G_j^* T)_{j+1,j-1} = 0$ ▷ remove the bulge
    $T \leftarrow G_j^* T G_j$ ▷ bulge moves to $(j+2, j)$, or leaves the matrix if $j = m-1$

Each step changes only two rows and two columns, so one QR step still costs $O(m^2)$, now in a single sweep through the matrix.

14.6 The Implicit Q Theorem

We close with the statement of the theorem that justifies bulge chasing. Its proof is given in Section 15.2.

Definition 93 (Unreduced Hessenberg Matrix)

An upper Hessenberg matrix H is *unreduced* if all its subdiagonal entries are nonzero, $h_{j+1,j} \neq 0$ for $j = 1, \ldots, m-1$. Equivalently, H is not block upper triangular.

If some $h_{j+1,j} = 0$, then H is block upper triangular and its eigenvalue problem splits into those of the two diagonal blocks, so we may always assume H to be unreduced.

Theorem 94 (Implicit Q Theorem)

Assume that Q is orthogonal and $Q^\top A Q = H$ is unreduced upper Hessenberg. Then columns $2, \ldots, m$ of Q are uniquely determined by the first column of Q, up to signs.

The name refers to how the theorem is used. To know the whole transformation Q, it suffices to know its first column and that the result is upper Hessenberg; Q never has to be computed explicitly. For bulge chasing, both the two-sweep algorithm and the one-sweep algorithm produce an upper Hessenberg $T_k = U_k^* T_{k-1} U_k$, and in both, $U_k \vec{e}_1 = G_1 \vec{e}_1$ with the same G_1. The theorem then says that the two U_k, and hence the two T_k, agree up to signs (Section 15.3).

14.7 (SUPPLEMENT) Further Topics

14.7.1 The Hermitian Case: Tridiagonal Form

If A is Hermitian, so is $H = Q^* A Q$, and a Hermitian Hessenberg matrix is tridiagonal. Exploiting the zeros and the symmetry reduces the cost from $\frac{10}{3}m^3$ to $\sim \frac{4}{3}m^3$ flops, and each subsequent QR step on a tridiagonal matrix costs only $O(m)$ [1, Lecture 26], [2, Section 4.4.7].

14.7.2 Stability

Like Householder QR, Algorithm 21 is backward stable: the computed $\tilde{H}$ and the exactly unitary $\tilde{Q}$ defined by the computed reflection vectors satisfy $\tilde{Q}\tilde{H}\tilde{Q}^* = A + \delta A$ with $\|\delta A\| / \|A\| = O(\varepsilon)$ [1, Theorem 26.1]. So the eigenvalues of $\tilde{H}$ are the exact eigenvalues of a nearby matrix.

14.7.3 The "Bad Idea" Revisited

Applying a full Householder reflector from both sides, as in the first step of Householder QR, destroys the zeros it creates. It does, however, typically make the entries below the diagonal smaller; this is exactly what one step of the QR iteration does, which is the sense in which the QR algorithm "triangularizes in the limit" [1, Lecture 26].

14.7.4 In Practice

LAPACK computes the Hessenberg reduction with blocked Householder updates (`xGEHRD`) and the QR iteration with multishift bulge chasing and aggressive early deflation (`xHSEQR`). In Julia, `hessenberg(A)` returns the reduction, and `eigvals(A)` runs the whole pipeline.

Lecture 15

Shifted QR

Supplementary reading:

- [2] Section 4.4.8 (Theorem 4.9, the implicit Q theorem).
- [2] Sections 4.4.4 and 4.4.5 (QR iteration with a shift, making it practical).
- [2] Section 5.3.1 (Wilkinson's shift).
- [1] Lecture 33 (Arnoldi iteration).

15.1 Recap: QR Iteration on Hessenberg Matrices

We begin with a recap of Lecture 14. We first reduce A once to upper Hessenberg form $H = Q^\top A Q$ (Section 14.3). Each step of the QR iteration on $T_0 = H$ then consists of a QR factorization by Givens rotations followed by the reverse product (Section 14.4),

$$R_k = G_{m-1}^\top \cdots G_1^\top T_{k-1}, \qquad T_k = R_k G_1 \cdots G_{m-1}, \tag{15.1}$$

and costs $O(m^2)$ because T_k is again upper Hessenberg.

The board compares two orders of applying the same rotations. In the *two-sweep* order all left multiplications are done before any right multiplication,

$$G_1^\top T_{k-1}, \quad G_2^\top G_1^\top T_{k-1}, \quad \ldots, \quad R_k, \quad R_k G_1, \quad R_k G_1 G_2, \quad \ldots, \quad T_k, \tag{15.2}$$

while *bulge chasing* (Section 14.5) applies every rotation from both sides as soon as it is known,

$$G_1^\top T_{k-1} G_1, \quad G_2^\top G_1^\top T_{k-1} G_1 G_2, \quad \ldots, \quad T_k. \tag{15.3}$$

By associativity both orders compute the same product *if they use the same rotations*. But in bulge chasing, $G_2, \ldots, G_{m-1}$ are chosen to remove the bulge that $G_1^\top T_{k-1} G_1$ creates at $(3,1)$, not to remove the subdiagonal entries of $G_1^\top T_{k-1}$. Why does this give the same T_k? The answer is the implicit Q theorem, stated at the end of Lecture 14; this lecture proves it, then turns to shifts.

15.2 Proof of the Implicit Q Theorem

Recall the statement (Theorem 94): if Q is orthogonal and $Q^\top AQ = H$ is unreduced upper Hessenberg, then columns $2, \dots, m$ of Q are uniquely determined by the first column of Q, up to signs.

Proof. We follow [2, Theorem 4.9]. Let both $Q^\top AQ = H$ and $V^\top AV = G$ be as in the theorem, with equal first columns $\vec{q}_1 = \vec{v}_1$. We want to show that

$$W \coloneqq V^\top Q = \operatorname{diag}(\pm 1, \dots, \pm 1), \tag{15.4}$$

since then $Q = VW$, i.e., $\vec{q}_i = \pm \vec{v}_i$ for every i.

W is orthogonal as a product of orthogonal matrices, and its first column is $W\vec{e}_1 = V^\top \vec{q}_1 = V^\top \vec{v}_1 = \vec{e}_1$. Moreover, using $GV^\top = V^\top AVV^\top = V^\top A$ and $AQ = QH$,

$$GW = GV^\top Q = V^\top AQ = V^\top QH = WH. \tag{15.5}$$

Write $\vec{w}_i$ for the ith column of W. Taking column i of both sides, and using that column i of the upper Hessenberg matrix H is zero below row $i+1$,

$$G\vec{w}_i = (GW)_{:,i} = (WH)_{:,i} = \sum_{j=1}^{i+1} \vec{w}_j H_{ji}. \tag{15.6}$$

The term $j = i+1$ is the only one that contains $\vec{w}_{i+1}$. Moving the others to the right-hand side gives

$$H_{i+1,i}\, \vec{w}_{i+1} = G\vec{w}_i - \sum_{j=1}^{i} H_{ji}\vec{w}_j. \tag{15.7}$$

We show by induction that each $\vec{w}_i$ is nonzero in entries $1, \dots, i$ only:

$$\vec{w}_i = \begin{bmatrix} \times \\ \vdots \\ \times \\ 0 \\ \vdots \\ 0 \end{bmatrix} \leftarrow i \tag{15.8}$$

This holds for $\vec{w}_1 = \vec{e}_1$. Suppose it holds for $\vec{w}_1, \dots, \vec{w}_i$. Then $G\vec{w}_i$ is a combination of the first i columns of G, which are zero below row $i+1$ because G is upper Hessenberg. The sum in (15.7) is zero below row i by the induction hypothesis. So the right-hand side of (15.7) is zero below row $i+1$. Since H is unreduced, $H_{i+1,i} \neq 0$, and dividing by it shows that $\vec{w}_{i+1}$ is zero below row $i+1$.

Hence W is upper triangular. Since W is also orthogonal, $W^{-1} = W^\top$ is both upper triangular (as the inverse of an upper triangular matrix) and lower triangular (as the transpose of one), so W is diagonal. An orthogonal diagonal matrix has $w_{ii}^2 = 1$, so $W = \operatorname{diag}(\pm 1, \dots, \pm 1)$. □

The proof has the same structure as the proof that the QR factorization is unique (Theorem 33). We take two solutions and look at the matrix W that converts one into the other. Showing that W is triangular and orthogonal forces it to be diagonal with entries of modulus one, and these are exactly the sign choices that cannot be fixed.

The recursion (15.7) is also a constructive version of the theorem. Given $\vec{q}_1$, the relation $AQ = QH$ determines $h_{1i}, \dots, h_{ii}$ by orthogonality and then $\vec{q}_{i+1}$ by normalization, one column at a time. This is the Arnoldi iteration [1, Lecture 33], and the step that divides by $h_{i+1,i}$ is where unreducedness is needed.

For $\mathbb{K} = \mathbb{C}$ the same proof works with Q and V unitary and $\top$ replaced by $*$. Then W is a unitary diagonal matrix, so each column of Q is determined up to a factor of modulus one.

15.3 Application to Bulge Chasing

Both the two-sweep algorithm and bulge chasing compute $T_k = U_k^\top T_{k-1} U_k$ with U_k a product of rotations $G_1 G_2 \cdots G_{m-1}$, where G_j acts on rows j and $j+1$. Only G_1 touches the first row, so $U_k \vec{e}_1 = G_1 \vec{e}_1$ in both cases. Moreover, G_1 is the same in both, since it is determined by the first column of T_{k-1} alone. Both results are upper Hessenberg. So if T_k is unreduced (otherwise we deflate), Theorem 94 shows that the two U_k agree up to the signs of their columns. The two T_k then agree up to a similarity $D^\top T_k D$ with $D = \operatorname{diag}(\pm 1, \dots, \pm 1)$, which only flips the signs of some rows and columns. As in Section 13.5, such a similarity leaves the diagonal and the moduli of all entries unchanged.

15.4 The Shifted QR Iteration

The QR iteration of Algorithm 20 can converge slowly, at the rate $|\lambda_{p+1}/\lambda_p|$ (Section 13.3). As for inverse iteration (Algorithm 16), the remedy is a shift. We compare the two iterations side by side:

$$\text{normal QR:} \quad \begin{aligned} T_{k-1} &= U_k R_k, \\ T_k &= R_k U_k, \end{aligned} \qquad \text{shifted QR:} \quad \begin{aligned} T_{k-1} - \mu_k I &= U_k R_k, \\ T_k &= R_k U_k + \mu_k I. \end{aligned} \tag{15.9}$$

The shift μ_k is subtracted before the factorization and added back afterwards. We check the two properties that made the unshifted iteration work.

The upper Hessenberg form is preserved. Subtracting and adding $\mu_k I$ changes only the diagonal. So $T_{k-1} - \mu_k I$ is upper Hessenberg, its QR factorization takes $m-1$ Givens rotations, and $R_k U_k$ is upper Hessenberg by Section 14.4. Each step still costs $O(m^2)$.

The eigenvalues are preserved. From the first equation, $R_k = U_k^*(T_{k-1} - \mu_k I)$, so

$$T_k = R_k U_k + \mu_k I = U_k^*(T_{k-1} - \mu_k I) U_k + \mu_k I = U_k^* T_{k-1} U_k, \tag{15.10}$$

since $U_k^*(\mu_k I) U_k = \mu_k I$. So T_k is again a unitary similarity transformation of T_{k-1}. Without adding back $\mu_k I$, the eigenvalues would be shifted by $-\mu_k$ in every step.

Tracking the eigenvectors. Since $T_k = U_k^* T_{k-1} U_k$ has exactly the same form as in the unshifted iteration, we can track Q_k as before (Section 13.5): accumulate $Q \leftarrow Q U_k$ after each step, so that $T_k = (U_1 \cdots U_k)^* H (U_1 \cdots U_k)$. Combined with the Hessenberg reduction, $A \approx (Q U_1 \cdots U_k)\, T_k\, (Q U_1 \cdots U_k)^*$ once T_k is nearly triangular.

15.5 Choosing the Shift

How does the shift affect convergence, and which μ_k should we choose? We first consider the ideal case.

Exact shift. Assume μ_k is an eigenvalue of T_{k-1}. Then $T_{k-1} - \mu_k I$ is singular. Assume moreover that all off-diagonal entries $t_{j+1,j}$ are nonzero, i.e., T_{k-1} is unreduced (otherwise we split the problem, see below). Then the first $m-1$ columns of $T_{k-1} - \mu_k I$ are linearly independent: rows $2, \dots, m$ of these columns form an upper triangular $(m-1) \times (m-1)$ matrix whose diagonal entries are the nonzero subdiagonal entries of T_{k-1}. In the QR factorization $T_{k-1} - \mu_k I = U_k R_k$, column j is a combination of $\vec{u}_1, \dots, \vec{u}_j$, with the coefficient r_{jj} on $\vec{u}_j$. If $r_{jj} = 0$ for some $j \leq m-1$, column j would lie in the span of the previous columns. Hence $r_{11}, \dots, r_{m-1,m-1} \neq 0$, and since R_k is singular, *the last entry must be zero*, $r_{mm} = 0$. As R_k is upper triangular, its whole last row vanishes:

$$\underbrace{T_{k-1} - \mu_k I}_{\text{unreduced Hessenberg}} = U_k \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 \end{bmatrix}}_{R_k}. \tag{15.11}$$

Now what happens to T_k? The last row of $R_k U_k$ is the last row of R_k times U_k, which is zero. Adding $\mu_k I$ gives

$$T_k = R_k U_k + \mu_k I = \begin{bmatrix} & \hat{T} & & \vec{t} \\ 0 & \cdots & 0 & \mu_k \end{bmatrix}, \tag{15.12}$$

with $\hat{T} \in \mathbb{K}^{(m-1)\times(m-1)}$ upper Hessenberg. So $t_{m,m-1} = 0$ and $t_{mm} = \mu_k$: with an exact shift, *one step* of the QR iteration splits off the eigenvalue μ_k. In the words of the board, we don't have to worry about the last entry any more.

We then continue on $\hat{T}$ alone. If a unitary Q acts on $\hat{T}$, then on the full matrix it acts as $\operatorname{diag}(Q, 1)$,

$$\begin{bmatrix} Q^* & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \hat{T} & \vec{t} \\ 0 & \mu_k \end{bmatrix} \begin{bmatrix} Q & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} Q^* \hat{T} Q & Q^* \vec{t} \\ 0 & \mu_k \end{bmatrix}, \tag{15.13}$$

so the eigenvalue already found is left untouched, and the problem size has decreased by one. The last column $Q^*\vec{t}$ only needs to be updated if we want the full Schur form, not just the eigenvalues.

Splitting. In practice μ_k is only close to an eigenvalue, and the subdiagonal entries become small rather than zero. Whenever a subdiagonal entry approaches 0, we set it to zero; T_k is then (approximately) block upper triangular,

$$T_k \approx \begin{bmatrix} (T_k)_{11} & (T_k)_{12} \\ 0 & (T_k)_{22} \end{bmatrix}, \tag{15.14}$$

and its eigenvalues are those of the two diagonal blocks, which are again upper Hessenberg. We recurse on $(T_k)_{11}$ and $(T_k)_{22}$ separately.

Algorithm 23 Practical QR iteration.

Input: $A \in \mathbb{K}^{m\times m}$

compute $H = Q^* A Q$ ▷ Algorithm 21, once
$T_0 \leftarrow H$, $k \leftarrow 0$
while not converged **do**
    $k \leftarrow k+1$, choose μ_k
    $T_{k-1} - \mu_k I = U_k R_k$ ▷ Givens QR, $O(m^2)$
    $T_k \leftarrow R_k U_k + \mu_k I$
    if some subdiagonal entry of T_k is ≈ 0 **then**
        split T_k into $(T_k)_{11}$ and $(T_k)_{22}$ and recurse on both

Which eigenvalue to target? The goal is to choose $\mu_k \approx$ an eigenvalue and to split up the problem wherever a subdiagonal entry is ≈ 0. But which eigenvalue? The answer on the board is $\mu_k \approx \lambda_m$, for two reasons.

1. The shift helps the eigenvalue λ_i to separate if $|\lambda_i - \mu| \ll |\lambda_j - \mu|$ for all $j \neq i$. Without a shift ($\mu = 0$) this is easiest if λ_i itself is small, i.e., for the eigenvalue of smallest modulus.
2. The QR iteration orders the eigenvalues by size along the diagonal of T_k (Theorem 90), with the smallest one, λ_m, at the bottom right. A shift close to a different eigenvalue would have to move that eigenvalue to the bottom, and *reordering the eigenvalues slows down convergence*.

So the shift should approximate the eigenvalue that is already converging in the bottom right corner, where we then look for $t_{m,m-1} \approx 0$.

15.6 Relationship with (Inverse) Power Iteration

We have seen that the QR iteration is equivalent to orthogonal iteration with $Q_0 = I$ (Section 13.5). The shifted QR iteration is likewise equivalent to a *shifted* orthogonal iteration. With $T_0 = H$, the first two steps read $H - \mu_1 I = U_1 R_1$, $T_1 = R_1 U_1 + \mu_1 I$, and $T_1 - \mu_2 I = U_2 R_2$. Hence

$$\begin{aligned} U_1 U_2 R_2 R_1 &= U_1 (T_1 - \mu_2 I) R_1 = U_1 (R_1 U_1 + \mu_1 I - \mu_2 I) R_1 \\ &= (H - \mu_1 I)((H - \mu_1 I) + \mu_1 I - \mu_2 I) \\ &= (H - \mu_1 I)(H - \mu_2 I), \end{aligned} \tag{15.15}$$

where the second line uses $U_1 R_1 = H - \mu_1 I$ twice: $U_1 R_1 U_1 R_1 = (H - \mu_1 I)^2$ and $U_1 (\mu_1 - \mu_2) R_1 = (\mu_1 - \mu_2)(H - \mu_1 I)$.

In the same way we can write, with $\bar{Q}_k \coloneqq U_1 \cdots U_k$ and $\bar{R}_k \coloneqq R_k \cdots R_1$,

$$(H - \mu_k I) \cdots (H - \mu_1 I) = \bar{Q}_k \bar{R}_k. \tag{15.16}$$

For the induction step, note that $T_{k-1} = \bar{Q}_{k-1}^* H \bar{Q}_{k-1}$ by the similarity relation of Section 15.4, so

$$\bar{Q}_k \bar{R}_k = \bar{Q}_{k-1} (T_{k-1} - \mu_k I) \bar{R}_{k-1} = (H - \mu_k I) \bar{Q}_{k-1} \bar{R}_{k-1}. \tag{15.17}$$

$\bar{Q}_k$ is unitary and $\bar{R}_k$ is upper triangular as a product of upper triangular matrices, so (15.16) is a QR factorization of the matrix polynomial $p_k(H) = (H - \mu_k I) \cdots (H - \mu_1 I)$. The factors commute, so their order does not matter.

Compared with the unshifted case, where $\bar{Q}_k$ is the Q factor of H^k, the power H^k is replaced by $p_k(H)$, which multiplies the eigenvector component for λ_j by $p_k(\lambda_j) = \prod_{i=1}^k (\lambda_j - \mu_i)$ instead of λ_j^k. If the shifts are close to an eigenvalue, p_k is tiny there, which is what accelerates convergence. Moreover, $T_k = \bar{Q}_k^* H \bar{Q}_k$ shows how to track the eigenvectors: $\bar{Q}_k$ accumulates all the transformations, and once T_k has converged to upper triangular form, $Q\bar{Q}_k$ contains the Schur vectors of A.

What about inverse iteration? Inverting (15.16) gives $p_k(H)^{-1} = \bar{R}_k^{-1} \bar{Q}_k^*$, and taking the adjoint,

$$\bar{Q}_k \bar{R}_k^{-*} = (H - \mu_k I)^{-*} \cdots (H - \mu_1 I)^{-*}, \tag{15.18}$$

where $X^{-*} \coloneqq (X^{-1})^*$. Inversion and the adjoint both reverse the order of a product, but the factors on the right commute, so the order does not matter. The right-hand side is exactly the operator of shifted inverse iteration (Algorithm 16), applied to H^* with shifts $\bar{\mu}_i$ (H^* has the eigenvalues $\bar{\lambda}_j$).

The difference to (15.16) is the triangular factor: $\bar{R}_k^{-1}$ is upper triangular, so $\bar{R}_k^{-*}$ is *lower* triangular. For an upper triangular factor, the first columns of the product only involve the first columns of $\bar{Q}_k$; for a lower triangular factor, the last columns only involve the last columns of $\bar{Q}_k$. In particular, the last column of $\bar{R}_k^{-*}$ is $(\bar{R}_k^{-*})_{mm}\vec{e}_m$ with $(\bar{R}_k^{-*})_{mm} = 1/\overline{(\bar{R}_k)_{mm}}$, so multiplying (15.18) by $\vec{e}_m$ gives

$$\bar{\vec{q}}_m = \overline{(\bar{R}_k)_{mm}}\,(H-\mu_k I)^{-*}\cdots(H-\mu_1 I)^{-*}\vec{e}_m, \tag{15.19}$$

where $\bar{\vec{q}}_m$ is the last column of $\bar{Q}_k$. So the same matrix $\bar{Q}_k$ carries out two iterations at once:

factorization	triangular factor	columns of $\bar{Q}_k$
$p_k(H) = \bar{Q}_k\bar{R}_k$	upper	first columns: orthogonal (power) iteration
$p_k(H)^{-*} = \bar{Q}_k\bar{R}_k^{-*}$	lower	last columns: shifted inverse iteration

The last column is shifted inverse iteration started from $\vec{e}_m$; for a single step this is the argument of [2, Section 4.4.4]. In short, *the QR iteration is both power and inverse power iteration*. This explains why the bottom right corner of T_k converges first, and why the shift should target λ_m (Section 15.5): a shift close to an eigenvalue makes inverse iteration converge fast (Theorem 63).

15.7 (SUPPLEMENT) Further Topics

15.7.1 The Rayleigh Quotient Shift

The board leaves the concrete choice of $\mu_k \approx \lambda_m$ open. The natural estimate is the bottom right entry, $\mu_k = t_{mm}^{(k-1)} = \vec{e}_m^* T_{k-1}\vec{e}_m$, which is the Rayleigh quotient of T_{k-1} at $\vec{e}_m$. This connects to Rayleigh quotient iteration (Algorithm 17): by the inverse iteration interpretation above, the last column of $\bar{Q}_k$ is inverse iteration with the shift taken from the current Rayleigh quotient. Demmel [2, Section 4.4.4] explains the resulting local *quadratic* convergence as follows. If $\eta = \|T_{k-1}(m, 1{:}m-1)\| \,/\, \|A\| \ll 1$, then setting the last row's off-diagonal part to zero perturbs an eigenvalue by $O(\eta\,\|A\|)$ (if it is well separated), so $|\lambda_m - t_{mm}| = O(\eta\,\|A\|)$. The next step shrinks the last row by the factor $|\lambda_m - \mu| \,/\, \min_{j\neq m}|\lambda_j - \mu| = O(\eta)$, giving $O(\eta^2)$.

15.7.2 The Wilkinson Shift

The Rayleigh quotient shift can stall. For $T_0 = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ we get $\mu_1 = t_{22} = 0$; T_0 is itself orthogonal, so its QR factorization is $U_1 = T_0$, $R_1 = I$, and $T_1 = R_1U_1 = T_0$. Nothing changes, because the eigenvalues ± 1 are equally far from the shift. The *Wilkinson shift* instead takes μ_k to be the eigenvalue of the trailing 2×2 block of T_{k-1} that is closer to t_{mm}; in the example this is ± 1, an exact eigenvalue, and one step suffices. For symmetric tridiagonal matrices, QR iteration with the Wilkinson shift is globally and at least linearly convergent, and asymptotically cubically convergent for almost all matrices [2, Theorem 5.8].

15.7.3 When Is a Subdiagonal Entry ≈ 0?

Since T_k is obtained from A by unitary similarity transformations in floating point, it already carries rounding errors of size $O(\varepsilon_{\text{mach}})\,\|A\|$. A subdiagonal entry below this size may as well be zero, and setting it to zero is a backward stable perturbation [2, Section 4.4.5]. The iteration ends when all diagonal blocks are 1×1 (or 2×2 for complex conjugate pairs of a real matrix).

15.7.4 Why the Implicit Q Theorem Matters for Shifts

With a shift μ, one QR step factors $T_{k-1} - \mu I = U_kR_k$ and sets $T_k = R_kU_k + \mu I = U_k^*T_{k-1}U_k$. By Theorem 94, U_k is determined by its first column, which is parallel to the first column of $T_{k-1} - \mu I$, i.e.,

to $(t_{11} - \mu, t_{21}, 0, \ldots, 0)^\top$. So G_1 is computed from these two numbers, and the rest of the step is the bulge chasing of Algorithm 22 on T_{k-1} itself; $T_{k-1} - \mu I$ is never formed. This is the *implicitly shifted* QR algorithm [2, Section 4.4.8]. For real matrices with complex conjugate eigenvalues, two such steps with shifts μ and $\bar{\mu}$ can be combined into a *double-shift* step that stays in real arithmetic (the Francis step) [2, Section 4.4.8].

15.7.5 The Krylov View

Why should the first column determine everything? From $AQ = QH$, column i gives $h_{i+1,i}\vec{q}_{i+1} = A\vec{q}_i - \sum_{j \le i} h_{ji}\vec{q}_j$. By induction, using $h_{i+1,i} \neq 0$,

$$\operatorname{span}(\vec{q}_1, \ldots, \vec{q}_k) = \operatorname{span}(\vec{q}_1, A\vec{q}_1, \ldots, A^{k-1}\vec{q}_1) \qquad \text{for } k = 1, \ldots, m. \tag{15.20}$$

The right-hand side is the Krylov subspace generated by A and $\vec{q}_1$, and depends on nothing else. Equivalently, the Krylov matrix $K = [\vec{q}_1, A\vec{q}_1, \ldots, A^{m-1}\vec{q}_1]$ satisfies $K = QR$ with $R = Q^\top K$ upper triangular and invertible. So Q is the orthogonal factor of the QR factorization of K, and the implicit Q theorem follows from the uniqueness of QR up to signs [1, Lecture 33]. The proof above is the same argument written out column by column.

15.7.6 A Direct Check for the First Bulge

For the first rotation after G_1 one can also see directly that the two algorithms agree. Write $G_1 = \left[\begin{smallmatrix} c & -s \\ s & c \end{smallmatrix}\right]$ in rows and columns $1, 2$, and let $X = G_1^\top T_{k-1}$. The first column of X is zero in rows $2, \ldots, m$, and the two-sweep algorithm chooses G_2 from (x_{22}, x_{32}). Right multiplication by G_1 replaces column 1 by $c\,(\text{column } 1) + s\,(\text{column } 2)$, so in XG_1 the bulge column has entries $s\,(x_{22}, x_{32})$ in rows $2, 3$. This vector is parallel to (x_{22}, x_{32}), so bulge chasing chooses the same G_2 up to sign (if $s = 0$, then $(T_{k-1})_{21} = 0$ and T_{k-1} was reduced). Continuing this argument down the matrix is possible but tedious; the implicit Q theorem handles all rotations at once.

Appendix A

Linear Algebra Basics

This appendix collects the first-course linear algebra (Math 54 level) that is used without comment throughout the course: matrix products, transposes, inverses, determinants, subspaces, eigenvalues, orthogonality, and symmetric matrices. Proofs are omitted; results proved in the main text are cross-referenced. Throughout, A and B are matrices of compatible sizes, c is a scalar, and $\vec{x}, \vec{y}$ are column vectors. Supplementary reading:

- [3] Chapters 2–7.
- [1] Lectures 1 and 2.
- [4] Section 7A.

A.1 Matrix Multiplication

Write $\vec{a}_j$ for the columns of $A \in \mathbb{K}^{m\times n}$ and $\vec{b}_i^{\top}$ for the rows of $B \in \mathbb{K}^{n\times p}$. The product AB can be read in four equivalent ways:

- *entries*: $(AB)_{ij} = \sum_k A_{ik}B_{kj}$;
- *columns*: $A\vec{x} = \sum_j x_j \vec{a}_j$ is a linear combination of the columns of A, and the j-th column of AB is A times the j-th column of B;
- *rows*: the i-th row of AB is the i-th row of A times B;
- *outer products*: $AB = \sum_{k=1}^{n} \vec{a}_k \vec{b}_k^{\top}$, a sum of rank-one matrices.

Multiplication is associative and distributive but not commutative. Multiplying by a diagonal matrix D on the left scales the rows (DA), on the right the columns (AD). Block matrices multiply like ordinary matrices as long as the block sizes are compatible.

A.2 Transpose and Conjugate Transpose

Definition 95 (Transpose and Conjugate Transpose)

For $A \in \mathbb{K}^{m\times n}$, the *transpose* $A^\top \in \mathbb{K}^{n\times m}$ and the *conjugate transpose* (or *adjoint*) $A^* \in \mathbb{K}^{n\times m}$ are

$$(A^\top)_{ij} = A_{ji}, \qquad (A^*)_{ij} = \overline{A_{ji}}. \tag{A.1}$$

For real matrices the two coincide.

Proposition 96 (Rules for the Transpose)

1. $(A^\top)^\top = A$.
2. $(A+B)^\top = A^\top + B^\top$ and $(cA)^\top = cA^\top$.
3. $(AB)^\top = B^\top A^\top$, and more generally $(A_1 A_2 \cdots A_k)^\top = A_k^\top \cdots A_2^\top A_1^\top$.
4. If A is invertible, so is $A^\top$, and $(A^\top)^{-1} = (A^{-1})^\top$, written $A^{-\top}$.
5. $\det A^\top = \det A$, $\operatorname{tr} A^\top = \operatorname{tr} A$, and $\operatorname{rank} A^\top = \operatorname{rank} A$.
6. $A^\top$ and A have the same characteristic polynomial, hence the same eigenvalues, but in general different eigenvectors.

The order reversal in (3) is the same as in $(AB)^{-1} = B^{-1}A^{-1}$: the factor applied last is undone first. Rule (4) follows from (3) by transposing $AA^{-1} = I$.

A.2.1 Vectors and Inner Products

Since a column vector is an $n \times 1$ matrix, the rules above apply to it directly.

- $(A\vec{x})^\top = \vec{x}^\top A^\top$.
- $\vec{x}^\top \vec{y} = \vec{y}^\top \vec{x}$, because $\vec{x}^\top \vec{y}$ is a 1×1 matrix, and a scalar is its own transpose.
- $(A\vec{x})^\top \vec{y} = \vec{x}^\top (A^\top \vec{y})$, i.e., $\langle A\vec{x}, \vec{y}\rangle = \left\langle \vec{x}, A^\top \vec{y}\right\rangle$. This *adjoint relation* moves a matrix from one side of an inner product to the other. It is the key step in deriving the normal equations (Lemma 34).

A.2.2 Block Matrices

A block matrix is transposed by transposing each block *and* the arrangement of the blocks:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^\top = \begin{bmatrix} A^\top & C^\top \\ B^\top & D^\top \end{bmatrix}. \tag{A.2}$$

The off-diagonal blocks B and C swap places.

A.2.3 The Complex Case

For complex matrices the conjugate transpose $A^* = \overline{A}^\top$ plays the role of the transpose. All rules of Proposition 96 carry over, with complex conjugates wherever a scalar appears:

- $(cA)^* = \bar{c}A^*$ and $(AB)^* = B^*A^*$;
- $\det A^* = \overline{\det A}$, and the eigenvalues of A^* are the complex conjugates of those of A;
- $\vec{x}^*\vec{y} = \overline{\vec{y}^*\vec{x}}$, which in general is *not* equal to $\vec{y}^*\vec{x}$;
- $\langle A\vec{x}, \vec{y}\rangle = \langle \vec{x}, A^*\vec{y}\rangle$ for the standard inner product $\langle \vec{x}, \vec{y}\rangle = \vec{x}^*\vec{y}$ (Lecture 11).

Hermitian versus symmetric. A matrix is *Hermitian* if $A^* = A$ and *symmetric* if $A^\top = A$. For real matrices $A^* = A^\top$, so the two notions coincide, and every result about Hermitian matrices applies to real symmetric ones. For complex matrices they differ: a complex symmetric matrix is in general not Hermitian. For example,

$$A = \begin{bmatrix} 1 & i \\ i & 1 \end{bmatrix} \tag{A.3}$$

satisfies $A^\top = A$, but $A^* = \left[\begin{smallmatrix} 1 & -i \\ -i & 1 \end{smallmatrix}\right] \neq A$, and its eigenvalues $1 \pm i$ are not real. Complex symmetric matrices therefore enjoy none of the special properties of Hermitian matrices (real eigenvalues, unitary diagonalization, the Rayleigh quotient characterization). In these notes $\mathbb{K}$ is $\mathbb{R}$ or $\mathbb{C}$, and "Hermitian" means "symmetric" when $\mathbb{K} = \mathbb{R}$.

A.2.4 Useful Consequences

- A^*A and AA^* are Hermitian and positive semidefinite, since $\vec{x}^*A^*A\vec{x} = \|A\vec{x}\|_2^2 \geq 0$.
- Q is unitary if and only if $Q^*Q = I$, i.e., $Q^* = Q^{-1}$ (Section B.2).
- A^* and A have the same singular values, so $\|A^*\|_2 = \|A\|_2$ and $\kappa_2(A^*) = \kappa_2(A)$.
- $\|A^*\|_1 = \|A\|_\infty$, since transposing turns column sums into row sums (Proposition 104).

Remark 97 (Common Mistakes)**.**

- Writing $(AB)^\top = A^\top B^\top$. The order must be reversed.
- Assuming that the product of two symmetric matrices is symmetric. In fact $(AB)^\top = B^\top A^\top = BA$, which in general differs from AB.
- Using $^\top$ where * is meant for complex matrices, e.g., writing $A^\top = A$ for "Hermitian".

A.3 Inverses, Elementary and Triangular Matrices

A square matrix A is invertible if $AB = BA = I$ for some $B = A^{-1}$; for square matrices a one-sided inverse is automatically two-sided. Then $(AB)^{-1} = B^{-1}A^{-1}$ and $(A^{-1})^{-1} = A$.

Theorem 98 (Invertible Matrix Theorem)

For $A \in \mathbb{K}^{n\times n}$ the following are equivalent:

1. A is invertible;
2. $A\vec{x} = \vec{b}$ has a unique solution for every $\vec{b}$;
3. $A\vec{x} = \vec{0}$ has only the solution $\vec{x} = \vec{0}$;
4. the columns (equivalently, the rows) of A are linearly independent;
5. $\operatorname{rank} A = n$;
6. $\det A \neq 0$;
7. 0 is not an eigenvalue of A;
8. $\sigma_{\min}(A) > 0$.

Elementary matrices. Each elementary row operation (swap two rows, scale a row, add a multiple of one row to another) is left multiplication by the matrix obtained by applying that operation to I; the corresponding column operation is right multiplication. Gaussian elimination is therefore a sequence of left multiplications, which is the starting point of LU factorization (Lecture 3). A *permutation matrix* P, a row-permuted identity, satisfies $P^{-1} = P^{\top}$.

Triangular matrices. Products and inverses of lower (upper) triangular matrices are lower (upper) triangular, and if the diagonal entries are all 1 they stay 1. A triangular matrix is invertible if and only if all diagonal entries are nonzero, and its eigenvalues are its diagonal entries.

A.4 Determinants

- For triangular A, $\det A = \prod_i a_{ii}$.
- Row operations: swapping two rows flips the sign, scaling a row by c scales $\det A$ by c, and adding a multiple of one row to another leaves $\det A$ unchanged. Hence if $PA = LU$ with unit lower triangular L, then $\det A = \pm \prod_i u_{ii}$, which is how determinants are computed in practice.
- $\det(AB) = \det A \det B$, $\det A^{-1} = 1/\det A$, $\det(cA) = c^n \det A$ for $A \in \mathbb{K}^{n\times n}$, and $|\det Q| = 1$ for unitary Q. In general $\det(A + B) \neq \det A + \det B$.
- For block triangular matrices, $\det \left[\begin{smallmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{smallmatrix}\right] = \det A_{11} \det A_{22}$.
- $\det A = \prod_i \lambda_i$ and $\operatorname{tr} A = \sum_i \lambda_i$, with eigenvalues counted with algebraic multiplicity.

A.5 Subspaces and Rank

For $A \in \mathbb{K}^{m\times n}$, the *range* (column space) $\mathcal{R}(A) = \{A\vec{x}\} \subseteq \mathbb{K}^m$ and the *null space* $\mathcal{N}(A) = \{\vec{x} : A\vec{x} = \vec{0}\} \subseteq \mathbb{K}^n$ are subspaces. A *basis* of a subspace is a linearly independent spanning set; all bases have the same size, the *dimension*.

- $\operatorname{rank} A = \dim \mathcal{R}(A)$, and column rank equals row rank, so $\operatorname{rank} A \le \min(m, n)$.
- *Rank–nullity*: $\operatorname{rank} A + \dim \mathcal{N}(A) = n$.
- $\mathcal{R}(A)^\perp = \mathcal{N}(A^*)$, so $\mathbb{K}^m = \mathcal{R}(A) \oplus \mathcal{N}(A^*)$ as an orthogonal direct sum.
- $\mathcal{N}(A^*A) = \mathcal{N}(A)$, hence $\operatorname{rank} A^*A = \operatorname{rank} A$, and A^*A is invertible if and only if A has linearly independent columns (Lecture 8).
- $\operatorname{rank}(AB) \le \min(\operatorname{rank} A, \operatorname{rank} B)$, and multiplying by an invertible matrix does not change the rank.

A.6 Eigenvalues and Diagonalization

λ is an *eigenvalue* of $A \in \mathbb{K}^{n\times n}$ with *eigenvector* $\vec{x} \neq \vec{0}$ if $A\vec{x} = \lambda\vec{x}$. The eigenvalues are the roots of the characteristic polynomial $\det(\lambda I - A)$, so there are exactly n of them in $\mathbb{C}$, counted with multiplicity.

- A real matrix may have complex eigenvalues; they come in conjugate pairs $\lambda, \bar{\lambda}$, with conjugate eigenvectors.
- If $A\vec{x} = \lambda\vec{x}$, then $A^k\vec{x} = \lambda^k\vec{x}$, $(A - \sigma I)\vec{x} = (\lambda - \sigma)\vec{x}$, and $A^{-1}\vec{x} = \lambda^{-1}\vec{x}$ for invertible A. These are the facts behind the power iteration, shifts, and inverse iteration (Lecture 10).
- *Similar* matrices $B = S^{-1}AS$ have the same characteristic polynomial, hence the same eigenvalues; eigenvectors transform as $\vec{x} \mapsto S^{-1}\vec{x}$. Row operations alone are *not* similarity transformations and change the eigenvalues, which is why eigenvalue algorithms transform $A \mapsto Q^*AQ$ from both sides.
- Eigenvectors belonging to distinct eigenvalues are linearly independent.
- The *algebraic multiplicity* of λ is its multiplicity as a root of the characteristic polynomial, and the *geometric multiplicity* is $\dim \mathcal{N}(A - \lambda I)$; always $1 \le$ geometric $\le$ algebraic.
- A is *diagonalizable*, $A = X\Lambda X^{-1}$, if and only if it has n linearly independent eigenvectors, if and only if the two multiplicities agree for every eigenvalue. Then $A^k = X\Lambda^k X^{-1}$. A matrix with n distinct eigenvalues is diagonalizable; a matrix that is not is called *defective* (Lecture 11).

A.7 Orthogonality and Projections

- A set of nonzero, mutually orthogonal vectors is linearly independent. In an orthonormal basis $\vec{q}_1, \dots, \vec{q}_n$ the coordinates are inner products: $\vec{y} = \sum_j (\vec{q}_j^*\vec{y})\vec{q}_j$.
- If $Q \in \mathbb{K}^{m\times n}$ has orthonormal columns, then $Q^*Q = I_n$ and $\|Q\vec{x}\|_2 = \|\vec{x}\|_2$. For $m > n$, however, $QQ^* \neq I_m$: it is the orthogonal projector onto $\mathcal{R}(Q)$.
- *Projection onto a line*: the projection of $\vec{y}$ onto $\operatorname{span}\{\vec{a}\}$ is $\frac{\vec{a}^*\vec{y}}{\vec{a}^*\vec{a}}\vec{a}$.
- *Orthogonal decomposition and best approximation*: for a subspace W, every $\vec{y}$ splits uniquely as $\vec{y} = \hat{\vec{y}} + \vec{z}$ with $\hat{\vec{y}} \in W$ and $\vec{z} \perp W$, and $\hat{\vec{y}}$ is the point of W closest to $\vec{y}$. If the columns of Q form an orthonormal basis of W, then $\hat{\vec{y}} = QQ^*\vec{y}$.
- *Gram–Schmidt* turns linearly independent columns into orthonormal ones and yields $A = QR$ (Lectures 5 and 7).

- *Least squares*: $\min_{\vec{x}} \left\| A\vec{x} - \vec{b} \right\|_2$ is attained exactly when $A^*A\vec{x} = A^*\vec{b}$; with $A = QR$ this becomes $R\vec{x} = Q^*\vec{b}$ (Lectures 5 and 8).

A.8 Hermitian Matrices, Quadratic Forms, and the SVD

- *Spectral theorem* (Theorem 70): a Hermitian A has real eigenvalues and an orthonormal basis of eigenvectors, $A = Q\Lambda Q^*$ with Q unitary and Λ real diagonal. Equivalently $A = \sum_i \lambda_i \vec{q}_i \vec{q}_i^*$ (spectral decomposition).
- A Hermitian A is *positive definite* if $\vec{x}^* A\vec{x} > 0$ for all $\vec{x} \neq \vec{0}$, equivalently if all its eigenvalues are positive; *semidefinite* versions use $\geq$. Changing variables $\vec{x} = Q\vec{y}$ turns the quadratic form into $\sum_i \lambda_i |y_i|^2$ (principal axes theorem).
- On the unit sphere, $\vec{x}^* A\vec{x}$ ranges over $[\lambda_{\min}, \lambda_{\max}]$, attained at the corresponding eigenvectors (Definition 55).
- *SVD*: every $A \in \mathbb{K}^{m\times n}$ factors as $A = U\Sigma V^*$ with U, V unitary and Σ diagonal with entries $\sigma_1 \geq \cdots \geq 0$. The σ_i^2 are the eigenvalues of A^*A, the columns $\vec{v}_i$ of V are its orthonormal eigenvectors, and $\vec{u}_i = A\vec{v}_i/\sigma_i$ for $\sigma_i > 0$. The number of nonzero σ_i is $\operatorname{rank} A$ (Section B.3).

Appendix B

Norms and Unitary Matrices

This appendix collects facts about unitary matrices and matrix norms that are used throughout the course, starting from Lecture 6. We begin with vector and matrix norms in general, following [1, Lecture 3], and then specialize to the 2-norm and the Frobenius norm. Supplementary reading:

- [1] Lectures 2, 3, and 5.
- [2] Sections 3.2.3 and 3.4.

B.1 Vector and Matrix Norms

Norms are the yardsticks of numerical linear algebra: every statement about the size of an error, the accuracy of an approximation, or the convergence of an iteration is made in some norm.

B.1.1 Vector Norms

Definition 99 (Vector Norm)

A *norm* on $\mathbb{K}^m$ is a function $\|\cdot\| : \mathbb{K}^m \to \mathbb{R}$ such that for all $\vec{x}, \vec{y} \in \mathbb{K}^m$ and all $\alpha \in \mathbb{K}$:

1. $\|\vec{x}\| \geq 0$, and $\|\vec{x}\| = 0$ only if $\vec{x} = \vec{0}$;
2. $\|\vec{x} + \vec{y}\| \leq \|\vec{x}\| + \|\vec{y}\|$ (the *triangle inequality*);
3. $\|\alpha\vec{x}\| = |\alpha| \, \|\vec{x}\|$.

In words, a nonzero vector has positive length, the length of a sum is at most the sum of the lengths, and scaling a vector scales its length by the same amount. The most important norms are the p-norms.

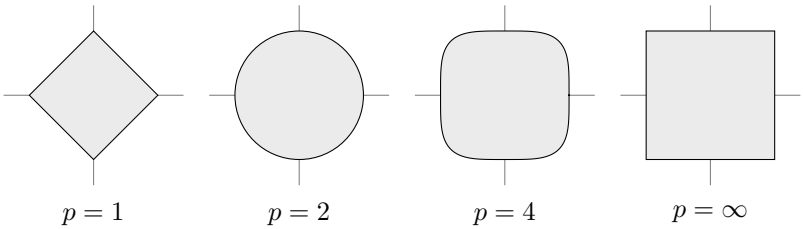
Definition 100 (p-Norms)

For $\vec{x} \in \mathbb{K}^m$,

$$\|\vec{x}\|_1 = \sum_{i=1}^{m} |x_i|\,, \qquad \|\vec{x}\|_2 = \left(\sum_{i=1}^{m} |x_i|^2\right)^{1/2} = \sqrt{\vec{x}^*\vec{x}}, \qquad \|\vec{x}\|_\infty = \max_{1\le i\le m} |x_i|\,, \tag{B.1}$$

and more generally $\|\vec{x}\|_p = (\sum_{i=1}^m |x_i|^p)^{1/p}$ for $1 \le p < \infty$.

A norm is determined by its closed unit ball $\{\vec{x} : \|\vec{x}\| \le 1\}$. For $m = 2$, this gives the following picture.



Unit balls of $\mathbb{R}^2$ in the p-norms. As p grows, the ball grows from the diamond ($p = 1$) to the square ($p = \infty$).

The 2-norm is the Euclidean length, and its unit ball is round. Since the balls are nested, the p-norms decrease as p increases; Corollary 108 below makes this precise, with constants depending on the dimension.

Aside from the p-norms, the most useful norms are the *weighted norms*. Given any norm $\|\cdot\|$ and a diagonal matrix $W = \operatorname{diag}(w_1, \dots, w_m)$ with all $w_i \ne 0$, we set

$$\|\vec{x}\|_W := \|W\vec{x}\|\,. \tag{B.2}$$

For example, the weighted 2-norm is $\|\vec{x}\|_W = \left(\sum_i |w_i x_i|^2\right)^{1/2}$, and its unit ball is an axis-aligned ellipse with semiaxes $1/|w_i|$. More generally, $\|W\vec{x}\|$ is a norm for *any* nonsingular W: properties 2 and 3 are inherited from $\|\cdot\|$ because W is linear, and $\|W\vec{x}\| = 0$ forces $W\vec{x} = \vec{0}$, hence $\vec{x} = \vec{0}$. Weighted norms are natural when the entries of $\vec{x}$ are measured in different units.

B.1.2 Induced Matrix Norms

An $m \times n$ matrix can be viewed as a vector in an mn-dimensional space, so any vector norm on $\mathbb{K}^{mn}$ measures the size of a matrix. More useful, however, are norms that measure what A *does* as a map from $\mathbb{K}^n$ to $\mathbb{K}^m$.

Definition 101 (Induced Matrix Norm)

Let $\|\cdot\|_{(n)}$ and $\|\cdot\|_{(m)}$ be norms on $\mathbb{K}^n$ and $\mathbb{K}^m$. The *induced matrix norm* of $A \in \mathbb{K}^{m\times n}$ is the smallest C such that $\|A\vec{x}\|_{(m)} \le C\,\|\vec{x}\|_{(n)}$ for all $\vec{x} \in \mathbb{K}^n$, that is,

$$\|A\|_{(m,n)} := \sup_{\vec{x}\ne\vec{0}} \frac{\|A\vec{x}\|_{(m)}}{\|\vec{x}\|_{(n)}} = \sup_{\|\vec{x}\|_{(n)}=1} \|A\vec{x}\|_{(m)}\,. \tag{B.3}$$

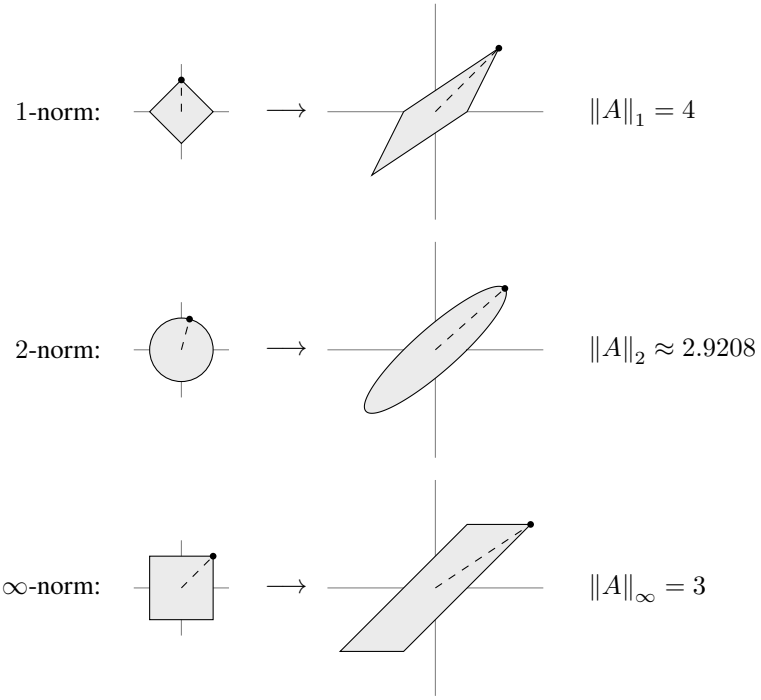
When both norms are p-norms, we write $\|A\|_p$.

So $\|A\|_{(m,n)}$ is the largest factor by which A can stretch a vector. The two expressions agree by property 3 of Definition 99: scaling $\vec{x}$ does not change the ratio, so it suffices to look at unit vectors. In particular, $\|A\|$ is determined by the image of the unit ball under A, which makes induced norms easy to visualize.

Example 102 (A 2×2 Matrix in Three Norms)**.** Take

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix}. \tag{B.4}$$

Since A is real, we may draw it as a map of $\mathbb{R}^2$. It sends $\vec{e}_1$ to its first column $(1,0)^\top$ and $\vec{e}_2$ to its second column $(2,2)^\top$. Mapping the unit balls of the 1-, 2-, and ∞-norms gives the following picture.



Left: unit balls of $\mathbb{R}^2$. Right: their images under A. The dashed vectors are the ones that A stretches the most in each norm.

In the 1-norm, the most amplified unit vector is $\vec{e}_2$, which is mapped to $(2,2)^\top$ with 1-norm 4. In the ∞-norm, it is $(1,1)^\top$, which is mapped to $(3,2)^\top$ with ∞-norm 3. In the 2-norm, the image of the unit circle is an ellipse, and $\|A\|_2$ is the length of its longest semiaxis, about 2.9208. This must be at least $\|A\vec{e}_2\|_2 = \sqrt{8} \approx 2.8284$; the exact value is $\sigma_{\max}(A)$ by Theorem 114.

Example 103 (The p-Norm of a Diagonal Matrix)**.** Let $D = \operatorname{diag}(d_1, \ldots, d_m)$. For $1 \le p < \infty$,

$$\|D\vec{x}\|_p^p = \sum_{i=1}^{m} |d_i x_i|^p \le \left(\max_i |d_i|\right)^p \|\vec{x}\|_p^p, \tag{B.5}$$

with equality for $\vec{x} = \vec{e}_i$ where i maximizes $|d_i|$; the case $p = \infty$ is the same argument with sums replaced by maxima. Hence $\|D\|_p = \max_i |d_i|$ for every p. Geometrically, D maps the 2-norm unit sphere to an ellipsoid with semiaxes $|d_i|$, and $\|D\|_2$ is the longest semiaxis. The SVD shows that *every* matrix maps the unit sphere to an ellipsoid; only the axes may be rotated.

For $p = 1$ and $p = \infty$, the induced norm has a closed form in terms of the entries.

Proposition 104 (The 1- and ∞-Norms of a Matrix)

Let $A \in \mathbb{K}^{m\times n}$ have columns $\vec{a}_1, \dots, \vec{a}_n$ and rows $\vec{\alpha}_1^*, \dots, \vec{\alpha}_m^*$. Then

$$\|A\|_1 = \max_{1\le j\le n} \|\vec{a}_j\|_1 \quad \text{(maximum column sum)}, \qquad \|A\|_\infty = \max_{1\le i\le m} \|\vec{\alpha}_i\|_1 \quad \text{(maximum row sum)}. \tag{B.6}$$

Proof. For the 1-norm, let $\|\vec{x}\|_1 = 1$. By the triangle inequality,

$$\|A\vec{x}\|_1 = \left\| \sum_{j=1}^n x_j \vec{a}_j \right\|_1 \le \sum_{j=1}^n |x_j| \, \|\vec{a}_j\|_1 \le \max_j \|\vec{a}_j\|_1 , \tag{B.7}$$

since the weights $|x_j|$ sum to 1. Taking $\vec{x} = \vec{e}_j$ for the column j with the largest 1-norm attains the bound.

For the ∞-norm, let $\|\vec{x}\|_\infty = 1$. Each entry of $A\vec{x}$ satisfies $|(A\vec{x})_i| \le \sum_j |A_{ij}| \, |x_j| \le \|\vec{\alpha}_i\|_1$, so $\|A\vec{x}\|_\infty \le \max_i \|\vec{\alpha}_i\|_1$. To attain it, pick the row i with the largest 1-norm and set $x_j = \overline{A_{ij}} / |A_{ij}|$ (and $x_j = 1$ if $A_{ij} = 0$). Then $\|\vec{x}\|_\infty = 1$ and $(A\vec{x})_i = \sum_j |A_{ij}| = \|\vec{\alpha}_i\|_1$. □

In the example above, the column sums are 1 and 4 and the row sums are 3 and 2, matching the picture. Both norms cost only $O(mn)$ operations, whereas $\|A\|_2$ requires the largest singular value. Computing $\|A\|_p$ for other p is harder still.

B.1.3 Cauchy–Schwarz and Hölder Inequalities

To bound matrix norms when $p \neq 1, \infty$, we bound inner products by p-norms.

Theorem 105 (Hölder and Cauchy–Schwarz Inequalities)

Let $1 \le p, q \le \infty$ with $1/p + 1/q = 1$. Then for all $\vec{x}, \vec{y} \in \mathbb{K}^m$,

$$|\vec{x}^* \vec{y}| \le \|\vec{x}\|_p \, \|\vec{y}\|_q . \tag{B.8}$$

The case $p = q = 2$ is the *Cauchy–Schwarz inequality* $|\vec{x}^*\vec{y}| \le \|\vec{x}\|_2 \, \|\vec{y}\|_2$.

Both bounds are tight: for every $\vec{x}$ there is a $\vec{y} \neq \vec{0}$ attaining equality. For Cauchy–Schwarz, $\vec{y} = \vec{x}$ works. Proofs can be found in any linear algebra text. These inequalities immediately give the 2-norm of two simple kinds of matrices.

Example 106 (The 2-Norm of a Row Vector)**.** Let $A = \vec{a}^*$ be a single row. By Cauchy–Schwarz, $\|A\vec{x}\|_2 = |\vec{a}^*\vec{x}| \le \|\vec{a}\|_2 \, \|\vec{x}\|_2$, with equality for $\vec{x} = \vec{a}$. Hence $\|\vec{a}^*\|_2 = \|\vec{a}\|_2$.

Example 107 (The 2-Norm of an Outer Product)**.** Let $A = \vec{u}\vec{v}^*$ with $\vec{u} \in \mathbb{K}^m$ and $\vec{v} \in \mathbb{K}^n$. For every $\vec{x} \in \mathbb{K}^n$,

$$\|A\vec{x}\|_2 = \|\vec{u}(\vec{v}^*\vec{x})\|_2 = \|\vec{u}\|_2 \, |\vec{v}^*\vec{x}| \le \|\vec{u}\|_2 \, \|\vec{v}\|_2 \, \|\vec{x}\|_2 , \tag{B.9}$$

with equality for $\vec{x} = \vec{v}$. Hence $\|\vec{u}\vec{v}^*\|_2 = \|\vec{u}\|_2 \, \|\vec{v}\|_2$. The Frobenius norm gives the same value: $\|\vec{u}\vec{v}^*\|_F^2 = \sum_{i,j} |u_i|^2 \, |v_j|^2 = \|\vec{u}\|_2^2 \, \|\vec{v}\|_2^2$, consistent with item 3 of Proposition 116 for $r = 1$.

Cauchy–Schwarz also lets us compare the p-norms in a fixed dimension.

Corollary 108 (Comparing p-Norms)**.** *For $\vec{x} \in \mathbb{K}^m$,*

$$\|\vec{x}\|_\infty \le \|\vec{x}\|_2 \le \|\vec{x}\|_1 \le \sqrt{m}\, \|\vec{x}\|_2 \le m\, \|\vec{x}\|_\infty , \tag{B.10}$$

and for $A \in \mathbb{K}^{m\times n}$,

$$\frac{1}{\sqrt{n}} \|A\|_\infty \le \|A\|_2 \le \sqrt{m}\, \|A\|_\infty , \qquad \frac{1}{\sqrt{m}} \|A\|_1 \le \|A\|_2 \le \sqrt{n}\, \|A\|_1 . \tag{B.11}$$

Proof. The first two inequalities hold because $\max_i |x_i|^2 \le \sum_i |x_i|^2 \le (\sum_i |x_i|)^2$. The third is Cauchy–Schwarz applied to the vector of the $|x_i|$ and the all-ones vector, and the fourth is $\sum_i |x_i|^2 \le m \max_i |x_i|^2$.

For the matrix bounds, chain the vector bounds through the definition. For instance,

$$\|A\vec{x}\|_2 \le \sqrt{m}\, \|A\vec{x}\|_\infty \le \sqrt{m}\, \|A\|_\infty \|\vec{x}\|_\infty \le \sqrt{m}\, \|A\|_\infty \|\vec{x}\|_2 \tag{B.12}$$

gives $\|A\|_2 \le \sqrt{m}\, \|A\|_\infty$, and $\|A\vec{x}\|_\infty \le \|A\vec{x}\|_2 \le \|A\|_2 \|\vec{x}\|_2 \le \sqrt{n}\, \|A\|_2 \|\vec{x}\|_\infty$ gives the other bound. The 1-norm bounds follow in the same way. □

In particular, all p-norms agree up to factors that depend only on the dimension. Such factors are harmless in $O(\varepsilon_{\text{mach}})$ statements, so a method that is stable in one p-norm is stable in all of them.

B.1.4 Bounding $\|AB\|$

Proposition 109 (Submultiplicativity of Induced Norms)

Let $\|\cdot\|_{(\ell)}$, $\|\cdot\|_{(m)}$, $\|\cdot\|_{(n)}$ be norms on $\mathbb{K}^\ell$, $\mathbb{K}^m$, $\mathbb{K}^n$, and let $A \in \mathbb{K}^{\ell\times m}$ and $B \in \mathbb{K}^{m\times n}$. Then

$$\|AB\|_{(\ell,n)} \le \|A\|_{(\ell,m)}\, \|B\|_{(m,n)} . \tag{B.13}$$

Proof. Apply the definition twice: $\|AB\vec{x}\|_{(\ell)} \le \|A\|_{(\ell,m)}\, \|B\vec{x}\|_{(m)} \le \|A\|_{(\ell,m)}\, \|B\|_{(m,n)}\, \|\vec{x}\|_{(n)}$. □

In particular, $\left\|A^k\right\| \le \|A\|^k$ for a square matrix in any induced norm. This is generally not an equality. For example, $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ has $\|A\|_p = 1$ for every p, but $A^2 = 0$.

A related bound concerns eigenvalues. If $A\vec{x} = \lambda\vec{x}$ with $\vec{x} \ne \vec{0}$, then $|\lambda|\, \|\vec{x}\| = \|A\vec{x}\| \le \|A\|\, \|\vec{x}\|$. So the *spectral radius* $\rho(A) \coloneqq \max\{|\lambda| : \lambda \text{ an eigenvalue of } A\}$ satisfies

$$\rho(A) \le \|A\| \tag{B.14}$$

in every induced norm. The same example shows that the inequality can be strict: $\rho(A) = 0$ but $\|A\| = 1$.

B.1.5 General Matrix Norms

Matrix norms do not have to be induced by vector norms. In general, a *matrix norm* is any function on $\mathbb{K}^{m\times n}$ that satisfies the three conditions of Definition 99 on this mn-dimensional vector space:

$$\|A\| \ge 0 \text{ with equality only if } A = 0, \qquad \|A + B\| \le \|A\| + \|B\| , \qquad \|\alpha A\| = |\alpha|\, \|A\| . \tag{B.15}$$

The most important matrix norm that is not induced by a vector norm is the Frobenius norm, the subject of Section B.4. Finally, the 2-norm and the Frobenius norm are both invariant under multiplication by unitary matrices; this is Theorem 111 below.

B.2 Unitary Transformations

The error analysis of Lecture 6 uses one property of unitary matrices: they do not change the length of a vector. In this section we collect this and a few related facts, which we also use in Lectures 5 and 7.

Definition 110 (Unitary and Orthogonal Matrices)

We say that a square matrix $Q \in \mathbb{K}^{m\times m}$ is *unitary* if $Q^*Q = I$. When $\mathbb{K} = \mathbb{R}$, this condition reads $Q^\top Q = I$, and we call Q *orthogonal*. We say that a tall matrix $\hat{Q} \in \mathbb{K}^{m\times n}$ has *orthonormal columns* if $\hat{Q}^*\hat{Q} = I_n$.

The condition $Q^*Q = I$ just says that the columns of Q are orthonormal. Since Q is square, it also gives $Q^{-1} = Q^*$ and $QQ^* = I$. In particular, inverting a unitary matrix costs nothing: the solution of $Q\vec{x} = \vec{b}$ is $\vec{x} = Q^*\vec{b}$.

Theorem 111 (Unitary Matrices Preserve the 2-Norm)

Let $Q \in \mathbb{K}^{m\times m}$ be unitary. Then for all $\vec{x}, \vec{y} \in \mathbb{K}^m$,

$$\langle Q\vec{x}, Q\vec{y}\rangle = \langle \vec{x}, \vec{y}\rangle\,, \qquad \|Q\vec{x}\|_2 = \|\vec{x}\|_2\,. \tag{B.16}$$

As a consequence, for any $A \in \mathbb{K}^{m\times n}$ and unitary $U \in \mathbb{K}^{m\times m}$, $V \in \mathbb{K}^{n\times n}$:

1. $\|UAV\|_2 = \|A\|_2$ and $\|UAV\|_F = \|A\|_F$.
2. UAV has the same singular values as A. In particular, $\kappa_2(UAV) = \kappa_2(A)$.
3. $\|Q\|_2 = \|Q^{-1}\|_2 = 1$ and $\kappa_2(Q) = 1$. All singular values of Q are 1, every eigenvalue of Q satisfies $|\lambda| = 1$, and $|\det Q| = 1$.

Proof. We have $\langle Q\vec{x}, Q\vec{y}\rangle = \vec{x}^*Q^*Q\vec{y} = \vec{x}^*\vec{y}$. Taking $\vec{y} = \vec{x}$ gives $\|Q\vec{x}\|_2 = \|\vec{x}\|_2$.

For item 1, the norm identity gives $\|UA\vec{x}\|_2 = \|A\vec{x}\|_2$ for every $\vec{x}$, so $\|UA\|_2 = \|A\|_2$. For the right factor, note that $\vec{y} = V\vec{x}$ runs over all unit vectors as $\vec{x}$ does, so

$$\|AV\|_2 = \sup_{\|\vec{x}\|_2=1} \|AV\vec{x}\|_2 = \sup_{\|\vec{y}\|_2=1} \|A\vec{y}\|_2 = \|A\|_2\,. \tag{B.17}$$

The statement for the Frobenius norm is item 1 of Proposition 116.

For item 2, let $A = W\Sigma Z^*$ be an SVD. Then $UAV = (UW)\Sigma(V^*Z)^*$. Products of unitary matrices are unitary, so this is an SVD of UAV with the same Σ.

For item 3, $Q = Q \cdot I \cdot I^*$ is an SVD with $\Sigma = I$. If $Q\vec{x} = \lambda\vec{x}$ with $\vec{x} \neq \vec{0}$, then $\|\vec{x}\|_2 = \|Q\vec{x}\|_2 = |\lambda|\,\|\vec{x}\|_2$, so $|\lambda| = 1$. Finally, $|\det Q|^2 = \det(Q^*Q) = \det I = 1$. □

The converse is also true: if $\|Q\vec{x}\|_2 = \|\vec{x}\|_2$ for every $\vec{x}$, then Q is unitary. (One recovers inner products from norms by the polarization identity.) In other words, unitary matrices are exactly the linear maps that preserve lengths. Geometrically, they are rotations and reflections.

B.2.1 Examples

Several matrices we have already seen are unitary.

- *Householder reflectors* $F = I - 2\vec{v}\vec{v}^*/(\vec{v}^*\vec{v})$. They are Hermitian, satisfy $F^2 = I$, and have $\det F = -1$.
- *Givens rotations* (Lecture 7). They have $\det = 1$ and change only two coordinates.
- *Permutation matrices*. So the row swaps of partial pivoting (Lecture 4) do not change any norms, and all of the instability of LU comes from the triangular factors.
- *Diagonal matrices* $D = \operatorname{diag}(d_1, \dots, d_m)$ with $|d_i| = 1$. These describe exactly the freedom in the QR factorization, since $A = (QD)(D^*R)$. The condition $r_{jj} > 0$ removes this freedom (Theorem 33).

B.2.2 Tall Matrices with Orthonormal Columns

Now let $\hat{Q} \in \mathbb{K}^{m\times n}$ with $m > n$ have orthonormal columns. The same one-line proof shows that $\left\|\hat{Q}\vec{x}\right\|_2 = \|\vec{x}\|_2$, and hence, as in item 1 of Theorem 111, $\left\|\hat{Q}B\right\|_2 = \|B\|_2$ and $\left\|\hat{Q}B\right\|_F = \|B\|_F$ for every $B \in \mathbb{K}^{n\times p}$. Taking conjugate transposes, the same holds for multiplication on the right by a matrix with orthonormal rows. However, $\hat{Q}^*$ does *not* preserve lengths. The matrix $\hat{Q}\hat{Q}^*$ is not the identity; it is the orthogonal projector onto $\mathcal{R}(\hat{Q})$. So $\left\|\hat{Q}^*\vec{y}\right\|_2 \le \|\vec{y}\|_2$, with equality only if $\vec{y} \in \mathcal{R}(\hat{Q})$.

This is exactly what makes QR useful for least squares. Take a full QR factorization $A = QR = \begin{bmatrix}\hat{Q} & \hat{Q}_\perp\end{bmatrix}\begin{bmatrix}\hat{R}\\ 0\end{bmatrix}$. Since Q^* preserves lengths, we can apply it inside the norm:

$$\left\|A\vec{x} - \vec{b}\right\|_2^2 = \left\|Q^*(A\vec{x} - \vec{b})\right\|_2^2 = \left\|\hat{R}\vec{x} - \hat{Q}^*\vec{b}\right\|_2^2 + \left\|\hat{Q}_\perp^*\vec{b}\right\|_2^2. \tag{B.18}$$

The second term does not depend on $\vec{x}$. So the minimizer solves $\hat{R}\vec{x} = \hat{Q}^*\vec{b}$, and the minimal residual is $\left\|\hat{Q}_\perp^*\vec{b}\right\|_2$ (compare Lecture 5).

B.2.3 Why Unitary Transformations Are Good for Computation

1. *One step does not amplify errors.* In Theorem 26 we have $\kappa(Q_k) = 1$, so each step has relative backward error $O(\varepsilon_{\text{mach}})$ with no extra factor.
2. *Many steps do not amplify errors.* A product of unitary matrices is unitary. So when we pull an error back through all the previous steps, its size does not change. The backward errors of n steps add up to $O(n\varepsilon_{\text{mach}})$ instead of multiplying (see Section 6.7).
3. *No element growth.* Every intermediate matrix has the same 2-norm as A, so there is nothing like the growth factor of LU. Compare this with Example 27: a triangular matrix with entries bounded by 1 can still have an exponentially large inverse, while $\left\|Q^{-1}\right\|_2 = \|Q\|_2 = 1$ always.
4. *Errors look the same before and after.* An error of size δ in $\vec{x}$ is an error of size δ in $Q\vec{x}$. So we can move errors between the input and the output without losing anything.

Note that this invariance holds for the 2-norm and the Frobenius norm, but not for other norms. For example, rotating $\vec{e}_1$ by $45°$ gives $(1/\sqrt{2}, 1/\sqrt{2})^\top$, whose ∞-norm is $1/\sqrt{2}$ and whose 1-norm is $\sqrt{2}$. This is why stability results for orthogonal methods are stated in the 2-norm. In other norms they still hold, but with extra constants that depend on the dimension.

As a final note, everything above assumes that the transformation is *exactly* unitary. A matrix that we compute column by column, such as the $\hat{Q}$ of modified Gram–Schmidt, is only unitary up to $O(\varepsilon_{\text{mach}}\kappa(A))$, and it

loses these advantages to the same extent. Householder's Q is different. Each reflector built from a computed vector $\tilde{\vec{v}}_k$ is exactly unitary, so rounding errors only change *which* unitary matrix we get, not *whether* it is unitary (Lecture 7).

B.3 The 2-Norm, Singular Values, and Condition Numbers

Most error bounds in this course are stated in the 2-norm. Here we record how it is expressed through the SVD; these facts are proved in Homework 1, Problem 3.

Definition 112 (Full and Reduced SVD)

Let $A \in \mathbb{K}^{m\times n}$ and $p = \min(m,n)$. A *full SVD* is a factorization

$$A = U\Sigma V^*, \qquad U \in \mathbb{K}^{m\times m}, \quad \Sigma \in \mathbb{R}^{m\times n}, \quad V \in \mathbb{K}^{n\times n}, \tag{B.19}$$

with U and V unitary and Σ "diagonal" of the same shape as A: only $\Sigma_{11}, \dots, \Sigma_{pp}$ can be nonzero, and these are the *singular values* $\sigma_1 \geq \cdots \geq \sigma_p \geq 0$. Dropping the zero blocks of Σ together with the columns of U or V they multiply gives the *reduced SVD*

$$A = \hat{U}\hat{\Sigma}\hat{V}^*, \qquad \hat{U} \in \mathbb{K}^{m\times p}, \quad \hat{\Sigma} = \operatorname{diag}(\sigma_1, \dots, \sigma_p) \in \mathbb{R}^{p\times p}, \quad \hat{V} \in \mathbb{K}^{n\times p}, \tag{B.20}$$

where $\hat{U}$ and $\hat{V}$ have orthonormal columns.

In the full SVD, Σ has a zero block below or beside its square diagonal part $D = \operatorname{diag}(\sigma_1, \dots, \sigma_p)$:

$$\Sigma = \begin{bmatrix} D \\ 0 \end{bmatrix} \ (m > n), \qquad \Sigma = \begin{bmatrix} D & 0 \end{bmatrix} \ (m < n). \tag{B.21}$$

The reduced form is more economical for computation, while the full form is more convenient in proofs, because square unitary U and V preserve norms in both directions (Theorem 111). Every matrix has an SVD, and the singular values are unique [1, Theorem 4.1].

Definition 113 (Induced 2-Norm)

The 2-*norm* of $A \in \mathbb{K}^{m\times n}$ is the matrix norm induced by the vector 2-norm (Definition 101),

$$\|A\|_2 \coloneqq \sup_{\vec{x}\neq\vec{0}} \frac{\|A\vec{x}\|_2}{\|\vec{x}\|_2}. \tag{B.22}$$

Theorem 114 (The 2-Norm Through the SVD)

Let $A \in \mathbb{K}^{m\times n}$ have singular values $\sigma_{\max} = \sigma_1 \geq \cdots \geq \sigma_{\min(m,n)} = \sigma_{\min}$. Then:

1. $\|A\|_2 = \sigma_{\max}$.
2. If $m \geq n$, then $\|A\vec{x}\|_2 \geq \sigma_{\min} \|\vec{x}\|_2$ for all $\vec{x}$, with equality for the last right singular vector.
3. If A is square and invertible, then $\left\|A^{-1}\right\|_2 = 1/\sigma_{\min}$, and hence

$$\kappa_2(A) \coloneqq \|A\|_2 \left\|A^{-1}\right\|_2 = \frac{\sigma_{\max}}{\sigma_{\min}}. \tag{B.23}$$

Proof. Take an SVD $A = U\Sigma V^*$ and put $\vec{y} = V^*\vec{x}$. Since U and V are unitary (Theorem 111), $\|A\vec{x}\|_2 = \|\Sigma\vec{y}\|_2$ and $\|\vec{y}\|_2 = \|\vec{x}\|_2$. Now

$$\|\Sigma\vec{y}\|_2^2 = \sum_{i=1}^{\min(m,n)} \sigma_i^2 |y_i|^2 . \tag{B.24}$$

The sum stops at $\min(m,n)$, not at n: for $m < n$ the last $n - m$ entries of $\vec{y}$ are multiplied by the zero block of Σ and do not contribute. In particular a wide matrix has a nontrivial null space, which is why item 2 needs $m \geq n$. This is at most $\sigma_{\max}^2 \|\vec{y}\|_2^2$, with equality for $\vec{y} = \vec{e}_1$, which gives item 1. If $m \geq n$, the sum runs over all n entries of $\vec{y}$, so it is at least $\sigma_{\min}^2 \|\vec{y}\|_2^2$, with equality for $\vec{y} = \vec{e}_n$; this gives item 2. For item 3, $A^{-1} = V\Sigma^{-1}U^*$ is an SVD up to reordering, so the singular values of A^{-1} are the $1/\sigma_i$, and the largest of them is $1/\sigma_{\min}$. □

For a tall matrix with full column rank we *define* $\kappa_2(A) \coloneqq \sigma_{\max}/\sigma_{\min}$; this is the condition number that governs least squares (Lecture 5). In both cases, $\kappa_2(A)$ is the worst-case condition number of the map $\vec{x} \mapsto A\vec{x}$ from Lecture 1:

$$\sup_{\vec{x}\neq\vec{0}} \frac{\|A\|_2 \|\vec{x}\|_2}{\|A\vec{x}\|_2} = \frac{\sigma_{\max}}{\sigma_{\min}}, \tag{B.25}$$

since $\|A\vec{x}\|_2 \,/\, \|\vec{x}\|_2$ is smallest, equal to $\sigma_{\min}$, for the last right singular vector. In words, $\kappa_2(A)$ is the ratio of the largest to the smallest stretching factor of A.

B.4 The Frobenius Norm

In Lecture 6 we bounded $\|L\|_2$ through the Frobenius norm. It is often the easiest matrix norm to compute and to estimate.

Definition 115 (Frobenius Norm)

The *Frobenius norm* of $A \in \mathbb{K}^{m\times n}$ is

$$\|A\|_F \coloneqq \left(\sum_{i=1}^{m} \sum_{j=1}^{n} |A_{ij}|^2 \right)^{1/2}. \tag{B.26}$$

In other words, $\|A\|_F$ is the Euclidean norm of A if we stack its columns into one long vector of length mn. Equivalently, $\|A\|_F^2 = \sum_j \|\vec{a}_j\|_2^2$ is the sum of the squared column norms. These squared column norms are

exactly the diagonal entries of A^*A, and the squared row norms are the diagonal entries of AA^*, so

$$\|A\|_F^2 = \operatorname{trace}(A^*A) = \operatorname{trace}(AA^*). \tag{B.27}$$

Proposition 116 (Properties of the Frobenius Norm)

Let $A \in \mathbb{K}^{m\times n}$ have rank r and singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$. Then:

1. *Unitary invariance.* $\|UAV\|_F = \|A\|_F$ for all unitary $U \in \mathbb{K}^{m\times m}$ and $V \in \mathbb{K}^{n\times n}$.
2. *Singular values.* $\|A\|_F = (\sigma_1^2 + \cdots + \sigma_r^2)^{1/2}$.
3. *Comparison with the 2-norm.* $\|A\|_2 \leq \|A\|_F \leq \sqrt{r}\,\|A\|_2 \leq \sqrt{\min(m,n)}\,\|A\|_2$.
4. *Compatibility and submultiplicativity.* $\|A\vec{x}\|_2 \leq \|A\|_F\,\|\vec{x}\|_2$, and $\|AB\|_F \leq \|A\|_2\,\|B\|_F \leq \|A\|_F\,\|B\|_F$.
5. *Only absolute values matter.* $\|A\|_F = \||A|\|_F$, and if $|A| \leq |B|$ entrywise, then $\|A\|_F \leq \|B\|_F$.

Proof. For item 1, using $U^*U = I$ and the cyclic property of the trace,

$$\|UAV\|_F^2 = \operatorname{trace}(V^*A^*U^*UAV) = \operatorname{trace}(V^*A^*AV) = \operatorname{trace}(A^*A) = \|A\|_F^2\,. \tag{B.28}$$

For item 2, take an SVD $A = U\Sigma V^*$. By item 1, $\|A\|_F = \|\Sigma\|_F = (\sum_i \sigma_i^2)^{1/2}$.

For item 3, use item 2 and $\|A\|_2 = \sigma_1$: we have $\sigma_1^2 \leq \sum_i \sigma_i^2 \leq r\sigma_1^2$.

For item 4, let $\vec{\alpha}_i^*$ be the i-th row of A. By the Cauchy–Schwarz inequality, $|\vec{\alpha}_i^*\vec{x}|^2 \leq \|\vec{\alpha}_i\|_2^2\,\|\vec{x}\|_2^2$. Summing over i gives $\|A\vec{x}\|_2^2 \leq \|A\|_F^2\,\|\vec{x}\|_2^2$. For the product, apply $\left\|A\vec{b}_j\right\|_2 \leq \|A\|_2\left\|\vec{b}_j\right\|_2$ to each column $\vec{b}_j$ of B and sum the squares. The last inequality then follows from $\|A\|_2 \leq \|A\|_F$.

Item 5 follows directly from the definition. □

Note that the Frobenius norm is *not* induced by any vector norm. Every induced norm satisfies $\|I\| = \sup_{\vec{x}\neq\vec{0}} \|\vec{x}\| / \|\vec{x}\| = 1$, but $\|I_n\|_F = \sqrt{n}$. Still, by item 4 it is compatible with the Euclidean norm on vectors.

B.4.1 Why It Is Useful in Error Analysis

- *It is cheap.* Computing $\|A\|_F$ takes $O(mn)$ operations, while $\|A\|_2 = \sigma_1$ requires the SVD (or an estimate of it). By item 3, the two norms differ by at most a factor of $\sqrt{\min(m,n)}$, and such dimension-dependent constants do not matter in $O(\varepsilon_{\text{mach}})$ statements.
- *It turns entrywise bounds into norm bounds.* The backward error of LU from Lecture 4 satisfies $|E| \leq m\varepsilon_{\text{mach}}\,|L|\,|U|$. By items 5 and 4,

$$\|E\|_F \leq m\varepsilon_{\text{mach}}\,\||L|\,|U|\|_F \leq m\varepsilon_{\text{mach}}\,\|L\|_F\,\|U\|_F\,. \tag{B.29}$$

  With partial pivoting, $\|L\|_F \leq m$ (Lecture 6), so everything depends on $\|U\|_F$, i.e., on the growth factor.
- *It is unitarily invariant.* Just like the 2-norm, the Frobenius norm of an error does not change under orthogonal transformations such as Householder reflectors.

B.4.2 Low-Rank Approximation

Since $\|A-B\|_F^2 = \sum_{i,j} |A_{ij}-B_{ij}|^2$ is a sum of squared entrywise errors, approximating a matrix in the Frobenius norm is a least squares problem. The Eckart–Young theorem also holds in this norm: the truncated SVD $A_k = \sum_{i\le k} \sigma_i \vec{u}_i \vec{v}_i^*$ satisfies

$$\min_{\operatorname{rank}(B)\le k} \|A-B\|_F = \|A-A_k\|_F = \left(\sigma_{k+1}^2 + \cdots + \sigma_r^2\right)^{1/2}. \tag{B.30}$$

In the 2-norm, the corresponding error is $\|A-A_k\|_2 = \sigma_{k+1}$ [2, §3.2.3]. We will come back to this when the course covers low-rank approximation.

Appendix C

The Floating-Point Model

This appendix collects background on floating-point arithmetic that is used throughout the course. Supplementary reading:

- [1] Lectures 13 and 14.
- [2] Section 1.5.
- [5] Section 2.4.

C.1 IEEE 754 Formats

The formats in common use are listed below; b_p counts the implicit bit, so a significand field with $b_p - 1$ stored bits provides b_p bits of precision.

Format	Total bits	Exponent bits	Stored significand bits	b_p	Bias	Decimal digits
Float64	64	11	52	53	1023	≈ 15.9
Float32	32	8	23	24	127	≈ 7.2
Float16	16	5	10	11	15	≈ 3.3
BFloat16	16	8	7	8	127	≈ 2.4

The exponent field uses a biased encoding, and its two extreme values are reserved:

- An all-zeros exponent encodes ± 0 (if the significand is also zero) or a *subnormal* number, which drops the implicit leading 1 and uses $0.f \times 2^{E_{\min}}$. Subnormals provide *gradual underflow*, which guarantees $x \neq y \implies x - y \neq 0$.
- An all-ones exponent encodes $\pm\infty$ (zero significand) or `NaN` (nonzero significand).

The normal range of Float64 is therefore roughly 2.2×10^{-308} to 1.8×10^{308}. IEEE defines five exceptions (invalid, division by zero, overflow, underflow, inexact), and specifies that ∞ and `NaN` propagate through arithmetic instead of halting it. Demmel [2, §1.5] stresses the value of this: an algorithm can run to completion and check its result at the end, instead of branching at every step. Note that `NaN != NaN` is always true, so one must test with `isnan`.

C.2 Machine Epsilon and the Model of Arithmetic

C.2.1 Two Conventions for Machine Epsilon

Two different quantities are both called "machine epsilon" in the literature, and they differ by a factor of 2.

Definition 117 (Machine Epsilon and Unit Roundoff)

Let b_p be the precision of a floating-point format.

- The *machine epsilon* $\varepsilon = \varepsilon_{\text{mach}} := 2^{-(b_p-1)}$ is the distance from 1 to the next larger floating-point number. For Float64, $\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$. This is the convention we use, as do Julia's `eps(Float64)`, and of C's `DBL_EPSILON`.
- The *unit roundoff* $u := 2^{-b_p} = \varepsilon/2$ is the largest relative rounding error, i.e., half a spacing. For Float64, $u = 2^{-53} \approx 1.11 \times 10^{-16}$. Trefethen and Bau [1] and Demmel [2] call this quantity *machine epsilon* ($\epsilon_{\text{machine}}$, resp. ε), and Golub and Van Loan [5] call it the unit roundoff. So when copying a bound from these books, their "machine epsilon" is our $u = \varepsilon_{\text{mach}}/2$.

The two quantities answer different questions: ε describes how dense the floating-point grid is, while u describes how much error rounding to that grid causes. They differ by a factor of 2 because rounding goes to the *nearest* floating-point number. On $[2^e, 2^{e+1})$ the spacing is $2^e\varepsilon$, a real number x in this interval is at most half a spacing from its nearest neighbour, and therefore

$$\frac{|\mathrm{fl}(x) - x|}{|x|} \le \frac{2^e\varepsilon/2}{2^e} = \frac{\varepsilon}{2} = u. \tag{C.1}$$

Both conventions survive for historical and practical reasons. Many early machines rounded by *chopping*, i.e., discarding the extra bits, and then the largest relative error is a whole spacing, so the two quantities coincided. They separated only once IEEE 754 made round to nearest the default. Since then, authors on error analysis (Trefethen–Bau, Higham, Golub–Van Loan) use u, because what they need is the bound on δ in $\mathrm{fl}(x) = x(1+\delta)$, whereas programming languages report ε, because the gap from 1 to `nextfloat(1.0)` is what can be observed directly.

Any statement of the form $O(\varepsilon)$ is unaffected by the choice, but constants copied from different textbooks, such as $|\delta| \le u$ or $\gamma_n = nu/(1-nu)$, must be read with care.

The IEEE default rounding mode is *round to nearest, ties to even*, under which both orders in Example 1 (Lecture 1) give 4. Non-associativity does not depend on the rounding mode, however. With ties to even,

$$(1+u)+u = 1 \qquad \text{but} \qquad 1+(u+u) = 1+\varepsilon, \tag{C.2}$$

since on the left each intermediate result is a tie that rounds back to 1, while on the right $u+u = \varepsilon$ and $1+\varepsilon$ are both exact.

C.2.2 The Fundamental Axiom

Error analysis needs rounding to be captured by a rule we can reason with. Let $\mathrm{fl}(x)$ denote the floating-point number nearest to x. Because the relative spacing is bounded, every x in the normal range satisfies

$$\mathrm{fl}(x) = x(1+\delta), \qquad |\delta| \le u. \tag{C.3}$$

IEEE requires $+$, $-$, $\times$, $\div$, and $\sqrt{\cdot}$ to be *correctly rounded*: the exact result is computed in infinite precision and then rounded once. This gives the following model, stated in the same form in [1, Lecture 13], [5, §2.4], and [2, §1.5].

Theorem 118 (Fundamental Axiom of Floating-Point Arithmetic)

For all floating-point numbers x, y and $\circledast \in \{+, -, \times, \div\}$,

$$\mathrm{fl}(x \circledast y) = (x \circledast y)(1+\delta), \qquad |\delta| \le u. \tag{C.4}$$

The axiom has an immediate *backward* reading. For addition,

$$\mathrm{fl}(x+y) = (x+y)(1+\delta) = x(1+\delta) + y(1+\delta), \tag{C.5}$$

so the computed sum is *exactly* the sum of two slightly perturbed inputs. Pushing errors back onto the inputs in this way is the main thread of error analysis in this course; see Section C.3.

C.2.3 Accumulation of Errors

A single operation incurs an error of size $O(u)$. For n operations, Golub and Van Loan write

$$\gamma_n \coloneqq \frac{nu}{1-nu} \approx nu \qquad (nu \ll 1). \tag{C.6}$$

Proposition 119 (Error in Inner Products)

For $\vec{x}, \vec{y} \in \mathbb{R}^n$,

$$\left|\mathrm{fl}(\vec{x}^\top \vec{y}) - \vec{x}^\top \vec{y}\right| \le \gamma_n \sum_{i=1}^{n} |x_i|\,|y_i| = \gamma_n \left|\vec{x}\right|^\top |\vec{y}|\,. \tag{C.7}$$

Similarly, for matrices, $|\mathrm{fl}(AB) - AB| \le \gamma_n |A|\,|B|$ entrywise.

The *shape* of this bound matters much more than its constant. The right-hand side involves $|\vec{x}|^\top |\vec{y}|$, not $\left|\vec{x}^\top \vec{y}\right|$. If all terms have the same sign, the two agree and the relative error is $O(nu)$, which is fine. If instead $\vec{x}^\top \vec{y}$ is a small number obtained by cancelling many terms of both signs, then $|\vec{x}|^\top |\vec{y}| \gg \left|\vec{x}^\top \vec{y}\right|$ and the relative error can be arbitrarily bad. The linear factor n is almost never the real issue (it is a very pessimistic worst case, and random errors grow more like $\sqrt{n}$). The real issue is always *cancellation*.

C.3 Backward Stability

The textbooks [1, Lecture 14] mostly use a stronger notion than the stability defined in Section 1.7 of Lecture 1.

Definition 120 (Backward Stability)

An algorithm $\hat{f}$ for f is *backward stable* if for every x there exists $\tilde{x}$ with

$$\hat{f}(x) = f(\tilde{x}) \qquad \text{and} \qquad \frac{|\tilde{x} - x|}{|x|} = O(u), \tag{C.8}$$

where $\tilde{x}$ is a hypothetical input constructed in the analysis, never computed.

In words: *the computed answer is the exact answer to a nearby problem*. Both halves of the definition matter.

- The two sides of $\hat{f}(x) = f(\tilde{x})$ are computed differently. The left side runs the algorithm on the computer, with rounding, on the true input x. The right side evaluates the exact mathematical function f, with no rounding at all, on the perturbed input $\tilde{x}$.
- The perturbation must be small, $|\tilde{x} - x| \,/\, |x| = O(u)$. Without this condition the definition would be empty, since for most f some $\tilde{x}$ reproduces any output whatsoever.

Backward stability does not make the rounding errors disappear; it is a way of *accounting* for them. All errors committed inside the algorithm are pushed back to the input and reinterpreted as a perturbation $\tilde{x} - x$ of the data. The backward reading of Theorem 118 is the simplest example: $\mathrm{fl}(x + y)$ is the exact sum of $\tilde{x} = x(1 + \delta)$ and $\tilde{y} = y(1 + \delta)$, which are within relative distance u of x and y.

This accounting is useful because the input carries errors of size $O(u)$ anyway, from rounding when it is stored or from measurement. A backward stable algorithm gives the exact answer for some input that is indistinguishable from the one we have, which is as much as can be asked of any algorithm. Whatever error remains is due to the problem, not the algorithm. Backward stability is the gold standard of numerical linear algebra because it cleanly separates the problem from the algorithm, and it yields the classical rule of thumb

$$\text{forward relative error} \lesssim \kappa_f(x) \cdot u, \tag{C.9}$$

or, in a form that is easier to remember,

$$\text{accuracy} \approx \text{condition number (problem)} \times \text{backward error (algorithm)}. \tag{C.10}$$

A backward stable algorithm still returns inaccurate answers for ill-conditioned problems, and this is *not the algorithm's fault*; it cannot be improved upon. Conversely, when a result is inaccurate, the first step should be to estimate the condition number, not to suspect the code.

Concept	Belongs to	Question it answers	Measure
Conditioning	problem f and input x	how much input perturbations are intrinsically amplified	$\kappa_f(x)$
Stability	algorithm $\hat{f}$	whether the implementation amplifies errors further	backward error $O(u)$
Accuracy	final result	whether the computed number is right	$\lesssim \kappa \cdot u$

Some common condition numbers:

Problem	κ	Remark
$f(x) = \sqrt{x}$	$1/2$	always well-conditioned; even shrinks errors
$f(x) = x^n$	n	grows linearly with the power
$f(x) = e^x$	$\lvert x\rvert$	ill-conditioned for large $\lvert x\rvert$
$f(x_1, x_2) = x_1 - x_2$	$\dfrac{\lvert x_1\rvert + \lvert x_2\rvert}{\lvert x_1 - x_2\rvert}$	cancellation (componentwise perturbations)
$\vec{x} \mapsto A\vec{x}$	$\dfrac{\lVert A\rVert \, \lVert\vec{x}\rVert}{\lVert A\vec{x}\rVert} \le \kappa(A)$	depends on the direction of $\vec{x}$
solve $A\vec{x} = \vec{b}$	$\kappa(A) = \lVert A\rVert \, \lVert A^{-1}\rVert$	
polynomial roots	can be enormous	Wilkinson's polynomial

For $f : \mathbb{R}^n \to \mathbb{R}^m$ one replaces f' by the Jacobian $J(\vec{x})$: the absolute condition number is $\hat{\kappa} = \lVert J(\vec{x})\rVert$ and the relative condition number is $\kappa = \lVert J(\vec{x})\rVert \, \lVert\vec{x}\rVert \, / \, \lVert f(\vec{x})\rVert$ [1, Lecture 12].

Bibliography

[1] L. N. Trefethen and D. Bau III. *Numerical Linear Algebra*. SIAM, 1997.

[2] J. W. Demmel. *Applied Numerical Linear Algebra*. SIAM, 1997.

[3] D. C. Lay, S. R. Lay, and J. J. McDonald. *Linear Algebra and Its Applications*, 6th edition. Pearson, 2021.

[4] S. Axler. *Linear Algebra Done Right*, 4th edition. Springer, 2024.

[5] G. H. Golub and C. F. Van Loan. *Matrix Computations*, 3rd edition. Johns Hopkins University Press, 1996.

[6] M. L. Overton. *Numerical Computing with IEEE Floating Point Arithmetic*. SIAM, 2001.

[7] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd edition. SIAM, 2002.

[8] N. J. Higham. *Functions of Matrices: Theory and Computation*. SIAM, 2008.